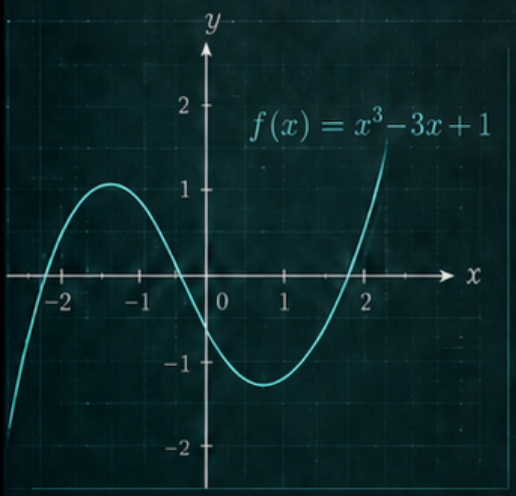
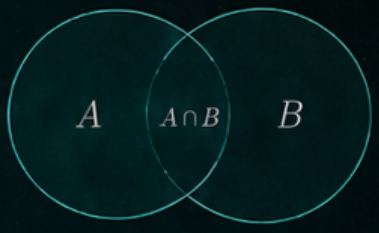
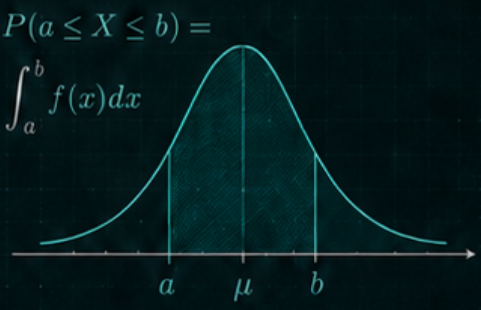


$$\begin{bmatrix} 1 & 2 & 3 & \cdots & n \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \cdots & a_{mn} \end{bmatrix}$$

0110 1011 0101 1100
1001 0100 1110 0011
0101 1101 0010 1011
1100 0101 1011 0100
0011 1100 1010 0111
1011 0010 1101 1001



$$|A \cup B| = |A| + |B| - |A \cap B|$$



MATEMATIK FOR SOFTWARE-INGENIØRER



ARITMETIK OG FUNKTIONER



TALSYSTEMER OG BINÆRE TAL



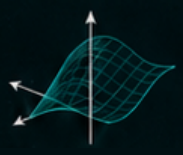
MÆNGDE-LÆRE



SANDSYNLIGHED OG STATISTIK



LINEÆR ALGEBRA



DIFFERENTIAL-REGNING I EN OG FLERE VARIABLE



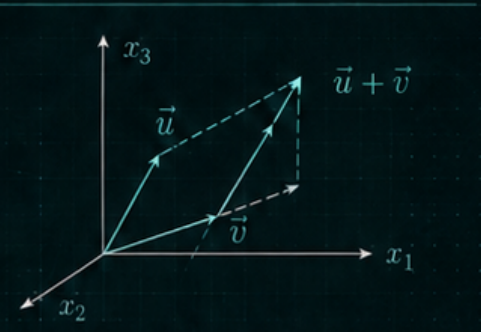
BOOLESK ALGEBRA OG LOGISKE GATES

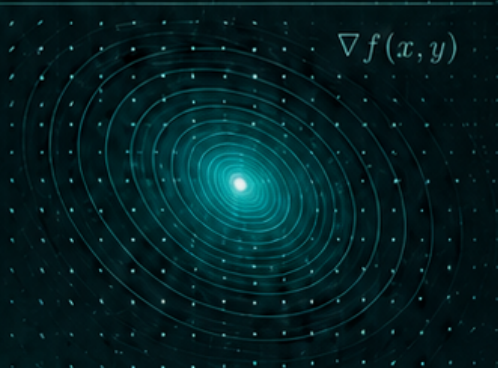


TIDSKOMPLEKSITET OG KODEANALYSE

```
for i = 1 to n
  for j = 1 to n
    sum = sum + A[i][j]
  end for
end for
```

$$T(n) = O(n^2)$$







RICHARD BROOKS

Indhold

I Grundlæggende begreber **1**

Kapitel 0 Grundlæggende aritmetik **2**

0.1 Faktorisering og regnearternes hierarki . . . 2
0.2 Brøker . . . 3
0.3 Potenser, rødder og irrationelle rødder . . . 6
0.4 Brug af formler og indsættelse . . . 9
0.5 Omskrivning af formler . . . 10

Kapitel 1 Funktioner **14**

1.1 Funktioners egenskaber . . . 14
1.2 Grafisk identifikation af funktionstyper . . . 19
1.3 Logaritmer . . . 24

Kapitel 2 Talsystemer **31**

2.1 Binære tal . . . 31
2.2 Oktal og hexadecimal . . . 41
2.3 Omregning mellem talsystemer . . . 43

Kapitel 3 Mængdelære **48**

3.1 Hvad er en mængde? . . . 48
3.2 Vigtige mængder: talsystemerne . . . 49
3.3 Relationer mellem mængder . . . 51
3.4 Egenskaber ved mængder . . . 53
3.5 Mængdeoperationer . . . 54
3.6 Kartesiske produkter og tupler . . . 57
3.7 Bevis for mængdeligheder . . . 58
3.8 Repræsentation af mængder i computeren . . . 61

II Sandsynlighedsregning og statistik **62**

Kapitel 4 Kombinatorik og sandsynlighedsteori **63**

4.1 Udfaldsrum og hændelser . . . 63
4.2 Tælleprincipper . . . 65
4.3 Grundlæggende sandsynlighedsregning . . . 71
4.4 Sandsynlighed for sammensatte hændelser og mængdeoperationer . . . 73

Kapitel 5 Betinget sandsynlighed og Bayes' sætning **77**

5.1 Betinget sandsynlighed . . . 77
5.2 Multiplikationsreglen og loven om total sandsynlighed . . . 82

5.3 Uafhængighed . . . 85
5.4 Bayes' sætning . . . 88

Kapitel 6 Beskrivende statistik 91
6.1 Introduktion til beskrivende statistik . . . 91
6.2 Beskrivelse af datasæt . . . 93
6.3 Opsummering af data . . . 99
6.4 Forståelse af datafordelinger . . . 105

III Lineær algebra 112

Kapitel 7 Lineære ligninger i lineær algebra 113
7.1 Lineære ligningssystemer . . . 113
7.2 Elementære rækkeoperationer og echelonformer . . . 119

Kapitel 8 Vektorer og matricer 124
8.1 Vektorer i $\mathbb{R}^2$ og $\mathbb{R}^n$ . . . 124
8.2 Matrixligninger . . . 129
8.3 Løsningsmængder for lineære ligningssystemer . . . 134
8.4 Lineær uafhængighed . . . 138

Kapitel 9 Matrixalgebra 141
9.1 Regning med matricer . . . 141
9.2 Den transponerede matrix . . . 147
9.3 Invertible matricer . . . 149
9.4 Sætningen om invertible matricer . . . 156

IV Differentialregning 158

Kapitel 10 Differentialregning 159
10.1 Grænseværdier og kontinuitet: grundlaget for differentialregning . . . 160
10.2 Den afledte: et mål for ændringshastigheden . . . 162
10.3 Differentiationsregler . . . 163
10.4 Anvendelser af afledte . . . 171
10.5 Den anden afledte: krumning og vendepunkter . . . 178

Kapitel 11 Funktioner af flere variable og gradienter 179
11.1 Funktioner af flere variable . . . 179
11.2 Partielle afledede og gradienten . . . 181
11.3 Anvendelser af gradienten . . . 184
11.4 Anvendelse: Træning af en model med gradientnedstigning . . . 185

V Supplerende emner **188**

Kapitel 12 Grundlæggende begreber i talteori **189**
12.1 Taltyper . . . 190
12.2 Egenskaber ved hele tal og modulær aritmetik . . . 193

Kapitel 13 Boolesk algebra **198**
13.1 Introduktion til boolesk algebra . . . 198
13.2 Grundlæggende operationer på bit . . . 199
13.3 Booleske funktioner . . . 202
13.4 Logiske gates . . . 207

Kapitel 14 Tidskompleksitet **210**
14.1 Introduktion til kompleksitetsanalyse . . . 210
14.2 Funktioners vækst . . . 211
14.3 Asymptotisk analyse . . . 215
14.4 Videre analyse . . . 221

Kapitel 15 Teknikker til kodeanalyse **230**
15.1 Pseudokode . . . 231
15.2 Analyse af kodekonstruktioner . . . 235
15.3 Analyse af løkker . . . 245
15.4 Analyse af kode . . . 257

Litteratur **266**

Bilag A Vigtige begreber **267**

Bilag B Summer **294**

Del I

Grundlæggende begreber

Kapitel 0 Grundlæggende aritmetik

En solid forståelse af de grundlæggende regneoperationer er fundamentet for mere avancerede matematiske emner. Aritmetik handler om at behandle tal ved hjælp af grundlæggende operationer som addition, subtraktion, multiplikation og division. Disse operationer er ikke kun vigtige i hverdagens beregninger. De er også byggestenene i mere komplekse matematiske metoder.

I dette kapitel gennemgår vi de centrale teknikker inden for algebraisk regning: faktorisering og regnearternes hierarki, regning med brøker, potenser og rødder, brug af formler og indsættelse samt omskrivning af formler. Når du behersker disse færdigheder, er du klar til at studere funktioner og de emner, der følger.

0.1 Faktorisering og regnearternes hierarki

Husk, at når vi beregner et produkt som

$$a(b+c) = ab + ac$$

ganger vi faktoren a ind på hvert led i parentesen. Dette kalder vi **den distributive lov**. Det er dog ofte en fordel eller ligefrem nødvendigt at udføre den omvendte operation, som kaldes faktorisering. Ved faktorisering skriver vi summen $ab + ac$ på faktoriseret form $a(b + c)$.

Matematisk betragtes udtrykket $a(b + c)$ som mere reduceret eller »bedre« end $ab + ac$. For at forstå hvorfor må vi skelne mellem **led** og **faktorer**. Led er adskilt af addition eller subtraktion, mens faktorer er adskilt af multiplikation eller division. For eksempel består udtrykket

$$8 - 5 + 3$$

af tre led (8, -5 og 3), mens udtrykket

$$2 \times 5 \times 3$$

består af tre faktorer (2, 5 og 3). Se på udtrykket

$$5 + 2 + 7 \times 8$$

som består af tre led (5, 2 og 7×8). Bemærk, at et af leddene selv indeholder to faktorer (7 og 8). På samme måde består udtrykket

$$5(2 + 4)(-9)$$

af tre faktorer (5, $2 + 4$ og -9), hvor faktoren $2 + 4$ indeholder to led.

Fordelen ved at skrive matematiske udtryk på faktoriseret form er, at de så kan reduceres mere effektivt. Det bliver tydeligere i løbet af kapitlet, især når vi arbejder med brøker og ligninger. Her er nogle eksempler på faktorisering:

Eksempel 0.1 Faktorisering

$$\begin{aligned} ab - ac &= a(b - c) \\ -ab - ac &= a(-b - c) \\ -ab - ac &= -a(b + c) \\ ab - ac + aa - aa &= a(b - c + a - a) \\ abc - ab + aba &= ab(c - 1 + a) \\ ab - abc - a &= a(b - bc - 1) \end{aligned}$$

Når vi udregner matematiske udtryk, er det afgørende at følge en bestemt rækkefølge for operationerne for at få korrekte resultater. Operationerne udføres i denne rækkefølge:

1. **Parenteser**: Udfør først alle operationer inde i parenteser.
2. **Potenser og rødder**: Udregn derefter potenser og rødder.
3. **Multiplikation og division**: Udfør så multiplikation og division fra venstre mod højre i den rækkefølge, de optræder.
4. **Addition og subtraktion**: Udfør til sidst addition og subtraktion fra venstre mod højre i den rækkefølge, de optræder.

Lad os se på et eksempel for hver operation for at illustrere regnearternes hierarki:

Eksempel 0.2 Parenteser
Udregn udtrykket: $(2 + 3) \times 4$

$$(2 + 3) \times 4 = 5 \times 4 = 20$$

Eksempel 0.3 Potenser og rødder
Udregn udtrykket: $3^2 + \sqrt{16}$

$$3^2 + \sqrt{16} = 9 + 4 = 13$$

Eksempel 0.4 Multiplikation og division
Udregn udtrykket: $6 \div 2 \times 3$. Ifølge det almindelige regnearternes hierarki udfører vi division og multiplikation fra venstre mod højre:

$$6 \div 2 \times 3 = (6 \div 2) \times 3 = 3 \times 3 = 9$$

Eksempel 0.5 Addition og subtraktion
Udregn udtrykket: $8 - 3 + 2$

$$8 - 3 + 2 = 5 + 2 = 7$$

0.2 Brøker

Per definition består en brøk altid af (mindst) to faktorer. Den første faktor kalder vi **tælleren**. Det er brøkens »øverste del«. Den nederste del kalder vi **nævneren**. Den måske vigtigste regel for regning med brøker er, at to

brøker kun kan lægges sammen eller trækkes fra hinanden, hvis de har samme nævner. Desuden må nævneren aldrig være lig med 0.

Eksempel 0.6

$$\frac{1}{x^2-2}$$

Her skal vi sikre, at $x^2 - 2 \neq 0$. Det betyder, at vi kun må bruge værdier, der er forskellige fra $\pm\sqrt{2}$.

Proposition 0.1 (Regneregler for brøker)

For $a, b, c, m \in \mathbb{R}$, hvor $a, b, c, m \neq 0$ når det er nødvendigt, gælder følgende identiteter:

(1) $\frac{a}{b} \times m = \frac{am}{b}$

(2) $\frac{a}{b} \div m = \frac{a}{bm}$

(3) $m \div \frac{a}{b} = \frac{mb}{a}$

(4) $\frac{a}{b} \times \frac{c}{a} = \frac{c}{b}$

(5) $\frac{a}{b} \div \frac{c}{a} = \frac{a^2}{bc}$

(6) $\frac{a}{b} = \frac{ac}{bc}$

(7) $\frac{a}{b} + \frac{c}{a} = \frac{a^2+bc}{ab}$

For at forlænge eller forkorte en brøk skal vi gange eller dividere med det samme tal i både tæller og nævner:

Eksempel 0.7 Forlængelse og forkortning af brøker

$$\frac{72}{144} = \frac{72 \div 12}{144 \div 12} = \frac{6}{12} = \frac{6 \div 6}{12 \div 6} = \frac{1}{2}$$

$$\frac{2}{3} = \frac{2 \times 6}{3 \times 6} = \frac{12}{18}$$

$$\frac{2x}{2xx} = \frac{2}{2x}$$

$$\frac{3a}{6a+3b} = \frac{3a}{3(2a+b)} = \frac{a}{2a+b}$$

Når vi faktoriserer et udtryk, skal alle led divideres eller ganges på samme måde. Det betyder, at følgende udtryk ikke kan reduceres yderligere, selv om det kan være fristende:

$$\frac{a}{2a+b} \neq \frac{1}{2+b}$$

For at faktorisere $\frac{a}{2a+b}$ korrekt skal du dividere a op i alle led i nævneren:

$$\frac{a}{2a+b} = \frac{1}{2+\frac{b}{a}}$$

Som vist ovenfor er multiplikation af brøker ligetil: Man ganger tællerne med hinanden og nævnerne med hinanden.

Eksempel 0.8 Multiplikation af brøker

Se på brøkerne $\frac{a}{b}$ og $\frac{c}{d}$. Deres produkt er:

$$\frac{a}{b} \times \frac{c}{d} = \frac{a \times c}{b \times d}$$

Hvis for eksempel $a = 2$, $b = 3$, $c = 4$ og $d = 5$, så er:

$$\frac{2}{3} \times \frac{4}{5} = \frac{2 \times 4}{3 \times 5} = \frac{8}{15}$$

Eksempel 0.9 Division af brøker

$$\frac{6}{7} \div \frac{4}{21} = \frac{6}{7} \times \frac{21}{4} = \frac{126}{28} = \frac{63}{14} = \frac{9}{2}$$

$$\frac{1}{2} \div 3 = \frac{1}{2} \div \frac{3}{1} = \frac{1}{2} \times \frac{1}{3} = \frac{1}{6}$$

$$\frac{2}{3} \div \frac{8}{9} = \frac{2}{3} \times \frac{9}{8} = \frac{18}{24} = \frac{3}{4}$$

$$\frac{8}{9} \div 16 = \frac{8}{9} \times \frac{1}{16} = \frac{8}{144} = \frac{4}{72} = \frac{1}{18}$$

Addition og subtraktion af brøker volder ofte flere problemer end multiplikation og division. Nøglen er at finde en fællesnævner for brøkerne og huske reglen ovenfor om forlængelse af brøker.

Eksempel 0.10 Addition og subtraktion af brøker

$$\frac{1}{5} + \frac{2}{5} = \frac{1+2}{5} = \frac{3}{5}$$

$$\frac{1}{4} + \frac{2}{3} = \frac{1 \times 3}{4 \times 3} + \frac{2 \times 4}{3 \times 4} = \frac{3}{12} + \frac{8}{12} = \frac{3+8}{12} = \frac{11}{12}$$

$$\frac{7}{12} - \frac{5}{8} = \frac{7 \times 8}{12 \times 8} - \frac{5 \times 12}{8 \times 12} = \frac{56}{96} - \frac{60}{96} = \frac{56-60}{96} = \frac{-4}{96} = \frac{-1}{24}$$

Bemærk: Gør aldrig sådan:

$$\frac{a}{b} + \frac{c}{d} = \frac{a+c}{b+d}$$

Eksempel 0.11

$$\frac{x}{x-1} \times \frac{2}{x(x+4)} = \frac{2x}{(x-1)x(x+4)} = \frac{2}{(x-1)(x+4)}$$

$$\frac{2}{x-1} \div \frac{x}{x-1} = \frac{2}{x-1} \times \frac{x-1}{x} = \frac{2(x-1)}{(x-1)x} = \frac{2}{x}$$

$$\frac{x+1}{x^2+2} + \frac{x-6}{x^2+2} = \frac{x+1+x-6}{x^2+2} = \frac{2x-5}{x^2+2}$$

Husk, at når der står et minustegn foran en brøk, skal alle fortegn i tælleren ændres. Det svarer til, at man ændrer alle fortegn inde i en parentes, når der står et minustegn foran den.

Se på udtrykket:

$$-\frac{a-b}{c}$$

For at håndtere minustegnet korrekt ændrer vi alle fortegn i tælleren:

$$-\frac{a-b}{c} = \frac{-a+b}{c}$$

Hvis for eksempel $a = 5$ og $b = 3$, så er:

$$-\frac{5-3}{c} = \frac{-5+3}{c} = \frac{-2}{c}$$

Eksempel 0.12

$$\frac{x+1}{x^2+2} - \frac{x-6}{x^2+2} = \frac{x+1-x+6}{x^2+2} = \frac{7}{x^2+2}$$

0.3 Potenser, rødder og irrationelle rødder

En **eksponent** er en kort måde at skrive gentagen multiplikation af det samme tal på:

Eksempel 0.13 Potensopløftning

$$4 \times 4 \times 4 \times 4 \times 4 = 4^5$$

$$x \times x \times x \times x \times x = x^5$$

Rødder er tæt knyttet til potensopløftning. Over de reelle tal er kvadratroden $\sqrt{x}$ defineret som det ikke-negative tal, hvis kvadrat er x. Den er derfor kun defineret, når $x \geq 0$. Hvis man kvadrerer et reelt tal og derefter tager kvadratroden, får man følgelig tallets absolutte værdi: $\sqrt{x^2} = |x|$.

Eksempel 0.14 Uddragning af rod

$$\sqrt{a} \times \sqrt{a} = (\sqrt{a})^2 = a$$

$$\sqrt{a} = b \iff a = b^2 \text{ og } b \geq 0$$

En **irrationel rod** (på engelsk *surd*) er en rod, der både er reel og irrational. Eksempler er $\sqrt{2}$, $\sqrt{3}$, $\sqrt{5}$ og $\sqrt{6}$.

Tal kan opløftes i andre potenser end 2, for eksempel i tredje potens eller i fjerde potens, 100. potens og så videre. Tilsvarende kan man tage kubikroden af et tal, fjerderoden, den 100. rod og så videre. For at angive en anden rod end kvadratroden bruger man det samme rodtegn, men indsætter et tal, kaldet rodeksponenten, i rodtegnet. Tallet placeres typisk i den del af tegnet, der ligner et »flueben«.

Eksempel 0.15 Rodeksponent og argument

$$4^3 = 64 \iff \sqrt[3]{64} = 4$$

I dette eksempel er »3« i rodtegnet rodens **rodeksponent**. »64« kaldes argumentet for **roden**, også kendt som **radikanden**. Da kvadratrødder er den mest almindelige type rødder, udelader man normalt rodeksponenten for kvadratrødder. Selv om »$\sqrt[2]{2}$« teknisk set er korrekt, bruges det sjældent i praksis.

Proposition 0.2 (Regneregler for rødder)

(1) $\sqrt[n]{xy} = \sqrt[n]{x} \times \sqrt[n]{y}$ hvor $x, y \geq 0$

(2) $\sqrt[n]{\frac{x}{y}} = \frac{\sqrt[n]{x}}{\sqrt[n]{y}}$ hvor $x \geq 0$ og $y > 0$

(3) $\sqrt{x^2} = |x|$ hvor $x \in \mathbb{R}$

(4) $(\sqrt[n]{x})^n = x$ hvor $x \geq 0$, eller hvor $x < 0$ og n er ulige

(5) $\sqrt[n]{-x} = -\sqrt[n]{x}$ hvor $x \geq 0$ og $n \in \mathbb{N}$ er ulige

♠

At opløfte et tal i en **potens**, også kaldet **potensopløftning**, er en grundlæggende matematisk operation. Som vi så ovenfor, ganger man et tal med sig selv et bestemt antal gange. **Grundtallet** er det tal, der ganges, og **eksponenten** angiver, hvor mange gange grundtallet indgår som faktor. For eksempel betyder a^n, at grundtallet a ganges med sig selv n gange. Potensopløftning er et stærkt værktøj i matematikken, og nogle få vigtige regler styrer, hvordan det bruges.

Proposition 0.3 (Potensregneregler for heltallige eksponenter)

Lad $n, m \in \mathbb{Z}$. Følgende identiteter gælder, når alle de indgående udtryk er definerede. Især skal et grundtal med negativ eksponent være forskelligt fra nul.

(1) $x^n \cdot x^m = x^{n+m}$,

(2) $\frac{x^n}{x^m} = x^{n-m}$ med $x \neq 0$,

(3) $x^n \cdot y^n = (xy)^n$,

(4) $\frac{x^n}{y^n} = \left(\frac{x}{y}\right)^n$ med $y \neq 0$,

(5) $(x^n)^m = x^{nm}$,

(6) $x^1 = x$.

♠

Nogle af disse regler gør det muligt at udvide potensbegrebet, så eksponenten kan være et vilkårligt helt tal. Hvis man sætter $n = m$ i regel (2), får man:

$$\frac{x^n}{x^n} = x^{n-n} = x^0$$

For $x \neq 0$ er brøken $\frac{x^n}{x^n}$ lig med 1, så vi får

$$x^0 = 1$$

Dermed er potensbegrebet udvidet til at omfatte $n \in \mathbb{N} \cup \{0\}$. Hvis man nu igen sætter $n = 0$ i regel (2), får man:

$$\frac{x^0}{x^m} = x^{0-m} = x^{-m}$$

Men ifølge den forrige udregning er $x^0 = 1$. Derfor får man

$$\frac{1}{x^m} = x^{-m}$$

Som vist i Proposition 0.3 giver disse regler en fuldstændig ramme for at regne med udtryk med vilkårlige

heltallige eksponenter. Som en konsekvens kan følgende yderligere regler udledes:

Proposition 0.4 (Flere potensregneregler for heltallige eksponenter)

Lad $n, m \in \mathbb{Z}$. Så gælder følgende (med $x, y \in \mathbb{R}$ og forskellige fra nul, hvor det er angivet):

(7) $x^0 = 1 \qquad x \neq 0$

(8) $\dfrac{1}{x^m} = x^{-m} \qquad x \neq 0$

♠

Lad os illustrere disse regler med et par eksempler.

Eksempel 0.16 Reducér følgende udtryk

$$\left(3xy^6\right)^3 = 3^3 \cdot x^3 \cdot \left(y^6\right)^3 = 27x^3y^{18}$$

Eksempel 0.17 Reducér følgende udtryk

$$\frac{a^{-4}b^3}{a^7b^{-5}} = \frac{a^{-4}}{a^7} \cdot \frac{b^3}{b^{-5}} = a^{-4-7} \cdot b^{3-(-5)} = a^{-11} \cdot b^8 = \frac{b^8}{a^{11}}$$

For positive tal x kan potensbegrebet udvides yderligere, så det også gælder, når eksponenten er et rationalt tal. Ethvert rationalt tal $r \in \mathbb{Q}$ kan skrives som $r = \dfrac{m}{n}$, hvor $m \in \mathbb{Z}$ og $n \in \mathbb{N}$. For $x > 0$ definerer vi nu

$$y = x^r = x^{\frac{m}{n}}$$

Heraf får man (blandt andet ved hjælp af regel (5)):

$$y^n = \left(x^{\frac{m}{n}}\right)^n = x^{\frac{m}{n} \cdot n} = x^m$$

Ved at bruge rodbegrebet får vi til sidst

$$y^n = x^m \quad \Longleftrightarrow \quad y = \sqrt[n]{x^m}$$

Bemærk, at fordi $x > 0$, er også $y > 0$. Vi er nu klar til at formulere ***det udvidede potensbegreb***.

Definition 0.1 (Det udvidede potensbegreb)

Lad $m \in \mathbb{Z}$ og lad $n \in \mathbb{N}$, så $\dfrac{m}{n} \in \mathbb{Q}$. Så gælder følgende:

$$x^{\frac{m}{n}} = \sqrt[n]{x^m} \qquad x > 0$$

Mere specifikt gælder

$$x^{\frac{1}{n}} = \sqrt[n]{x} \qquad x > 0$$

♣

Nævneren i en rational eksponent svarer til rodens rodeksponent, mens tælleren forbliver eksponent for grundtallet. Omvendt kan en rods rodeksponent omdannes til nævneren i en eksponent i et tilsvarende potensudtryk. Denne egenskab gør det muligt at omskrive ethvert rodudtryk til potensform, hvilket er et stærkt værktøj til reduktion.

Eksempel 0.18

$$\sqrt[5]{x^3} = x^{\frac{3}{5}} \quad \text{mod} \quad \sqrt[3]{x^5} = x^{\frac{5}{3}}$$

$$\frac{1}{\sqrt[7]{x^3}} = x^{-\frac{3}{7}}$$

$$\frac{1}{\sqrt[3]{x^2}} = \left(x^2\right)^{-\frac{1}{3}} = x^{-\frac{2}{3}}$$

Denne egenskab kan også vendes om: Enhver rational eksponent kan skrives om til et rodudtryk ved at bruge nævneren som rodens rodeksponent. Når vi kan skifte mellem potensform og rodform, kan vi udregne udtryk, som tidligere var svære at håndtere, ved at omskrive dem til rødder.

Eksempel 0.19

$$27^{-\frac{4}{3}} = \frac{1}{\sqrt[3]{27^4}} = \frac{1}{\left(\sqrt[3]{27}\right)^4} = \frac{1}{3^4} = \frac{1}{81}$$

En af de største fordele ved at omskrive et rodudtryk til potensform er, at vi så kan bruge alle potensreglerne til at reducere udtrykket. De følgende eksempler viser, hvordan forskellige regler kan bruges til at reducere udtryk med rationale eksponenter.

Eksempel 0.20

$$a^{\frac{2}{3}}b^{\frac{1}{2}}a^{\frac{1}{6}}b^{\frac{1}{5}} = a^{\frac{2}{3}+\frac{1}{6}}b^{\frac{1}{2}+\frac{1}{5}} = a^{\frac{4}{6}+\frac{1}{6}}b^{\frac{5}{10}+\frac{2}{10}} = a^{\frac{5}{6}}b^{\frac{7}{10}}$$

$$\left(x^{\frac{1}{3}}x^{\frac{2}{5}}\right)^{\frac{3}{4}} = x^{\frac{1}{3}\times\frac{3}{4}}x^{\frac{2}{5}\times\frac{3}{4}} = x^{\frac{3}{12}}x^{\frac{6}{20}} = x^{\frac{1}{4}}x^{\frac{3}{10}} = x^{\frac{11}{20}}$$

$$\frac{x^{\frac{4}{2}}x^{\frac{4}{6}}x^{\frac{1}{2}}x^{\frac{5}{6}}}{x^{\frac{7}{2}}x^0} = x^{\frac{4}{2}+\frac{4}{6}+\frac{1}{2}+\frac{5}{6}-\frac{7}{2}} = x^{\frac{1}{2}}$$

$$\left(25x^{\frac{1}{3}}x^{\frac{2}{5}}\right)^{-\frac{1}{2}} = \left(25x^{\frac{11}{15}}\right)^{-\frac{1}{2}} = \frac{1}{5x^{\frac{11}{30}}}$$

Det er vigtigt at huske, at når vi reducerer udtryk med rationale eksponenter, bruger vi de samme potensregler som for heltallige eksponenter. Den eneste forskel er, at vi også skal overholde reglerne for brøker.

0.4 Brug af formler og indsættelse

I ingeniørfagene er fysiske størrelser ofte knyttet til hinanden gennem formler. Formlerne består af variable og konstanter, som repræsenterer de pågældende fysiske størrelser. For at udregne en formel skal man indsætte talværdier for variablene.

Ohms lov er for eksempel en formel, der forbinder spændingen v over en modstand med resistansen R og strømmen i, der løber gennem den. Formlen er givet ved

$$v = iR$$

Med denne formel kan vi beregne spændingen v, hvis værdierne af i og R er kendte. Hvis for eksempel $i = 13\,\text{A}$ og $R = 5\,\Omega$, så er

$$v = iR = (13)(5) = 65$$

Spændingen er altså 65 V.

Eksemplet viser, hvor vigtigt det er at holde nøje øje med enhederne for de fysiske størrelser, der indgår. En formel er kun gyldig, hvis man bruger et konsistent sæt enheder.

Eksempel 0.21 Indsættelse i formler

Den kinetiske energi K for et legeme med massen M, som bevæger sig med farten v, kan beregnes med formlen:

$$K = \frac{1}{2}Mv^2$$

Beregn den kinetiske energi for et legeme med en masse på 5 kg, som bevæger sig med en fart på $2\,\mathrm{m\,s^{-1}}$.

Løsning:

$$K = \frac{1}{2}Mv^2 = \frac{1}{2}(5)(2^2) = 10$$

◀

I SI-systemet er enheden for energi joule, så legemets kinetiske energi er 10 joule.

Eksempel 0.22 Indsættelse i formler

Arealet A af en cirkel med radius r kan beregnes med formlen $A = \pi r^2$.

Hvis cirklens diameter d er kendt, kan man i stedet bruge den tilsvarende formel:

$$A = \frac{\pi d^2}{4}$$

Beregn arealet af en cirkel med en diameter på 0.1 m. Værdien af π er indbygget i din lommeregner.

Løsning:

$$A = \frac{\pi(0.1)^2}{4} = 0.00785\,\mathrm{m^2}$$

◀

Eksempel 0.23 Indsættelse i formler

Rumfanget V af en cirkulær cylinder er lig med dens tværsnitsareal A ganget med dens længde h.

Beregn rumfanget af en cylinder med en diameter på 0.1 m og en længde på 0.3 m.

Løsning:

$$V = Ah = \frac{\pi(0.1)^2}{4} \times 0.3 = 0.00236$$

Rumfanget er $0.00236\,\mathrm{m^3}$. ◀

0.5 Omskrivning af formler

I formlen for cirklens areal, $A = \pi r^2$, er variablen A isoleret. En variabel er isoleret, hvis den står alene på den ene side af lighedstegnet, normalt på venstre side, og ikke optræder andre steder i formlen. Hvis vi bliver bedt om at isolere r i formlen eller løse den med hensyn til r, skal vi omskrive ligningen, så r bliver isoleret. Når man omskriver en formel, skal enhver operation, der udføres på den ene side, også udføres på den anden side. Der er fem centrale regler, som skal følges i denne proces.

Regler for omskrivning af formler

Følgende operationer kan udføres på begge sider af formlen:

- Læg den samme størrelse til på begge sider
- Træk den samme størrelse fra på begge sider
- Gang begge sider med den samme størrelse forskellig fra nul – husk at gange alle led
- Divider begge sider med den samme størrelse forskellig fra nul – husk at dividere alle led
- Anvend den samme funktion på begge sider, og kontrollér samtidig funktionens definitionsmængde, og om operationen kan vendes om

De første fire operationer bevarer ækvivalensen, når deres angivne begrænsninger er opfyldt. At anvende en funktion, for eksempel at kvadrere, kan ikke nødvendigvis vendes om: For eksempel medfører $x = 2$, at $x^2 = 4$, men $x^2 = 4$ tillader også $x = -2$. Løsninger, der er fundet efter en operation, som ikke kan vendes om, skal derfor kontrolleres i den oprindelige ligning.

Eksempel 0.24 Omskriv formlen $p = 5t - 17$, så t bliver isoleret.

Løsning: For at isolere t på venstre side går vi frem trinvis efter de fem regler. Læg først 17 til på begge sider af ligningen $p = 5t - 17$:

$$p + 17 = 5t - 17 + 17$$

Ved reduktion får vi:

$$p + 17 = 5t$$

Divider derefter begge sider med 5 for at isolere t:

$$\frac{p + 17}{5} = t$$

Formlen for t er altså:

$$t = \frac{p + 17}{5}$$

◀

Eksempel 0.25 Omskriv formlen $\sqrt{2q} = p$, og løs den med hensyn til q.

Løsning: Kvadrér først begge sider for at fjerne kvadratroden omkring $2q$. Bemærk, at $(\sqrt{2q})^2 = 2q$. Det giver:

$$2q = p^2$$

Divider derefter begge sider med 2 for at løse med hensyn til q:

$$q = \frac{p^2}{2}$$

◀

Eksempel 0.26 Omskriv formlen $v = \sqrt{t^2 + w}$, og løs den med hensyn til w. Følg disse trin for at isolere w:

Løsning:

a. Kvadrér først begge sider for at fjerne kvadratroden omkring $t^2 + w$:
$$v^2 = t^2 + w$$
b. Træk derefter t^2 fra på begge sider for at isolere w:
$$v^2 - t^2 = w$$
c. Skriv til sidst formlen for w:
$$w = v^2 - t^2$$

◀

Eksempel 0.27 Omskriv formlen $x = \frac{1}{y}$, og løs den med hensyn til y.

Løsning: For at isolere y bemærker vi, at y står i nævneren. Hvis vi ganger begge sider med y, forsvinder brøken:

$$yx = y \times \frac{1}{y}$$

Det reduceres til:

$$yx = 1$$

Divider til sidst begge sider med x for at løse med hensyn til y:

$$y = \frac{1}{x}$$

Alternativt kan du blot invertere begge sider direkte og få:

$$y = \frac{1}{x}$$

◀

Eksempel 0.28 Isolér R i formlen:

$$\frac{2}{R} = \frac{3}{x + y}$$

Løsning: Da R står i en brøk, inverterer vi begge sider:

$$\frac{R}{2} = \frac{x + y}{3}$$

Ved at gange begge sider med 2 får vi: $R = \frac{2(x + y)}{3}$ ◀

Eksempel 0.29 Isolér R i formlen:

$$\frac{1}{R} = \frac{1}{R_1} + \frac{1}{R_2}$$

Løsning: De to led på højre side kan samles:

$$\frac{1}{R_1} + \frac{1}{R_2} = \frac{R_2 + R_1}{R_1 R_2}$$

Formlen bliver så:

$$\frac{1}{R} = \frac{R_2 + R_1}{R_1 R_2}$$

Til sidst inverterer vi begge sider og får:

$$R = \frac{R_1 R_2}{R_2 + R_1}$$

◀

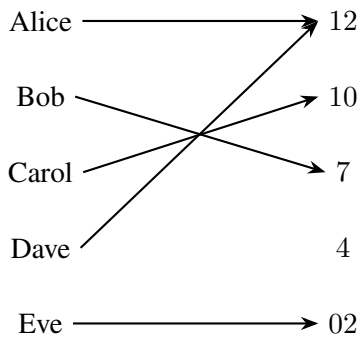
Kapitel 1 Funktioner

Funktioner er et centralt begreb i matematikken og danner bro mellem aritmetik og mere avanceret matematisk analyse. En funktion kan opfattes som en særlig sammenhæng mellem to mængder, hvor hvert input (fra definitionsmængden) er knyttet til præcis ét output (fra dispositionsmængden). Når vi forstår funktioner og deres egenskaber, kan vi modellere og løse virkelige problemer med større præcision og fleksibilitet.

I dette kapitel introducerer vi funktionsbegrebet, herunder definitioner, notation og vigtige egenskaber. Derefter undersøger vi almindelige funktionstyper grafisk og udvikler de beslægtede begreber logaritmer og løsning af eksponential- og logaritmeligninger. Disse begreber danner grundlag for mere avancerede matematiske emner senere i bogen.

1.1 Funktioners egenskaber

I matematikken knytter en funktion hvert element i én mængde til et bestemt element i en anden mængde (som kan være den samme mængde). Se for eksempel på et hold i Matematik for softwareingeniører, hvor hver studerende får en karakter fra mængden $\{12, 10, 7, 4, 02\}$. Antag, at karakterne er som følger:



Figur 1.1: Eksempel på en funktion, der afbilder navne over i tal.

Denne tildeling af karakterer, som er illustreret i Figur 1.1, er et eksempel på en funktion.

Funktioner spiller en central rolle i matematik og datalogi. De definerer diskrete strukturer som følger og strenge, og de bruges til at analysere algoritmers tidskompleksitet. Mange computerprogrammer er designet til at beregne funktionsværdier. Rekursive funktioner, der er defineret ved hjælp af sig selv, har særlig stor betydning i datalogi. Dette afsnit giver et overblik over de grundlæggende funktionsbegreber, som er nødvendige i matematik for softwareingeniører.

Definition 1.1

Lad A og B være ikke-tomme mængder. En funktion f fra A til B er en tildeling af præcis ét element i B til hvert element i A. Vi skriver $f(a) = b$, hvis b er det entydige element i B, som funktionen f tildeler elementet a i A. Hvis f er en funktion fra A til B, skriver vi $f : A \to B$.

Bemærkning: Funktioner kaldes undertiden også afbildninger eller transformationer.

Funktioner kan angives på forskellige måder. Nogle gange angiver vi tildelingerne eksplicit, som vist i Figur 1.1. Ofte bruges en forskrift som $f(x) = x + 1$ til at definere en funktion. I andre tilfælde kan funktionen være givet

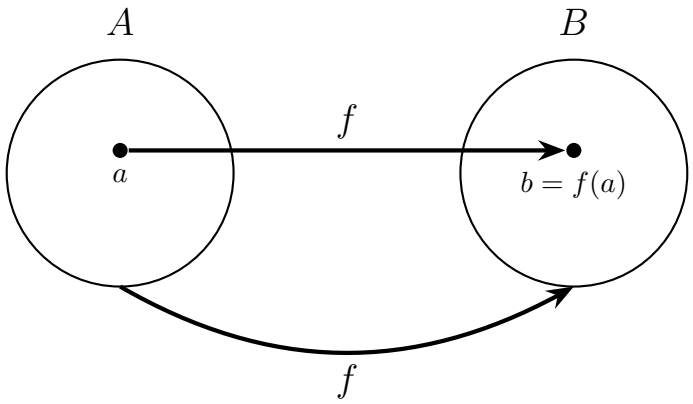
ved et computerprogram.

Definition 1.2

Hvis f er en funktion fra A til B, siger vi, at A er **definitionsmængden** for f, og at B er **dispositionsmængden** for f. Hvis $f(a) = b$, siger vi, at b er **billedet** af a, og at a er et **urbillede** af b. **Værdimængden**, eller billedmængden, for f er mængden af alle billeder af elementer i A. Hvis f er en funktion fra A til B, siger vi desuden, at f **afbilder** A ind i B.

Når vi definerer en funktion, angiver vi dens definitionsmængde, dens dispositionsmængde og afbildningen af elementerne i definitionsmængden over i dispositionsmængden. To funktioner er ens, hvis de har samme definitionsmængde og samme dispositionsmængde og afbilder hvert element i definitionsmængden over i det samme element i dispositionsmængden.

Det er vigtigt at bemærke, at en ændring af definitionsmængden eller dispositionsmængden giver en anden funktion. Tilsvarende giver en ændring af afbildningen af elementerne også en anden funktion.



Figur 1.2: En funktion f, der afbilder et element a i mængden A over i et element $b = f(a)$ i mængden B.

De følgende eksempler illustrerer forskellige funktioner. I hvert eksempel beskriver vi definitionsmængden, dispositionsmængden, værdimængden og tildelingen af værdier til elementerne i definitionsmængden.

Eksempel 1.1 Hvad er definitionsmængden, dispositionsmængden og værdimængden for den funktion, der tildeler de studerende karakterer, og som blev introduceret i begyndelsen af dette afsnit?

Løsning: Lad G være den funktion, der tildeler en studerende på vores hold i matematik for softwareingeniører en karakter. Bemærk for eksempel, at $G(\text{Alice}) = 12$. Definitionsmængden for G er mængden {Alice, Bob, Carol, Dave, Eve }, og dispositionsmængden er mængden $\{12, 10, 7, 4, 02\}$. Værdimængden for G er mængden $\{12, 10, 7, 02\}$, fordi alle karakterer undtagen 4 er tildelt mindst én studerende. ◀

Eksempel 1.2 Lad f være den funktion, der til en bitstreng af længde 2 eller mere tildeler strengens sidste to bit. For eksempel er $f(11010) = 10$. Så er definitionsmængden for f mængden af alle bitstrenge af længde 2 eller mere, og både dispositionsmængden og værdimængden er mængden $\{00, 01, 10, 11\}$.

Eksempel 1.3 Lad $f : \mathbb{Z} \to \mathbb{Z}$ tildele et helt tal dets kvadrat. Så er $f(x) = x^2$, hvor definitionsmængden for f er mængden af alle hele tal, dispositionsmængden for f er mængden af alle hele tal, og værdimængden for f er mængden af alle hele tal, der er kvadrattal, nemlig $\{0, 1, 4, 9, \ldots\}$.

Injektive og surjektive funktioner

I matematikken er funktioner et grundlæggende begreb, der bruges til at beskrive sammenhængen mellem to mængder. Men ikke alle funktioner opfører sig ens. For at forstå forskellene introducerer vi begreberne injektive og surjektive funktioner.

Nogle funktioner tildeler aldrig den samme værdi til to forskellige elementer i definitionsmængden. Sådanne funktioner kaldes **injektive**.

Definition 1.3 (Injektive funktioner)

En funktion $f : A \to B$ kaldes **injektiv**, hvis forskellige elementer i A afbildes over i forskellige elementer i B. Med andre ord: Hvis $f(a_1) = f(a_2)$, så er $a_1 = a_2$. Denne egenskab sikrer, at to forskellige elementer i A aldrig afbildes over i det samme element i B.

♣

For en reel funktion, hvis graf er tegnet i et koordinatsystem, gælder, at funktionen er injektiv, hvis og kun hvis ingen vandret linje skærer grafen mere end én gang. Dette kriterium kaldes den vandrette linjetest.

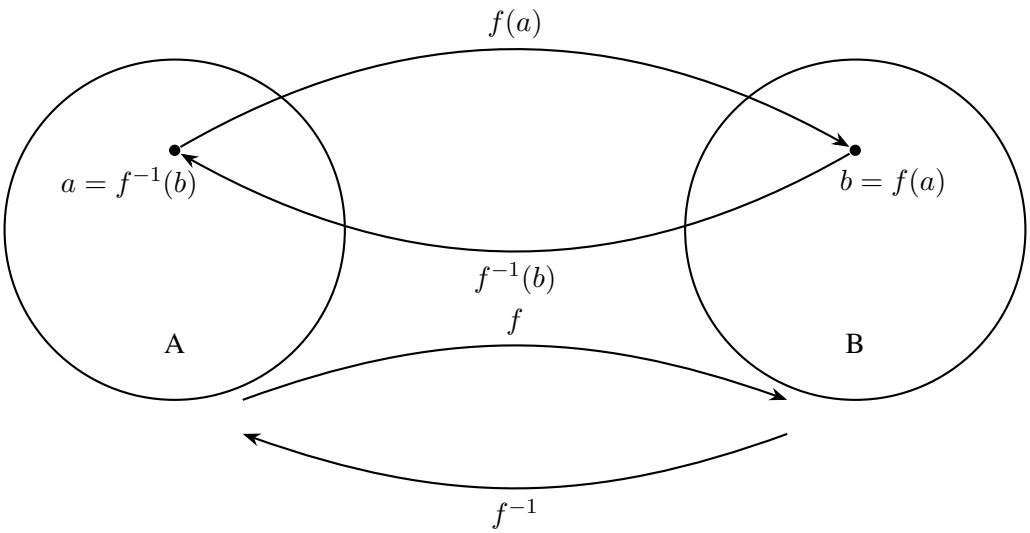
Definition 1.4 (Surjektive funktioner)

En funktion $f : A \to B$ kaldes **surjektiv**, hvis hvert element i B er billede af mindst ét element i A. Med andre ord findes der for hvert $b \in B$ mindst ét $a \in A$, så $f(a) = b$. Denne egenskab sikrer, at funktionen »dækker« hele mængden B.

♣

Inverse funktioner

Se nu på en funktion $f : A \to B$, som er både injektiv og surjektiv. Fordi f er surjektiv, er hvert element i B billede af et element i A. Fordi f er injektiv, er hvert element i B desuden billede af ét entydigt bestemt element i A. Denne entydige korrespondance gør det muligt at definere en ny funktion fra B til A, som »vender« den afbildning om, der er givet ved f.



Figur 1.3: Funktionen f^{-1} er den inverse funktion til f.

Denne nye funktion kaldes den **inverse funktion** (eller omvendte funktion) til f og betegnes $f^{-1} : B \to A$.

Den inverse funktion f^{-1} har følgende egenskaber:

$$f(f^{-1}(b)) = b \quad \text{for alle } b \in B.$$

$$f^{-1}(f(a)) = a \quad \text{for alle } a \in A.$$

Vi kan sammenfatte disse overvejelser i følgende definition

Definition 1.5 (Inverse funktioner)

Lad f være en bijektion fra mængden A til mængden B. Den inverse funktion til f er den funktion, der til et element b i B tildeler det entydige element a i A, for hvilket $f(a) = b$. Den inverse funktion til f betegnes f^{-1}. Altså er $f^{-1}(b) = a$, når $f(a) = b$.

Disse egenskaber viser, at f^{-1} reelt ophæver virkningen af f ved at afbilde hvert element i B tilbage i det tilsvarende element i A.

Eksempel 1.4

Lad $f : \mathbb{R} \to \mathbb{R}$ være defineret ved $f(x) = 2x + 3$. Vi kan kontrollere, at f er både injektiv og surjektiv:

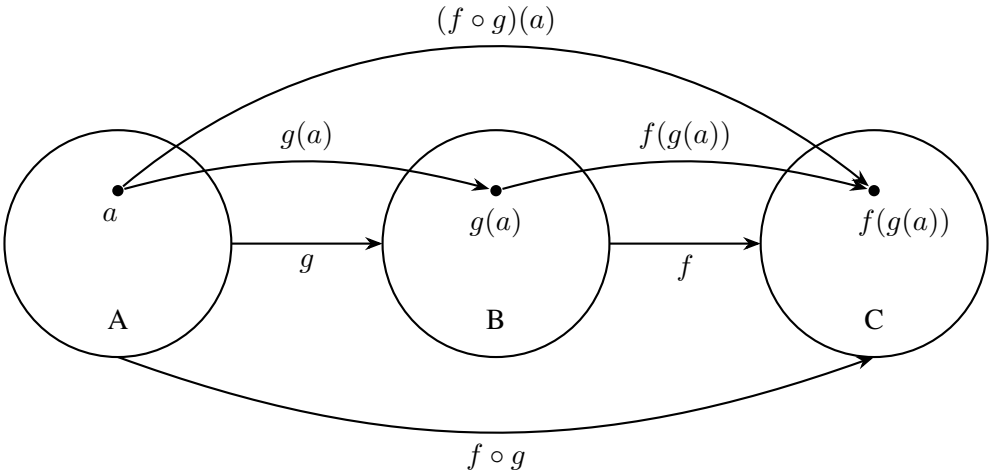
- **Injektiv**: Hvis $f(x_1) = f(x_2)$, så er $2x_1 + 3 = 2x_2 + 3$. Når vi trækker 3 fra på begge sider, får vi $2x_1 = 2x_2$, og division med 2 giver $x_1 = x_2$. Altså er f injektiv.
- **Surjektiv**: For et vilkårligt $y \in \mathbb{R}$ kan vi løse $y = 2x + 3$ med hensyn til x og får $x = \frac{y-3}{2}$. Da dette x findes for alle y, er f surjektiv.

Da f er både injektiv og surjektiv, har den en invers funktion f^{-1}, der er defineret ved

$$f^{-1}(y) = \frac{y-3}{2}.$$

Sammensatte funktioner

I matematikken kan funktioner kombineres til nye funktioner. En vigtig måde at kombinere funktioner på er sammensætning af funktioner. At sammensætte to funktioner består i bund og grund i at anvende den ene funktion på resultaterne af den anden.



Figur 1.4: Sammensætningen af f og g.

Definition 1.6

Lad $f : B \to C$ og $g : A \to B$ være to funktioner. Den **sammensatte funktion** af f og g, som betegnes $f \circ g$, er en funktion fra A til C, der er defineret ved

$$(f \circ g)(x) = f(g(x)),$$

for alle $x \in A$. ♣

Den sammensatte funktion $f \circ g$ betyder med andre ord, at du først anvender funktionen g på inputtet x og derefter anvender funktionen f på resultatet $g(x)$. Sammensætning af funktioner er vist i Figur 1.4.

Eksempel 1.5

Se på funktionerne $f(x) = 2x + 3$ og $g(x) = x^2$. Den sammensatte funktion $f \circ g$ er givet ved:

$$(f \circ g)(x) = f(g(x)) = f(x^2) = 2x^2 + 3.$$

Her kvadrerer funktionen $g(x)$ inputtet x, hvorefter funktionen $f(x)$ ganger resultatet med 2 og lægger 3 til.

Lad os nu bytte om på rækkefølgen i sammensætningen og beregne $g \circ f$:

$$(g \circ f)(x) = g(f(x)) = g(2x + 3) = (2x + 3)^2.$$

Bemærk, at $f \circ g$ og $g \circ f$ generelt er forskellige funktioner. Det viser, at sammensætning af funktioner ikke er kommutativ.

Eksempel 1.6 Lad g være funktionen fra mængden $\{a, b, c\}$ til sig selv, så $g(a) = b, g(b) = c$ og $g(c) = a$. Lad f være funktionen fra mængden $\{a, b, c\}$ til mængden $\{1, 2, 3\}$, så $f(a) = 3, f(b) = 2$ og $f(c) = 1$. Hvad er sammensætningen af f og g, og hvad er sammensætningen af g og f?

Løsning: Sammensætningen $f \circ g$ er defineret ved $(f \circ g)(a) = f(g(a)) = f(b) = 2$, $(f \circ g)(b) = f(g(b)) = f(c) = 1$ og $(f \circ g)(c) = f(g(c)) = f(a) = 3$.

Bemærk, at $g \circ f$ ikke er defineret, fordi værdimængden for f ikke er en delmængde af definitionsmængden for g. ◀

Eksempel 1.7 Lad f og g være funktionerne fra mængden af hele tal til mængden af hele tal, der er defineret ved $f(x) = 2x + 3$ og $g(x) = 3x + 2$. Hvad er sammensætningen af f og g? Hvad er sammensætningen af g og f?

Løsning: Både sammensætningen $f \circ g$ og sammensætningen $g \circ f$ er defineret. Desuden er

$$(f \circ g)(x) = f(g(x)) = f(3x + 2) = 2(3x + 2) + 3 = 6x + 7$$

og

$$(g \circ f)(x) = g(f(x)) = g(2x + 3) = 3(2x + 3) + 2 = 6x + 11.$$

◀

Bemærkning: Selv om $f \circ g$ og $g \circ f$ begge er defineret for funktionerne f og g i det foregående eksempel, er de ikke ens. Sammensætning af funktioner er med andre ord generelt ikke kommutativ.

Når en funktion sammensættes med sin inverse funktion, uanset i hvilken rækkefølge, får man en identitetsfunktion. Antag for at se dette, at f er en bijektion fra mængden A til mængden B. Så findes den inverse funktion f^{-1}, og den er en bijektion fra B til A. Den inverse funktion vender den oprindelige funktions korrespondance om, så $f^{-1}(b) = a$, når $f(a) = b$, og $f(a) = b$, når $f^{-1}(b) = a$.

Derfor er

$$\left(f^{-1} \circ f\right)(a) = f^{-1}(f(a)) = f^{-1}(b) = a,$$

og

$$\left(f \circ f^{-1}\right)(b) = f\left(f^{-1}(b)\right) = f(a) = b.$$

Følgelig er $f^{-1} \circ f = I_A$ og $f \circ f^{-1} = I_B$, hvor I_A og I_B er identitetsfunktionerne på henholdsvis mængden A og mængden B. Det vil sige, at $\left(f^{-1}\right)^{-1} = f$.

Eksempel 1.8 Hvis $f : \mathbb{R} \to \mathbb{R}$ er defineret ved $f(x) = 2x + 3$, så er $f^{-1}(x) = \frac{x-3}{2}$. Den sammensatte funktion $f \circ f^{-1}$ er da identitetsfunktionen $I_{\mathbb{R}}$ på de reelle tal, dvs. $f\left(f^{-1}(x)\right) = x$ for alle $x \in \mathbb{R}$.

1.2 Grafisk identifikation af funktionstyper

Det er grundlæggende i matematikken at forstå, hvordan forskellige typer af funktioner opfører sig. Funktioner kan klassificeres ud fra mønstrene i deres grafer, som giver værdifuld indsigt i deres egenskaber. I dette afsnit undersøger vi forskellige typer af funktioner, herunder lineære funktioner, andengradsfunktioner, eksponentialfunktioner med flere. Ved at undersøge deres grafer kan vi identificere vigtige træk som skæringspunkter med akserne, hældninger, krumning og asymptotisk opførsel, så vi effektivt kan skelne mellem de forskellige funktionstyper.

Lineære funktioner

Den generelle ligning for en lineær funktion er givet ved

$$y = ax + b \quad \text{(ofte skrevet som } y = mx + b\text{)},$$

hvor a (eller m) er hældningen, og b er skæringen med y-aksen. Definitionsmængden for funktionen er alle reelle tal. Ligningen står på hældning-skæringsform, fordi a (eller m) angiver hældningen, og b angiver skæringen med y-aksen. Hvis $a = 0$, reduceres funktionen til $y = b$, som er en konstant funktion.

Grundfunktionen for en lineær ligning er

$$y = x.$$

Den transformerede funktion kan skrives på punkt-hældningsform som

$$y = y_1 + a(x - x_1),$$

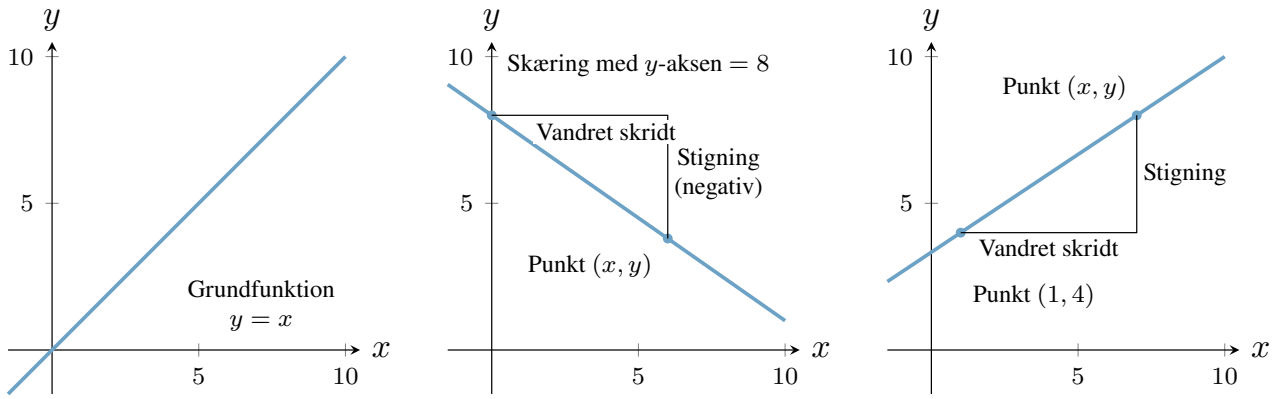
hvor grafen går gennem punktet (x_1, y_1) og har hældningen a. I denne form gælder:

- a er den lodrette skalering (hældningen),
- y_1 angiver den lodrette forskydning,
- x_1 angiver den vandrette forskydning.

Punkt-hældningsformen kan også skrives som

$$y - y_1 = a(x - x_1),$$

hvor koordinaterne til det faste punkt (x_1, y_1) optræder med negativt fortegn. Formen $y = y_1 + a(x - x_1)$ udtrykker y eksplicit ved x, hvilket gør den lettere at indtaste på en grafregner.



$y = -0.7x + 8$ $\qquad$ $y = 4 + \frac{2}{3}(x - 1)$

Figur 1.5: Lineære funktioner

Grafen for en lineær funktion er en ret linje. Grundfunktionen $y = x$ er vist til venstre i Figur 1.5, hældning-skæringsformen i midten og punkt-hældningsformen til højre.

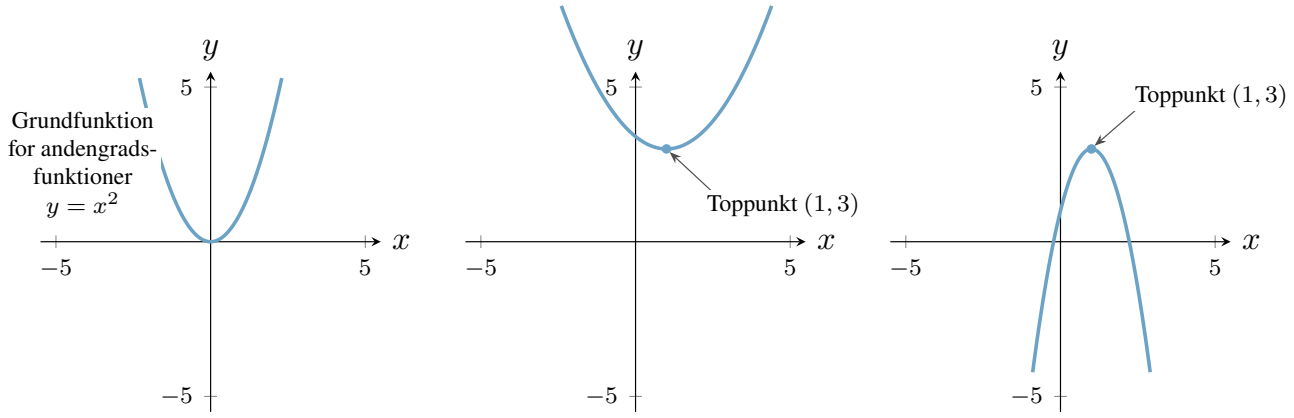
For hældning-skæringsformen gælder: »Start i b på y-aksen, gå x enheder vandret, og stig ax enheder lodret.« For punkt-hældningsformen gælder: »Start i (x_1, y_1), gå $(x - x_1)$ enheder vandret, og stig $a(x - x_1)$ enheder lodret.«

Andengradsfunktioner

Den generelle ligning for en andengradsfunktion er givet ved

$$y = ax^2 + bx + c,$$

hvor $a \neq 0$, og a, b og c er konstanter. Definitionsmængden for funktionen er alle reelle tal.



$y = 3 + 0.4(x - 1)^2$ $\qquad$ $y = 3 - 2(x - 1)^2$

Figur 1.6: Andengradsfunktioner

Grundfunktionen for en andengradsligning er

$$y = x^2,$$

hvor parablens toppunkt ligger i origo $(0, 0)$.

Den transformerede funktion kan skrives på toppunktsform som

$$y = k + a(x - h)^2,$$

hvor parablens toppunkt ligger i (h, k). I denne form gælder:

- k angiver den lodrette forskydning,
- h angiver den vandrette forskydning,
- a angiver den lodrette skalering.

Toppunktsformen kan også skrives som

$$y - k = a(x - h)^2,$$

men når y udtrykkes eksplicit ved x, er ligningen lettere at indtaste på en grafregner.

Grafen for en andengradsfunktion er en parabel (af det græske ord for »langs en bolds bane«). Parablen er konveks (krummer opad), hvis $a > 0$, og konkav (krummer nedad), hvis $a < 0$. Denne opførsel er illustreret i Figur 1.6.

Potensfunktioner

Den generelle ligning for en potensfunktion er givet ved

$$y = ax^b,$$

hvor a og b er konstanter forskellige fra nul. Funktionens definitionsmængde afhænger af værdien af b:

- Hvis $b > 0$, er definitionsmængden alle reelle tal.
- Hvis $b < 0$, udelukker definitionsmængden $x = 0$ for at undgå division med nul.
- Hvis b ikke er et helt tal, udelukker definitionsmængden som regel negative tal for at undgå at tage rødder af negative tal.

I de fleste anvendelser er definitionsmængden begrænset til ikke-negative tal.

Grundfunktionen for en potensfunktion er

$$y = x^b.$$

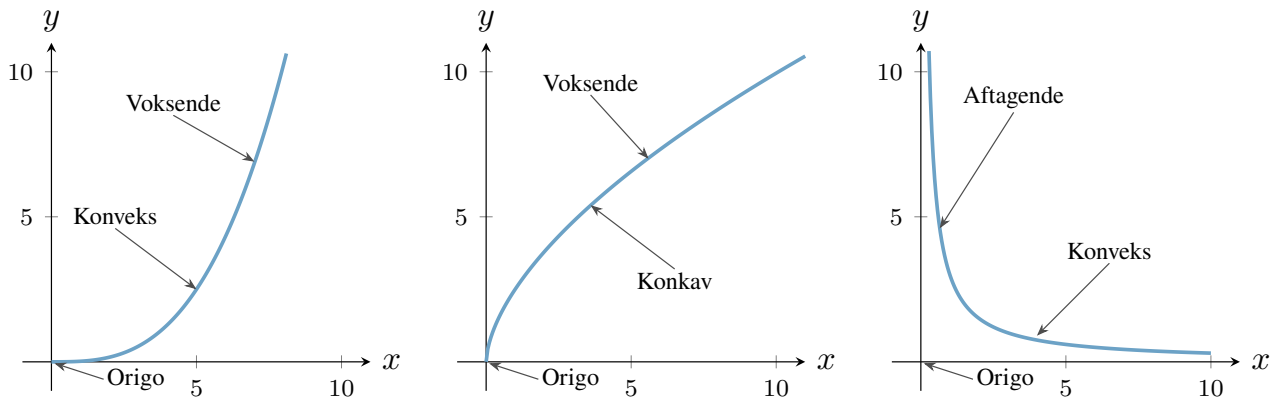
For den generelle potensfunktion $y = ax^b$ gælder:

- Hvis $b > 0$, varierer y ligefrem med b'te potens af x, dvs. y er ligefrem proportional med b'te potens af x.
- Hvis $b < 0$, varierer y omvendt med b'te potens af x, dvs. y er omvendt proportional med b'te potens af x.

Skaleringsfaktoren a fungerer som proportionalitetskonstant.

Den forskudte form af en potensfunktion er

$$y = d + a(x - c)^b,$$



$y = 0.02x^3$
Eksponent større end 1

$y = 2.5x^{0.6}$
Eksponent mellem 0 og 1

$y = 3x^{-1}$
Negativ eksponent

Figur 1.7: Potensfunktioner

hvor c og d er henholdsvis den vandrette og den lodrette forskydning. Dette kan sammenlignes med de forskudte former af lineære funktioner og andengradsfunktioner:

$$y = y_1 + a(x - x_1) \qquad \text{(lineær funktion)},$$
$$y = k + a(x - h)^2 \qquad \text{(andengradsfunktion)}.$$

Medmindre andet er angivet, betyder »potensfunktion« den ikke-forskudte form, $y = ax^b$.

Figur 1.7 viser graferne for potensfunktioner for forskellige værdier af b. I alle tilfælde er $a > 0$. Grafens form og krumning afhænger af værdien af b:

- Hvis $b > 0$, går grafen gennem origo.
- Hvis $b < 0$, har grafen akserne som asymptoter.
- Funktionen er voksende, hvis $b > 0$, og aftagende, hvis $b < 0$.
- Grafen er konveks (krummer opad), hvis $b > 1$ eller $b < 0$, og konkav (krummer nedad), hvis $0 < b < 1$.

Grafens krumning beskriver, hvor hurtigt y vokser. For $b > 0$ betyder konveks, at y vokser med en stigende hastighed, mens konkav betyder, at y vokser med en aftagende hastighed.

Eksponentialfunktioner

Den generelle ligning for en eksponentialfunktion er givet ved

$$y = ab^x,$$

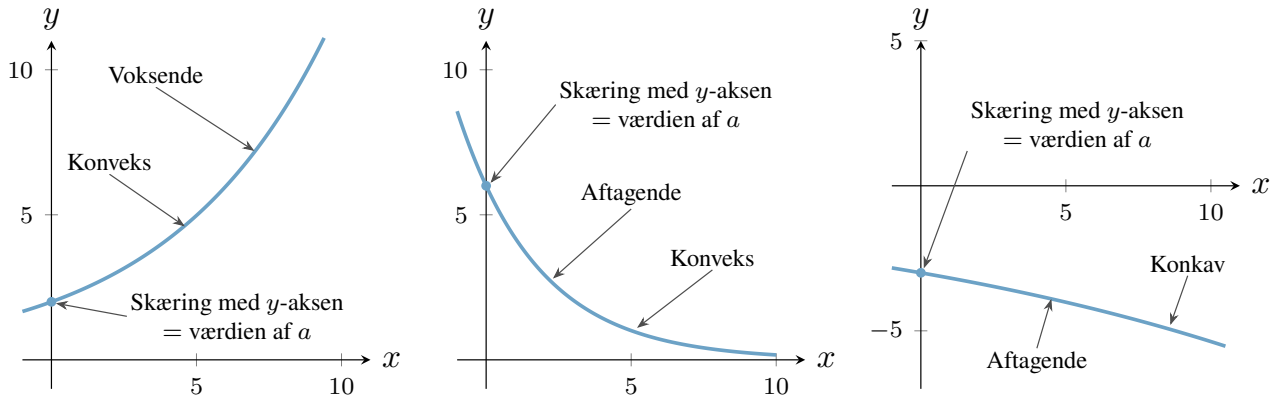
hvor a og b er konstanter, $a \neq 0$, $b > 0$ og $b \neq 1$. Definitionsmængden for funktionen er alle reelle tal.

Grundfunktionen for en eksponentialligning er

$$y = b^x,$$

hvor asymptoten er x-aksen.

I ligningen $y = ab^x$ siger vi, at »y varierer eksponentielt med x«. Det betyder, at y ændres med en konstant faktor b, hver gang x øges med én enhed.



$y = 2 \cdot 1.2^x$
Grundtal større end 1

$y = 6 \cdot 0.7^x$
Grundtal mellem 0 og 1

$y = -3 \cdot 1.06^x$
Negativ koefficient

Figur 1.8: Eksponentialfunktioner

Den forskudte form af eksponentialfunktionen er

$$y = ab^x + c,$$

hvor asymptoten er linjen $y = c$. Medmindre andet er angivet, betyder »eksponentialfunktion« den ikke-forskudte form $y = ab^x$.

Figur 1.8 illustrerer eksponentialfunktioner for forskellige værdier af a og b. Grafens vigtigste egenskaber er:

- Konstanten a er grafens skæring med y-aksen.
- Funktionen er voksende, hvis $b > 1$, og aftagende, hvis $0 < b < 1$, forudsat at $a > 0$.
- Hvis $a < 0$, er funktionens opførsel omvendt: Den er aftagende, hvis $b > 1$, og voksende, hvis $0 < b < 1$.
- Grafen er konveks (krummer opad), hvis $a > 0$, og konkav (krummer nedad), hvis $a < 0$.

Matematikere bruger ofte en af to særlige konstanter som grundtal for en eksponentialfunktion: enten 10, som er grundtallet i titalssystemet, eller det naturligt forekommende tal e, som er tilnærmelsesvis 2.71828. Disse grundtal har stor betydning i mange matematiske anvendelser.

Definition 1.7 (Særlige eksponentialfunktioner)

$y = a \cdot 10^{bx}$ eksponentialfunktion med grundtal 10

$y = a \cdot e^{bx}$ naturlig eksponentialfunktion (grundtal e),

hvor a og b er konstanter, og definitionsmængden er alle reelle tal.

For at generalisere eksponentialfunktionen ganges variablen i eksponenten ofte med en konstant. De (ikke-forskudte) generelle former af disse eksponentialfunktioner er givet nedenfor:

$$y = a \cdot 10^{bx} \quad \text{og} \quad y = a \cdot e^{bx}$$

Funktionerne kan generaliseres yderligere ved at indføre forskydninger i både x- og y-retningen. De forskudte former er:

$$y = a \cdot 10^{b(x-c)} + d \quad \text{og} \quad y = a \cdot e^{b(x-c)} + d$$

Eksponentialfunktionen med grundtal e har især en betydelig fordel i differentialregning, fordi ændringshastigheden for e^x er lig med e^x selv.

1.3 Logaritmer

Ethvert positivt tal kan skrives som en potens af 10. For eksempel er

$$3 = 10^{0.477\ldots}$$
$$5 = 10^{0.6989\ldots}$$
$$15 = 10^{1.1760\ldots}$$

Eksponenterne $0.4771\ldots$, $0.6989\ldots$ og $1.1760\ldots$ kaldes 10-talslogaritmerne af henholdsvis 3, 5 og 15:

$$\log 3 = 0.4771\ldots$$
$$\log 5 = 0.6989\ldots$$
$$\log 15 = 1.1760\ldots$$

For bedre at forstå, hvad logaritmer betyder, kan du trykke `LOG` 3 på din lommeregner. Du får:

$$\log 3 = 0.477121254\ldots$$

Opløft derefter 10 til denne potens uden at afrunde. Du får:

$$10^{0.477121254\ldots} = 3$$

Potenser af 10 har de sædvanlige egenskaber for potenser. For eksempel er

$$\begin{aligned} 15 = (3)(5) &= \left(10^{0.4771\ldots}\right)\left(10^{0.6990\ldots}\right) \\ &= 10^{0.4771\ldots+0.6990\ldots} \\ &= 10^{1.1761\ldots} \end{aligned}$$

Det betyder, at $10^{0.4771\ldots+0.6990\ldots} = 10^{1.1761\ldots}$. Her lægger du eksponenterne sammen og beholder det samme grundtal. Du kan kontrollere med din lommeregner, at $10^{1.1761\ldots}$ er tilnærmelsesvis 15; de viste decimaler er afrundet.

Af dette eksempel kan du slutte, at logaritmer har de samme egenskaber som eksponenter. Det er forventeligt, fordi logaritmer *er* eksponenter. For eksempel er

$$\log(3 \cdot 5) = \log 3 + \log 5 \quad \textit{Logaritmen af et produkt er summen af faktorernes logaritmer.}$$

Ud fra de tidligere angivne værdier kan du også vise, at:

$$\log \frac{15}{3} = \log 15 - \log 3 \quad \textit{Logaritmen af en kvotient.}$$

Denne egenskab er rimelig, fordi man dividerer potenser med samme grundtal ved at trække eksponenterne fra hinanden:

$$\frac{15}{3} = \frac{10^{1.1760\ldots}}{10^{0.477\ldots}} = 10^{1.1760\ldots-0.4771\ldots} = 10^{0.6989\ldots} = 5$$

Da en potens kan skrives som et produkt, kan du finde logaritmen af en potens på følgende måde:

$$\begin{aligned} \log 3^4 &= \log(3 \cdot 3 \cdot 3 \cdot 3) = \log 3 + \log 3 + \log 3 + \log 3 \\ &= 4 \log 3 \quad \textit{Saml ensartede led.} \end{aligned}$$

Logaritmen af en potens er eksponenten gange logaritmen af det tal, der opløftes til potensen. For at kontrollere resultatet kan du bemærke, at $3^4 = 81$. Tryk $4 \times$ `LOG` 3 på din lommeregner, og du vil se, at det giver $1.9084\ldots$.

Definition 1.8 (10-talslogaritmer)

$$\log x = y \iff 10^y = x, \qquad x > 0$$

Med ord: $\log x$ er eksponenten i den potens af 10, der giver x

♣

Ordet logaritme kommer af de græske ord *logos*, der betyder »forhold«, og *arithmos*, der betyder »tal«. Før lommeregneren blev opfundet, blev 10-talslogaritmer beregnet tilnærmelsesvis ved hjælp af uendelige rækker og opført i tabeller. Produkter med mange faktorer, som

$$(357)(4.367)(22.4)(3.142)$$

kunne beregnes ved at lægge faktorernes logaritmer sammen i stedet for møjsommeligt at gange flere talpar sammen. John Napier introducerede logaritmerne i begyndelsen af det syttende århundrede, og Henry Briggs udviklede og udgav efterfølgende omfattende tabeller over titalslogaritmer (med grundtal 10).

Regneregler for 10-talslogaritmer

For $x > 0$, $y > 0$ og $r \in \mathbb{R}$ gælder følgende regler:

- Logaritmen af et produkt:
  $$\log(xy) = \log x + \log y$$
  Med ord: log af et produkt er summen af faktorernes log.
- Logaritmen af en kvotient:
  $$\log \frac{x}{y} = \log x - \log y$$
  Med ord: log af en kvotient er tællerens log minus nævnerens log.
- Logaritmen af en potens:
  $$\log x^r = r \log x$$
  Med ord: log af en potens er eksponenten gange log af det tal, der opløftes til potensen.

Eksempel 1.9 Find x, hvis $\log_{10} 10^{3.721} = x$

Løsning: Ifølge definitionen er logaritmen eksponenten til 10. Altså er $x = 3.721$. ◀

Eksempel 1.10 Find x, hvis $0.258 = 10^x$

Løsning: Ifølge definitionen er x, eksponenten til 10, logaritmen af 0.258.

$$x = \log_{10} 0.258 = -0.5883\ldots$$

◀

Det vigtigste, du skal huske om logaritmer, er dette

En logaritme er en eksponent.

Logaritmer med vilkårligt grundtal: reglen for basisskift

Hvis $x = 10^y$, er y 10-talslogaritmen af x. Tilsvarende gælder, at hvis $x = 2^y$, er y 2-talslogaritmen af x. Den eneste forskel på disse logaritmer er det tal, der fungerer som grundtal. For at skelne mellem logaritmer med

forskellige grundtal skrives grundtallet som et sænket tegn efter forkortelsen »log«. For eksempel:

$$3 = \log_2 8 \Leftrightarrow 2^3 = 8,$$
$$4 = \log_3 81 \Leftrightarrow 3^4 = 81,$$
$$2 = \log_{10} 100 \Leftrightarrow 10^2 = 100.$$

Symbolet $\log_2 8$ udtales »logaritmen med grundtal 2 af 8«. Symbolet $\log_{10} 100$ er naturligvis det samme som $\log 100$, som defineret i det foregående afsnit. Bemærk, at en logaritme i alle tilfælde er en eksponent.

Definition 1.9 (Logaritme med vilkårligt grundtal)

Algebraisk:

$\log_b x = y$ hvis og kun hvis $b^y = x$, hvor $b > 0, b \neq 1$, og $x > 0$

Med ord:

$\log_b x = y$ betyder, at y er den eksponent til b, der giver x som resultat.

♣

Måden, du udtaler logaritmesymbolet på, giver dig en måde at huske definitionen på. De næste to eksempler viser, hvordan du gør.

Eksempel 1.11 Skriv $\log_5 c = a$ på eksponentiel form.

Løsning:

Tænk sådan:

- »$\log_5 \ldots$« læses »logaritmen med grundtal 5 …«, dvs. 5 er grundtallet.
- En logaritme er en eksponent. Da log er lig med a, må a være eksponenten.
- Det »resultat«, man får af 5^a, er logaritmens argument, som betegnes c.

Skriv kun dette:

$$5^a = c$$

◀

Eksempel 1.12 Antag, at $z > 0$, $z \neq 1$ og $m > 0$, og skriv $z^4 = m$ på logaritmisk form.

Løsning: $\log_z m = 4$ ◀

To grundtal for logaritmer bruges så ofte, at de har deres egen tast på de fleste lommeregnere. Det ene er 10-talslogaritmen, også kaldet titalslogaritmen, som blev behandlet i det foregående afsnit. Det andet er logaritmen med grundtal e, kaldet den naturlige logaritme, hvor $e = 2.71828\ldots$ er et naturligt forekommende tal (ligesom π), som vil være en fordel i dine videre matematikstudier.

Symbolet $\ln x$ (udtales »el-en af x«) bruges om naturlige logaritmer og er defineret som:

$$\ln x = \log_e x$$

Definition 1.10 (Titalslogaritme og naturlig logaritme)

Titalslogaritme: Symbolet $\log x$ betyder $\log_{10} x$.

Naturlig: Symbolet $\ln x$ betyder $\log_e x$, hvor e er en konstant lig med $2.71828182845\ldots$

♣

Eksempel 1.13 Find $\log_5 17$. Kontrollér dit svar med en passende numerisk metode.

Løsning: Lad $x = \log_5 17$.

$$
\begin{aligned}
5^x &= 17 \\
\log_{10} 5^x &= \log_{10} 17 \\
x \log_{10} 5 &= \log_{10} 17 \\
x &= \frac{\log_{10} 17}{\log_{10} 5} = 1.7603\ldots \\
\log_5 17 &= 1.7603\ldots \\
5^{1.7603\ldots} &= 17
\end{aligned}
$$

◀

Bemærk i dette eksempel, at 5-talslogaritmen af et tal er ligefrem proportional med 10-talslogaritmen af samme tal. Eksemplets konklusion kan udtrykkes således:

$$\log_5 17 = \frac{1}{\log_{10} 5} \cdot \log_{10} 17 = 1.4306\ldots \log_{10} 17$$

For at finde 5-talslogaritmen af et vilkårligt tal skal du blot gange tallets 10-talslogaritme med $1.4306\ldots$ (dvs. dividere med $\log_{10} 5$).

Denne proportionale sammenhæng kaldes reglen for basisskift. Ud fra resultaterne i Eksempel 1.13 kan du skrive:

$$\log_5 17 = \frac{\log_{10} 17}{\log_{10} 5}$$

Bemærk, at logaritmen med det ønskede grundtal står isoleret på ligningens venstre side, mens de to logaritmer på højre side har samme grundtal – typisk et, som findes på din lommeregner. Boksen nedenfor viser reglen for grundtallene a og b med argumentet x:

Reglen for basisskift for logaritmer

$$\log_a x = \frac{\log_b x}{\log_b a} \quad \text{eller} \quad \log_a x = \frac{1}{\log_b a} (\log_b x)$$

hvor $a > 0$, $a \neq 1$, $b > 0$, $b \neq 1$ og $x > 0$.

Eksempel 1.14 Find $\ln 29$ ved hjælp af reglen for basisskift med 10-talslogaritmer. Kontrollér dit svar direkte ved at trykke $\ln 29$ på din lommeregner.

Løsning:

$$\ln 29 = \frac{\log 29}{\log e} = \frac{1.4623\ldots}{0.4342\ldots} = 3.3672\ldots$$

Direkte: $\ln 29 = 3.3672\ldots$,

hvilket stemmer med det svar, vi fik ved hjælp af reglen for basisskift. ◀

Logaritmeregler

Lad $b > 0$ med $b \neq 1$, og lad $x > 0$, $y > 0$ og $r \in \mathbb{R}$.
Logaritmen af en potens:

$$\log_b x^r = r \log_b x$$

Med ord: Logaritmen af en potens er produktet af eksponenten og logaritmen af det tal, der opløftes til potensen. Logaritmen af et produkt:

$$\log_b(xy) = \log_b x + \log_b y$$

Med ord: Logaritmen af et produkt er summen af faktorernes logaritmer. Logaritmen af en kvotient:

$$\log_b \frac{x}{y} = \log_b x - \log_b y$$

Med ord: Logaritmen af en kvotient er tællerens logaritme minus nævnerens logaritme.

Løsning af eksponential- og logaritmeligninger

Logaritmer giver en metode til at løse en ligning med en variabel i eksponenten eller en ligning, der allerede indeholder logaritmer. Det viser vi i de næste eksempler.

Eksempel 1.15 Løs eksponentialligningen $7^{3x} = 983$ algebraisk ved hjælp af logaritmer.

Løsning:

$7^{3x} = 983$

$\log 7^{3x} = \log 983$ `Tag 10-talslogaritmen på begge sider.`

$3x \log 7 = \log 983$ `Brug reglen for logaritmen af en potens.`

$x = \dfrac{\log 983}{3 \log 7}$ `Dividér begge sider med koefficienten til x.`

$x = 1.1803\ldots$

◀

Eksempel 1.16 Løs ligningen

$$\log_2(x-1) + \log_2(x-3) = 3$$

Løsning:

$$\log_2(x-1)+\log_2(x-3)=3$$
$$\log_2[(x-1)(x-3)]=3 \qquad \texttt{Brug reglen for logaritmen af et produkt.}$$
$$2^3=(x-1)(x-3) \qquad \texttt{Brug definitionen af logaritmen.}$$
$$8=x^2-4x+3 \qquad \texttt{Gang parenteserne ud.}$$
$$x^2-4x-5=0 \qquad \texttt{Få nul på den ene side. Brug lighedens symmetriegenskab.}$$
$$(x-5)(x+1)=0 \qquad \texttt{Løs ved faktorisering.}$$
$$x=5 \quad \text{eller} \quad x=-1$$

Vi skal være forsigtige her, fordi løsningerne i sidste trin er løsninger til andengradsligningen, og vi skal sikre os, at de også er løsninger til den oprindelige logaritmeligning. Kontrollér ved at indsætte løsningerne i den oprindelige ligning.

Hvis $x=5$, så er
$$\log_2(5-1)+\log_2(5-3)$$
$$=\log_2 4+\log_2 2$$
$$=2+1=3$$

Hvis $x=-1$, så er
$$\log_2(-1-1)+\log_2(-1-3)$$
$$=\log_2(-2)+\log_2(-4)$$
hvilket ikke er defineret.

◀

Eksempel 1.17 Løs ligningen
$$e^{2x}-3e^x+2=0$$

Løsning:
$$e^{2x}-3e^x+2=0$$
$$(e^x)^2-3e^x+2=0$$

Vi indser, at dette er en andengradsligning i variablen e^x. Med løsningsformlen for andengradsligningen får du
$$e^x=\frac{+3\pm\sqrt{9-4(2)}}{2}=\frac{3\pm1}{2}$$
$e^x=2$ eller $e^x=1$

Nu skal du løse disse to ligninger.

$e^x=2$	$e^x=1$
$x=\ln 2=0.6931\ldots$	$x=0$

Kontrol:

$e^{2\ln 2}-3e^{\ln 2}+2$	$(e^0)^2-3e^0+2$
$=\left(e^{\ln 2}\right)^2-3e^{\ln 2}+2$	$=1^2-3(1)+2=0$
$=2^2-3(2)+2=0$	

Begge løsninger er korrekte. ◀

Eksempel 1.18 Løs logaritmeligningen $\ln(x+3)+\ln(x+5)=0$

Løsning:

$$\ln(x+3)+\ln(x+5)=0$$
$$\ln[(x+3)(x+5)]=0$$
$$(x+3)(x+5)=e^0=1$$
$$x^2+8x+15=1$$
$$x^2+8x+14=0$$
$$x=-2.5857\ldots \quad \text{eller} \quad x=-5.4142\ldots$$

Kontrol:

$x=-2.5857\ldots$:

$$\ln(-2.5857\ldots+3)+\ln(-2.5857\ldots+5)$$
$$=\ln(0.4142\ldots)+\ln(2.4142\ldots)$$
$$=-0.8813\ldots+0.8813\ldots=0$$

hvilket er i orden.

$x=-5.4142\ldots$:

$$\ln(-5.4142\ldots+3)+\ln(-5.4142\ldots+5)$$
$$=\ln(-2.4142\ldots)+\ln(-0.4142\ldots)$$

hvilket ikke er defineret.

Den eneste gyldige løsning er $x=-2.5857\ldots$. ◀

Kapitel 2 Talsystemer

Vi er så vant til at arbejde i titalssystemet, at vi ofte glemmer, at det er en forholdsvis ny opfindelse, som engang blev betragtet som revolutionerende. Det er på tide at se nøje på, hvordan vi repræsenterer tal. Normalt bruger vi titalssystemet, hvor et tal som 3459 er en kort skrivemåde for $3 \times 1000 + 4 \times 100 + 5 \times 10 + 9$. Hvert ciffers position er afgørende, fordi den gør det muligt at skelne mellem værdier som 30 og 3. Titalssystemet er et **positionssystem**, dvs. det har faste positioner for enere, tiere, hundreder og så videre. Et ciffers position angiver den multiplikator (en potens af ti), som cifferet skal ganges med, og hver position har en værdi, der er ti gange så stor som værdien af positionen til højre for den.

Bemærk, at vi kan spare plads ved at skrive 1000 som 10^3, hvor eksponenten 3 angiver antallet af nuller. Altså er $100000 = 10^5$. Hvis eksponenten er negativ, repræsenterer den en brøk, fx $10^{-3} = \frac{1}{1000}$. Det mest geniale ved positionssystemet var måske indførelsen af decimalpunktummet, som gør det muligt at medtage decimalbrøker. For eksempel svarer tallet 123.456 til:

$$1 \times 100 + 2 \times 10 + 3 \times 1 + 4 \times \frac{1}{10} + 5 \times \frac{1}{100} + 6 \times \frac{1}{1000}.$$

Det kan illustreres således:

Multiplikator:	$\ldots$	10^2	10^1	10^0	.	10^{-1}	10^{-2}	10^{-3}	$\ldots$
Cifre:	$\ldots$	1	2	3	.	4	5	6	$\ldots$
					↑				
					Decimalpunktum:				

Der er imidlertid ingen grund til, at vi nødvendigvis skal bruge potenser af 10, dvs. grundtallet 10. Babylonierne brugte for eksempel grundtallet 60, og grundtallet 12 var meget udbredt i middelalderens Europa. I dag er de mest anvendte talsystemer sammenfattet i Tabel 2.1

Talsystem	Symboler	Grundtal	Yderligere oplysninger
Decimal	0-9	10	-
Binær	0, 1	2	-
Hexadecimal	0-9, A-F	16	$\mathrm{A} \equiv 10, \mathrm{B} \equiv 11, \mathrm{C} \equiv 12, \mathrm{D} \equiv 13, \mathrm{E} \equiv 14, \mathrm{F} \equiv 15$
Oktal	0-7	8	-

Tabel 2.1: Oversigt over almindelige talsystemer

Vi begynder med totalssystemet, som også er det system, vi behandler mest indgående i dette kapitel.

2.1 Binære tal

I totalssystemet udtrykker vi tal ved potenser af 2 i stedet for potenser af 10 som i titalssystemet. For nogle tal er det let. Husk, at $2^0 = 1$,

Som i titalssystemet skriver vi tallet, så cifrets position angiver potensen: Den første plads efter det binære punktum er 2^{-1}-pladsen, den næste er 2^{-2}-pladsen og så videre. For at omregne et decimaltal til et binært tal kan vi bruge operatoren `mod`.

Decimaltal		Som potenser af 2	Potens af 2				Binært tal
			3	2	1	0	
8	=	2^3	1	0	0	0	1000
7	=	$2^2 + 2^1 + 2^0$	0	1	1	1	111
6	=	$2^2 + 2^1$	0	1	1	0	110
5	=	$2^2 + 2^0$	0	1	0	1	101
4	=	2^2	0	1	0	0	100
3	=	$2^1 + 2^0$	0	0	1	1	11
2	=	2^1	0	0	1	0	10
1	=	2^0	0	0	0	1	1

Tabel 2.2: Decimaltal i binær repræsentation

Se som eksempel på 88 i titalssystemet, dvs. 88_{10}. Vi vil gerne skrive det som et binært tal. Vi dividerer gentagne gange tallet med 2, bruger heltalskvotienten i næste trin og noterer resten. Se nedenfor:

Trin nr. n	x_n	$\lfloor x_n/2 \rfloor$	$x_n \bmod 2$
0	88	44	0
1	44	22	0
2	22	11	0
3	11	5	1
4	5	2	1
5	2	1	0
6	1	0	1

Tabel 2.3: Omregning af decimaltallet 88 til binær form

Når vi skriver den sidste kolonne i omvendt rækkefølge, dvs. nedefra og op, får vi 1011000, som er den binære form af 88, dvs. $88_{10} = 1011000_2$.

Binære tal med cifre efter punktummet er mindre almindelige, men sagtens mulige. Således er 101.1011 blot $2^2+2^0+2^{-1}+2^{-3}+2^{-4}$, hvilket efter lidt regning giver 5.6875. Vi har set, hvordan man omregner heltalsdelen af et decimaltal til et binært tal, og vi kan gøre det samme med en decimalbrøk. Se på 0.6875. Som før opstiller vi en tabel:

Trin nr. n	x_n	$x_n \times 2$	$\lfloor x_n \times 2 \rfloor$
0	0.6875	1.375	1
1	0.375	0.75	0
2	0.75	1.5	1
3	0.5	1	1

Tabel 2.4: Omregning af decimalbrøken 0.6875 til binær form

Når vi læser nedad, får vi $0.6875_{10} = 0.1011_2$.

Binær opløsning

Fremgangsmåden fra det foregående afsnit kaldes **binær opløsning** og betegner repræsentationen af et tal i totalssystemet (grundtal 2). Ethvert decimaltal kan skrives som en sum af potenser af 2, hvor hver potens svarer til et binært ciffer (en bit) i tallets binære form.

Lad os igen se på decimaltallet 88. For at finde dets binære opløsning finder vi den største potens af 2, der er mindre end eller lig med 88, og fortsætter med at trække potenser af 2 fra, indtil vi når 0.

Først bemærker vi, at $2^6 = 64$ er den største potens af 2, der er mindre end 88:

$$88 = 64 + 24$$

Dernæst finder vi, at $2^4 = 16$ er den største potens af 2, der er mindre end 24:

$$24 = 16 + 8$$

Til sidst passer $2^3 = 8$ præcis med resten:

$$8 = 8 + 0$$

Altså har vi:

$$88 = 2^6 + 2^4 + 2^3$$

I binær form repræsenteres hver af disse potenser af 2 af et »1« på den tilsvarende plads, mens der står »0« på de pladser, hvor ingen potens af 2 bidrager:

$$88_{10} = 1011000_2$$

Sammenfattende:

- $2^6 = 64$ svarer til det »1« længst til venstre i den binære opløsning.
- $2^4 = 16$ svarer til det næste »1«.
- $2^3 = 8$ svarer til det næste »1«.
- De øvrige cifre er »0«, fordi 2^5, 2^2, 2^1 og 2^0 ikke bidrager til værdien 88.

Den binære opløsning af 88 er altså 1011000_2. Denne måde at repræsentere tal på er grundlæggende i datalogi og digital elektronik, hvor binær repræsentation er standarden for lagring og behandling af data.

Binære operationer

Binære operationer er grundlæggende regneoperationer, der udføres på binære tal. Disse operationer er afgørende i computere og digitale systemer, fordi de udgør fundamentet for, hvordan computere behandler og manipulerer data.

Binær addition, subtraktion og multiplikation svarer til de tilsvarende operationer i titalssystemet, men følger enklere regler, fordi totalssystemet kun har få cifre. For eksempel følger binær addition disse regler:

$$0 + 0 = 0, \quad 0 + 1 = 1, \quad 1 + 0 = 1, \quad 1 + 1 = 10$$

Her giver $1 + 1$ resultatet 10_2, hvilket betyder 0 med 1 i mente til den næste, højere bit. Binær subtraktion og multiplikation følger tilsvarende enkle regler, som er lette at implementere i digitale systemer.

XOR-operationen (eksklusivt OR) er en anden vigtig binær operation. XOR giver 1, hvis de to bit, der sammenlignes, er forskellige, og 0, hvis de er ens:

$$0 \oplus 0 = 0, \quad 0 \oplus 1 = 1, \quad 1 \oplus 0 = 1, \quad 1 \oplus 1 = 0$$

Ved binær addition bruges XOR-operationen til at lægge to bit sammen uden at tage hensyn til en eventuel mente fra en foregående bit. Det skyldes, at XOR reelt udfører addition modulo 2, hvilket passer præcis med, hvordan binær addition fungerer. For eksempel:

Bit 1	Bit 2	XOR (sum)	AND (mente)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

I tilfældet $1 + 1$ giver XOR summen 0, og en AND-operation (som registrerer menten) giver menten 1, hvilket resulterer i det binære tal 10.

$$\begin{aligned} 0 + 0 &= 0 \\ 0 + 1 &= 1 \\ 1 + 1 &= 10 \quad \text{så vi har 1 i mente og skriver et nul} \\ 1 + 1 + 1 &= 1 + (1 + 1) = 1 + 10 = 11. \end{aligned}$$

Vi kan opstille det på stort set samme måde som en addition i titalssystemet:

	1	1	0	1	0	1	
+	1	0	1	1	1	0	
1	1	0	0	0	1	1	Sum
↑				↑			

Den højre pil viser, hvor vi har 1 i mente. Den venstre pil viser, hvor vi har $1 + 1 + 1$, så vi har 1 i mente og 1 tilbage.

Som vi skal se nedenfor, får vi ofte brug for at håndtere flere menter. Det kan gøres på to måder, der minder om de metoder, vi kender fra titalssystemet. Vi forklarer dem med et eksempel.

Metode 1: Kolonnevis binær addition med flere menter

Se på

$$\begin{array}{cccccc} & 1 & 1 & 1 & 1 & 1 \\ + & 1 & 1 & 1 & 0 & 1 \\ + & 1 & 1 & 1 & 0 & 1 \\ + & 1 & 1 & 1 & 1 & 1 \\ \hline \end{array}$$

Trin 1: Læg kolonnen længst til højre sammen
Begynd med at lægge bittene længst til højre sammen:

$$1 + 1 + 1 + 1 = 100_2 \quad (\text{ som er binært for } 4)$$

Når vi læser resultatet fra højre mod venstre (dvs. fra *mindst betydende bit* (LSB) til *mest betydende bit*(MSB)), gør vi følgende:

- skriv 0
- overfør 0 som mente til næste kolonne
- overfør 1 som mente til tredje kolonne

Du ender med

$$\begin{array}{cccccc} & & & {\scriptstyle 1} & {\scriptstyle 0} & \\ & 1 & 1 & 1 & 1 & 1 \\ + & 1 & 1 & 1 & 0 & 1 \\ + & 1 & 1 & 1 & 0 & 1 \\ + & 1 & 1 & 1 & 1 & 1 \\ \hline & & & & & 0 \end{array}$$

Trin 2: Læg den anden kolonne fra højre sammen
Læg dernæst den anden kolonne sammen:

$$0 + 1 + 0 + 0 + 1 = 10_2 \quad (\text{ som er binært for } 2)$$

Når vi læser resultatet fra LSB til MSB:

- skriv 0
- overfør 1 som mente til næste kolonne

Du ender med

$$\begin{array}{cccccc} & & & {\scriptstyle 1} & & \\ & & & {\scriptstyle 1} & & \\ & 1 & 1 & 1 & 1 & 1 \\ + & 1 & 1 & 1 & 0 & 1 \\ + & 1 & 1 & 1 & 0 & 1 \\ + & 1 & 1 & 1 & 1 & 1 \\ \hline & & & & 0 & 0 \end{array}$$

Trin 3: Læg den tredje kolonne fra højre sammen
Læg nu den tredje kolonne sammen:

$$1 + 1 + 1 + 1 + 1 + 1 = 110_2 \quad (\text{ som er binært for } 6)$$

Når vi læser resultatet fra LSB til MSB:

- skriv 0
- overfør 1 som mente til fjerde kolonne
- overfør 1 som mente til femte kolonne

Du ender med

Vores sum indtil videre:

$$\begin{array}{cccccc} & & {\scriptstyle 1} & {\scriptstyle 1} & & \\ & & 1 & 1 & 1 & 1 & 1 \\ + & & 1 & 1 & 1 & 0 & 1 \\ + & & 1 & 1 & 1 & 0 & 1 \\ + & & 1 & 1 & 1 & 1 & 1 \\ \hline & & & & 0 & 0 & 0 \end{array}$$

Trin 4: Læg den fjerde kolonne fra højre sammen
Gå videre til den fjerde kolonne:

$$1 + 1 + 1 + 1 + 1 = 101_2 \quad (\text{ som er binært for } 5)$$

Når vi læser resultatet fra LSB til MSB:

- skriv 1
- overfør 0 som mente til femte kolonne
- overfør 1 som mente til sjette kolonne

Du ender med

$$\begin{array}{cccccc} & & & {\scriptstyle 0} & & \\ & {\scriptstyle 1} & {\scriptstyle 1} & & & \\ & 1 & 1 & 1 & 1 & 1 \\ + & 1 & 1 & 1 & 0 & 1 \\ + & 1 & 1 & 1 & 0 & 1 \\ + & 1 & 1 & 1 & 1 & 1 \\ \hline & & 1 & 0 & 0 & 0 \end{array}$$

Trin 5: Læg kolonnen længst til venstre sammen
Læg kolonnen længst til venstre sammen:

$$1 + 1 + 1 + 1 + 1 = 101_2 \quad (\text{ som er binært for } 5)$$

Når vi læser resultatet fra LSB til MSB:

- skriv 1
- overfør 0 som mente til sjette kolonne
- overfør 1 som mente til syvende kolonne

Det giver

$$\begin{array}{cccccc} & {\scriptstyle 1\ 1} & {\scriptstyle 0} & & & \\ & 1 & 1 & 1 & 1 & 1 \\ + & & 1 & 1 & 1 & 0 & 1 \\ + & & 1 & 1 & 1 & 0 & 1 \\ + & & 1 & 1 & 1 & 1 & 1 \\ \hline & & 1 & 1 & 0 & 0 & 0 \end{array}$$

Trin 6: Læg de resterende menter sammen
Læg til sidst de resterende menter sammen:

$$\begin{array}{cccccc} & & 1 & 1 & 1 & 1 & 1 \\ + & & 1 & 1 & 1 & 0 & 1 \\ + & & 1 & 1 & 1 & 0 & 1 \\ + & & 1 & 1 & 1 & 1 & 1 \\ \hline & 1 & 1 & 1 & 1 & 0 & 0 & 0 \end{array}$$

Det følgende eksempel viser hele processen, hvor forskellige farver skelner mellem kolonnerne og de menter, de giver anledning til. Bemærk, at de to sidste cifre i summen er sorte, fordi de ikke stammer fra en bestemt kolonne, men udelukkende fremkommer af menterne.

$$\begin{array}{cccccc} & {\scriptstyle 1\ 1} & {\scriptstyle 0\ 1} & {\scriptstyle 0} & {\scriptstyle 1} & {\scriptstyle 1\ 1\ 0} \\ & 1 & 1 & 1 & 1 & 1 \\ + & & 1 & 1 & 1 & 0 & 1 \\ + & & 1 & 1 & 1 & 0 & 1 \\ + & & 1 & 1 & 1 & 1 & 1 \\ \hline & 1 & 1 & 1 & 1 & 0 & 0 & 1 \end{array}$$

Metode 2: Direkte summation og forenkling

Vi illustrerer den anden metode med det samme eksempel. I det foregående tilfælde overførte vi selve det binære tal til de næste kolonner. I denne metode skriver vi 0, hvis summen er lige, og 1, hvis summen er ulige. For hvert multiplum af 2 i summen overfører vi 1 som mente til de næste kolonner, og vi fortsætter sådan for hver kolonne, idet menterne indgår i beregningen af den pågældende kolonne.

Trin 1: Læg kolonnen længst til højre sammen
Læg bittene i kolonne 1 sammen (talt fra LSB):

$$1 + 1 + 1 + 1 = 100_2 \quad (\text{ som er binært for } 4)$$

Når vi læser resultatet fra LSB til MSB:

- skriv 0
- overfør 1 som mente for det første multiplum af 2
- overfør 1 som mente for det andet multiplum af 2

Det giver

$$\begin{array}{cccccc} & & & & {\scriptstyle 1}\,{\scriptstyle 1} & \\ & 1 & 1 & 1 & 1 & 1 \\ + & 1 & 1 & 1 & 0 & 1 \\ + & 1 & 1 & 1 & 0 & 1 \\ + & 1 & 1 & 1 & 1 & 1 \\ \hline & & & & & 0 \end{array}$$

Trin 2: Læg den anden kolonne fra højre sammen
Læg bittene i kolonne 2 sammen (talt fra LSB):

$$1 + 1 + 1 + 1 = 100_2 \quad (\text{ som er binært for } 4)$$

Når vi læser resultatet fra LSB til MSB:

- skriv 0
- overfør 1 som mente for det første multiplum af 2
- overfør 1 som mente for det andet multiplum af 2

Det giver

$$\begin{array}{cccccc} & & & {\scriptstyle 1} & & \\ & & & {\scriptstyle 1} & & \\ & 1 & 1 & 1 & 1 & 1 \\ + & 1 & 1 & 1 & 0 & 1 \\ + & 1 & 1 & 1 & 0 & 1 \\ + & 1 & 1 & 1 & 1 & 1 \\ \hline & & & & 0 & 0 \end{array}$$

Trin 3: Læg den tredje kolonne fra højre sammen Læg bittene i kolonne 3 sammen (talt fra LSB):

$$1 + 1 + 1 + 1 + 1 + 1 = 110_2 \quad (\text{ som er binært for } 6)$$

Når vi læser resultatet fra LSB til MSB:

- skriv 0
- overfør 1 som mente for det første multiplum af 2
- overfør 1 som mente for det andet multiplum af 2
- overfør 1 som mente for det tredje multiplum af 2

Det giver

$$\begin{array}{cccccc} & & {\scriptstyle 1} & & & \\ & & {\scriptstyle 1} & & & \\ & & {\scriptstyle 1} & & & \\ & 1 & 1 & 1 & 1 & 1 \\ + & 1 & 1 & 1 & 0 & 1 \\ + & 1 & 1 & 1 & 0 & 1 \\ + & 1 & 1 & 1 & 1 & 1 \\ \hline & & & 0 & 0 & 0 \end{array}$$

Trin 4: Læg den fjerde kolonne fra højre sammen

Læg bittene i kolonne 4 sammen (talt fra LSB):

$$1+1+1+1+1+1+1 = 111_2 \quad (\text{ som er binært for } 7)$$

Når vi læser resultatet fra LSB til MSB:

- skriv 1
- overfør 1 som mente for det første multiplum af 2
- overfør 1 som mente for det andet multiplum af 2
- overfør 1 som mente for det tredje multiplum af 2

Det giver

$$\begin{array}{cccccc} & & {\scriptstyle 1} & & & \\ & & {\scriptstyle 1} & & & \\ & & {\scriptstyle 1} & & & \\ & 1 & 1 & 1 & 1 & 1 \\ + & 1 & 1 & 1 & 0 & 1 \\ + & 1 & 1 & 1 & 0 & 1 \\ + & 1 & 1 & 1 & 1 & 1 \\ \hline & & 1 & 0 & 0 & 0 \end{array}$$

Trin 5: Læg kolonnen længst til venstre sammen

Læg bittene i kolonnen længst til venstre sammen:

$$1+1+1+1+1+1+1 = 111_2 \quad (\text{ som er binært for } 7)$$

Når vi læser resultatet fra LSB til MSB:

- skriv resultatet i binær form, dvs. 1 1 1

Det giver

$$
\begin{array}{cccccccc}
 & & & 1 & 1 & 1 & 1 & 1 \\
+ & & & 1 & 1 & 1 & 0 & 1 \\
+ & & & 1 & 1 & 1 & 0 & 1 \\
+ & & & 1 & 1 & 1 & 1 & 1 \\
\hline
 & 1 & 1 & 1 & 1 & 0 & 0 & 0
\end{array}
$$

hvilket svarer til det resultat, vi fik ovenfor. Selv om det ikke er vist eksplicit her, fungerer subtraktion på tilsvarende måde.

Med en af disse metoder til at håndtere flere menter kan vi også gange to binære tal med hinanden.

Multiplikation i totalssystemet

Multiplikation i totalssystemet er teknisk set lettere end multiplikation i titalssystemet. I binære operationer arbejder vi udelukkende med to cifre: 0 og 1. Det betyder, at både multiplikatoren[1] og multiplikanden kun består af 0'er og 1'er. Fremgangsmåden til at finde det binære produkt svarer til almindelig multiplikation i titalssystemet. De fire mulige produkter af enkelte binære cifre er:

$0 \times 0 = 0$

$0 \times 1 = 0$

$1 \times 0 = 0$

$1 \times 1 = 1$

$1 \times 10_2 = 10_2$ (multiplikation med grundtallet 10_2 tilføjer et 0 til sidst)

Det sidste trin betyder, at $101_2 \times 10_2 = 1010_2$, hvilket svarer til tilfældet i titalssystemet: $143_{10} \times 10_{10} = 1430_{10}$.

Vi illustrerer fremgangsmåden med et par eksempler.

Eksempel 2.1

$$
\begin{array}{cccccc}
 & (1) & & & 1 & 0 & 0 \\
\times & (2) & & & & 1 & 1 \\
\hline
 & (3) & & & 1 & 0 & 0 \\
+ & (4) & & 1 & 0 & 0 & 0 \\
\hline
 & (5) & & 1 & 1 & 0 & 0
\end{array}
$$

Trinene er:

- Gang multiplikanden (linje 1) med multiplikatorens LSB (linje 2), som her er 1.
- Skriv resultatet i linje 3.
- Tilføj et 0 i linje 4 for at tage højde for forskydningen til den næste potens af 2 i multiplikatoren.

[1] »Multiplikanden« er det tal, der skal ganges, og »multiplikatoren« er det tal, der ganges med.

- Gang multiplikanden (linje 1) med multiplikatorens næste bit (linje 2), som her også er 1.
- Skriv dette resultat i linje 4 foran det 0, du tilføjede før.
- Læg til sidst værdierne i linje 3 og 4 sammen som beskrevet i det foregående afsnit for at få det endelige resultat i linje 5.

Vi giver yderligere to eksempler:

Eksempel 2.2

$$
\begin{array}{rcccccc}
 & & & & 1 & 0 & 1 \\
\times & & & 1 & 0 & 1 & 1 \\
\hline
 & & & & 1 & 0 & 1 \\
+ & & & 1 & 0 & 1 & 0 \\
+ & & 0 & 0 & 0 & 0 & 0 \\
+ & 1 & 0 & 1 & 0 & 0 & 0 \\
\hline
 & 1 & 1 & 0 & 1 & 1 & 1
\end{array}
$$

Eksempel 2.3

$$
\begin{array}{rcccccccc}
 & & & 1 & 0 & 0 & 1 & 1 & 1 & 0 \\
\times & & & & & & & 1 & 0 & 1 \\
\hline
 & & & 1 & 0 & 0 & 1 & 1 & 1 & 0 \\
+ & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\
\hline
 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0
\end{array}
$$

Bemærk i Eksempel 2.3, at vi har udeladt den række af nuller, som multiplikatorens anden værdi ville have givet, og bemærk desuden, at vi har tilføjet to 0'er, før vi gentog multiplikanden i summen.

Binær multiplikation er ligesom binær addition en central operation i computeraritmetik. Når processen deles op i overskuelige trin – først gange de enkelte bit og derefter lægge resultaterne sammen – bliver det tydeligt, hvor meget den ligner de multiplikationsmetoder, vi bruger i titalssystemet. Den største forskel er, at det er enkelt og effektivt at arbejde i totalssystemet, hvor kun cifrene 0 og 1 indgår.

Vi bemærker, hvordan binær multiplikation bygger på binær addition. Hvert trin, som består af forskydninger og summer, er i bund og grund gentagne additioner, justeret med potenser af to. En solid forståelse af binær addition fører derfor naturligt til en bedre forståelse af binær multiplikation og dens anvendelser.

2.2 Oktal og hexadecimal

Ottetalssystemet (oktal) er et talsystem med grundtal 8, der bruger cifrene 0 til og med 7. Det er nært beslægtet med totalssystemet, som har grundtal 2. Forbindelsen mellem de to består i, hvor let binære tal kan omregnes til oktale tal og omvendt. Hvert oktalt ciffer svarer til præcis tre binære cifre (bit), hvilket gør omregninger ligetil. For eksempel kan det binære tal »110« omregnes direkte til det oktale ciffer »6«. På grund af denne

nære sammenhæng bruges oktal ofte som en kortform for binær i datalogi, især i sammenhænge, hvor det bliver lettere at læse og fortolke binære data, når de binære cifre grupperes tre og tre.

$$12_8 = 1 \cdot 8^1 + 2 \cdot 8^0 = 10_{10}$$
$$3021_8 = 3 \cdot 8^3 + 0 \cdot 8^2 + 2 \cdot 8^1 + 1 \cdot 8^0 = 1553_{10}$$

Da 8 er 2^3, kan vi udtrykke det i binær form:

$3 \rightarrow 011$

$0 \rightarrow 000$

$2 \rightarrow 010$

$1 \rightarrow 001$

Altså er $3021_8 = 011000010001_2 = 11000010001_2$

Vi får det endelige resultat ved at fjerne de foranstillede nuller.

Sekstentalssystemet (hexadecimal) er et talsystem med grundtal 16, der bruger seksten forskellige symboler: cifrene 0-9 og bogstaverne A-F, hvor A repræsenterer 10, B repræsenterer 11 og så videre op til F, som repræsenterer 15. Hexadecimal er nært beslægtet med binær, fordi hvert hexadecimalt ciffer svarer præcis til fire binære cifre (bit). Denne direkte sammenhæng gør det let at omregne mellem de to systemer. For eksempel svarer det hexadecimale ciffer »A« til den binære sekvens »1010«. Fordi grupperingen er så effektiv, bruges hexadecimal ofte i datalogi som en mere kompakt og læsevenlig måde at repræsentere binære data på, især til fx hukommelsesadresser og farvekoder i webdesign.

$$123_{16} = 1 \cdot 16^2 + 2 \cdot 16^1 + 3 \cdot 16^0 = 256 + 32 + 3 = 291_{10}$$
$$A2E_{16} = 10 \cdot 16^2 + 2 \cdot 16^1 + 14 \cdot 16^0 = 2560 + 32 + 14 = 2606_{10}$$

Da 16 er 2^4, kan vi for eksempel udtrykke $A2E_{16}$ i binær form:

$A \rightarrow 1010$

$2 \rightarrow 0010$

$E \rightarrow 1110$

Altså er $A2E_{16} = 101000101110_2$

Tilsvarende får vi $5EB52_{16}$ som

$5 \rightarrow 0101$

$E \rightarrow 1110$

$B \rightarrow 1011$

$5 \rightarrow 0101$

$2 \rightarrow 0010$

Altså er $5EB52_{16} = 0101\,1110\,1011\,0101\,0010_2 = 10111101011010100 10_2$

Grupperne med fire bit gør omregningen gennemskuelig. I den endelige repræsentation kan det ene foranstillede nul udelades uden at ændre værdien.

2.3 Omregning mellem talsystemer

Det er vigtigt i datalogi og digital elektronik at forstå, hvordan man omregner tal mellem totalssystemet, titalssystemet, ottetalssystemet og sekstentalssystemet. Hvert system har sit eget grundtal og sine egne anvendelser. Her er en trinvis vejledning, der hjælper dig med at omregne tal fra ét system til et andet.

Fra decimal til binær

Sådan omregner du et decimaltal til binær form:

1. Dividér decimaltallet med 2.
2. Notér resten (den er 0 eller 1).
3. Dividér kvotienten med 2, og notér resten.
4. Gentag, indtil kvotienten er 0.
5. Det binære tal er rækken af rester læst nedefra og op.

Eksempel 2.4 Omregn 23_{10} til binær form.

Løsning:

$$\begin{aligned} 23 \div 2 &= 11 \quad \text{rest } 1 \\ 11 \div 2 &= 5 \quad \text{rest } 1 \\ 5 \div 2 &= 2 \quad \text{rest } 1 \\ 2 \div 2 &= 1 \quad \text{rest } 0 \\ 1 \div 2 &= 0 \quad \text{rest } 1 \end{aligned}$$

Altså er $23_{10} = 10111_2$. ◀

Fra decimal til oktal

Sådan omregner du et decimaltal til oktal form:

1. Dividér decimaltallet med 8.
2. Notér resten.
3. Dividér kvotienten med 8, og notér resten.
4. Gentag, indtil kvotienten er 0.
5. Det oktale tal er rækken af rester læst nedefra og op.

Eksempel 2.5 Omregn 78_{10} til oktal form.

Løsning:

$$78 \div 8 = 9 \quad \text{rest } 6$$
$$9 \div 8 = 1 \quad \text{rest } 1$$
$$1 \div 8 = 0 \quad \text{rest } 1$$

Altså er $78_{10} = 116_8$. ◀

Fra decimal til hexadecimal

Sådan omregner du et decimaltal til hexadecimal form:

1. Dividér decimaltallet med 16.
2. Notér resten (brug A, B, C, D, E, F for henholdsvis resterne 10, 11, 12, 13, 14, 15).
3. Dividér kvotienten med 16, og notér resten.
4. Gentag, indtil kvotienten er 0.
5. Det hexadecimale tal er rækken af rester læst nedefra og op.

Eksempel 2.6 Omregn 255_{10} til hexadecimal form.

Løsning:

$$255 \div 16 = 15 \quad \text{rest } 15 \quad (\text{F})$$
$$15 \div 16 = 0 \quad \text{rest } 15 \quad (\text{F})$$

Altså er $255_{10} = FF_{16}$. ◀

Fra binær til decimal

Sådan omregner du et binært tal til decimal form:

1. Gang hver bit med 2 opløftet til bittens position, idet du starter med 0 længst til højre.
2. Læg alle produkterne sammen.

Bemærk, at dette svarer til metoden med binær opløsning, som er beskrevet ovenfor.

Eksempel 2.7 Omregn 1101_2 til decimal form.

Løsning:

$$1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 8 + 4 + 0 + 1 = 13_{10}$$

◀

Fra binær til oktal

Sådan omregner du et binært tal til oktal form:

1. Gruppér de binære cifre tre og tre, idet du starter fra højre. Tilføj foranstillede nuller, hvis det er nødvendigt.

2. Omregn hver gruppe med tre binære cifre til det tilsvarende oktale ciffer.

Eksempel 2.8 Omregn 110110_2 til oktal form.

Løsning:

$110 \rightarrow 6$

$110 \rightarrow 6$

Altså er $110110_2 = 66_8$. ◀

Fra binær til hexadecimal

Sådan omregner du et binært tal til hexadecimal form:

1. Gruppér de binære cifre fire og fire, idet du starter fra højre. Tilføj foranstillede nuller, hvis det er nødvendigt.
2. Omregn hver gruppe med fire binære cifre til det tilsvarende hexadecimale ciffer.

Eksempel 2.9 Omregn 10110101_2 til hexadecimal form.

Løsning:

$1011 \rightarrow B$

$0101 \rightarrow 5$

Altså er $10110101_2 = B5_{16}$. ◀

Fra oktal til binær

Sådan omregner du et oktalt tal til binær form: Omregn hvert oktalt ciffer til det tilsvarende binære tal med 3 bit!

Eksempel 2.10 Omregn 57_8 til binær form.

Løsning:

$5 \rightarrow 101$

$7 \rightarrow 111$

Altså er $57_8 = 101111_2$. ◀

Fra oktal til decimal

Sådan omregner du et oktalt tal til decimal form:

1. Gang hvert ciffer med 8 opløftet til cifrets position, idet du starter med 0 længst til højre.
2. Læg alle produkterne sammen.

Eksempel 2.11 Omregn 157_8 til decimal form.

Løsning:

$$1 \cdot 8^2 + 5 \cdot 8^1 + 7 \cdot 8^0 = 64 + 40 + 7 = 111_{10}$$

◀

Fra oktal til hexadecimal

Sådan omregner du et oktalt tal til hexadecimal form:

1. Omregn først det oktale tal til binær form.
2. Omregn derefter det binære tal til hexadecimal form ved at gruppere de binære cifre fire og fire.

Eksempel 2.12 Omregn 157_8 til hexadecimal form.

Løsning:

$1 \rightarrow 001$

$5 \rightarrow 101$

$7 \rightarrow 111$

Altså er $157_8 = 001101111_2 = 6F_{16}$. ◀

Fra hexadecimal til binær

Sådan omregner du et hexadecimalt tal til binær form: Omregn hvert hexadecimalt ciffer til det tilsvarende binære tal med 4 bit.

Eksempel 2.13 Omregn $2B_{16}$ til binær form.

Løsning:

$2 \rightarrow 0010$

$B \rightarrow 1011$

Altså er $2B_{16} = 00101011_2$. ◀

Fra hexadecimal til decimal

Sådan omregner du et hexadecimalt tal til decimal form:

1. Gang hvert ciffer med 16 opløftet til cifrets position, idet du starter med 0 længst til højre.
2. Læg alle produkterne sammen.

Eksempel 2.14 Omregn $2B_{16}$ til decimal form.

Løsning:

$$2 \cdot 16^1 + 11 \cdot 16^0 = 32 + 11 = 43_{10}$$

◀

Fra hexadecimal til oktal

Sådan omregner du et hexadecimalt tal til oktal form:

1. Omregn først det hexadecimale tal til binær form.
2. Omregn derefter det binære tal til oktal form ved at gruppere de binære cifre tre og tre.

Eksempel 2.15 Omregn $2B_{16}$ til oktal form.

Løsning:

$$2 \rightarrow 0010$$
$$B \rightarrow 1011$$

Altså er $2B_{16} = 00101011_2 = 53_8$. ◀

Afsluttende bemærkninger om omregning

Begrebet opløsning spiller en central rolle i disse omregninger. Uanset om du opløser et decimaltal i dets binære, oktale eller hexadecimale form eller omregner et binært tal til dets oktale eller hexadecimale modstykke, udtrykker du mere eller mindre tallet ved potenser af grundtallet. Metoden med opløsning er i bund og grund den samme for alle systemer, da den går ud på rekursivt at dividere med den højeste potens af grundtallet:

$$\begin{aligned} 7562_{10} &= 1 \cdot 16^3 + 3466 = 1 \cdot 16^3 + 13 \cdot 16^2 + 138 \\ &= 1 \cdot 16^3 + 13 \cdot 16^2 + 8 \cdot 16^1 + 10 \cdot 16^0 = 1\,D\,8\,A \end{aligned}$$

Når du forstår disse opløsninger og sammenhængene mellem talsystemerne, kan du effektivt skifte mellem dem, så du kan repræsentere og behandle data i det format, der passer bedst til den givne situation.

Kapitel 3 Mængdelære

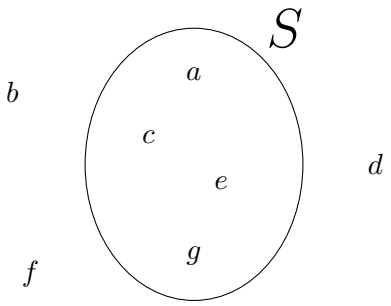
I softwareudvikling arbejder vi hele tiden med samlinger af ting: brugere i et system, poster i en database eller knuder i et netværk. Mængdelæren giver os det formelle matematiske sprog til at beskrive og håndtere disse samlinger præcist og klart. Den er fundamentet for mange centrale begreber i datalogi – fra databaseforespørgselssprog som SQL til datastrukturer og algoritmisk logik.

Dette kapitel introducerer mængdelærens grundlæggende principper. Vi begynder med den enkle, intuitive idé om en mængde og ser på den formelle notation, der bruges til at definere mængder. Derefter gennemgår vi de væsentligste relationer mellem mængder, fx delmængder, og de centrale operationer, der bruges til at kombinere dem, herunder foreningsmængder, fællesmængder og komplementer. Disse operationer svarer direkte til de logiske operatorer (`OR`, `AND`, `NOT`), der styrer forløbet i din kode. Til sidst ser vi på ordnede samlinger, der kaldes tupler, og introducerer en enkel metode til at bevise, at to mængder er lig med hinanden.

3.1 Hvad er en mængde?

En mængde er en uordnet samling af forskellige objekter. Objekterne i en mængde kaldes dens **elementer** eller **medlemmer**. Vi kan tænke på en mængde som en simpel beholder, hvor ting er samlet, og hvor rækkefølgen, vi skriver dem i, ikke har betydning. For eksempel kan mængden af primærfarver skrives som $\{\text{rød}, \text{gul}, \text{blå}\}$ eller lige så godt som $\{\text{blå}, \text{rød}, \text{gul}\}$.

Mængder er en hjørnesten i moderne matematik og et grundlæggende begreb i datalogi. De udgør det logiske grundlag for alt fra databaseforespørgselssprog og datastrukturer til specifikationen af typer i programmeringssprog.



Elementerne i mængden S er a, c, e og g.

Figur 3.1: En mængde S med fire elementer. Objekterne b, d og f er ikke elementer i S.

Angivelse af en mængde

Der er to primære måder at beskrive en mængde på: ved eksplicit at opremse dens elementer eller ved at angive en egenskab, som elementerne skal opfylde.

Listenotation

Den mest direkte måde at definere en mængde på er at opremse alle dens elementer mellem krøllede parenteser, $\{\}$. Det kaldes **listenotation**.

For eksempel:

- Mængden af alfabetets første fem bogstaver er $A = \{a, b, c, d, e\}$.
- Mængden af de første tre positive hele tal er $C = \{1, 2, 3\}$.
- En mængde kan indeholde forskellige typer af elementer: $D = \{\text{Alice}, 42, \pi\}$.

Når vi bruger listenotation, gælder der to grundlæggende regler:

1. **Rækkefølgen har ingen betydning.** En mængde er alene bestemt af de elementer, den indeholder, ikke af den rækkefølge, de er opremset i. For eksempel er $\{1, 2, 3\}$ præcis den samme mængde som $\{3, 1, 2\}$.
2. **Hvert element skal være entydigt.** Et element er enten med i en mængde, eller også er det ikke. At opremse et element mere end én gang er overflødigt og ændrer ikke mængden. For eksempel er mængden $\{a, a, b, c, c\}$ blot $\{a, b, c\}$.

Mængdebyggernotation

Når det er upraktisk eller umuligt at opremse alle elementer (fx for uendelige mængder), bruger vi **mængdebyggernotation**. Denne metode definerer en mængde ved at angive en egenskab eller regel, som elementerne skal opfylde. Notationen bruger en lodret streg '|' eller et kolon ':', som læses »sådan at«.

Den generelle struktur er:

$$\{\text{variabel} \mid \text{en egenskab, som variablen skal opfylde}\}$$

For eksempel:

- $A = \{l \mid l \text{ er en vokal i det engelske alfabet}\}$
  Det læses: »A er mængden af alle elementer l, sådan at l er en vokal i det engelske alfabet.« Det er en anden måde at skrive $A = \{a, e, i, o, u\}$ på.
- $C = \{n \mid n \in \mathbb{Z} \text{ og } 0 < n < 4\}$
  Det læses: »C er mængden af alle tal n, sådan at n er et helt tal, og n er større end 0 og mindre end 4.« Det definerer mængden $\{1, 2, 3\}$.
- $E = \{x \mid x \text{ er et lige helt tal}\}$
  Det definerer den uendelige mængde af alle lige hele tal: $\{\ldots, -4, -2, 0, 2, 4, \ldots\}$.

Mængdebyggernotation er meget effektiv, fordi den gør det muligt at definere store, komplekse eller endda uendelige mængder med en kort og præcis beskrivelse.

3.2 Vigtige mængder: talsystemerne

Tal er fundamentet for matematik og beregning. De forskellige kategorier af tal, vi bruger i hverdagen – fra at tælle ting til at måle kontinuerte størrelser – kan defineres formelt som mængder. Det er afgørende for enhver softwareingeniør at forstå disse grundlæggende mængder, fordi de ligger til grund for datatyper, aritmetisk logik og numeriske algoritmer.

De vigtigste talmængder

Følgende mængder er blandt de vigtigste i matematikken og bruges overalt i naturvidenskab og ingeniørarbejde.

Naturlige tal ($\mathbb{N}$) Mængden af positive hele tal, som bruges til at tælle: $\{1, 2, 3, \dots\}$. Nogle gange defineres denne mængde, så den også indeholder 0. På grund af denne tvetydighed er det ofte tydeligere at angive *positive hele tal* eller *ikke-negative hele tal*.

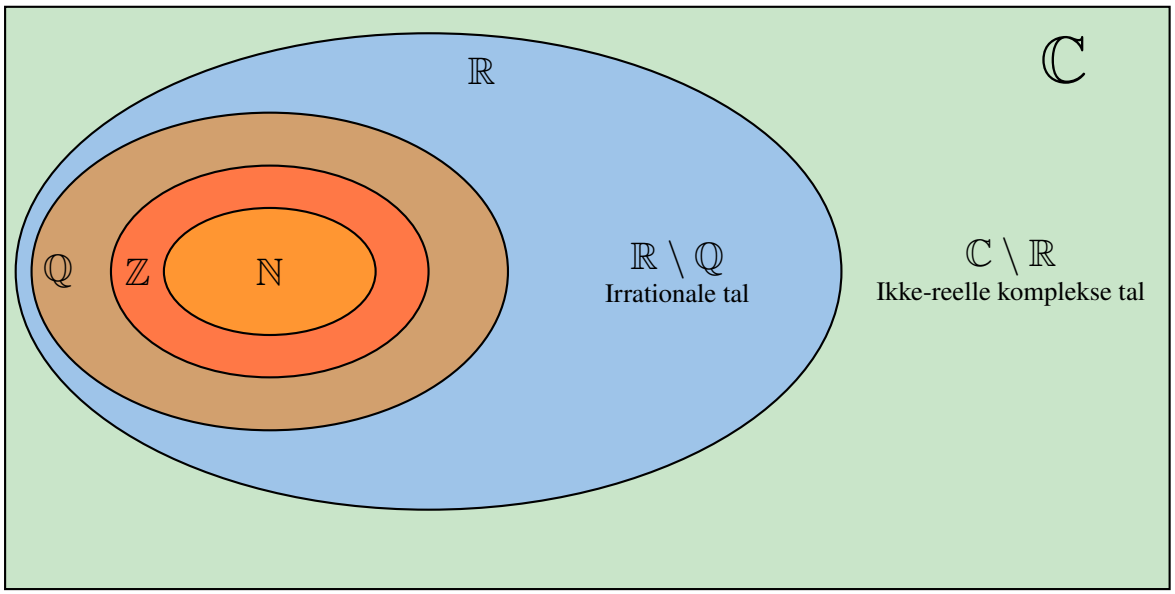
Hele tal ($\mathbb{Z}$) Mængden af alle positive og negative hele tal, inklusive nul: $\{\dots, -2, -1, 0, 1, 2, \dots\}$.

Rationale tal ($\mathbb{Q}$) Mængden af alle tal, der kan skrives som en brøk $\frac{p}{q}$, hvor p og q er hele tal og $q \neq 0$. Den omfatter alle hele tal samt alle endelige og periodiske decimalbrøker. Eksempler: $\frac{1}{2}$, -5, 0.25.

Reelle tal ($\mathbb{R}$) Mængden af alle tal på tallinjen. Den omfatter både rationale tal og irrationale tal (som π eller $\sqrt{2}$), der ikke kan skrives som simple brøker.

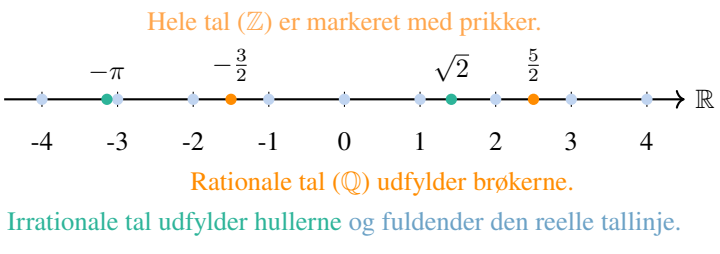
Komplekse tal ($\mathbb{C}$) Mængden af alle tal, der kan skrives på formen $a + bi$, hvor a og b er reelle tal, og i er den imaginære enhed, som opfylder $i^2 = -1$.

Det er vigtigt at kunne visualisere relationerne mellem disse mængder for at forstå dem. Et Venn-diagram viser hierarkiet i, hvordan mængderne ligger inden i hinanden, mens en tallinje illustrerer, hvordan de dækker eller udfylder kontinuummet af værdier.



Figur 3.2: Et Venn-diagram, der illustrerer den hierarkiske relation mellem de vigtigste talmængder.

Bemærkning: Venn-diagrammet i Figur 3.2 viser, at mængden af reelle tal ($\mathbb{R}$) udelukkende består af rationale ($\mathbb{Q}$) og irrationale tal. Der findes intet reelt tal, som hverken er det ene eller det andet. Derfor betegnes mængden af irrationale tal formelt $\mathbb{R} \setminus \mathbb{Q}$, hvilket betyder »mængden af alle reelle tal bortset fra de rationale tal«.



Figur 3.3: Den reelle tallinje, udfyldt med hele tal, rationale tal og irrationale tal.

Intervalnotation

I mange anvendelser har vi brug for at henvise til et sammenhængende udsnit af de reelle tal. **Intervalnotation** er en praktisk kortform til at beskrive sådanne delmængder af $\mathbb{R}$.

Et interval er bestemt ved sine to endepunkter. Vi bruger firkantede parenteser '[' ']' til at angive, at et endepunkt er med i mængden, og runde parenteser '(' ')' til at angive, at det ikke er med.

Lukket interval: Indeholder begge endepunkter.

$$[a, b] = \{x \in \mathbb{R} \mid a \leq x \leq b\}$$

Åbent interval: Indeholder ingen af endepunkterne.

$$(a, b) = \{x \in \mathbb{R} \mid a < x < b\}$$

Halvåbne intervaller: Indeholder det ene endepunkt, men ikke det andet.

$$[a, b) = \{x \in \mathbb{R} \mid a \leq x < b\} \quad \text{og} \quad (a, b] = \{x \in \mathbb{R} \mid a < x \leq b\}$$

Eksempel 3.1 Intervalnotation

- $[-2, 3]$ er mængden af alle reelle tal fra -2 til 3, inklusive -2 og 3.
- $(-2, 3)$ er mængden af alle reelle tal mellem -2 og 3.
- $[0, 100)$ betegner alle tal fra 0 op til (men ikke med) 100.

Intervaller kan også være ubegrænsede, så de strækker sig mod plus eller minus uendelig (∞). Da uendelig ikke er et tal, udelukkes det altid med en rund parentes.

$$(a, \infty) = \{x \in \mathbb{R} \mid x > a\} \quad \text{og} \quad (-\infty, b] = \{x \in \mathbb{R} \mid x \leq b\}$$

Eksempel 3.2 Ubegrænset interval

- $(3, \infty)$ betegner alle reelle tal større end 3.
- $(-\infty, 5)$ betegner alle reelle tal mindre end 5.
- $(-\infty, 0]$ betegner mængden af alle ikke-positive reelle tal.

3.3 Relationer mellem mængder

At forstå en mængde handler ikke kun om dens elementer, men også om, hvordan den forholder sig til andre mængder. Dette afsnit definerer de grundlæggende relationer, som gør det muligt at sammenligne og klassificere mængder.

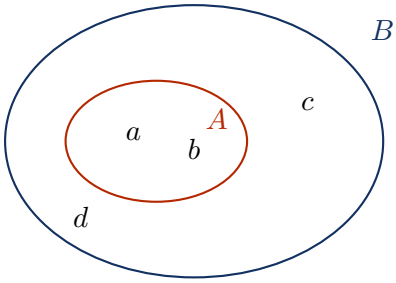
Delmængder og ægte delmængder

En af de mest grundlæggende relationer er inklusion, hvor én mængde er indeholdt i en anden.

Definition 3.1 (Delmængde)

En mængde A er en **delmængde** af en mængde B, hvis ethvert element i A også er et element i B. Vi skriver det som $A \subseteq B$.

Hvis fx $A = \{1, 2\}$ og $B = \{1, 2, 3\}$, så gælder $A \subseteq B$, fordi ethvert element i A også er med i B. Ifølge denne definition er enhver mængde en delmængde af sig selv (dvs. $A \subseteq A$).



Figur 3.4: Mængden $A = \{a, b\}$ er en delmængde af mængden $B = \{a, b, c, d\}$, hvilket skrives $A \subseteq B$.

Nogle gange vil vi angive, at en mængde er en delmængde af en anden, men ikke er lig med den.

Definition 3.2 (Ægte delmængde)

En mængde A er en **ægte delmængde** af en mængde B, hvis $A \subseteq B$ og $A \neq B$. Det betyder, at B skal indeholde mindst ét element, som ikke er med i A. Vi skriver det som $A \subset B$.

I det foregående eksempel indeholder B elementet 3, som ikke er med i A. Derfor kan vi sige, at A er en ægte delmængde af B, dvs. $A \subset B$.

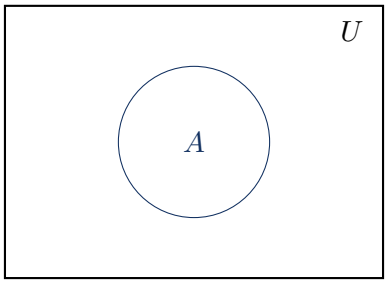
Universalmængden og den tomme mængde

To særlige mængder udgør mængdelærens yderpunkter: mængden, der indeholder alt, og mængden, der ikke indeholder noget.

Definition 3.3 (Universalmængde)

Universalmængden, som betegnes U, er mængden af alle mulige elementer, der er i betragtning i en given sammenhæng. Alle andre mængder i den sammenhæng betragtes som delmængder af universalmængden.

I Venn-diagrammer tegnes universalmængden som et rektangel, der omslutter alle andre mængder. Hvis vi fx taler om hele tal, er universalmængden $U = \mathbb{Z}$. Hvis vi taler om studerende på et universitet, er U mængden af alle indskrevne studerende.



Figur 3.5: Universalmængden U indeholder alle de elementer og mængder, der er i betragtning.

Definition 3.4 (Den tomme mængde)

Den tomme mængde (eller **nulmængden**) er den entydige mængde, der ikke indeholder nogen elementer. Den betegnes $\emptyset$ eller $\{\}$.

Den tomme mængde har en afgørende egenskab:

Den tomme mængde er en delmængde af enhver mængde.

Det skyldes, at der ikke findes noget element i $\emptyset$, som ikke er med i en vilkårlig anden mængde A. For enhver mængde A gælder derfor altid $\emptyset \subseteq A$.

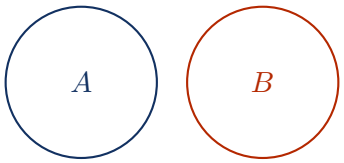
Disjunkte mængder

Nogle gange har mængder slet ingen sammenhæng, fordi de er helt adskilte.

Definition 3.5 (Disjunkte mængder)

To mængder A og B er **disjunkte**, hvis de ikke har nogen elementer til fælles. Deres fællesmængde er med andre ord den tomme mængde: $A \cap B = \emptyset$.

De lige heltal $E = \{\dots, -2, 0, 2, \dots\}$ og de ulige heltal $O = \{\dots, -3, -1, 1, 3, \dots\}$ er disjunkte.



Disjunkte mængder

Figur 3.6: Mængderne A og B er disjunkte, fordi de ikke overlapper.

3.4 Egenskaber ved mængder

Ud over relationerne mellem mængder kan vi også beskrive deres indre egenskaber. De to mest grundlæggende egenskaber er en mængdes størrelse (dens kardinalitet) og samlingen af alle dens mulige delmængder (dens potensmængde).

Kardinalitet

Den mest grundlæggende egenskab ved en endelig mængde er dens størrelse.

Definition 3.6 (Kardinalitet)

Kardinaliteten af en endelig mængde A, som betegnes $|A|$, er antallet af forskellige elementer i mængden.

For eksempel:

- Hvis $A = \{a, b, c, d\}$, så er $|A| = 4$.
- Hvis $B = \{n \in \mathbb{Z} \mid 0 < n < 5\}$, så er $B = \{1, 2, 3, 4\}$ og $|B| = 4$.
- For den tomme mængde gælder $|\emptyset| = 0$.

Begrebet kardinalitet kan udvides til uendelige mængder, men det er et mere avanceret emne, der ligger uden for dette kapitels rammer. Til vores formål er kardinaliteten blot antallet af elementer.

Potensmængder

En af de mest effektive konstruktioner i mængdelæren er idéen om at danne en mængde, der indeholder alle mulige delmængder af en anden mængde.

Definition 3.7 (Potensmængde)

Potensmængden af en mængde A, som betegnes $\mathcal{P}(A)$, er mængden af alle delmængder af A. Elementerne i potensmængden er altså selv mængder. ♣

Potensmængden indeholder altid den tomme mængde ($\emptyset$) og mængden A selv.

Eksempel 3.3 Bestemmelse af potensmængden af $A = \{1, 2, 3\}$
For at bestemme $\mathcal{P}(A)$ opremser vi alle mulige delmængder af A, grupperet efter deres kardinalitet:

- Delmængder af størrelse 0: $\{\emptyset\}$
- Delmængder af størrelse 1: $\{\{1\}, \{2\}, \{3\}\}$
- Delmængder af størrelse 2: $\{\{1, 2\}, \{1, 3\}, \{2, 3\}\}$
- Delmængder af størrelse 3: $\{\{1, 2, 3\}\}$

Når vi samler alle disse delmængder i én mængde, får vi potensmængden:

$$\mathcal{P}(A) = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}\}$$

Bemærkning: For en endelig mængde A med kardinalitet $|A| = n$ er kardinaliteten af dens potensmængde $|\mathcal{P}(A)| = 2^n$.

Denne formel er afgørende i datalogi. Grunden er, at vi for hvert af de n elementer i mængden A kan træffe et binært valg: Enten **medtager** vi elementet i en delmængde, eller også **udelader** vi det. Med to valg for hvert af de n elementer er der i alt $2 \times 2 \times \cdots \times 2$ (n gange), dvs. 2^n, mulige kombinationer, hvilket svarer til det samlede antal mulige delmængder.

3.5 Mængdeoperationer

Ligesom vi kan udføre aritmetiske operationer på tal, kan vi udføre operationer på mængder og derved danne nye mængder. Disse operationer udgør grundlaget for mængdealgebraen. For en softwareingeniør er den vigtigste indsigt, at mængdeoperationer er en direkte parallel til de logiske operationer i **boolesk algebra**. Hver regel, du lærer for mængder, har en tilsvarende regel i logik og i design af digitale kredsløb.

Dualiteten mellem mængder og boolesk algebra

For en fast universalmængde er algebraen af dens delmængder med foreningsmængde, fællesmængde og komplement en boolesk algebra. Operationerne svarer til logisk OR, AND og NOT som følger:

Mængdelære		**Boolesk algebra/logik**
Foreningsmængde ($\cup$)	$\Longleftrightarrow$	Boolesk sum ($+$) eller `OR`
Fællesmængde ($\cap$)	$\Longleftrightarrow$	Boolesk produkt ($\cdot$) eller `AND`
Komplement (A^c)	$\Longleftrightarrow$	Komplementering ($\overline{x}$) eller `NOT`
Universalmængden (U)	$\Longleftrightarrow$	Sand (1)
Den tomme mængde ($\emptyset$)	$\Longleftrightarrow$	Falsk (0)

Vi gennemgår nu de primære mængdeoperationer og holder denne dualitet for øje.

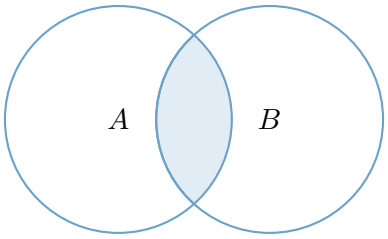
Fællesmængde

Fællesmængden af to mængder indeholder kun de elementer, der er fælles for begge mængder. Den svarer til den logiske operator `AND`.

Definition 3.8 (Fællesmængde)

Fællesmængden af mængderne A og B, som betegnes $A \cap B$, er mængden af alle elementer, der både er med i A og i B.

$$A \cap B = \{x \mid x \in A \text{ og } x \in B\}$$



Figur 3.7: Det skraverede område angiver fællesmængden $A \cap B$.

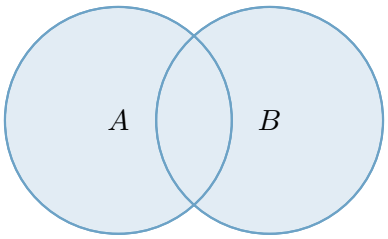
Foreningsmængde

Foreningsmængden af to mængder indeholder alle de elementer, der forekommer i mindst én af mængderne (eller i begge). Den svarer til den logiske operator `OR`.

Definition 3.9 (Foreningsmængde)

Foreningsmængden af mængderne A og B, som betegnes $A \cup B$, er mængden af alle elementer, der er med i A, i B eller i begge.

$$A \cup B = \{x \mid x \in A \text{ eller } x \in B\}$$



Figur 3.8: Det skraverede område angiver foreningsmængden $A \cup B$.

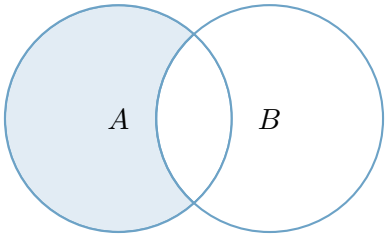
Mængdedifferens

Differensen mellem to mængder indeholder de elementer, der er med i den første mængde, men *ikke* i den anden.

Definition 3.10 (Mængdedifferens)

Differensen mellem mængden A og mængden B, som betegnes $A \setminus B$, er mængden af alle elementer, der er med i A, men ikke i B.

$$A \setminus B = \{x \mid x \in A \text{ og } x \notin B\}$$



Figur 3.9: Det skraverede område angiver differensen $A \setminus B$.

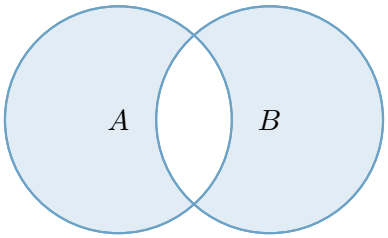
Symmetrisk differens

Den symmetriske differens indeholder alle elementer, der er med i den ene eller den anden mængde, men ikke i begge. Den svarer til den logiske operator `XOR`.

Definition 3.11 (Symmetrisk differens)

Den **symmetriske differens** af mængderne A og B, som betegnes $A \oplus B$, er mængden af de elementer, der er med i en af mængderne, men ikke i deres fællesmængde.

$$A \oplus B = (A \cup B) \setminus (A \cap B)$$



Figur 3.10: Det skraverede område angiver den symmetriske differens $A \oplus B$.

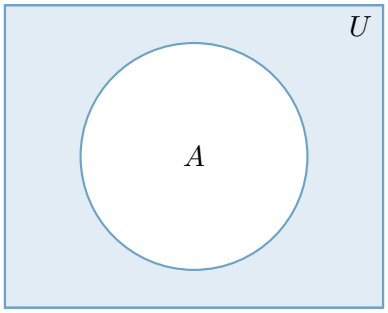
Komplement

Komplementet til en mængde indeholder alle de elementer i universalmængden, som *ikke* er med i mængden selv. Det svarer til den logiske operator `NOT`.

Definition 3.12 (Komplement)

Komplementet til en mængde A, som betegnes A^c, er mængden af alle elementer i universalmængden U, der ikke er med i A.

$$A^c = U \setminus A$$



Figur 3.11: Det skraverede område angiver komplementet A^c.

3.6 Kartesiske produkter og tupler

Mængder er uordnede samlinger, men i datalogi har vi ofte brug for at arbejde med ordnede samlinger, fx koordinater, databaseposter eller strukturerede data. Det kartesiske produkt er den mængdeoperation, der gør det muligt at danne sådanne ordnede strukturer.

Kartesisk produkt

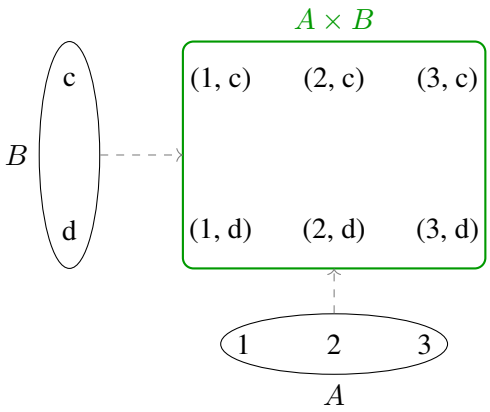
Det kartesiske produkt danner en ny mængde ud fra to eller flere eksisterende mængder. Den nye mængde består af alle mulige ordnede kombinationer af deres elementer.

Definition 3.13 (Kartesisk produkt)

Det **kartesiske produkt** af mængderne A og B, som betegnes $A \times B$, er mængden af alle mulige ordnede par (a, b), hvor det første element a kommer fra A, og det andet element b kommer fra B.

$$A \times B = \{(a, b) \mid a \in A \text{ og } b \in B\}$$

Navnet kommer fra det kartesiske koordinatsystem, hvor ethvert punkt i en 2D-plan kan repræsenteres ved et ordnet par (x, y) fra det kartesiske produkt af de reelle tal, $\mathbb{R} \times \mathbb{R}$.



Figur 3.12: Det kartesiske produkt af $A = \{1, 2, 3\}$ og $B = \{c, d\}$ er en mængde med 6 ordnede par.

Bemærkning: Rækkefølgen af mængderne i et kartesisk produkt har betydning. Mængden $A \times B$ er generelt ikke lig med mængden $B \times A$. For eksempel er parret $(1, c)$ med i $A \times B$, mens parret $(c, 1)$ ville være med i $B \times A$.

Tupler

Elementerne i et kartesisk produkt kaldes **tupler**. Hvis produktet involverer n mængder, kaldes elementerne **n-tupler**.

- En **2-tupel**, fx (a, b), kaldes oftere et **ordnet par**.
- En **3-tupel** har formen (a, b, c).
- En **n-tupel** har formen $(a_1, a_2, \ldots, a_n)$.

Det afgørende kendetegn ved en tupel er, at **rækkefølgen har betydning**. Det gør tupler grundlæggende forskellige fra mængder.

- For mængder: $\{1, 2, 3\} = \{3, 2, 1\}$

- For tupler: $(1, 2, 3) \neq (3, 2, 1)$

Denne egenskab gør tupler velegnede til at repræsentere data, hvor et elements position har betydning, fx en databasepost '(UserID, Name, Email)'.

3.7 Bevis for mængdeligheder

I softwareudvikling er det afgørende at kunne forenkle kompleks betinget logik for at skrive effektiv og læsbar kode. På samme måde har vi i mængdelæren ofte brug for at bevise, at to forskellige udtryk beskriver præcis den samme mængde. Der er to primære metoder til dette: medlemskabstabeller og mængdeidentiteter.

Metode 1: Medlemskabstabeller

En medlemskabstabel er et værktøj til at bevise, at to mængdeudtryk er lig med hinanden, ved at kontrollere alle mulige kombinationer af et elements medlemskab af de indgående mængder.

Metoden er mængdelærens pendant til at bruge en **sandhedstabel** i boolesk algebra. I stedet for at kontrollere `TRUE` eller `FALSE` kontrollerer vi, om et element er med i en mængde (angivet med 1) eller ikke er med (angivet med 0). Hvis søjlerne for to forskellige mængdeudtryk er ens i alle rækker, er det bevist, at udtrykkene er lig med hinanden.

Eksempel 3.4 Vis, at $A \cap B = B \setminus (B \setminus A)$
For at bevise denne lighed opstiller vi en medlemskabstabel for alle kombinationer af medlemskab af mængderne A og B.

A	B	$A \cap B$	$B \setminus A$	$B \setminus (B \setminus A)$
1	1	**1**	0	**1**
1	0	**0**	0	**0**
0	1	**0**	1	**0**
0	0	**0**	0	**0**

Løsning: (Eksempel 3.4) Da søjlen for $A \cap B$ er identisk med søjlen for $B \setminus (B \setminus A)$ for alle mulige kombinationer af medlemskab, er de to udtryk lig med hinanden. ◀

Metode 2: Mængdeidentiteter

Medlemskabstabeller er effektive, men de kan blive meget store, når antallet af mængder vokser. En mere algebraisk fremgangsmåde er at bruge **mængdeidentiteter** til at forenkle det ene udtryk, indtil det svarer til det andet. Disse identiteter er grundlæggende love, der bestemmer, hvordan mængdeoperationer opfører sig.

Tabel 3.1 viser de vigtigste mængdeidentiteter. Læg mærke til den slående lighed med lovene i boolesk algebra – de har nøjagtig samme struktur.

Tabel 3.1: Grundlæggende mængdeidentiteter og deres modstykker i boolesk algebra.

Navn	**Mængdeidentitet**	**Boolesk identitet**
Identitetslove	$A \cup \emptyset = A$ $A \cap U = A$	$x + 0 = x$ $x \cdot 1 = x$
Dominanslove	$A \cup U = U$ $A \cap \emptyset = \emptyset$	$x + 1 = 1$ $x \cdot 0 = 0$
Idempotenslove	$A \cup A = A$ $A \cap A = A$	$x + x = x$ $x \cdot x = x$
Komplementlove	$A \cup A^c = U$ $A \cap A^c = \emptyset$	$x + \overline{x} = 1$ $x \cdot \overline{x} = 0$
Kommutative love	$A \cup B = B \cup A$ $A \cap B = B \cap A$	$x + y = y + x$ $x \cdot y = y \cdot x$
Associative love	$(A \cup B) \cup C = A \cup (B \cup C)$ $(A \cap B) \cap C = A \cap (B \cap C)$	$(x + y) + z = x + (y + z)$ $(x \cdot y) \cdot z = x \cdot (y \cdot z)$
Distributive love	$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$	$x \cdot (y + z) = (x \cdot y) + (x \cdot z)$ $x + (y \cdot z) = (x + y) \cdot (x + z)$
De Morgans love	$(A \cap B)^c = A^c \cup B^c$ $(A \cup B)^c = A^c \cap B^c$	$\overline{x \cdot y} = \overline{x} + \overline{y}$ $\overline{x + y} = \overline{x} \cdot \overline{y}$
Absorptionslove	$A \cup (A \cap B) = A$ $A \cap (A \cup B) = A$	$x + (x \cdot y) = x$ $x \cdot (x + y) = x$

Eksempel 3.5 Bevis for $A \cup (B \cap A^c) = A \cup B$ ved hjælp af identiteter

$$\begin{aligned} A \cup (B \cap A^c) &= (A \cup B) \cap (A \cup A^c) && \text{jf. den distributive lov} \\ &= (A \cup B) \cap U && \text{jf. komplementloven} \\ &= A \cup B && \text{jf. identitetsloven} \end{aligned}$$

Eksempel 3.6 Afgør, om $(A \cap B) \cup (A \cap B^c) = A$

Løsning: Vi forenkler venstre side ved hjælp af mængdeidentiteter:

$$\begin{aligned} (A \cap B) \cup (A \cap B^c) &= A \cap (B \cup B^c) && \text{jf. den distributive lov} \\ &= A \cap U && \text{jf. komplementloven} \\ &= A && \text{jf. identitetsloven} \end{aligned}$$

Da venstre side kan forenkles til A, har vi $(A \cap B) \cup (A \cap B^c) = A$. **Udtrykkene er lig med hinanden.** ◀

Eksempel 3.7 Afgør, om $(A \cup B) \cap (A^c \cup B^c) = A \cap B$

Løsning: Vi forenkler venstre side:

$$\begin{aligned} (A \cup B) \cap (A^c \cup B^c) &= (A \cup B) \cap (A \cap B)^c && \text{jf. De Morgans lov} \\ &= (A \cup B) \cap (A \cap B)^c \end{aligned}$$

Det er alle de elementer, der er med i A eller B, men ikke i både A og B på samme tid. Det er den symmetriske differens $A \oplus B$, ikke $A \cap B$.

Som et konkret modeksempel lader vi universalmængden være $U = \{1, 2, 3\}$ med $A = \{1, 2\}$ og $B = \{2, 3\}$:

- $(A \cup B) \cap (A^c \cup B^c) = \{1, 2, 3\} \cap \{3, 1\} = \{1, 3\}$
- $A \cap B = \{2\}$

Da $\{1, 3\} \neq \{2\}$, **er udtrykkene ikke lig med hinanden.** ◀

Eksempel 3.8 Afgør, om $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$

Løsning: Dette er faktisk den distributive lov for foreningsmængde over fællesmængde. Lad os eftervise det:

$$\begin{aligned} (A \cup B) \cap (A \cup C) &= A \cup (B \cap C) && \text{jf. den distributive lov} \end{aligned}$$

Vi har altså $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$. **Udtrykkene er lig med hinanden.** ◀

Eksempel 3.9 Afgør, om $(A \cap B^c) \cup (A^c \cap B) = (A \cup B) \cap (A \cap B)^c$

Løsning: Venstre side er den symmetriske differens $A \oplus B$. Vi undersøger højre side:

$$\begin{aligned} (A \cup B) \cap (A \cap B)^c &= (A \cup B) \cap (A^c \cup B^c) && \text{jf. De Morgans lov} \\ &= (A \cap A^c) \cup (A \cap B^c) \cup (B \cap A^c) \cup (B \cap B^c) && \text{jf. den distributive lov} \\ &= \emptyset \cup (A \cap B^c) \cup (A^c \cap B) \cup \emptyset && \text{jf. komplementloven} \\ &= (A \cap B^c) \cup (A^c \cap B) && \text{jf. identitetsloven} \end{aligned}$$

Da begge sider er lig med $(A \cap B^c) \cup (A^c \cap B)$, **er udtrykkene lig med hinanden.** ◀

3.8 Repræsentation af mængder i computeren

Mængdelæren giver et abstrakt sprog for samlinger, men datalogi kræver konkrete og effektive måder at implementere idéerne på. For endelige universalmængder er en af de mest elegante og effektive metoder at repræsentere mængder med **bitstrenge**. Teknikken kræver to betingelser:

1. Universalmængden U skal være endelig.
2. Elementerne i U skal have en fast, aftalt rækkefølge.

Lad $U = \{a_1, a_2, \ldots, a_n\}$. Enhver delmængde $A \subseteq U$ kan repræsenteres ved en bitstreng af længde n, hvor den i'te bit er 1, hvis $a_i \in A$, og 0, hvis $a_i \notin A$.

Eksempel 3.10 Mængder repræsenteret som bitstrenge
Lad universalmængden være $U = \{1, 2, 3, 4, 5, 6, 7, 8\}$.

- Mængden $A = \{1, 3, 4, 8\}$ repræsenteres ved bitstrengen `10110001`.
- Mængden $B = \{2, 3, 8\}$ repræsenteres ved bitstrengen `01100001`.
- Mængden af alle lige tal, $\{2, 4, 6, 8\}$, er `01010101`.
- Den tomme mængde, $\emptyset$, er `00000000`.

Den virkelige styrke ved denne repræsentation er, at mængdeoperationer svarer direkte til ekstremt hurtige bitvise operationer på lavt niveau, som processorer kan udføre i en enkelt cyklus.

Mængdeoperationer som bitvise operationer

Lad bitstrengene for mængderne A og B være s_A og s_B.

- **Foreningsmængde** ($A \cup B$) svarer til en bitvis `OR`-operation.
- **Fællesmængde** ($A \cap B$) svarer til en bitvis `AND`-operation.
- **Komplement** (A^c) svarer til en bitvis `NOT`-operation (enerkomplement).
- **Symmetrisk differens** ($A \oplus B$) svarer til en bitvis `XOR`-operation.

Eksempel 3.11 Mængdeoperationer udført med bitstrenge
Med $U = \{1, 2, 3, 4, 5, 6, 7, 8\}$ lader vi $A = \{1, 3, 4, 8\}$ og $B = \{2, 3, 8\}$.

	Mængderepræsentation	**Bitstrengsrepræsentation**
Mængden A	$\{1, 3, 4, 8\}$	`10110001`
Mængden B	$\{2, 3, 8\}$	`01100001`
$A \cup B$	$\{1, 2, 3, 4, 8\}$	`10110001 OR 01100001 = 11110001`
$A \cap B$	$\{3, 8\}$	`10110001 AND 01100001 = 00100001`
A^c	$\{2, 5, 6, 7\}$	`NOT 10110001 = 01001110`

Bitstrengsrepræsentationen er grundlæggende inden for mange områder af databehandling, herunder filrettigheder i styresystemer, databaseindeksering, netværksprotokoller og grafikprogrammering, fordi den gør det muligt at håndtere og forespørge samlinger med maksimal effektivitet.

Del II

Sandsynlighedsregning og statistik

Kapitel 4 Kombinatorik og sandsynlighedsteori

Forestil dig, at du skal danne hold på 3 til et semesterprojekt i en klasse med 45 studerende. I første omgang har rækkefølgen, du vælger holdmedlemmerne i, ingen betydning, så du skal kun tænke på kombinationer. Der er hele 14 190 måder at danne et hold på 3 ud af 45 studerende på!

Det følgende semester introduceres de studerende til projektstyring med Scrum, hvor hvert hold skal have tre bestemte roller: Scrum Master, Product Owner og Development Team. Denne lille ændring – at rollerne angives – forvandler pludselig problemet fra en simpel *kombination* til en *permutation*. Nu springer antallet af mulige måder at fordele rollerne på op til 85 140!

De 45 studerende er frustrerede over det enorme antal muligheder og holder en fest for at slappe af. Da de er velopdragne, beslutter de, at alle skal give hånd til alle andre præcis én gang. Efter et par minutter regner de det samlede antal håndtryk ud – 990. De studerende bliver igen overraskede over, hvor hurtigt noget så simpelt som at give hånd kan løbe op.

Som aftenen skrider frem, foreslår en studerende en sjov leg – en lodtrækning om fem dørpræmier, som alle er forskellige. Med 45 fremmødte studerende og kun fem præmier bliver chancen for ikke at vinde noget bekymrende høje 89 procent. De studerende indser hurtigt, at oddsene ikke er på deres side.

En mindre gruppe er ikke klar til at opgive lykken og beslutter at kaste en mønt 10 gange i håb om at få præcis fem gange krone og vinde spillet. Men da de finder ud af, at sandsynligheden for, at det sker, kun er omkring 25 procent, falder humøret yderligere.

De studerende konkluderer, at i stedet for at stole på tilfældet er det på tide at gå i dybden med kombinatorik og sandsynlighedsteori. Med denne viden kan de bedre forudsige udfald og undgå fremtidige skuffelser både til fester og i projektplanlægningen.

4.1 Udfaldsrum og hændelser

Et **stokastisk eksperiment** er et eksperiment, der kan føre til forskellige udfald, selv når det gentages under de samme betingelser. Denne tilfældighed er et grundlæggende aspekt af mange ingeniøropgaver.

Tænk på det på denne måde: Lad os sige, at du tester hastigheden af et websted under forskellige forhold. Nogle gange indlæses det hurtigt, andre gange er det langsommere. Selv om du bruger den samme kode og server, får ting som netværkstrafik eller belastning af serveren resultaterne til at variere fra gang til gang.

Definition 4.1 (Stokastisk eksperiment)

Et stokastisk eksperiment er et eksperiment, der kan give forskellige resultater, selv om du gør alting på samme måde hver gang.

♣

Eller forestil dig, at du måler signalstyrken i en trådløs enhed. Du får måske lidt forskellige målinger hver gang på grund af ting som interferens eller små ændringer i omgivelserne.

Denne tilfældighed viser sig overalt, fra ydelsestest af software til eksperimenter inden for elektroteknik. Det er vigtigt at forvente den og tage højde for den i sine overvejelser. Ellers risikerer du at træffe beslutninger på

grundlag af ufuldstændige eller misvisende data. Når du tager højde for tilfældig variation, kan du lave bedre forudsigelser og design.

For at modellere og analysere et stokastisk eksperiment er det afgørende at kende mængden af mulige udfald. I sandsynlighedsteorien kaldes denne mængde **udfaldsrummet**, og den betegnes S. Et udfaldsrum kan enten være **diskret** (bestå af en endelig eller tælleligt uendelig mængde af udfald) eller **kontinuert** (indeholde et interval af reelle tal). Den præcise definition af et udfaldsrum afhænger ofte af formålet med analysen.

Et **udfald** er ét muligt resultat af det stokastiske eksperiment, og en **hændelse** er en vilkårlig delmængde af udfaldsrummet, som kan bestå af ét eller flere udfald. Nedenfor følger nogle eksempler, der illustrerer disse begreber:

Eksempel 4.1 Netværksforsinkelse

Se på et eksperiment, hvor du måler forsinkelsen (latensen) af datapakker i et netværk. Udfaldsrummet kan defineres ud fra typen af målinger:

- Hvis latensen måles som et positivt reelt tal, er udfaldsrummet kontinuert:
  $S = \{x \mid x > 0\}$.
- Hvis det vides, at latensen ligger mellem 10 og 100 millisekunder, kan udfaldsrummet indsnævres til:
  $S = \{x \mid 10 \leq x \leq 100\}$.
- Hvis formålet er at inddele latensen i lav, middel eller høj, bliver udfaldsrummet diskret:
  $S = \{\text{lav}, \text{middel}, \text{høj}\}$.
- Ved en simpel vurdering af, om latensen overholder en standardgrænse, kan udfaldsrummet reduceres til:
  $S = \{\text{bestået}, \text{fejlet}\}$.

Hvert udfald i disse udfaldsrum repræsenterer én mulig måling af latensen, og hændelser kan defineres som mængder af udfald, fx »latensen er høj«.

Det er grundlæggende i sandsynlighedsregning at forstå, hvad udfaldsrum, udfald og hændelser er, fordi det gør det muligt at definere og arbejde med sandsynligheder for komplekse scenarier i forskellige ingeniørsammenhænge. Lad os opsummere disse centrale begreber:

Definition 4.2 (Udfaldsrum, udfald og hændelser)

Udfaldsrum: Mængden af alle mulige udfald af et stokastisk eksperiment kaldes udfaldsrummet og betegnes S. Udfaldene kan være diskrete eller kontinuerte afhængigt af eksperimentets karakter.
Udfald: Ét muligt resultat af et stokastisk eksperiment.
Hændelse: En vilkårlig delmængde af udfaldsrummet, som kan bestå af ét eller flere udfald. ♣

Eksempel 4.2 Test af en softwareudgivelse

Forestil dig en softwaretestproces, hvor hvert testtilfælde enten kan bestås eller fejle. Udfaldsrummet for et enkelt testtilfælde er diskret og kan skrives som:

$$S = \{\text{bestået}, \text{fejlet}\}.$$

Hvis du kører tre testtilfælde, er det samlede udfaldsrum for alle mulige udfald:

$$S = \{(\text{bestået}, \text{bestået}, \text{bestået}), (\text{bestået}, \text{bestået}, \text{fejlet}), (\text{bestået}, \text{fejlet}, \text{bestået}),$$
$$(\text{bestået}, \text{fejlet}, \text{fejlet}), (\text{fejlet}, \text{bestået}, \text{bestået}), (\text{fejlet}, \text{bestået}, \text{fejlet}),$$
$$(\text{fejlet}, \text{fejlet}, \text{bestået}), (\text{fejlet}, \text{fejlet}, \text{fejlet})\}.$$

Dette udfaldsrum omfatter alle følger af udfald for de tre test.

- Det samlede antal mulige udfald er $2^3 = 8$.
- Hændelsen »mindst én test fejler« omfatter fx (fejlet, bestået, bestået) og (bestået, fejlet, fejlet) samt alle andre udfald med mindst én fejlet test.

Eksempel 4.3 Komponentkvalitet i produktionen
En virksomhed producerer elektroniske komponenter, og hver komponent testes for, om den overholder kvalitetsstandarderne. Testen kan give et af tre udfald: `bestået`, `marginal` eller `fejlet`. Udfaldsrummet er:

$$S = \{\text{bestået}, \text{marginal}, \text{fejlet}\}.$$

Hændelse: Antag, at vi er interesseret i hændelsen, at en komponent ikke består kvalitetstesten. Denne hændelse er en mængde af udfald:

$$E = \{\text{marginal}, \text{fejlet}\}.$$

Det betyder, at alle de operationer, vi har defineret på mængder, kan overføres direkte til operationer på hændelser:

- $A \cup B$: hændelsen, at mindst én af A og B indtræffer.
- $A \cap B$: hændelsen, at både A og B indtræffer.
- $\overline{A}$ eller A^c: hændelsen, at A ikke indtræffer.
- $A \setminus B$ eller $A - B$: hændelsen, at A indtræffer, men B ikke gør.
- $A \Delta B$ eller $A \oplus B$: hændelsen, at enten A eller B indtræffer, men ikke begge (symmetrisk differens).

Sandsynlighedsteorien bygger altså direkte på mængdelæren: En sandsynlighed tildeler et numerisk mål til disse delmængder af udfaldsrummet. Senere i kapitlet ser vi, hvordan dette mål defineres og bruges til at ræsonnere om usikkerhed.

Vi kan opsummere overvejelserne i dette afsnit i følgende tabel:

I denne bog bruger vi notationen A^c for komplementet til mængden A, som omfatter alle elementer, der ikke er med i A.

4.2 Tælleprincipper

I mange problemer inden for matematik, datalogi og ingeniørfag er det vigtigt at bestemme, på hvor mange måder bestemte hændelser kan indtræffe. Uanset om du ordner elementer, udvælger grupper eller navigerer gennem komplekse scenarier, giver tælleteknikker de grundlæggende værktøjer til at løse disse problemer. Teknikkerne rækker ud over simpel aritmetik og gør det muligt at besvare spørgsmål som:

- På hvor mange måder kan vi ordne en mængde af objekter?
- Ad hvor mange forskellige veje kan en proces forløbe?

Operation	Boolesk algebra	Logik	Mængdelære
NOT	$\overline{x}$	$\neg x$	A^c eller A'
OR	$+$	$\vee$	$\cup$
AND	$\cdot$	$\wedge$	$\cap$
NAND	$\overline{x \cdot y}$	$\neg(x \wedge y)$	$(A \cap B)^c$ eller $\overline{A \cap B}$
NOR	$\overline{x + y}$	$\neg(x \vee y)$	$(A \cup B)^c$ eller $\overline{A \cup B}$
XOR (symmetrisk differens)	$x \oplus y$	$(x \wedge \neg y) \vee (\neg x \wedge y)$	$A \triangle B$ eller $A \oplus B$
Differens	$x \cdot \overline{y}$	$x \wedge \neg y$	$A - B$ eller $A \setminus B$

Tabel 4.1: Sammenligning af operatorer i boolesk algebra, logik og mængdelære

- Hvad er sandsynligheden for, at en bestemt hændelse indtræffer, når der er flere muligheder?

Tælleteknikker som permutationer, kombinationer og multiplikationsreglen hjælper os med at opgøre disse muligheder systematisk.

Multiplikationsreglen

Vi begynder med at se på det mest grundlæggende tælleprincip, **multiplikationsreglen**:

Sætning 4.1 (Multiplikationsreglen)

Lad en handling være beskrevet som en følge af k trin. Antag følgende betingelser:

- Der er n_1 måder at udføre trin 1 på.
- Der er n_2 måder at udføre trin 2 på for hver måde at udføre trin 1 på.
- Der er n_3 måder at udføre trin 3 på for hver mulig udførelse af de to første trin, og tilsvarende for hvert af de efterfølgende trin.

Så er det samlede antal måder at udføre hele handlingen på givet ved:

$$n_1 \times n_2 \times \cdots \times n_k.$$

♡

Eksempel 4.4 Antag, at du skal vælge et måltid på en restaurant. Du har følgende muligheder:

- 3 valgmuligheder til hovedretten.
- 4 valgmuligheder til tilbehøret.
- 2 valgmuligheder til drikkevaren.

Ifølge multiplikationsreglen er det samlede antal måder at vælge et måltid på:

$$3 \times 4 \times 2 = 24.$$

Der er altså 24 forskellige måltidskombinationer at vælge imellem.

Eksempel 4.5 Tilvalg til biler

En bilproducent leverer biler med udvalgte tilvalg. Hver bil bestilles

- med eller uden automatgear

- med eller uden soltag
- med ét ud af tre valgmuligheder for stereoanlæg
- med én ud af fire udvendige farver

Hvis udfaldsrummet består af mængden af alle mulige biltyper, hvad er så antallet af udfald i udfaldsrummet?

Løsning: Ved hjælp af multiplikationsreglen kan vi beregne det samlede antal mulige biltyper ved at gange antallet af valgmuligheder for hvert tilvalg:

- 2 valgmuligheder for gearkassen (med eller uden automatgear)
- 2 valgmuligheder for soltaget (med eller uden soltag)
- 3 valgmuligheder for stereoanlægget
- 4 valgmuligheder for den udvendige farve

Det samlede antal mulige biltyper er derfor:

$$2 \times 2 \times 3 \times 4 = 48$$

Der er altså 48 forskellige mulige biltyper i udfaldsrummet. ◀

Tilbagelægning og rækkefølge ved optælling

Dernæst ser vi på en vigtig skelnen i tælleprincipperne, nemlig mellem optælling med og uden tilbagelægning:

Definition 4.3 (Optælling med og uden tilbagelægning)

Når vi tæller antallet af måder at udvælge objekter fra en mængde på, er der to almindelige scenarier:

- **Med tilbagelægning**: Et objekt kan udvælges mere end én gang.
- **Uden tilbagelægning**: Når et objekt er udvalgt, kan det ikke vælges igen.

♣

Eksempel 4.6 Antag, at du har en pose med 5 kugler i forskellige farver. Du trækker 2 kugler:

- **Med tilbagelægning**: Den første kugle lægges tilbage i posen, før den anden trækkes. Der er $5 \times 5 = 25$ mulige udfald.
- **Uden tilbagelægning**: Den første kugle lægges ikke tilbage, så antallet af udfald er $5 \times 4 = 20$.

Eksempel 4.7 Tre personer trækker kort efter tur fra et almindeligt spil kort med 52 kort. Målet er at finde spar es. Lad os se på de to scenarier: med tilbagelægning og uden tilbagelægning.

Uden tilbagelægning

I dette scenarie lægges et trukket kort ikke tilbage i bunken, så det samlede antal tilgængelige kort bliver mindre efter hver trækning.

- Første trækning: Den første person har en chance på $\frac{1}{52}$ for at trække spar es.
- Anden trækning: Hvis den første person ikke trækker spar es, er der nu 51 kort tilbage, og den anden person har en chance på $\frac{1}{51}$ for at trække spar es.
- Tredje trækning: Hvis spar es ikke er trukket af de to første personer, har den tredje person en chance på $\frac{1}{50}$ for at trække det.

Sandsynlighederne ændrer sig ved hver trækning, fordi det samlede antal kort bliver mindre, og tidligere trukne kort ikke er tilgængelige.

Med tilbagelægning

I dette scenarie lægges hvert trukket kort tilbage i bunken, som blandes, før den næste person trækker. Det samlede antal kort er dermed konstant.

- Første trækning: Den første person har en chance på $\frac{1}{52}$ for at trække spar es.
- Anden trækning: Da kortet lægges tilbage og blandes ind i bunken igen, har den anden person også en chance på $\frac{1}{52}$ for at trække spar es.
- Tredje trækning: Tilsvarende har den tredje person en chance på $\frac{1}{52}$ for at trække spar es.

Sandsynlighederne er de samme ved hver trækning, fordi bunken bringes tilbage til sin oprindelige tilstand efter hver trækning.

Eksempel 4.8 Forestil dig, at du har en gruppe på 5 studerende: Alice, Bob, Charlie, David og Eve. Du skal udvælge 2 af dem i forskellige scenarier, som illustrerer, hvornår rækkefølgen af udvælgelsen har betydning, og hvornår den ikke har.

- **Rækkefølgen har betydning**: At vælge Alice som anfører og Bob som viceanfører er et andet udfald end at vælge Bob som anfører og Alice som viceanfører.

Forestil dig nu, at du blot udvælger 2 studerende til en læsegruppe uden bestemte roller. Her har rækkefølgen ingen betydning.

- **Rækkefølgen har ingen betydning:** At vælge Alice og Bob betragtes som det samme udfald som at vælge Bob og Alice; der er ingen forskel på de to rækkefølger, da der ikke er tildelt roller.

Eksempel 4.8 illustrerer forskellen mellem *permutationer* og *kombinationer*, to grundlæggende tælleprincipper, som er meget udbredte i sandsynlighedsteori og kombinatorik.

Definition 4.4 (Permutation og kombination)

- **Permutation (rækkefølgen har betydning)**: Forskellige rækkefølger tælles som forskellige udfald, hvilket giver et større antal.
- **Kombination (rækkefølgen har ingen betydning)**: Rækkefølger behandles som ens, hvilket giver et mindre antal.

♣

Vi ser først på permutationer, som bruges, når rækkefølgen af udvælgelsen har betydning.

Permutationer

Se på en mængde af elementer, fx $S = \{a, b, c\}$. En permutation af elementerne er en ordnet følge af elementerne. For eksempel er abc, acb, bac, bca, cab og cba alle permutationerne af elementerne i S.

Proposition 4.1 (Permutationer af n forskellige objekter)

Antallet af ordnede opstillinger (permutationer) af n forskellige objekter er

$$n! = n \cdot (n-1) \cdot \cdots \cdot 2 \cdot 1.$$

♠

Resultatet er en direkte anvendelse af multiplikationsreglen. For at danne en permutation begynder du med at vælge et element til den første plads blandt alle n elementer. Derefter vælger du et element til den anden plads blandt de resterende $n-1$ elementer, så til den tredje plads blandt de resterende $n-2$ elementer og fortsætter på denne måde, indtil alle pladser er besat. Sådanne opstillinger kaldes ofte lineære permutationer.

Eksempel 4.9 Man siger, at enhver blanding af et spil kort kun er forekommet én gang i historien. Det skyldes, at antallet af måder at blande et spil med 52 kort på er 52!, hvilket er et astronomisk stort tal.

$$52! \approx 8.07 \times 10^{67}$$

Eksempel 4.10 Antag, at du har 5 forskellige bøger på en hylde. Du vil stille dem i en anden rækkefølge. Antallet af måder at stille bøgerne på er

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

Der er tilfælde, hvor vi kun er interesseret i at ordne en delmængde af elementerne fra en større mængde. Formlen for antallet af sådanne opstillinger følger også af multiplikationsreglen.

Sætning 4.2 (Permutationer af delmængder)

For hele tal $n \geq r \geq 0$ er antallet af ordnede udvælgelser af r forskellige objekter blandt n forskellige objekter

$$P_r^n = P(n,r) = n \cdot (n-1) \cdots (n-r+1) = \frac{n!}{(n-r)!}.$$

♡

Eksempel 4.11 Antag, at du har 5 forskellige bøger på en hylde, og at du vil stille 3 af dem i en anden rækkefølge. Antallet af måder at stille de 3 bøger på er

$$P_3^5 = 5 \times 4 \times 3 = 60$$

Eksempel 4.12 Der er 10 deltagere i en konkurrence. Kun tre vinder, nemlig 1., 2. eller 3. præmie. Hvad er de mulige resultater?

Løsning: Antallet af måder at uddele præmierne på er antallet af permutationer af 3 objekter udvalgt blandt 10, dvs.

$$P_3^{10} = \frac{10!}{(10-3)!} = \frac{10!}{7!} = 10 \times 9 \times 8 = 720$$

Der er altså 720 mulige udfald af præmieuddelingen. ◀

Kombinationer

Når rækkefølgen af udvælgelsen ikke har betydning, bruger vi begrebet kombinationer. Kombinationer bruges, når vi er interesseret i at udvælge en delmængde af elementer fra en større mængde uden hensyn til den rækkefølge, de udvælges i. Lad os begynde med et par eksempler, der illustrerer begrebet kombinationer.

Eksempel 4.13 Antag, at du har en gruppe på 5 studerende: Alice, Bob, Charlie, David og Eve. Du skal udvælge 2 af dem til at danne en læsegruppe. Rækkefølgen, du vælger de studerende i, har ingen betydning. De mulige

kombinationer er:

- Alice og Bob
- Alice og Charlie
- Alice og David
- Alice og Eve
- Bob og Charlie
- Bob og David
- Bob og Eve
- Charlie og David
- Charlie og Eve
- David og Eve

Rækkefølgen af de studerende i læsegruppen har ingen betydning, så kombinationerne betragtes som ens.

Eksempel 4.14 Maria har tre billetter til en koncert. Hun vil selv bruge den ene billet. De to andre billetter kan hun så tilbyde til hvilke som helst af fire venner (Ann, Beth, Chris, Dave). På hvor mange måder kan 2 personer udvælges blandt 4 til at gå til koncert?

Eksempel 4.15 Et printkort har fire forskellige pladser, hvor en komponent kan placeres. Hvis tre ens komponenter skal placeres på kortet, hvor mange forskellige designs er så mulige?

Løsning: Da der kun kan placeres én komponent på hver plads, begrænser placeringen af en komponent på en plads straks valgmulighederne for den næste komponent.

1. Besæt plads 1, 2 og 3.
2. Besæt plads 1, 2 og 4.
3. Besæt plads 1, 3 og 4.
4. Besæt plads 2, 3 og 4.

◀

Disse eksempler illustrerer begrebet kombinationer, hvor rækkefølgen af udvælgelsen ikke har betydning. Formlen for kombinationer udledes af formlen for permutationer ved at dividere med antallet af måder at ordne de r elementer på.

Sætning 4.3 (Kombinationer)

Antallet af kombinationer af r elementer udvalgt fra en mængde med n forskellige elementer er givet ved

$$C_r^n = \binom{n}{r} = \frac{n!}{r!(n-r)!}$$

♡

Dette kaldes også **binomialkoefficienten**, som betegnes $\binom{n}{r}$ og læses »n over r«. Den kaldes binomialkoefficienten, fordi den optræder i binomialsætningen, som udvikler potenser af et binomium (et toleddet udtryk):

Sætning 4.4 (Binomialsætningen)

I algebra bruges binomialkoefficienten til at udvikle potenser af binomier. Ifølge binomialsætningen gælder

$$(a+b)^n = \sum_{k=0}^{n} \binom{n}{k} a^k b^{n-k}$$

♡

Sætningen siger, at udviklingen af binomiet $(a+b)^n$ er summen af leddene $\binom{n}{k}a^k b^{n-k}$ for $k = 0, 1, 2, \ldots, n$. Binomialkoefficienten $\binom{n}{k}$ angiver antallet af måder at vælge k elementer fra en mængde med n elementer på. Vi vil ikke lægge yderligere vægt på binomialsætningen her, men den er et grundlæggende begreb i algebra og kombinatorik og bruges i vid udstrækning i sandsynlighedsteori.

Vi afslutter gennemgangen af tælleprincipper med principperne for optælling med tilbagelægning.

Proposition 4.2 (Med tilbagelægning)

Ved udvælgelse blandt n typer gælder:

- **Ordnet med tilbagelægning**: n^r udfald.
- **Uordnet med tilbagelægning**: $\binom{n+r-1}{r}$ udfald.

♠

4.3 Grundlæggende sandsynlighedsregning

Sandsynlighed er et mål for, hvor sandsynligt eller hvor stor chancen er for, at hændelser i forbindelse med et stokastisk eksperiment indtræffer. Når du fx hører »Chancen for regn i dag er 30%«, udtrykker det usikkerhed om hændelsen, at det regner. Sandsynligheder ligger mellem 0 og 1 (eller tilsvarende mellem 0% og 100%). En umulig hændelse har sandsynlighed 0, og hele udfaldsrummet har sandsynlighed 1. I generelle sandsynlighedsmodeller kan en hændelse dog have sandsynlighed 0 uden at være tom eller sandsynlighed 1 uden at indeholde alle mulige udfald; sådanne hændelser kaldes henholdsvis næsten umulige og næsten sikre.

Sandsynligheder kan fortolkes på forskellige måder:

- **Objektiv (eller klassisk) sandsynlighed**: Denne tilgang, der ofte kaldes klassisk sandsynlighed, bruges, når udfaldene er lige sandsynlige, fx ved kast med en fair terning eller en mønt. Sandsynlighederne tildeles ud fra antagelsen om, at alle udfald har lige stor chance for at indtræffe. Når man fx kaster en fair sekssidet terning, er sandsynligheden for at slå en 3'er $\frac{1}{6}$, fordi der er 6 lige sandsynlige udfald (1, 2, 3, 4, 5, 6), og kun ét af dem er en 3'er. Sandsynligheden er den samme for alle iagttagere.
- **Relativ hyppighed (empirisk sandsynlighed)**: Empirisk sandsynlighed bygger på observationer fra eksperimenter frem for teoretiske beregninger. Hvis en softwaretester fx kører en belastningstest på en server 100 gange, og serveren går ned 7 gange, er den empiriske sandsynlighed for et nedbrud $\frac{7}{100} = 0.07$. Denne tilgang bygger på faktiske data frem for antagelser eller intuition.
- **Subjektiv sandsynlighed**: Den afspejler vores personlige overbevisning eller grad af tiltro til et udfald. Forskellige personer kan tildele den samme hændelse forskellige sandsynligheder ud fra deres viden eller synsvinkel. Du og dine venner diskuterer Danmarks chancer for at vinde VM. Ud fra holdets seneste præstationer og styrke vurderer du chancen til 10%. En mere optimistisk ven sætter den derimod til 20%, mens en anden kun giver 5%, fordi der er stærkere konkurrenter. Det illustrerer subjektiv sandsynlighed, hvor hver persons vurdering varierer ud fra personlige overbevisninger og forudindtagethed frem for objektive data.

Når vi tildeler sandsynligheder, er det afgørende, at summen af alle sandsynligheder i et eksperiment er 1. Det

sikrer overensstemmelse med fortolkningen som relativ hyppighed.

Vi begynder med at opstille sandsynlighedsregningens aksiomer, som danner grundlaget for, hvordan sandsynligheder tildeles hændelser. Aksiomerne fastlægger de grundlæggende egenskaber, som ethvert sandsynlighedsmål skal opfylde.

Aksiom 4.1 (Sandsynlighedsregningens aksiomer)

- **Aksiom 1:** For enhver hændelse A gælder $0 \leq P(A) \leq 1$.
- **Aksiom 2:** Sandsynligheden for udfaldsrummet S er $P(S) = 1$.
- **Aksiom 3:** Hvis $A_1, A_2, A_3, \cdots$ er disjunkte hændelser, så gælder $P\left(A_1 \cup A_2 \cup A_3 \cdots\right) = P\left(A_1\right) + P\left(A_2\right) + P\left(A_3\right) + \cdots$

♡

Egenskaben $0 \leq P(A) \leq 1$ svarer til kravet om, at en relativ hyppighed skal ligge mellem 0 og 1. Egenskaben $P(S) = 1$ er en følge af, at der ved hvert forsøg i et eksperiment indtræffer et udfald fra udfaldsrummet. Derfor er den relative hyppighed af S lig med 1. Egenskab 3 indebærer, at hvis hændelserne A_1 og A_2 ikke har nogen udfald til fælles, er den relative hyppighed af udfald i $A_1 \cup A_2$ summen af de relative hyppigheder af udfaldene i A_1 og A_2.

I de næste afsnit ser vi nærmere på sandsynligheden for hændelser, og hvordan den beregnes.

Sandsynligheden for en hændelse

Sandsynligheden for en hændelse er et mål for, hvor sandsynligt det er, at hændelsen indtræffer. Den betegnes $P(A)$, hvor A er hændelsen, og den ligger mellem 0 og 1. Den tomme hændelse har sandsynlighed 0, og udfaldsrummet har sandsynlighed 1.

Definition 4.5 (Klassisk sandsynlighed for lige sandsynlige udfald)

Hvis et endeligt udfaldsrum består af lige sandsynlige udfald, er sandsynligheden for en hændelse A forholdet mellem antallet af udfald i A og det samlede antal udfald i udfaldsrummet:

$$P(A) = \frac{\text{Antal gunstige udfald}}{\text{Samlet antal udfald}}$$

♣

Eksempel 4.16 Antag, at du tester et softwaremodul med 10 forskellige testtilfælde. Det vides, at 3 af testtilfældene fejler på grund af en fejl i koden. Hvis du vælger et tilfældigt testtilfælde og kører det, hvad er så sandsynligheden for, at det valgte testtilfælde fejler?

Løsning: Her er hændelsen A »testtilfældet fejler«.

- Antal gunstige udfald (fejlende testtilfælde) $= 3$
- Samlet antal udfald (testtilfælde i alt) $= 10$

Ved hjælp af formlen får vi:

$$P(A) = \frac{\text{Antal gunstige udfald}}{\text{Samlet antal udfald}} = \frac{3}{10} = 0.3$$

Sandsynligheden for, at et tilfældigt valgt testtilfælde fejler, er altså 0.3, hvilket betyder, at der er 30% chance

for fejl.

◀

Eksempel 4.17 Forestil dig et softwareudviklingsmiljø med 50 filer, som består af 20 Python-scripts, 15 Java-filer og 15 konfigurationsfiler. Hvis du vælger en tilfældig fil at redigere, hvad er så sandsynligheden for, at filen er et Python-script?

Løsning: Her er hændelsen A »den valgte fil er et Python-script«.

- Antal gunstige udfald (Python-scripts) $= 20$
- Samlet antal udfald (filer i alt) $= 50$

Ved hjælp af formlen får vi:

$$P(A) = \frac{\text{Antal gunstige udfald}}{\text{Samlet antal udfald}} = \frac{20}{50} = 0.4$$

Sandsynligheden for at vælge et Python-script er altså 0.4, hvilket betyder, at der er 40% chance for at vælge en Python-fil blandt filerne.

◀

4.4 Sandsynlighed for sammensatte hændelser og mængdeoperationer

Sammensatte hændelser dannes ved at anvende de grundlæggende mængdeoperationer på enkelte hændelser. Vi møder ofte foreningsmængder af hændelser som $A \cup B$, fællesmængder af hændelser som $A \cap B$ og komplementer af hændelser som A^c. Disse sammensatte hændelser er ofte af særlig interesse, og deres sandsynligheder kan tit udledes af sandsynlighederne for de enkelte hændelser, de er sammensat af. Det er vigtigt at forstå disse mængdeoperationer for at kunne beregne sandsynligheden for sammensatte hændelser korrekt. I dette afsnit ser vi på, hvordan foreningsmængder af hændelser og andre mængdeoperationer kan bruges til at bestemme sandsynlighederne for mere komplekse hændelser.

Når vi arbejder med hændelser, svarer fællesmængden til `AND`, mens foreningsmængden svarer til `OR`. Sandsynligheden for fællesmængden af hændelserne A og B, som betegnes $P(A \cap B)$, kan også skrives $P(A, B)$ eller $P(AB)$.

Ud fra sandsynlighedsregningens aksiomer kan vi udlede følgende regler for sandsynligheder:

Sætning 4.5 (Regler for sandsynlighed)

- **Komplementreglen:** Sandsynligheden for komplementet til hændelse A er
  $$P(A^c) = 1 - P(A)$$
- **Reglen for den tomme mængde:** Sandsynligheden for den tomme mængde er 0, dvs.
  $$P(\emptyset) = 0$$
- **Additionsreglen:** For to vilkårlige hændelser A og B er sandsynligheden for foreningsmængden af hændelserne A og B givet ved
  $$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

- **Differensreglen:** Sandsynligheden for differensen mellem hændelserne A og B er givet ved
  $$P(A - B) = P(A) - P(A \cap B)$$
- **Delmængdereglen:** Hvis A er en delmængde af B ($A \subset B$), så gælder
  $$P(A) \leq P(B)$$

.

♡

Vi kan udlede komplementreglen ved at bemærke følgende:

$$\begin{aligned} 1 &= P(S) && \text{(aksiom 2)} \\ &= P(A \cup A^c) && \text{(definitionen af komplement)} \\ &= P(A) + P(A^c) && \text{(da } A \text{ og } A^c \text{ er disjunkte)} \end{aligned}$$

Da $\emptyset = S^c$, kan vi bruge komplementreglen til at slutte, at $P(\emptyset) = 1 - P(S) = 0$. Det er intuitivt, fordi en hændelse pr. definition indtræffer, når udfaldet af det stokastiske eksperiment er en del af hændelsen. Men da den tomme mængde ikke indeholder nogen elementer, kan intet udfald af eksperimentet nogensinde tilhøre den, og derfor er dens sandsynlighed nul.

Differensreglen kan udledes ved at vise, at $P(A) = P(A \cap B) + P(A - B)$. Bemærk, at de to mængder $A \cap B$ og $A - B$ er disjunkte, og at deres foreningsmængde er A. Ifølge sandsynlighedsregningens tredje aksiom gælder derfor

$$\begin{aligned} P(A) &= P\left((A \cap B) \cup (A - B)\right) && \text{(da } A = (A \cap B) \cup (A - B)) \\ &= P(A \cap B) + P(A - B) && \text{(da } A \cap B \text{ og } A - B \text{ er disjunkte)} \end{aligned}$$

Additionsreglen får vi ved at bemærke, at A og $B - A$ er disjunkte mængder, og at deres foreningsmængde er $A \cup B$. Dermed gælder

$$\begin{aligned} P(A \cup B) &= P(A \cup (B - A)) && \text{(da } A \cup B = A \cup (B - A)) \\ &= P(A) + P(B - A) && \text{(da } A \text{ og } B - A \text{ er disjunkte)} \\ &= P(A) + P(B) - P(A \cap B) && \text{(jf. differensreglen)} \end{aligned}$$

Endelig er delmængdereglen en direkte følge af, at hvis $A \subset B$, kan B skrives som foreningsmængden af A og $B - A$. Da A og $B - A$ er disjunkte, har vi $P(B) = P(A) + P(B - A) \geq P(A)$.

Vi afslutter afsnittet med nogle eksempler, der viser, hvordan disse regler anvendes til at beregne sandsynligheder for sammensatte hændelser.

Eksempel 4.18 En virksomhed har afgivet bud på to store byggeprojekter. Virksomhedens direktør vurderer, at sandsynligheden for at vinde den første kontrakt er 0.6, sandsynligheden for at vinde den anden kontrakt er 0.4, og sandsynligheden for at vinde begge kontrakter er 0.2.

(a) Hvad er sandsynligheden for, at virksomheden vinder mindst én kontrakt?
(b) Hvad er sandsynligheden for, at virksomheden vinder den første kontrakt, men ikke den anden?
(c) Hvad er sandsynligheden for, at virksomheden ikke vinder nogen af kontrakterne?
(d) Hvad er sandsynligheden for, at virksomheden vinder præcis én kontrakt?

Løsning: Lad A være hændelsen, at virksomheden vinder den første kontrakt, og B hændelsen, at virksomheden vinder den anden kontrakt. Vi har givet:

- $P(A) = 0.6$
- $P(B) = 0.4$
- $P(A \cap B) = 0.2$

(a) Sandsynligheden for, at virksomheden vinder mindst én kontrakt, er sandsynligheden for foreningsmængden af hændelserne A og B. Ifølge additionsreglen er:

$$\begin{aligned} P(A \cup B) &= P(A) + P(B) - P(A \cap B) \\ &= 0.6 + 0.4 - 0.2 \\ &= 0.8 \end{aligned}$$

Sandsynligheden for, at virksomheden vinder mindst én kontrakt, er altså 0.8.

(b) Sandsynligheden for, at virksomheden vinder den første kontrakt, men ikke den anden, er sandsynligheden for differensen mellem hændelserne A og B. Ifølge differensreglen er:

$$\begin{aligned} P(A - B) &= P(A) - P(A \cap B) \\ &= 0.6 - 0.2 \\ &= 0.4 \end{aligned}$$

Sandsynligheden for, at virksomheden vinder den første kontrakt, men ikke den anden, er altså 0.4.

(c) Sandsynligheden for, at virksomheden ikke vinder nogen af kontrakterne, er sandsynligheden for komplementet til foreningsmængden af hændelserne A og B. Ifølge komplementreglen er:

$$\begin{aligned} P((A \cup B)^c) &= 1 - P(A \cup B) \\ &= 1 - 0.8 \\ &= 0.2 \end{aligned}$$

Sandsynligheden for, at virksomheden ikke vinder nogen af kontrakterne, er altså 0.2.

(d) Sandsynligheden for, at virksomheden vinder præcis én kontrakt, er sandsynligheden for differensen mellem foreningsmængden af hændelserne A og B og fællesmængden af hændelserne A og B. Ifølge differensreglen er:

$$\begin{aligned} P((A \cup B) - (A \cap B)) &= P(A \cup B) - P(A \cap B) \\ &= 0.8 - 0.2 \\ &= 0.6 \end{aligned}$$

Sandsynligheden for, at virksomheden vinder præcis én kontrakt, er altså 0.6.

◀

Eksempel 4.19

- Der er 60 procents chance for, at det regner i dag.
- Der er 50 procents chance for, at det regner i morgen.
- Der er 30 procents chance for, at det ikke regner nogen af dagene.

(a) Sandsynligheden for, at det regner i dag eller i morgen.

(b) Sandsynligheden for, at det regner i dag og i morgen.

(c) Sandsynligheden for, at det regner i dag, men ikke i morgen.

(d) Sandsynligheden for, at det regner enten i dag eller i morgen, men ikke begge dage.

Løsning:

(a) Lad A være hændelsen, at det regner i dag, og B hændelsen, at det regner i morgen.
Vi har givet:

$$\begin{aligned} P(A \cup B) &= 1 - P\left((A \cup B)^c\right) && \text{jf. komplementreglen} \\ &= 1 - P\left(A^c \cap B^c\right) && \text{jf. De Morgans lov} \\ &= 1 - 0.3 \\ &= 0.7 \end{aligned}$$

Sandsynligheden for, at det regner i dag eller i morgen, er altså 0.7.

(b) Sandsynligheden for, at det regner i dag og i morgen: Det er $P(A \cap B)$. For at bestemme den bemærker vi, at

$$\begin{aligned} P(A \cap B) &= P(A) + P(B) - P(A \cup B) \\ &= 0.6 + 0.5 - 0.7 \\ &= 0.4 \end{aligned}$$

(c) Sandsynligheden for, at det regner i dag, men ikke i morgen: Det er $P\left(A \cap B^c\right)$.

$$\begin{aligned} P(A \cap B^c) &= P(A - B) \\ &= P(A) - P(A \cap B) \\ &= 0.6 - 0.4 \\ &= 0.2 \end{aligned}$$

(d) Sandsynligheden for, at det regner enten i dag eller i morgen, men ikke begge dage: Det er $P(A - B) + P(B - A)$. Vi har allerede fundet $P(A - B) = .2$. På tilsvarende måde kan vi bestemme $P(B - A)$:

$$\begin{aligned} P(B - A) &= P(B) - P(B \cap A) \\ &= 0.5 - 0.4 \\ &= 0.1 \end{aligned}$$

Dermed er

$$\begin{aligned} P(A - B) + P(B - A) &= 0.2 + 0.1 \\ &= 0.3 \end{aligned}$$

◀

Kapitel 5 Betinget sandsynlighed og Bayes' sætning

I dette kapitel introducerer vi mere avancerede begreber inden for sandsynlighedsregning. Vi begynder med **betinget sandsynlighed**, som vurderer sandsynligheden for, at en hændelse indtræffer, *givet* at en anden hændelse allerede har fundet sted. I forlængelse heraf introducerer vi **multiplikationsreglen**, et centralt princip til at bestemme sandsynligheden for, at flere hændelser indtræffer efter hinanden. Dernæst ser vi på **loven om total sandsynlighed**, som gør det muligt at opdele og beregne sandsynligheder på tværs af forskellige scenarier eller partitioner af udfaldsrummet. For bedre at kunne skelne mellem, hvordan hændelser påvirker hinanden, undersøger vi **afhængige og uafhængige hændelser**. Her afklarer vi, hvordan forekomsten af én hændelse påvirker eller ikke påvirker en anden.

Til sidst ser vi nærmere på **Bayes' sætning**, et stærkt værktøj til at opdatere sandsynligheder i lyset af ny evidens.

5.1 Betinget sandsynlighed

Se på følgende scenarie: I en bestemt population har 5% af individerne en bestemt sygdom. Sandsynligheden for, at en tilfældigt udvalgt person har sygdommen, er derfor 5%:

$$P(D) = 0.05,$$

Her betegner D hændelsen, at personen har sygdommen. $P(D)$ kaldes *a priori-sandsynligheden*: vores vurdering før yderligere information.

Forestil dig nu, at vi udvælger en tilfældig person og får at vide, at personen er testet positiv for sygdommen. Hvordan skal vi med denne ekstra information opdatere sandsynligheden for, at personen faktisk har sygdommen? Hvad er med andre ord sandsynligheden for, at en person har sygdommen, givet at personen er testet positiv? Lad T betegne hændelsen, at en person testes positiv. Denne betingede sandsynlighed skrives som:

$$P(D \mid T),$$

hvilket er sandsynligheden for D, givet at T er indtruffet. Den opdaterede sandsynlighed $P(D \mid T)$ kaldes *a posteriori-sandsynligheden*, fordi den udtrykker vores reviderede opfattelse af hændelsens sandsynlighed, efter at vi har taget den nye evidens i betragtning. Intuitivt er det rimeligt at forvente, at $P(D \mid T)$ er større end a priori-sandsynligheden $P(D)$. Men hvad er den præcise værdi af $P(D \mid T)$? Før vi introducerer en generel formel, ser vi på et simpelt eksempel.

Eksempel 5.1 Jeg kaster en fair terning. Lad A være hændelsen, at udfaldet er et primtal, dvs. $A = \{2, 3, 5\}$. Lad desuden B være hændelsen, at udfaldet er større end eller lig med 3, dvs. $B = \{3, 4, 5, 6\}$.

(a) Hvad er sandsynligheden for A, $P(A)$?
(b) Hvad er sandsynligheden for A givet B, $P(A \mid B)$?

Løsning:

(a) Sandsynligheden for A er antallet af udfald i A divideret med det samlede antal udfald. Da der er 3 primtal og 6 mulige udfald, har vi:

$$P(A) = \frac{3}{6} = \frac{1}{2}$$

(b) Sandsynligheden for A givet B er antallet af udfald i fællesmængden af A og B divideret med antallet af udfald i B. Da udfaldene i fællesmængden af A og B er $\{3, 5\}$, og udfaldene i B er $\{3, 4, 5, 6\}$, har vi:

$$P(A \mid B) = \frac{|A \cap B|}{|B|} = \frac{|\{3, 5\}|}{|\{3, 4, 5, 6\}|} = \frac{1}{2}$$

◀

Nu hvor vi har forstået det grundlæggende eksempel, kan vi generalisere fremgangsmåden og udlede en mere generel formel for betinget sandsynlighed. Med udgangspunkt i det konkrete tilfælde kan vi omskrive udtrykket ved at dividere både tæller og nævner med det samlede antal mulige udfald, $|S|$, som vist nedenfor:

$$P(A \mid B) = \frac{|A \cap B|}{|B|} = \frac{\frac{|A \cap B|}{|S|}}{\frac{|B|}{|S|}} = \frac{P(A \cap B)}{P(B)}$$

Forklaring:

- **Tæller:** $|A \cap B|$ er antallet af udfald, hvor både hændelse A og hændelse B indtræffer. Ved at dividere med $|S|$ omdannes dette antal til en sandsynlighed, $P(A \cap B)$.
- **Nævner:** $|B|$ er antallet af udfald, hvor hændelse B indtræffer. Ved tilsvarende at dividere med $|S|$ får vi sandsynligheden for hændelse B, som betegnes $P(B)$.

Den betingede sandsynlighed $P(A \mid B)$ kan altså udtrykkes som forholdet mellem den fælles sandsynlighed for A og B og sandsynligheden for B.

Selv om udledningen ovenfor forudsætter et endeligt udfaldsrum med lige sandsynlige udfald, er den resulterende formel bemærkelsesværdigt generel. Den gælder, uanset om udfaldsrummet er endeligt eller uendeligt, og uanset om udfaldene er lige sandsynlige eller ej. Denne generalitet gør formlen til en hjørnesten i sandsynlighedsteorien.

Definition 5.1 (Betinget sandsynlighed)

For to vilkårlige hændelser A og B med $P(B) > 0$ er den betingede sandsynlighed for A givet B defineret som:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}, \qquad P(B) > 0$$

♣

Her er intuitionen bag formlen. Når vi ved, at hændelse B er indtruffet, udelukker vi reelt alle udfald, der ikke er en del af B. Derved bliver vores oprindelige udfaldsrum reduceret til delmængden B.

I dette nye, begrænsede udfaldsrum B kan hændelse A kun indtræffe, hvis udfaldet ligger i fællesmængden af A og B, som betegnes $A \cap B$. For at bestemme den betingede sandsynlighed $P(A \mid B)$ beregner vi forholdet mellem sandsynligheden for, at både A og B indtræffer, og sandsynligheden for, at B indtræffer alene. Matematisk udtrykkes det som:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$

Denne formulering sikrer, at sandsynlighederne i det nye udfaldsrum B er normaliserede, dvs. at den samlede sandsynlighed summer til 1. Se for eksempel på den betingede sandsynlighed for B givet B:

$$P(B \mid B) = \frac{P(B \cap B)}{P(B)} = \frac{P(B)}{P(B)} = 1$$

Dette resultat bekræfter intuitivt, at hvis B er indtruffet, er det sikkert, at B indtræffer, dvs. sandsynligheden er 1.

Bemærk, at den elementære formel for $P(A \mid B)$ ikke er defineret, når $P(B) = 0$, fordi den ville kræve division med nul. En hændelse med sandsynlighed nul behøver ikke at være tom i en kontinuert sandsynlighedsmodel, men betingning på sådanne hændelser kræver mere avancerede definitioner, som ligger uden for dette kapitels rammer.

Vi kan også formulere sandsynlighedsregningens aksiomer ved hjælp af betinget sandsynlighed. Aksiomerne er som følger:

Aksiom 5.1 (Aksiomer for betinget sandsynlighed)

- **Aksiom 1:** For enhver hændelse A gælder $0 \leq P(A \mid B) \leq 1$.
- **Aksiom 2:** Den betingede sandsynlighed for B givet B er 1, dvs. $P(B \mid B) = 1$.
- **Aksiom 3:** Hvis $A_1, A_2, A_3, \cdots$ er disjunkte hændelser, så gælder
$$P\left(A_1 \cup A_2 \cup A_3 \cdots \mid B\right) = P\left(A_1 \mid B\right) + P\left(A_2 \mid B\right) + P\left(A_3 \mid B\right) + \cdots$$

♡

Ud fra disse aksiomer kan vi også udlede andre regler for betinget sandsynlighed:

Sætning 5.1 (Regler for betinget sandsynlighed)

- **Komplementreglen:** Sandsynligheden for komplementet til hændelse A givet C er
$$P\left(A^c \mid C\right) = 1 - P(A \mid C)$$
- **Reglen for den tomme mængde:** Sandsynligheden for den tomme mængde givet en hændelse C er 0, dvs.
$$P(\emptyset \mid C) = 0$$
- **Additionsreglen:** For to vilkårlige hændelser A og B er sandsynligheden for foreningsmængden af hændelserne A og B givet en anden hændelse C
$$P(A \cup B \mid C) = P(A \mid C) + P(B \mid C) - P(A \cap B \mid C)$$
- **Differensreglen:** Sandsynligheden for differensen mellem hændelserne A og B givet en anden hændelse C er
$$P(A - B \mid C) = P(A \mid C) - P(A \cap B \mid C)$$
- **Delmængdereglen:** Hvis $A \subset B$, så gælder
$$P(A \mid C) \leq P(B \mid C)$$

♡

Det følgende eksempel viser, hvordan betinget sandsynlighed anvendes, og introducerer begrebet **kontingenstabeller** samt **falsk positive** og **falsk negative**.

Eksempel 5.2 En forsker vil vurdere, hvor effektiv en diagnostisk test er til at påvise nyresygdom hos patienter med forhøjet blodtryk. Hun udfører derfor testen på en stikprøve af 137 patienter, som er inddelt således:

- **67 patienter** med en bekræftet diagnose af nyresygdom.
- **70 patienter**, som vides at være raske (dvs. ikke har nyresygdom).

Den diagnostiske test giver et af to mulige resultater for hver patient:

- **Positiv**: Angiver, at patienten har nyresygdom.
- **Negativ**: Angiver, at patienten ikke har nyresygdom.

Resultaterne er opsummeret i følgende kontingenstabel:

	Testresultat		
Faktisk tilstand	Positiv	Negativ	I alt
Nyresygdom	44	23	67
Rask	10	60	70
I alt	54	83	137

I dette eksperiment er:

- **Sandt positive**: Patienter, der har nyresygdom og er testet positive.
- **Falsk negative**: Patienter, der har nyresygdom, men er testet negative.
- **Falsk positive**: Raske patienter, der er testet positive.
- **Sandt negative**: Raske patienter, der er testet negative.

Bestem følgende sandsynligheder:

(a) Sandsynligheden for at have nyresygdom, $P(D)$.
(b) Sandsynligheden for en positiv test, $P(T^+)$.
(c) Sandsynligheden for en negativ test, $P(T^-)$.
(d) Hvis en person har nyresygdom, hvad er så sandsynligheden for, at personen testes positiv for sygdommen?
(e) Bestem sandsynligheden for, at en patient har nyresygdom givet et positivt testresultat, $P(D \mid T^+)$.
(f) Bestem sandsynligheden for, at en patient ikke har nyresygdom givet et negativt testresultat, $P(\text{Rask} \mid T^-)$.
(g) Vurdér den diagnostiske tests samlede nøjagtighed.

Løsning: Ud fra kontingenstabellen kan vi beregne følgende sandsynligheder:

(a) **Sandsynligheden for nyresygdom,** $P(D)$:

$$\begin{aligned} P(D) &= \frac{\text{Antal patienter med nyresygdom}}{\text{Samlet antal patienter}} \\ &= \frac{67}{137} \approx 0.4891 \text{ eller } 48.91\% \end{aligned}$$

(b) **Sandsynligheden for en positiv test,** $P(T^+)$:

$$\begin{aligned} P(T^+) &= \frac{\text{Antal positive tests}}{\text{Samlet antal patienter}} \\ &= \frac{54}{137} \approx 0.3942 \text{ eller } 39.42\% \end{aligned}$$

(c) **Sandsynligheden for en negativ test,** $P(T^-)$:

$$\begin{aligned} P(T^-) &= \frac{\text{Antal negative tests}}{\text{Samlet antal patienter}} \\ &= \frac{83}{137} \approx 0.6058 \text{ eller } 60.58\% \end{aligned}$$

(d) **Sandsynligheden for en positiv test givet nyresygdom,** $P(T^+ \mid D)$:

$$P(T^+ \mid D) = \frac{44}{67} \approx 0.6567$$

(e) **Sandsynligheden for nyresygdom givet en positiv test,** $P(D \mid T^+)$:

$$\begin{aligned} P(D \mid T^+) &= \frac{P(D \cap T^+)}{P(T^+)} \\ &= \frac{44}{54} = \frac{22}{27} \approx 0.8148 \end{aligned}$$

(f) **Sandsynligheden for at være rask givet en negativ test,** $P(\textbf{Rask} \mid T^-)$:

$$P(\text{Rask} \mid T^-) = \frac{P(\text{Rask} \cap T^-)}{P(T^-)} = \frac{60}{83} \approx 0.7229$$

(g) **Den diagnostiske tests samlede nøjagtighed**: Den diagnostiske tests samlede nøjagtighed er andelen af korrekte diagnoser, dvs. summen af sandt positive og sandt negative divideret med det samlede antal patienter:

$$\text{Samlet nøjagtighed} = \frac{44 + 60}{137} = \frac{104}{137} \approx 0.7591$$

Beregningerne beskriver flere forskellige mål for testens ydeevne. Sensitiviteten, som er sandsynligheden for et positivt resultat blandt patienter med nyresygdom, er $44/67 \approx 65.67\%$. Specificiteten, som er sandsynligheden for et negativt resultat blandt raske patienter, er $60/70 \approx 85.71\%$. Derimod er $P(D \mid T^+) = 81.48\%$ den positive prædiktive værdi, og $P(\text{Rask} \mid T^-) = 72.29\%$ er den negative prædiktive værdi. Den samlede nøjagtighed er 75.91%. ◀

Her er et klassisk sandsynlighedsproblem, som kaldes tobørnsproblemet. Scenariet optræder i forskellige udgaver i litteraturen, og hver udgave har små drejninger, som fører til forskellige resultater. Før du ser på udregningerne, så brug et øjeblik på at komme med dine egne forudsigelser – du bliver måske overrasket over resultaterne!

Eksempel 5.3 Se på en familie med to børn. Vi er interesserede i de mulige kombinationer af børnenes biologiske køn. Udfaldsrummet for denne situation er:

$$S = \{(G, G), (G, B), (B, G), (B, B)\}$$

hvor G betegner en biologisk pige (af engelsk »girl«), og B betegner en biologisk dreng (af engelsk »boy«). For enkelhedens skyld antager vi, at alle fire udfald er lige sandsynlige.

(a) Hvad er sandsynligheden for, at begge børn er piger, givet at det første barn er en pige?

(b) Vi spørger faren: »Har du mindst én datter?« Han svarer »Ja!« Hvad er sandsynligheden for, at begge børn er piger, givet denne ekstra information? Hvad er med andre ord sandsynligheden for, at begge børn er piger, givet at vi ved, at mindst ét af dem er en pige?

Løsning: Lad A være hændelsen, at begge børn er piger, dvs. $A = \{(G, G)\}$. Lad B være hændelsen, at det første barn er en pige, dvs. $B = \{(G, G), (G, B)\}$. Lad til sidst C være hændelsen, at mindst ét af børnene er en pige, dvs. $C = \{(G, G), (G, B), (B, G)\}$. Da udfaldene er lige sandsynlige, kan vi skrive

$$P(A) = \frac{1}{4}$$
$$P(B) = \frac{2}{4} = \frac{1}{2}$$
$$P(C) = \frac{3}{4}$$

(a) Hvad er sandsynligheden for, at begge børn er piger, givet at det første barn er en pige? Det er $P(A \mid B)$, så vi kan skrive

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}$$
$$= \frac{P(A)}{P(B)} \quad (\text{ da } A \subset B)$$
$$= \frac{\frac{1}{4}}{\frac{1}{2}} = \frac{1}{2}$$

(b) Hvad er sandsynligheden for, at begge børn er piger, givet at vi ved, at mindst ét af dem er en pige? Det er $P(A \mid C)$, så vi kan skrive

$$P(A \mid C) = \frac{P(A \cap C)}{P(C)}$$
$$= \frac{P(A)}{P(C)} \quad (\text{ da } A \subset C)$$
$$= \frac{\frac{1}{4}}{\frac{3}{4}} = \frac{1}{3}$$

◀

Bemærkning: Mange vil intuitivt gætte på, at både $P(A \mid B)$ og $P(A \mid C)$ er 50%. Men selv om $P(A \mid B) = 50\%$, er $P(A \mid C)$ kun 33%. Det viser, hvor kontraintuitiv sandsynlighed kan være. Nøglen er at indse, at hændelse B er en delmængde af hændelse C. Konkret udelukker B udfaldet (B, G), som er med i C. Derfor har C flere udfald, der ikke ligger i A, end B har, hvilket giver en mindre $P(A \mid C)$ sammenlignet med $P(A \mid B)$.

Resultatet i del (b) forudsætter, at den givne information er præcis hændelsen C, »mindst ét barn er en pige«. Hvis faren blev udvalgt eller spurgt efter en anden tilfældig procedure, kan den betingede sandsynlighed ændre sig. Når man fortolker dette problem, skal man derfor angive den procedure, der frembringer informationen.

5.2 Multiplikationsreglen og loven om total sandsynlighed

Multiplikationsreglen er et grundlæggende princip i sandsynlighedsteorien, som gør det muligt at beregne sandsynligheden for, at flere hændelser indtræffer efter hinanden. Reglen er særlig nyttig, når hændelserne er afhængige, dvs. når forekomsten af én hændelse påvirker sandsynligheden for den efterfølgende hændelse.

Sætning 5.2 (Multiplikationsreglen)

Hvis $P(A) > 0$ og $P(B) > 0$, er sandsynligheden for, at begge hændelser indtræffer, givet ved:

$$P(A \cap B) = P(A)P(B \mid A) = P(B)P(A \mid B)$$

♡

Eksempel 5.4 Antag, at et softwareudviklingsteam tester en ny funktion. Testen foregår i to trin: enhedstest (A) og integrationstest (B). Sandsynligheden for, at en fejl opdages under enhedstesten, er $P(A) = 0.3$. Hvis en fejl opdages under enhedstesten, er sandsynligheden for, at den også opdages under integrationstesten, $P(B \mid A) = 0.7$. Hvad er sandsynligheden for, at en fejl opdages i begge testtrin?

Løsning: Med multiplikationsreglen får vi:

$$P(A \cap B) = P(A) \cdot P(B \mid A) = 0.3 \times 0.7 = 0.21$$

Sandsynligheden for, at en fejl opdages både under enhedstesten og integrationstesten, er altså 0.21. ◀

Eksempel 5.5 En virksomhed har indført et sikkerhedssystem i to lag for at beskytte mod netværksindbrud: **firewall (A)** og **indtrængningsdetektionssystem (IDS) (B)**. Sandsynligheden for, at et indbrudsforsøg opdages af firewallen, er $P(A) = 0.4$. Hvis indbruddet slipper igennem firewallen, er sandsynligheden for, at det opdages af IDS'en, $P(B \mid A^c) = 0.6$. Givet at firewallen opdager indbruddet, er den betingede sandsynlighed for, at IDS'en også opdager det, $P(B \mid A) = 0.8$.

(a) Hvad er sandsynligheden for, at et indbrud opdages af mindst ét af sikkerhedslagene?
(b) Hvad er sandsynligheden for, at et indbrud opdages af begge sikkerhedslag?

Løsning:

(a) **Sandsynligheden for at blive opdaget af mindst ét lag:**
For at finde sandsynligheden for, at indbruddet opdages af mindst ét lag, beregner vi sandsynligheden for, at et indbrud slipper uopdaget gennem begge lag, og trækker den fra 1.

$$P(\text{Uopdaget}) = P(A^c) \cdot P(B^c \mid A^c) = (1 - 0.4) \cdot (1 - 0.6) = 0.6 \cdot 0.4 = 0.24$$

Sandsynligheden for at opdage indbruddet med mindst ét lag er derfor:

$$P(\text{Opdaget}) = 1 - P(\text{Uopdaget}) = 1 - 0.24 = 0.76$$

(b) **Sandsynligheden for at blive opdaget af begge lag:**
Med multiplikationsreglen får vi:

$$P(A \cap B) = P(A) \cdot P(B \mid A) = 0.4 \cdot 0.8 = 0.32$$

Sandsynligheden for at opdage indbruddet med mindst ét lag er 0.76, mens sandsynligheden for at opdage indbruddet med både firewallen og IDS'en er 0.32. Denne forskel understreger, hvor vigtigt det er at tage højde for afhængigheder mellem hændelser, når man beregner sandsynligheder.

◀

Bemærkning: Man kunne fejlagtigt antage, at de to lag er uafhængige, og beregne $P(A \cap B) = P(A) \cdot P(B) = 0.4 \cdot 0.68 = 0.272$, hvor $P(B) = 0.4 \cdot 0.8 + 0.6 \cdot 0.6 = 0.68$. Da sandsynligheden for, at IDS'en opdager indbruddet, afhænger af firewallens resultat, bruger vi i stedet $P(B \mid A)$ og får det korrekte resultat 0.32.

I nogle situationer afhænger sandsynligheden for en hændelse af forskellige betingelser. Hvis vi kender de betingede sandsynligheder under disse forskellige scenarier, kan vi bestemme hændelsens samlede sandsynlighed. Se for eksempel på halvlederproduktion, hvor vi definerer:

- A som hændelsen, at en chip er stærkt forurenet
- B som hændelsen, at et produkt, der bruger chippen, svigter

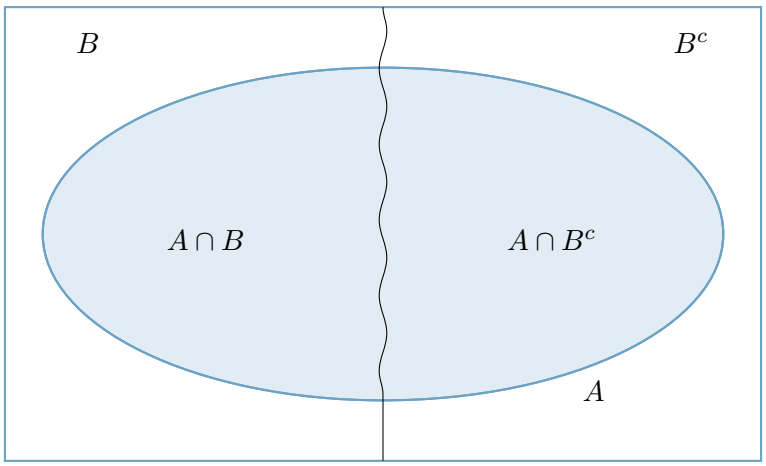
Sandsynligheden for svigt for en chip, der ikke er forurenet, er $P(B \mid A^c) = 0.005$. Hvis chippen derimod er udsat for et højt forureningsniveau, er sandsynligheden for svigt $P(B \mid A) = 0.10$. Antag, at 20% af chippene i en given produktionsserie er stærkt forurenede ($P(A) = 0.20$). Hvad er sandsynligheden for, at et produkt, der bruger en af disse chips, svigter?

Sandsynligheden for svigt afhænger af, om chippen har været udsat for høj forurening eller ej. Enhver hændelse A kan opdeles i to gensidigt udelukkende dele: én, der skærer B, og én, der skærer komplementet B^c. Matematisk kan vi udtrykke det som:

$$A = (B \cap A) \cup (B^c \cap A)$$

Denne opdeling er vist i Venn-diagrammet i Figur 5.1. Da B og B^c er gensidigt udelukkende, er deres fællesmængder med A også gensidigt udelukkende. Ved at bruge reglen for sandsynligheden for foreningsmængden af gensidigt udelukkende hændelser og multiplikationsreglen kan vi derfor udlede den totale sandsynlighed således:

$$P(A) = P(A \cap B) + P(A \cap B^c)$$



Figur 5.1: $P(A) = P(A \cap B) + P(A \cap B^c)$

Vi opsummerer dette i følgende sætning:

Sætning 5.3 (Loven om total sandsynlighed)

For vilkårlige hændelser A og B, hvor B og B^c udgør en partition af udfaldsrummet S, er den totale sandsynlighed for hændelse A givet ved:

$$\begin{aligned} P(A) &= P(A \cap B) + P(A \cap B^c) \\ &= P(B)P(A \mid B) + P(B^c)P(A \mid B^c) \end{aligned}$$

For enhver hændelse A og enhver partition $B_1, B_2, \ldots, B_n$ af udfaldsrummet, hvor $B_i \cap B_j = \emptyset$ for alle $i \neq j$ og $\bigcup_{i=1}^{n} B_i = S$, er den totale sandsynlighed for hændelse A givet ved:

$$P(A) = \sum_{i=1}^{n} P(A \mid B_i)P(B_i)$$

♡

Lad os se på nogle eksempler.

Eksempel 5.6 En virksomhed producerer to typer produkter: **produkt A** og **produkt B**. Sandsynligheden for, at et produkt er defekt, er $P(D \mid A) = 0.05$ for produkt A og $P(D \mid B) = 0.10$ for produkt B. Virksomheden fremstiller 60% af sine produkter som produkt A og 40% som produkt B. Hvad er sandsynligheden for, at et tilfældigt udvalgt produkt er defekt?

Løsning: Med loven om total sandsynlighed får vi:

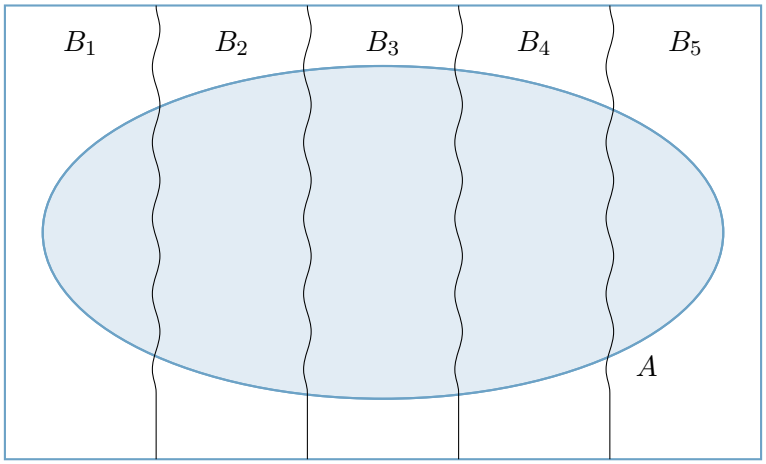
$$P(D) = P(D \mid A)P(A) + P(D \mid B)P(B)$$

Vi indsætter de givne værdier:

$$P(D) = 0.05 \times 0.60 + 0.10 \times 0.40 = 0.03 + 0.04 = 0.07$$

Sandsynligheden for, at et tilfældigt udvalgt produkt er defekt, er altså 0.07. ◀

En grafisk fremstilling af, hvordan en hændelse B opdeles på en samling af gensidigt udelukkende og udtømmende hændelser, er vist i Figur 5.2. Hændelsen A er opdelt i fem gensidigt udelukkende hændelser B_1, B_2, B_3, B_4, B_5, som tilsammen udgør udfaldsrummet S. Den totale sandsynlighed for hændelse A beregnes som summen af de betingede sandsynligheder for A givet hver del B_i i partitionen ganget med sandsynligheden $P(B_i)$ for hver del.



Figur 5.2: $P(A) = \sum_{i=1}^{5} P(A \mid B_i)P(B_i)$

Eksempel 5.7 Et universitet udbyder tre typer kurser: **online**, **hybride** og **fremmødekurser**. Sandsynligheden for, at en studerende dumper et kursus, er $P(F \mid \text{Online}) = 0.10$ for onlinekurser, $P(F \mid \text{Hybrid}) = 0.05$ for hybride kurser og $P(F \mid \text{Fremmøde}) = 0.02$ for fremmødekurser. Universitetet udbyder 50% af sine kurser online, 30% som hybride kurser og 20% som fremmødekurser. Hvad er sandsynligheden for, at en tilfældigt udvalgt studerende dumper et kursus?

Løsning: Med loven om total sandsynlighed får vi:

$$P(F) = P(F \mid \text{Online})P(\text{Online}) + P(F \mid \text{Hybrid})P(\text{Hybrid}) + P(F \mid \text{Fremmøde})P(\text{Fremmøde})$$

Vi indsætter de givne værdier:

$$P(F) = 0.10 \times 0.50 + 0.05 \times 0.30 + 0.02 \times 0.20 = 0.05 + 0.015 + 0.004 = 0.069$$

Sandsynligheden for, at en tilfældigt udvalgt studerende dumper et kursus, er altså 0.069. ◀

5.3 Uafhængighed

Lad A være hændelsen, at det regner i morgen, med $P(A) = \frac{1}{3}$. Antag desuden, at jeg kaster en fair mønt, og lad B være hændelsen, at den lander med krone opad, så $P(B) = \frac{1}{2}$.

Se nu på sandsynligheden $P(A \mid B)$. Hvad tror du, den er? Du gætter måske intuitivt på, at $P(A \mid B) = P(A) = \frac{1}{3}$, og det er korrekt! Udfaldet af møntkastet har ingen indflydelse på vejret. Det betyder, at sandsynligheden for A er uændret, uanset om B indtræffer eller ej. Scenariet illustrerer begrebet uafhængige hændelser: To hændelser er uafhængige, hvis forekomsten af den ene ikke giver nogen information om den anden.

Lad os nu formalisere definitionen af uafhængighed.

Definition 5.2 (Uafhængighed)

To hændelser A og B er uafhængige, hvis

$$P(A \cap B) = P(A)P(B).$$

Når $P(B) > 0$, er dette ensbetydende med $P(A \mid B) = P(A)$, og når $P(A) > 0$, er det ensbetydende med $P(B \mid A) = P(B)$.

♣

Sammenfattende kan uafhængighed forstås på to ækvivalente måder: »Uafhængighed betyder, at sandsynligheden for fællesmængden af to hændelser kan findes ved blot at gange deres individuelle sandsynligheder,« eller alternativt: »Uafhængighed betyder, at den betingede sandsynlighed for den ene hændelse givet den anden er den samme som den oprindelige, ubetingede sandsynlighed.«

Lemma 5.1

Hvis A og B er uafhængige, så gælder:

- A og B^c er uafhængige,
- A^c og B er uafhængige,
- A^c og B^c er uafhængige.

♡

Når vi arbejder med sandsynligheden for foreningsmængden af flere uafhængige hændelser, $A_1, A_2, \ldots, A_n$, er det ofte lettere at finde sandsynligheden for deres fællesmængde end for deres foreningsmængde. I sådanne situationer kan De Morgans lov være meget nyttig:

$$A_1 \cup A_2 \cup \cdots \cup A_n = (A_1^c \cap A_2^c \cap \cdots \cap A_n^c)^c$$

Ved hjælp af denne sammenhæng kan vi udtrykke sandsynligheden for foreningsmængden som:

$$\begin{aligned} P(A_1 \cup A_2 \cup \cdots \cup A_n) &= 1 - P(A_1^c \cap A_2^c \cap \cdots \cap A_n^c) \\ &= 1 - P(A_1^c)\,P(A_2^c) \cdots P(A_n^c) \quad \text{(da hændelserne } A_i \text{ er uafhængige)} \\ &= 1 - (1 - P(A_1))\,(1 - P(A_2)) \cdots (1 - P(A_n)). \end{aligned}$$

Sætning 5.4 (Uafhængighed og De Morgans lov)

Hvis $A_1, A_2, \cdots, A_n$ er uafhængige, så gælder

$$P(A_1 \cup A_2 \cup \cdots \cup A_n) = 1 - (1 - P(A_1))\,(1 - P(A_2)) \cdots (1 - P(A_n))$$

♡

Advarsel! En almindelig misforståelse er at forveksle uafhængighed med disjunkthed. Det er dog grundlæggende forskellige begreber. To hændelser A og B er disjunkte, hvis $A \cap B = \emptyset$. Hvis begge hændelser har positiv sandsynlighed, medfører disjunkthed afhængighed: Hvis vi ved, at den ene er indtruffet, er den anden udelukket. Undtagelsen er, når mindst én af de disjunkte hændelser har sandsynlighed nul. I det tilfælde er produktkriteriet for uafhængighed opfyldt.

Vi kan udvide begrebet uafhængighed til flere hændelser. En samling hændelser $A_1, A_2, \ldots, A_n$ betragtes som uafhængige, hvis forekomsten af en vilkårlig delmængde af hændelserne ikke giver nogen information om forekomsten af de øvrige hændelser. Matematisk udtrykkes det således:

Begreb	Beskrivelse	Centrale formler
Disjunkte	Hændelserne A og B kan ikke indtræffe samtidig	$A \cap B = \emptyset$ $P(A \cup B) = P(A) + P(B)$
Uafhængige	Forekomsten af B giver ingen information om A	$P(A \mid B) = P(A)$ $P(B \mid A) = P(B)$ $P(A \cap B) = P(A) \cdot P(B)$

Tabel 5.1: Sammenligning af disjunkte og uafhængige hændelser.

Definition 5.3 (Uafhængighed af flere hændelser)

Hændelserne $A_1, A_2, \ldots, A_k$ er indbyrdes uafhængige, hvis det for enhver delmængde $J \subseteq \{1, \ldots, k\}$ med mindst to indeks gælder, at

$$P\left(\bigcap_{j \in J} A_j\right) = \prod_{j \in J} P(A_j).$$

Det er ikke tilstrækkeligt kun at kræve lighedstegnet for fællesmængden af alle k hændelser.

♣

Eksempel 5.8 Spillerens fejlslutning

Spillerens fejlslutning er en almindelig kognitiv bias, som opstår, når man tror, at udfaldet af en tilfældig hændelse påvirkes af tidligere udfald. Fejlslutningen ses ofte i forbindelse med spil, hvor man fejlagtigt antager, at sandsynligheden for en hændelse påvirkes af tidligere hændelser.

En mand kaster en fair mønt otte gange og noterer ved hvert kast, om det giver krone (H) eller plat (T). Efter hvilken af følgende sekvenser af møntkast er det mest sandsynligt, at manden får krone (H) i sit næste kast? Denne:

TTTTTTTT

eller denne:

HHTHTTHH

Svaret er ingen af dem, som det fremgår her:

$$P\left(H_9 \mid T_1 T_2 \ldots T_8\right) = \frac{P\left(H_9 \cap T_1 \cap T_2 \cap \cdots \cap T_8\right)}{P\left(T_1 \cap T_2 \cap \cdots \cap T_8\right)} = \frac{\left(\frac{1}{2}\right)^9}{\left(\frac{1}{2}\right)^8} = \frac{1}{2}$$

Prøv at undgå at falde i spillerens fejlslutning. Hvis en fair mønt for eksempel er landet på plat otte gange i træk, er det let at tro, at krone »står for tur« i næste kast. Men hvert kast er uafhængigt, og sandsynligheden er den samme – der er stadig 50% chance for krone eller plat, uanset tidligere resultater.

5.4 Bayes' sætning

Vi er nu klar til at introducere et af de stærkeste værktøjer inden for betinget sandsynlighed: Bayes' regel (eller sætning). Reglen er særlig nyttig, når vi kender $P(A \mid B)$, men vil finde $P(B \mid A)$. Ud fra definitionen af betinget sandsynlighed har vi:

$$P(A \mid B) \cdot P(B) = P(A \cap B) = P(B \mid A) \cdot P(A)$$

Ved at dividere begge sider med $P(A)$ får vi:

$$P(B \mid A) = \frac{P(A \mid B) \cdot P(B)}{P(A)}$$

Denne formel er velkendt som Bayes' regel. I mange tilfælde har vi brug for loven om total sandsynlighed for at beregne $P(A)$ i Bayes' regel. Derfor præsenteres Bayes' regel ofte på formen:

$$P(B \mid A) = \frac{P(A \mid B) \cdot P(B)}{P(A \mid B) \cdot P(B) + P(A \mid B^c) \cdot P(B^c)}$$

eller mere generelt

$$P\left(B_j \mid A\right) = \frac{P\left(A \mid B_j\right) \cdot P\left(B_j\right)}{\sum_i P\left(A \mid B_i\right) \cdot P\left(B_i\right)}$$

hvor $B_1, B_2, \ldots, B_n$ udgør en partition af udfaldsrummet.

Sætning 5.5 (Bayes' sætning)

For to vilkårlige hændelser A og B, hvor $P(A) \neq 0$, har vi

$$P(B \mid A) = \frac{P(A \mid B) \cdot P(B)}{P(A)} = \frac{P(A \mid B) \cdot P(B)}{P(A \mid B) \cdot P(B) + P(A \mid B^c) \cdot P(B^c)}$$

Hvis $B_1, B_2, B_3, \cdots$ udgør en partition af udfaldsrummet S, og A er en vilkårlig hændelse med $P(A) \neq 0$, har vi

$$P\left(B_j \mid A\right) = \frac{P\left(A \mid B_j\right) \cdot P\left(B_j\right)}{\sum_i P\left(A \mid B_i\right) \cdot P\left(B_i\right)}$$

♡

Vi begynder med et berygtet eksempel, som fremhæver betydningen af Bayes' sætning.

Eksempel 5.9 Det falsk positive paradoks

Forestil dig en sjælden sygdom, som rammer omkring 1 ud af hver 10 000 personer. Der findes en test til at påvise sygdommen, og selv om testen er meget nøjagtig, er den ikke perfekt. Konkret gælder:

- Sandsynligheden for, at testen viser et positivt resultat (som tyder på sygdom), når personen ikke har sygdommen, er 2%.
- Sandsynligheden for, at testen viser et negativt resultat (som tyder på, at der ikke er sygdom), når personen har sygdommen, er 1%.

Antag nu, at en tilfældigt udvalgt person bliver testet, og at resultatet er positivt. Hvad er sandsynligheden for, at personen faktisk har sygdommen?

Løsning: Lad D være hændelsen, at personen har sygdommen, lad T^+ være hændelsen, at testresultatet er

positivt, og lad T^- være hændelsen, at det er negativt. Vi har fået oplyst:

$$P(D) = \frac{1}{10\,000}$$
$$P\left(T^+ \mid D^c\right) = 0.02 \quad \text{(Falsk positiv-rate)}$$
$$P\left(T^- \mid D\right) = 0.01 \quad \text{(Falsk negativ-rate)}$$

Vi vil beregne $P(D \mid T^+)$. Først har vi brug for $P(T^+ \mid D)$, som er sandsynligheden for et sandt positivt resultat. Da en person med sygdommen enten testes positiv eller negativ, har vi:

$$P(T^+ \mid D) = 1 - P(T^- \mid D) = 1 - 0.01 = 0.99$$

Med Bayes' regel får vi nu:

$$\begin{aligned} P(D \mid T^+) &= \frac{P(T^+ \mid D)P(D)}{P(T^+ \mid D)P(D) + P\left(T^+ \mid D^c\right)P\left(D^c\right)} \\ &= \frac{0.99 \times 0.0001}{0.99 \times 0.0001 + 0.02 \times (1 - 0.0001)} \\ &= \frac{0.000099}{0.000099 + 0.02 \times 0.9999} \\ &= \frac{0.000099}{0.000099 + 0.019998} \\ &\approx 0.0049 \end{aligned}$$

Resultatet betyder, at der er mindre end en halv procents chance for, at personen faktisk har sygdommen. Trods det positive testresultat er det sygdommens lave forekomst og testens falsk positiv-rate, der fører til dette kontraintuitive resultat. ◀

Eksempel 5.10 Bayesianske netværk bruges ofte på højteknologiske producenters websteder for at hjælpe kunder med hurtigt at diagnosticere problemer med deres produkter. En printerproducent bruger for eksempel data fra testresultater til at finde mulige årsager til printerfejl. Printerfejl hænger primært sammen med tre typer problemer: hardware, software og andre problemer (som stik og forbindelser). Sandsynlighederne for disse problemer er som følger:

- Sandsynligheden for et hardwareproblem: $P(H) = 0.1$
- Sandsynligheden for et softwareproblem: $P(S) = 0.6$
- Sandsynligheden for en anden type problem: $P(O) = 0.3$

Sandsynligheden for, at en printer fejler, givet hver type problem, er:

- Sandsynligheden for fejl givet et hardwareproblem: $P(F \mid H) = 0.9$
- Sandsynligheden for fejl givet et softwareproblem: $P(F \mid S) = 0.2$
- Sandsynligheden for fejl givet et andet problem: $P(F \mid O) = 0.5$

Givet at en kunde oplever en printerfejl og bruger producentens websted til at diagnosticere problemet, hvad er så den mest sandsynlige årsag til problemet?

Løsning: For at bestemme den mest sandsynlige årsag til printerfejlen skal vi beregne sandsynligheden for hver type problem, givet at der er opstået en fejl. Det gør vi ved at bruge Bayes' sætning til at beregne a posteriori-sandsynlighederne.

Lad F betegne hændelsen, at printeren fejler. Vi vil finde:

$$P(H \mid F), \quad P(S \mid F), \quad P(O \mid F)$$

Trin 1: Beregn den totale sandsynlighed for fejl

Først bruger vi loven om total sandsynlighed til at finde $P(F)$:

$$P(F) = P(F \mid H) \cdot P(H) + P(F \mid S) \cdot P(S) + P(F \mid O) \cdot P(O)$$

Vi indsætter de givne værdier:

$$P(F) = (0.9 \times 0.1) + (0.2 \times 0.6) + (0.5 \times 0.3)$$
$$P(F) = 0.09 + 0.12 + 0.15 = 0.36$$

Trin 2: Brug Bayes' sætning til at finde a posteriori-sandsynlighederne

Brug nu Bayes' sætning for hver type problem:

1. Sandsynligheden for et hardwareproblem givet fejl:

$$P(H \mid F) = \frac{P(F \mid H) \cdot P(H)}{P(F)} = \frac{0.9 \times 0.1}{0.36} = \frac{0.09}{0.36} = 0.25$$

2. Sandsynligheden for et softwareproblem givet fejl:

$$P(S \mid F) = \frac{P(F \mid S) \cdot P(S)}{P(F)} = \frac{0.2 \times 0.6}{0.36} = \frac{0.12}{0.36} = 0.3333$$

3. Sandsynligheden for andre problemer givet fejl:

$$P(O \mid F) = \frac{P(F \mid O) \cdot P(O)}{P(F)} = \frac{0.5 \times 0.3}{0.36} = \frac{0.15}{0.36} = 0.4167$$

Trin 3: Fortolk resultaterne

- Hardwareproblem: 25% chance
- Softwareproblem: cirka 33.33% chance
- Andet problem: cirka 41.67% chance

Konklusion: Givet en printerfejl er den mest sandsynlige årsag et andet problem, for eksempel stik og forbindelser, med en sandsynlighed på cirka 41.67%.

◀

Kapitel 6 Beskrivende statistik

I softwareudvikling arbejder vi konstant med data: ydelsesmålinger, mønstre i brugeradfærd, systemers svartider og mål for kodekompleksitet. For at forstå disse oplysninger og uddrage meningsfuld indsigt har vi brug for værktøjer til at opsummere, organisere og visualisere data. Beskrivende statistik giver os disse grundlæggende værktøjer, så vi kan forstå vores datas egenskaber uden at drage slutninger om større populationer.

Forestil dig et softwareudviklingsteam, der analyserer ydelsen af sin webapplikation. Teamet indsamler svartider for 1000 brugerforespørgsler og finder værdier fra 50 millisekunder til 3.2 sekunder. Uden en ordentlig analyse er disse rådata overvældende og siger ikke meget. Ved at anvende beskrivende statistik kan teamet derimod fastslå, at den gennemsnitlige svartid er 450 millisekunder, at 75% af forespørgslerne er afsluttet inden for 600 millisekunder, og at der er nogle få usædvanligt langsomme forespørgsler, som kan tyde på ydelsesproblemer.

Dette kapitel introducerer de grundlæggende begreber i beskrivende statistik med fokus på mål, der hjælper os med at forstå centrale tendenser, variation og fordelingsmønstre i vores data. Vi undersøger, hvordan disse teknikker anvendes specifikt i softwareudvikling, fra analyse af algoritmers ydelse til forståelse af mønstre i brugeradfærd.

6.1 Introduktion til beskrivende statistik

Nogle gange begynder statistisk arbejde med eksisterende data, fx nedbørsmålinger, ledighedsprocenter eller BNP-tal, som vi derefter opsummerer og analyserer. I andre situationer skal data frembringes gennem et eksperiment eller en undersøgelse. Hvis en underviser fx vil sammenligne to undervisningsmetoder i et introducerende programmeringskursus, kan underviseren tilfældigt dele de studerende i to grupper, anvende en forskellig metode i hver gruppe og derefter sammenligne testresultaterne. Den tilfældige fordeling er afgørende: Den sikrer, at forskelle mellem grupperne ikke skyldes forhold, der fandtes i forvejen, fx evner, men derimod selve undervisningsmetoden.

Når data er indsamlet, bliver de opsummeret og visualiseret – for eksempel ved at angive det gennemsnitlige resultat for hver gruppe. Denne proces er kernen i **beskrivende statistik**: at organisere, præsentere og beskrive data på en meningsfuld måde.

Population og stikprøve

I statistisk analyse skelner vi mellem to centrale begreber:

Definition 6.1 (Population)

En **population** er den fuldstændige mængde af alle mulige observationer eller målinger, der er af interesse i en bestemt undersøgelse. I softwareudvikling kan det være alle mulige køretider for en algoritme, alle brugersessioner på et websted eller alle kodelinjer i et projekt. ♣

Definition 6.2 (Stikprøve)

En **stikprøve** er en delmængde af populationen, som vi faktisk observerer eller måler. På grund af praktiske begrænsninger arbejder vi ofte med stikprøver i stedet for hele populationer. ♣

I eksemplet med undervisningsmetoderne udgør de studerende i klasselokalet kun en *stikprøve* af den større *population* af potentielle studerende. Beskrivende statistik hjælper os med at forstå selve stikprøven, mens inferentielle metoder senere gør det muligt at bruge stikprøven til at drage konklusioner om den bredere population. Inferentiel statistik ligger uden for denne bogs rammer. I dette kapitel fokuserer vi på beskrivende statistik.

Eksempel 6.1 Analyse af softwareydelse

Se på en webapplikation, der betjener millioner af brugere dagligt. **Populationen** vil være svartiderne for alle mulige brugerforespørgsler. På grund af begrænsninger i beregnings- og lagerkapacitet indsamler vi dog måske kun svartider for en **stikprøve** på 10 000 forespørgsler om dagen. Stikprøven bør være repræsentativ for populationen, hvis vi skal kunne drage meningsfulde konklusioner.

Datatyper

Data kan inddeles i forskellige typer, som hver kræver forskellige statistiske tilgange:

Definition 6.3 (Kvalitative data)

Kvalitative data (også kaldet kategoriske data) består af ikke-numeriske oplysninger, der kan inddeles i kategorier. Eksempler er de programmeringssprog, der bruges i et projekt, brugertilfredshed (tilfreds/neutral/utilfreds) eller fejls alvorlighedsgrad (kritisk/høj/middel/lav).

♣

Den væsentlige forskel mellem **kvalitative** og **kvantitative** data ligger i, hvad de repræsenterer, og hvordan de analyseres. Kvalitative data er kategorier eller egenskaber, der ikke meningsfuldt kan udtrykkes som tal. Sådanne data bruges til at klassificere eller mærke elementer, fx programmeringssprog eller fejls alvorlighedsgrad. Kvantitative data består derimod af numeriske værdier, der repræsenterer antal eller målinger, fx svartider eller antal brugere. Fordi disse værdier er numeriske, kan man udføre matematiske operationer og statistisk analyse på dem. Det er grundlæggende at kunne skelne mellem kvalitative og kvantitative data, da det styrer valget af statistiske metoder og af, hvilke typer visualiseringer der er egnede til analysen.

Definition 6.4 (Kvantitative data)

Kvantitative data består af numeriske målinger, der kan ordnes og underkastes matematiske operationer. De kan yderligere opdeles i:

- **Diskrete**: Tællelige værdier (antal fejl, kodelinjer, brugersessioner)
- **Kontinuerte**: Målbare værdier, der kan antage enhver værdi inden for et interval (svartider, hukommelsesforbrug, CPU-udnyttelse)

♣

Eksempel 6.2

- **Kvalitative**: Programmeringssprog (Python, Java, C++), driftsmiljø (udvikling, staging, produktion)
- **Kvantitative, diskrete**: Antal commits pr. dag, kodelinjer, antal testtilfælde
- **Kvantitative, kontinuerte**: Svartid i millisekunder, hukommelsesforbrug i MB, CPU-udnyttelse i procent

Outliers og ekstreme data

I ethvert datasæt har de fleste værdier en tendens til at samle sig i et centralt område, men af og til møder vi værdier, der er meget større eller mindre end resten. De kaldes **outliers** eller **ekstremværdier**. Outliers kan opstå af mange grunde: målefejl, usædvanlige, men gyldige hændelser eller naturlig variation i data.

Hvis de fleste webforespørgsler fx er afsluttet på under 400 millisekunder, men én forespørgsel tager 1200 millisekunder, er værdien 1200 ms en outlier. Outliers kan have stor indflydelse på opsummerende statistikker som gennemsnittet, så data ser ud til at variere mere, eller så gennemsnittet flyttes væk fra det sted, hvor de fleste værdier ligger.

Det er vigtigt at holde øje med outliers, når man analyserer data, da de kan være tegn på interessante fænomener (fx en sjælden flaskehals i ydelsen), indtastningsfejl eller et behov for nærmere undersøgelse. Der findes formelle metoder til at finde outliers, som vi gennemgår senere, men ofte er et simpelt plot eller et blik på de sorterede data nok til at få øje på værdier, der »skiller sig ud« fra resten.

I praksis er det afgørende at forstå sammenhængen: Nogle gange er outliers fejl, der skal rettes eller fjernes, men andre gange er de den vigtigste del af historien.

6.2 Beskrivelse af datasæt

Når man præsenterer numeriske resultater, er klarhed og kortfattethed afgørende. Tabeller og grafer er særligt effektive, fordi de lader læseren hurtigt få fat i et datasæts vigtigste egenskaber. Visuelle opsummeringer og tabeller fremhæver ofte vigtige forhold som den samlede variationsbredde, hvor tæt værdierne er samlet, og om data ser symmetriske eller skæve ud.

Hyppighedstabeller og grafer

Det første skridt i en dataanalyse er ofte at organisere rådata i en hyppighedsfordeling, som viser, hvor ofte hver værdi (eller hvert interval af værdier) forekommer.

Definition 6.5 (Hyppighedsfordeling)

En **hyppighedsfordeling** er en tabel, der viser hyppigheden (antallet) af hver værdi eller klasse af værdier i et datasæt. Den kan angives som:

- **Absolut hyppighed**: Det faktiske antal forekomster
- **Relativ hyppighed**: Andelen af det samlede antal observationer
- **Kumuleret hyppighed**: Den løbende sum af hyppighederne

♣

Eksempel 6.3 Et datasæt med et forholdsvis lille antal forskellige værdier kan bekvemt præsenteres i en hyppighedstabel. Se fx på et datasæt med de månedlige startlønninger (afrundet til nærmeste tusind danske kroner) for 42 nyuddannede med en bachelorgrad i softwareudvikling.

Rådata er som følger:

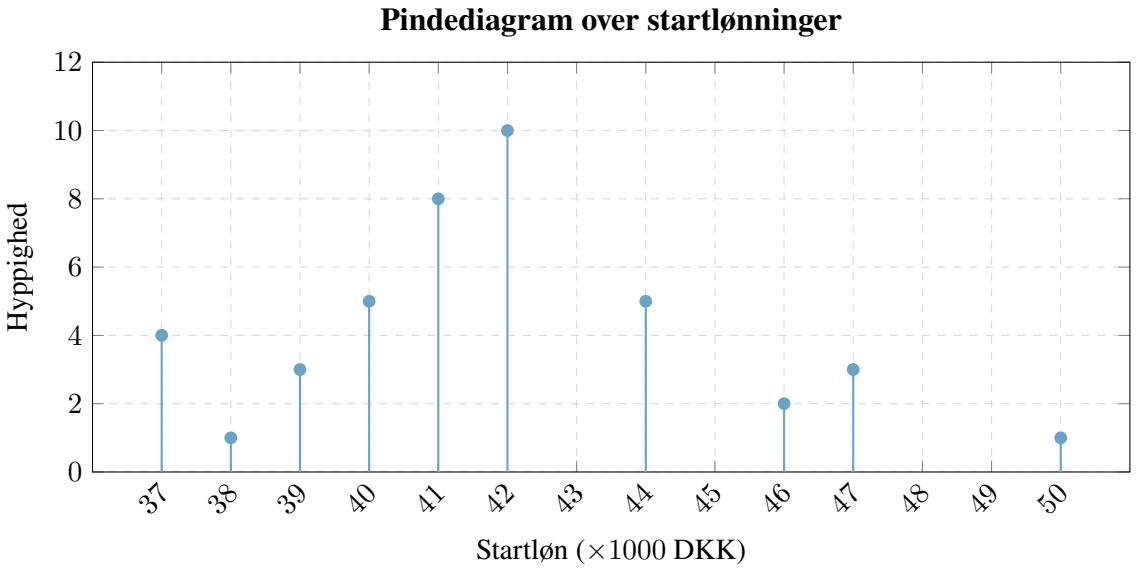
$$37, 37, 37, 37, 38, 39, 39, 39, 40, 40, 40, 40, 40, 41, 41, 41, 41, 41, 41, 41, 41,$$
$$42, 42, 42, 42, 42, 42, 42, 42, 42, 42, 44, 44, 44, 44, 44, 46, 46, 47, 47, 47, 50$$

Ud fra disse data kan vi opstille en hyppighedstabel. Tabel 6.1 fortæller os blandt andet, at den laveste startløn på 37 000 DKK blev givet til fire af de nyuddannede, mens den højeste løn på 50 000 DKK kun blev givet til én. Den hyppigste startløn var 42 000 DKK, som 10 af de nyuddannede fik.

Løn ($\times 1000$ **DKK**)	**Hyppighed**
37	4
38	1
39	3
40	5
41	8
42	10
44	5
46	2
47	3
50	1

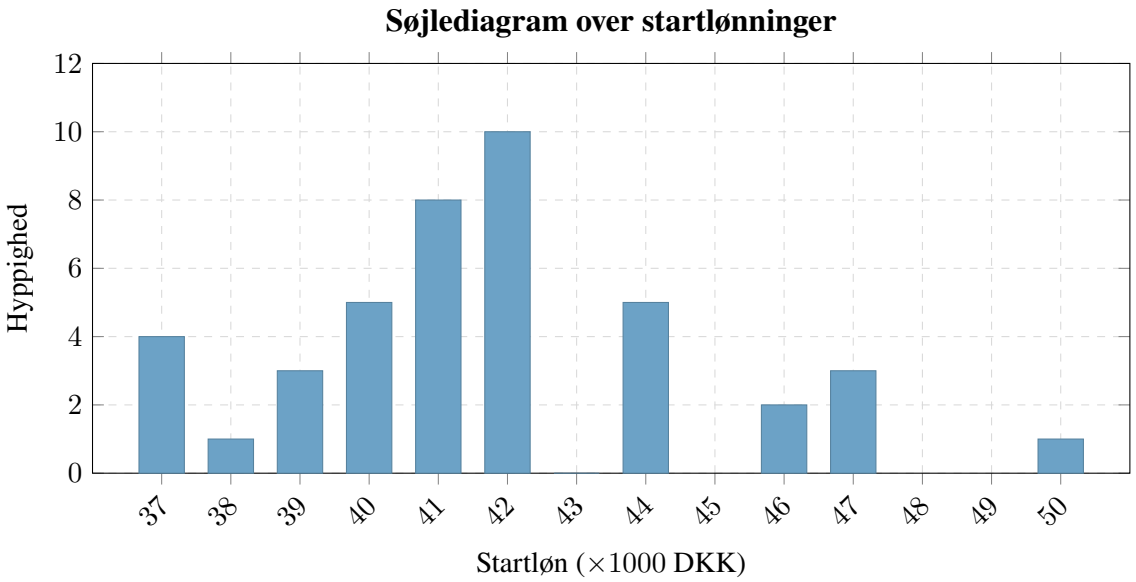
Tabel 6.1: Hyppighedsfordeling af månedlige startlønninger

Data fra en hyppighedstabel kan afbildes grafisk i et pindediagram, der afsætter de forskellige dataværdier på den vandrette akse og angiver deres hyppigheder med højden af lodrette linjer. Et pindediagram over dataene i Tabel 6.1 er vist i Figur 6.1.



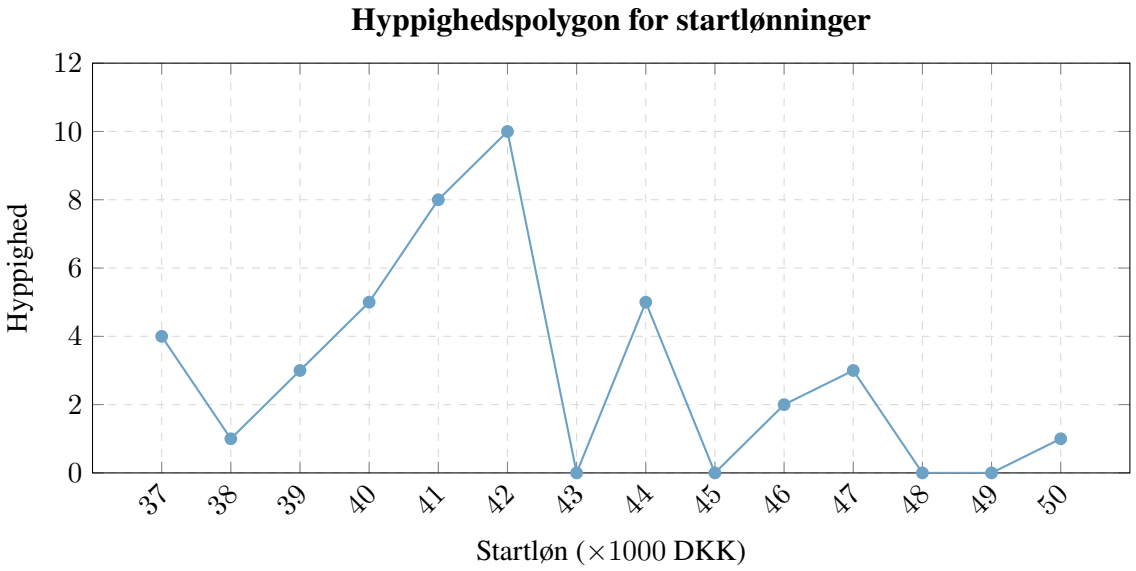
Figur 6.1: Et pindediagram, der viser hyppigheden af de forskellige startlønninger.

Når linjerne i et pindediagram gøres tykkere, kaldes grafen et søjlediagram. Figur 6.2 viser et søjlediagram over de samme lønninger.



Figur 6.2: Et søjlediagram over lønningerne.

En anden type graf, der bruges til at afbilde en hyppighedstabel, er hyppighedspolygonen, som afsætter hyppighederne af de forskellige dataværdier på den lodrette akse og derefter forbinder de afsatte punkter med rette linjestykker. Figur 6.3 viser en hyppighedspolygon for de samme lønninger.



Figur 6.3: En hyppighedspolygon for lønningerne, der forbinder punkterne fra hyppighedstabellen.

Nogle gange foretrækker vi den relative hyppighed frem for den absolutte hyppighed. Det er særligt nyttigt ved store datasæt, og når man vil sammenligne værdier fra datasæt af forskellig størrelse. En udvidelse af den relative hyppighed er den kumulerede relative hyppighed, som er den løbende sum af de relative hyppigheder. Tabel 6.2 viser hyppigheden, den relative hyppighed og den kumulerede relative hyppighed for lønningerne.

Løn	Hyppighed	Relativ hyppighed	Kumuleret rel. hyppighed
37	4	$4/42 \approx 0.095$	$4/42 \approx 0.095$
38	1	$1/42 \approx 0.024$	$5/42 \approx 0.119$
39	3	$3/42 \approx 0.071$	$8/42 \approx 0.190$
40	5	$5/42 \approx 0.119$	$13/42 \approx 0.310$
41	8	$8/42 \approx 0.190$	$21/42 = 0.500$
42	10	$10/42 \approx 0.238$	$31/42 \approx 0.738$
44	5	$5/42 \approx 0.119$	$36/42 \approx 0.857$
46	2	$2/42 \approx 0.048$	$38/42 \approx 0.905$
47	3	$3/42 \approx 0.071$	$41/42 \approx 0.976$
50	1	$1/42 \approx 0.024$	$42/42 = 1.000$

Tabel 6.2: Hyppighed, relativ hyppighed og kumuleret relativ hyppighed af månedlige startlønninger.

Grupperede data og histogrammer

Som vi så i Afsnit 6.2, er pindediagrammer og søjlediagrammer nyttige til at vise dataværdiers hyppigheder. Når et datasæt indeholder mange forskellige værdier, som pærernes levetider i Tabel 6.3, bliver det upraktisk at lave en hyppighedstabel for hver enkelt værdi. En mere effektiv fremgangsmåde er at gruppere data i et antal **klasseintervaller**, som også ofte kaldes **intervaller** (på engelsk *bins*).

Levetider (i timer)									
1067	919	1196	785	1126	936	918	1156	920	948
855	1092	1162	1170	929	950	905	972	1035	1045
1157	1195	1195	1340	1122	938	970	1237	956	1102
1022	978	832	1009	1157	1151	1009	765	958	902
923	1333	811	1217	1085	896	958	1311	1037	702
521	933	928	1153	946	858	1071	1069	830	1063
930	807	954	1063	1002	909	1077	1021	1062	1157
999	932	1035	944	1049	940	1122	1115	833	1320
901	1324	818	1250	1203	1078	890	1303	1011	1102
996	780	900	1106	704	621	854	1178	1138	951
1187	1067	1118	1037	958	760	1101	949	992	966
824	653	980	935	878	934	910	1058	730	980
844	814	1103	1000	788	1143	935	1069	1170	1067
1037	1151	863	990	1035	1112	931	970	932	904
1026	1147	883	867	990	1258	1192	922	1150	1091
1039	1083	1040	1289	699	1083	880	1029	658	912
1023	984	856	924	801	1122	1292	1116	880	1173
1134	932	938	1078	1180	1106	1184	954	824	529
998	996	1133	765	775	1105	1081	1171	705	1425
610	916	1001	895	709	860	1110	1149	972	1002

Tabel 6.3: Levetid i timer for 200 glødepærer.

Valget af, hvor mange intervaller man skal bruge, er en afvejning. Med for få intervaller risikerer vi at miste

vigtig information, fordi for mange forskellige værdier bliver grupperet sammen. Med for mange intervaller kan hyppighederne i hvert interval blive så små, at de ikke afslører noget tydeligt mønster i datas fordeling.

Det optimale antal intervaller er et subjektivt valg, der afhænger af datasættet, men det er typisk at vælge 5 til 10 intervaller. Det er også almindeligt, men ikke påkrævet, at bruge lige brede intervaller, da det ofte gør fortolkningen lettere.

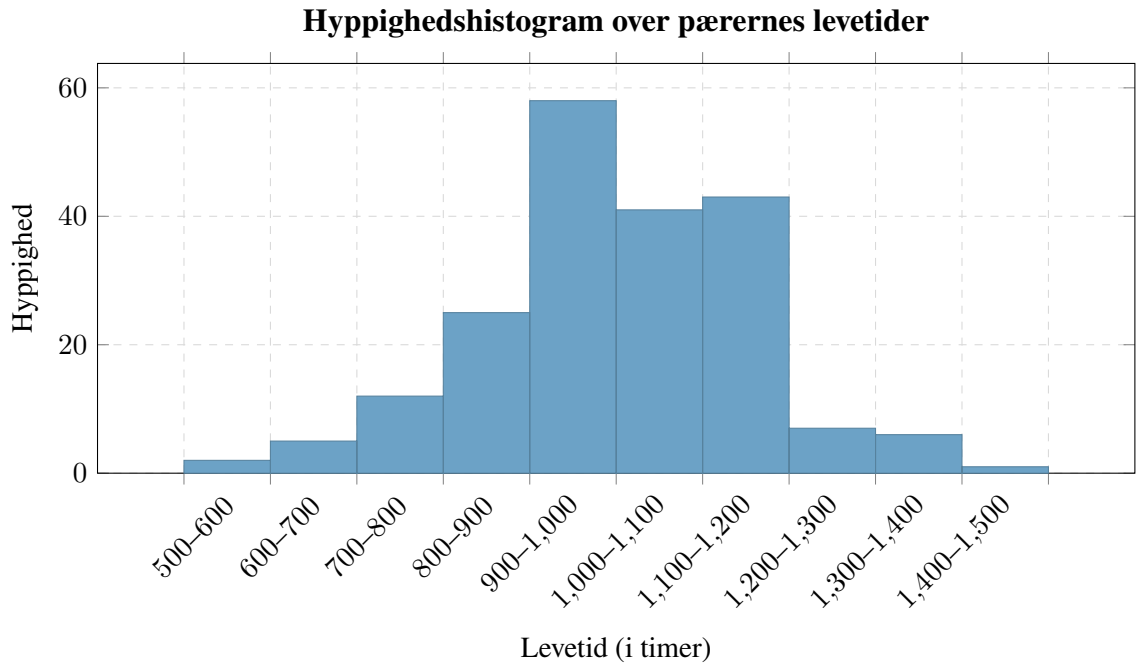
Et intervals endepunkter kaldes dets **klassegrænser**. Vi følger den almindelige **konvention om inklusion af venstre endepunkt**, som fastlægger, at et interval indeholder sin venstre grænse, men ikke sin højre. Intervallet 700–800 indeholder for eksempel enhver værdi x, hvor $700 \leq x < 800$.

Med denne fremgangsmåde er rådata fra Tabel 6.3 opsummeret i Tabel 6.4 med intervaller af længden 100, startende ved 500.

Klasseinterval	Hyppighed	Relativ hyppighed	Kumuleret rel. hyppighed
500–600	2	0.010	0.010
600–700	5	0.025	0.035
700–800	12	0.060	0.095
800–900	25	0.125	0.220
900–1000	58	0.290	0.510
1000–1100	41	0.205	0.715
1100–1200	43	0.215	0.930
1200–1300	7	0.035	0.965
1300–1400	6	0.030	0.995
1400–1500	1	0.005	1.000

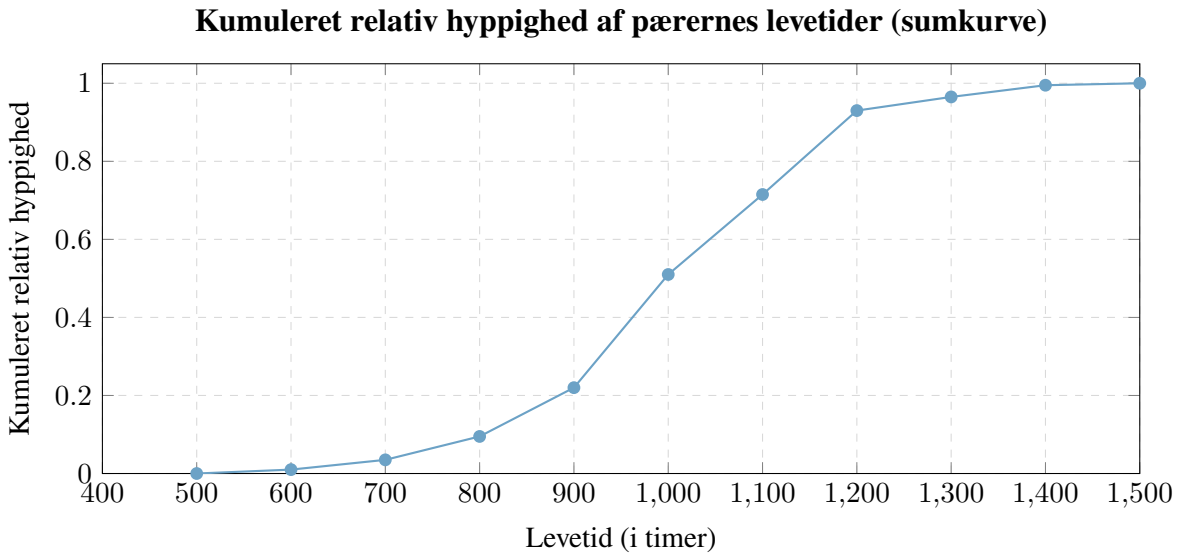
Tabel 6.4: Hyppighedsfordeling for levetiderne af 200 glødepærer.

Et søjlediagram, der viser hyppighederne af data grupperet i intervaller, kaldes et **histogram**. Et vigtigt kendetegn ved et histogram er, at søjlerne står direkte op ad hinanden, hvilket afspejler, at intervallerne er sammenhængende. Den lodrette akse kan angive enten den absolutte hyppighed (antallet af værdier i hvert interval) eller den relative hyppighed. Det første kaldes et **hyppighedshistogram**, det sidste et **relativt hyppighedshistogram**. Figur 6.4 viser et hyppighedshistogram for pærernes levetider.



Figur 6.4: Et hyppighedshistogram for pærernes levetider. De tilstødende søjler viser, hvordan data fordeler sig på sammenhængende intervaller.

Endelig er vi nogle gange interesserede i at afbilde den kumulerede hyppighed eller, oftere, den kumulerede relative hyppighed. En sådan graf, der ofte kaldes en **sumkurve** (på engelsk *ogive*), viser antallet eller andelen af datapunkter, der ligger på eller under en bestemt værdi.



Figur 6.5: En sumkurve, der viser den kumulerede relative hyppighed for pærernes levetider.

På en sumkurve repræsenterer hvert punkt på den vandrette akse en dataværdi, mens det tilsvarende punkt på den lodrette akse viser andelen af data, der er mindre end eller lig med denne værdi. Figur 6.5 viser fx den kumulerede relative hyppighed for pærernes levetider. Ud fra grafen kan vi hurtigt se, at omkring 50% af pærerne havde en levetid på under 1000 timer, at cirka 95% havde en levetid på under 1200 timer (den faktiske værdi er 93%), og at 100% af pærerne var gået i stykker efter 1500 timer.

6.3 Opsummering af data

Vi har set, hvordan man kan opsummere datafordelinger visuelt og numerisk. Nu fokuserer vi på to grundlæggende aspekter af beskrivelsen af et datasæt: dets *centrum* (eller typiske værdi) og dets *spredning* (eller variation).

Centralmål

Gennemsnittet (også kaldet middelværdien) er det mest anvendte centralmål. Definition 6.6 viser, hvordan man beregner et datasæts gennemsnit, og Eksempel 6.4 giver et eksempel.

Definition 6.6 (Aritmetisk gennemsnit)

For et datasæt med n værdier $x_1, x_2, \ldots, x_n$ er det **aritmetiske gennemsnit** defineret som:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i = \frac{x_1 + x_2 + \cdots + x_n}{n}$$

♣

Eksempel 6.4 Analyse af svartider

En webapplikation har registreret følgende svartider (i millisekunder) for 10 forespørgsler:

$$\{120, 150, 180, 200, 220, 250, 280, 300, 350, 400\}$$

Den gennemsnitlige svartid er:

$$\bar{x} = \frac{120 + 150 + 180 + 200 + 220 + 250 + 280 + 300 + 350 + 400}{10} = \frac{2450}{10} = 245 \text{ millisekunder}$$

Gennemsnittet inddrager alle værdier i et datasæt, hvilket gør det til et stærkt centralmål. Det er mest velegnet, når dataværdierne er af nogenlunde samme størrelsesorden, og når fordelingen er rimeligt symmetrisk uden ekstreme outliers. Under disse betingelser giver gennemsnittet en pålidelig opsummering af datasættet.

I analysen af webydelse giver den gennemsnitlige svartid fx et samlet billede af, hvor hurtigt et system i gennemsnit svarer. Hvis nogle få svartider er meget langsommere end resten, kan gennemsnittet dog give et misvisende indtryk af den typiske brugeroplevelse.

Ud over sin rolle som opsummerende statistik er gennemsnittet også grundlaget for mange statistiske metoder. Det er især centralt i definitionerne af varians og standardafvigelse, som måler datas spredning omkring gennemsnittet.

Bemærkning: Som nævnt tidligere, og som vi vender tilbage til, er gennemsnittet meget følsomt over for ekstremværdier eller outliers. Selv en enkelt usædvanligt stor eller lille observation kan flytte gennemsnittet betydeligt, så det i mindre grad afspejler datasættets typiske værdi. Det er derfor vigtigt at se på datas fordeling, før man udelukkende bruger gennemsnittet til at beskrive et datasæts centrum.

Medianen er den midterste værdi, når data er ordnet i stigende rækkefølge. Den er mindre følsom over for outliers end gennemsnittet.

Definition 6.7 (Median)

For et datasæt med n værdier ordnet i stigende rækkefølge er medianen defineret som:

$$\text{median} = \begin{cases} x_{\frac{n+1}{2}} & \text{hvis } n \text{ er ulige} \\ \frac{x_{\frac{n}{2}} + x_{\frac{n}{2}+1}}{2} & \text{hvis } n \text{ er lige} \end{cases}$$

♣

Eksempel 6.5 Median svartid

Vi bruger de samme svartider som i det foregående eksempel:

$$\{120, 150, 180, 200, 220, 250, 280, 300, 350, 400\}$$

Da $n = 10$ (lige), er medianen gennemsnittet af den 5. og 6. værdi:

$$\text{Median} = \frac{220 + 250}{2} = 235 \text{ millisekunder}$$

Eksempel 6.6

Se på de samme data med en ekstremværdi:

$$\{120, 150, 180, 200, 220, 250, 280, 300, 350, 2000\}$$

Gennemsnittet bliver: $\bar{x} = \frac{4050}{10} = 405$ millisekunder (stærkt påvirket af outlieren)

Medianen forbliver: $\frac{220+250}{2} = 235$ millisekunder (upåvirket af outlieren)

Fordi medianen deler et datasæt i to lige store halvdele, er den et robust centralmål, især når data indeholder outliers eller er skæve. I modsætning til gennemsnittet, som kan trækkes mod ekstremværdier, er medianen stort set upåvirket og giver i sådanne situationer et mere pålideligt billede af den »typiske« værdi.

Typetallet er den værdi, der forekommer oftest i et datasæt.

Definition 6.8 (Typetal)

Typetallet er den værdi, der forekommer hyppigst i et datasæt. Et datasæt kan have:

- **Intet typetal**: Hvis alle værdier forekommer lige hyppigt
- **Ét typetal (unimodalt)**: Hvis én værdi forekommer hyppigst
- **Flere typetal (multimodalt)**: Hvis to eller flere værdier deler den højeste hyppighed

♣

Når et datasæt har to typetal, kaldes det nogle gange et **bimodalt** datasæt.

Eksempel 6.7

Et softwareprojekt har følgende alvorlighedsgrader for fejl:

$$\{\text{Høj}, \text{Middel}, \text{Lav}, \text{Høj}, \text{Kritisk}, \text{Middel}, \text{Høj}, \text{Lav}, \text{Høj}, \text{Middel}\}$$

Optælling af hyppigheder:

- Høj: 4 forekomster
- Middel: 3 forekomster
- Lav: 2 forekomster
- Kritisk: 1 forekomst

Typetallet er »Høj«, da denne værdi forekommer hyppigst.

Typetallet er især nyttigt ved analyse af kategoriske eller diskrete data, hvor det ikke altid giver mening at beregne et gennemsnit eller en median. I softwareudvikling kan typetallet fx hjælpe med at finde den fejlkode, som et API oftest returnerer, det mest anvendte programmeringssprog i en kodebase eller den hyppigst rapporterede fejltype i et issue-tracking-system.

Hvornår bruges hvilket mål?

Mål	Bedst til	Begrænsninger
Gennemsnit	Kontinuerte data, symmetriske fordelinger	Følsomt over for outliers
Median	Data med outliers, skæve fordelinger	Mindre informativ ved symmetriske data
Typetal	Kategoriske data, diskrete data	Findes ikke altid eller er ikke entydigt

Tabel 6.5: Sammenligning af centralmål

Spredningsmål

Mens centralmål fortæller os om den typiske værdi, beskriver **spredningsmål**, hvor spredte data er. I softwareudvikling er det afgørende at forstå variation, når man skal vurdere konsistens og pålidelighed.

Variationsbredden er det simpleste spredningsmål.

Definition 6.9 (Variationsbredde)

Et datasæts **variationsbredde** er forskellen mellem den største og den mindste værdi:

$$\text{Variationsbredde} = x_{\max} - x_{\min}$$

Eksempel 6.8

For svartiderne: $\{120, 150, 180, 200, 220, 250, 280, 300, 350, 400\}$

$$\text{Variationsbredde} = 400 - 120 = 280 \text{ millisekunder}$$

Bemærkning: Variationsbredden er følsom over for outliers og giver ingen information om, hvordan værdierne er fordelt mellem yderpunkterne.

Variansen og **standardafvigelsen** er mere avancerede mål, der tager hensyn til alle datapunkter.

Definition 6.10 (Stikprøvevarians og standardafvigelse)

For en stikprøve med n værdier er **stikprøvevariansen**, betegnet s^2, defineret som:

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

Stikprøvestandardafvigelsen, betegnet s, er kvadratroden af variansen:

$$s = \sqrt{s^2} = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

Mens gennemsnittet angiver et datasæts centrum, måler **variansen** og **standardafvigelsen** dets variation eller spredning. En lille standardafvigelse betyder, at datapunkterne ligger tæt samlet omkring gennemsnittet, mens en stor standardafvigelse betyder, at de er mere spredte.

Stikprøvevariansen er en skaleret sum af de kvadrerede afvigelser fra gennemsnittet med divisoren $n - 1$. Kvadreringen af hver afvigelse tjener to formål: Den gør alle bidrag ikke-negative, og den giver større afvigelser større vægt. Beregningen efterlader dog variansen i kvadrerede enheder (fx millisekunder i anden), som ikke er intuitive at fortolke.

Det løser vi ved at bruge standardafvigelsen, som blot er kvadratroden af variansen. Dette afgørende skridt bringer målet tilbage til datas oprindelige enheder (fx millisekunder), så det kan fortolkes direkte. Kort sagt repræsenterer standardafvigelsen et datapunkts typiske afstand fra gennemsnittet.

Eksempel 6.9 Beregning af standardafvigelse

For svartiderne: $\{120, 150, 180, 200, 220, 250, 280, 300, 350, 400\}$ med $\bar{x} = 245$:

$$\begin{aligned} s^2 &= \frac{1}{9}[(120-245)^2 + (150-245)^2 + \cdots + (400-245)^2] \\ &= \frac{1}{9}[(-125)^2 + (-95)^2 + \cdots + (155)^2] \\ &= \frac{1}{9}[15625 + 9025 + \cdots + 24025] \\ &= \frac{1}{9} \times 70850 \approx 7872.22 \end{aligned}$$

$s = \sqrt{7872.22} \approx 88.73$ millisekunder

Følgende algebraiske identitet er ofte nyttig, når man beregner varians eller standardafvigelse i hånden:

Vi tager udgangspunkt i definitionen:

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

Ved hjælp af identiteten

$$\sum_{i=1}^{n}(x_i - \bar{x})^2 = \sum_{i=1}^{n} x_i^2 - \frac{\left(\sum_{i=1}^{n} x_i\right)^2}{n}$$

kan vi omskrive stikprøvevariansen til

$$s^2 = \frac{1}{n-1}\left(\sum_{i=1}^{n} x_i^2 - \frac{\left(\sum_{i=1}^{n} x_i\right)^2}{n}\right) = \frac{1}{n-1}\left(\sum_{i=1}^{n} x_i^2 - n\bar{x}^2\right)$$

Dette kaldes **regneformlen for varians**, da den gør det muligt at beregne s^2 direkte ud fra $\sum x_i$ og $\sum x_i^2$ (eller tilsvarende ud fra $\sum x_i^2$ og $\bar{x}$) uden at skulle beregne hver enkelt afvigelse fra gennemsnittet.

Variationskoefficienten gør det muligt at sammenligne variation på tværs af datasæt med forskellige skalaer.

Definition 6.11 (Variationskoefficient)

Variationskoefficienten er forholdet mellem standardafvigelsen og gennemsnittet:

$$CV = \frac{s}{\bar{x}} \times 100\%$$

♣

Eksempel 6.10 Sammenligning af variation

Se på to systemer:

- System A: Gennemsnitlig svartid = 100 ms, standardafvigelse = 20 ms
- System B: Gennemsnitlig svartid = 500 ms, standardafvigelse = 100 ms

$$CV_A = \frac{20}{100} \times 100\% = 20\%$$
$$CV_B = \frac{100}{500} \times 100\% = 20\%$$

Begge systemer har samme relative variation (20%), selv om deres absolutte standardafvigelser er forskellige.

Positionsmål

For at kunne introducere et spredningsmål, der er mindre følsomt over for outliers, skal vi først forstå positionsmål, som beskriver et datapunkts relative placering i et datasæt.

Definition 6.12 (Percentil)

Den p-te **percentil** er en værdi, som cirka $p\%$ af observationerne ligger på eller under. Der findes flere konventioner for beregning af percentiler, især for små stikprøver, så den valgte metode bør altid angives. ♣

Hvis en svartid fx ligger på 95.-percentilen, betyder det, at cirka 95% af alle svartider ligger på eller under denne værdi.

Eksempel 6.11 Beregning af percentil

Brug metoden med nærmeste rang til at finde 90.-percentilen for svartiderne:

1. Sortér data: $\{120, 150, 180, 200, 220, 250, 280, 300, 350, 400\}$
2. Rang: $\lceil 0.9 \times 10 \rceil = 9$
3. 90.-percentilen = 350 ms

Kvartiler er specifikke, meget anvendte percentiler, der deler data i fire lige store dele.

Definition 6.13 (Kvartiler)

Kvartiler deler et ordnet datasæt i fire lige store dele.

- **Q1 (første kvartil)**: 25.-percentilen.
- **Q2 (anden kvartil)**: 50.-percentilen. Det er datasættets **median**.

- **Q3 (tredje kvartil)**: 75.-percentilen.

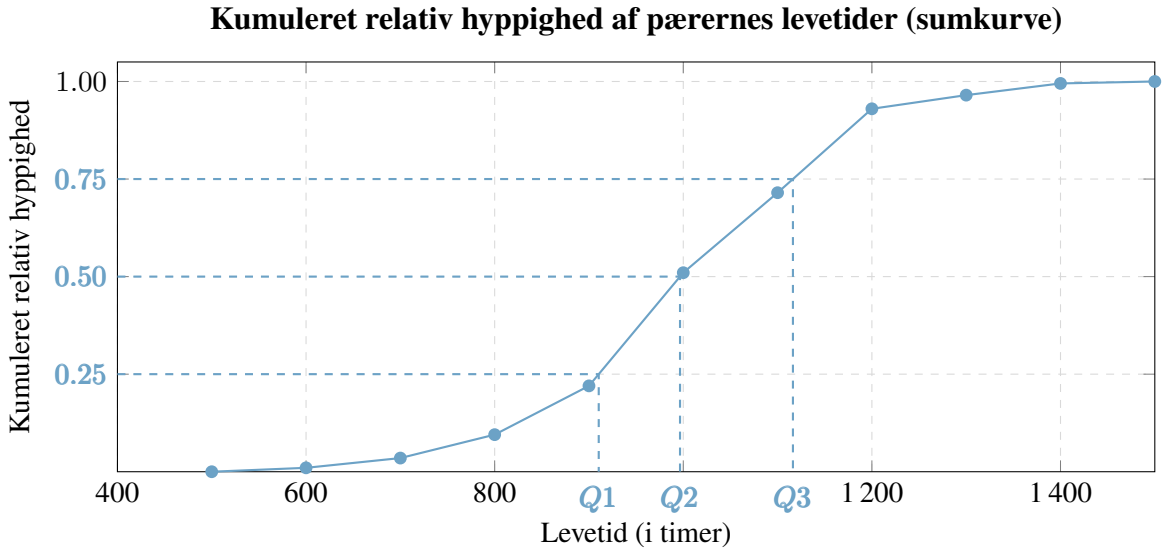
Eksempel 6.12 Beregning af kvartiler

For de sorterede svartider: $\{120, 150, 180, 200, 220, |250, 280, 300, 350, 400\}$

- Den nederste halvdel er $\{120, 150, 180, 200, 220\}$. Medianen af denne halvdel er $Q1 = 180$ ms.
- Den øverste halvdel er $\{250, 280, 300, 350, 400\}$. Medianen af denne halvdel er $Q3 = 300$ ms.

Eksempel 6.13 Kvartiler aflæst på en sumkurve

Kvartiler kan aflæses direkte på en sumkurve. Som eksempel bruger vi igen pærernes levetider. Aflæst på sumkurven i Figur 6.6 er de omtrentlige kvartiler $Q1 \approx 900$, $Q2$ (medianen) ≈ 1000 og $Q3 \approx 1100$ timer.



Figur 6.6: En sumkurve for pærernes levetider med kvartilerne $Q1$, $Q2$ og $Q3$ markeret med stiplede linjer.

Percentiler deler data i 100 lige store dele, mens kvartiler deler data i fire lige store dele.

Kvartilafstanden er et robust spredningsmål, som generelt er langt mindre følsomt over for ekstreme observationer end den fulde variationsbredde eller standardafvigelsen.

Definition 6.14 (Kvartilafstand)

Kvartilafstanden (IQR) måler spredningen af de midterste 50% af data og er defineret som

$$IQR = Q_3 - Q_1,$$

hvor Q_1 og Q_3 er første og tredje kvartil. En større IQR betyder, at den centrale del af data er mere spredt, mens en mindre IQR betyder, at den er tættere samlet.

Eksempel 6.14 Beregning af IQR

For svartiderne: $\{120, 150, 180, 200, 220, 250, 280, 300, 350, 400\}$

- Nederste halvdel: $\{120, 150, 180, 200, 220\} \rightarrow Q1 = 180$
- Øverste halvdel: $\{250, 280, 300, 350, 400\} \rightarrow Q3 = 300$

- $IQR = 300 - 180 = 120$ millisekunder

Opsummerende statistik

Før vi afslutter dette kapitel, gennemgår vi de opsummerende statistikker, som er behandlet i dette afsnit. Tabel 6.6 er et hypotetisk eksempel for et separat datasæt med 1000 svartider fra et websted. De underliggende observationer er ikke angivet, så værdierne er medtaget for at illustrere opbygningen af en opsummerende tabel og ikke som en beregning, der kan efterprøves ud fra de ti foregående observationer.

Eksempel 6.15

For et datasæt med 1000 svartider fra et websted:

Mål	Værdi
Antal	1000
Gennemsnit	245.3 ms
Median	238.0 ms
Typetal	220.0 ms
Standardafvigelse	95.6 ms
Varians	9139.4 ms²
Minimum	45.0 ms
Maksimum	1200.0 ms
Variationsbredde	1155.0 ms
Q1	180.0 ms
Q3	300.0 ms
IQR	120.0 ms
Skævhed	1.2 (positiv skævhed)
Kurtosis	2.8 (leptokurtisk)

Tabel 6.6: Fuldstændig statistisk opsummering

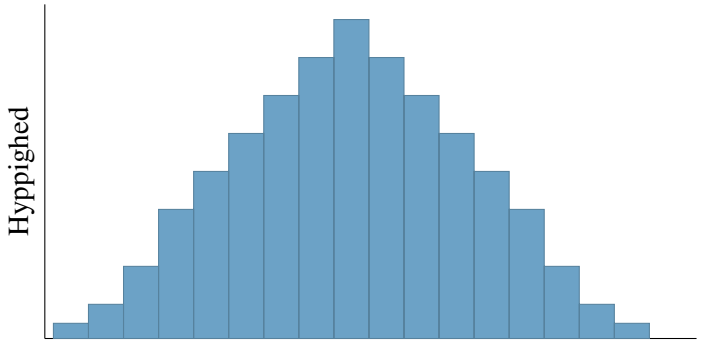
Tabellen giver et fuldstændigt billede af data og er ofte et første skridt i analysen af et datasæt. Bemærk, at skævhed gennemgås i næste afsnit, mens kurtosis ikke forklares i denne bog. Vi har valgt at medtage dem her for fuldstændighedens skyld, da de ofte bruges i analysen af et datasæt – selv om det ligger uden for denne bogs rammer at beregne dem.

6.4 Forståelse af datafordelinger

Når vi har visualiseret vores data med histogrammer og boksplot, kan vi begynde at analysere fordelingens *form*. Formen fortæller os om de underliggende mønstre i data og hjælper os med at vælge de rette statistiske værktøjer. I softwareudvikling er det afgørende at forstå fordelingen af målinger som svartider eller fejlrater, når man skal fastlægge referenceniveauer for ydelse og opdage afvigelser. Først genopfrisker vi begrebet outliers.

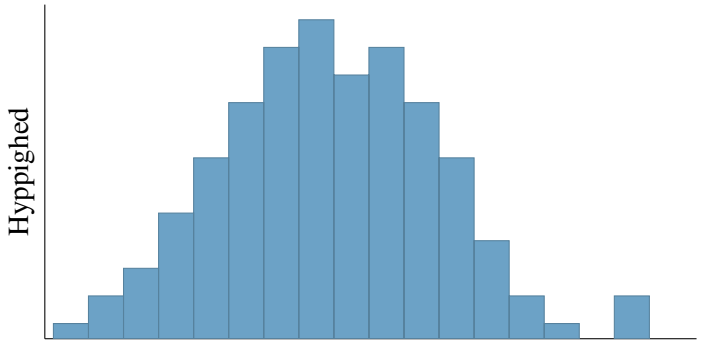
Normale datasæt

Mange store datasæt giver histogrammer med en karakteristisk overordnet form: Hyppighederne topper nær stikprøvens median og aftager symmetrisk på begge sider, så der dannes en karakteristisk **klokkeformet kurve**. Datasæt med denne egenskab kaldes **normale**, og deres histogrammer kaldes **normale histogrammer** (se Figur 6.7).



Figur 6.7: Et histogram over et normalt datasæt, der viser den karakteristiske klokkeform.

I softwaresystemer er perfekt normale fordelinger sjældne, men nogle målinger kan være **tilnærmelsesvis normale**. Svartiden (latensen) for en stærkt optimeret, intern mikrotjeneste, der udfører en ensartet opgave, kan fx være tilnærmelsesvis normal. Et histogram over et tilnærmelsesvis normalt datasæt ser ud som Figur 6.8.



Figur 6.8: Et histogram over et datasæt, der er tilnærmelsesvis normalt. Selv om det ikke er perfekt symmetrisk, følger det den overordnede klokkeform.

Mange datasæt fra virkeligheden, fra menneskers højde til målefejl, følger netop denne form. Om sådanne data siger man, at de er **normalfordelte**.

Definition 6.15 (Normalfordeling)

En **normalfordeling** er en symmetrisk, klokkeformet fordeling, hvor middelværdi, median og typetal alle er ens og ligger i centrum. Dens form er fuldstændig bestemt af dens middelværdi (μ) og standardafvigelse (σ).

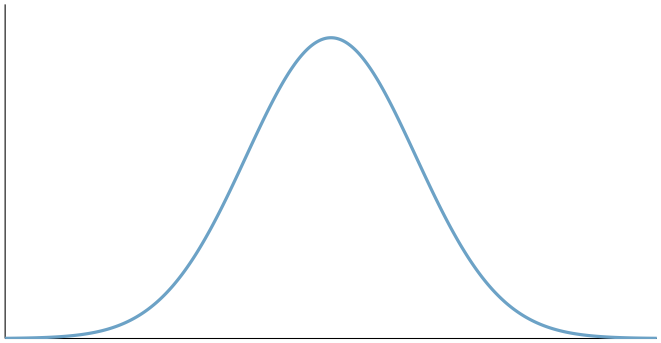
♣

Et histogram er et skøn over formen af den fordeling, som har frembragt observationerne. Med flere data og et passende valg af intervalbredde kan det give et mere detaljeret billede af den underliggende fordeling, men det bliver ikke nødvendigvis normalt. Om en normalfordelingsmodel er passende, afhænger af den proces, der frembringer data.

Sætning 6.1 (Store tals lov)

For uafhængige og identisk fordelte observationer med en endelig forventet værdi konvergerer stikprøvens gennemsnit mod populationens middelværdi, når stikprøvestørrelsen vokser. ♡

Store tals lov handler om stikprøvens gennemsnit, ikke om et histograms form. Hvis den underliggende population selv er normalfordelt, har dens tæthedsfunktion følgende form:



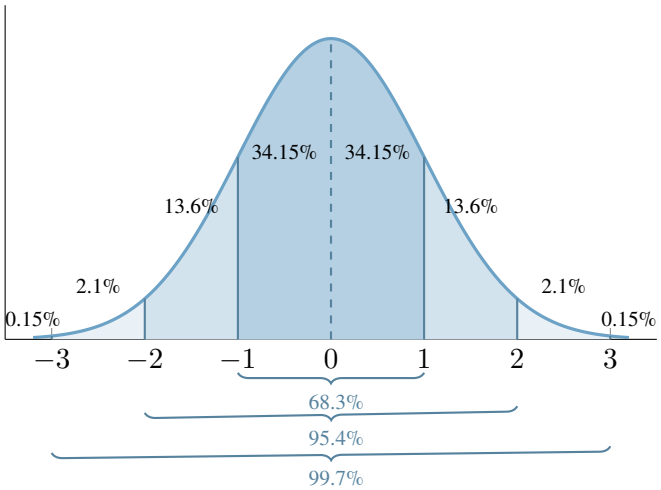
Figur 6.9: Normalfordelingskurven.

For data, der er tilnærmelsesvis normale, kan vi bruge en effektiv tommelfingerregel til at forstå spredningen uden at se på hvert eneste datapunkt. Den kaldes **den empiriske regel**.

Sætning 6.2 (Den empiriske regel)

Hvis et datasæt er tilnærmelsesvis normalt med stikprøvegennemsnit $\bar{x}$ og stikprøvestandardafvigelse s, gælder:

- Cirka **68.3%** af observationerne ligger inden for 1 standardafvigelse fra gennemsnittet ($\bar{x} \pm s$).
- Cirka **95.4%** af observationerne ligger inden for 2 standardafvigelser fra gennemsnittet ($\bar{x} \pm 2s$).
- Cirka **99.7%** af observationerne ligger inden for 3 standardafvigelser fra gennemsnittet ($\bar{x} \pm 3s$). ♡



Figur 6.10: Den empiriske regel vist på en normalfordelingskurve. De indre bånd markerer $\pm 1\sigma$ (68.3%), $\pm 2\sigma$ (95.4%) og $\pm 3\sigma$ (99.7%).

Skæve datasæt

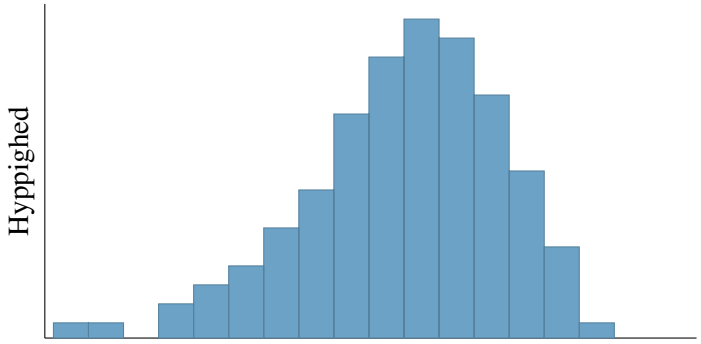
Når et datasæt ikke er symmetrisk omkring stikprøvens median, beskrives det som **skævt**. En lang hale, der strækker sig mod højre, betyder, at fordelingen er **højreskæv**; en lang hale mod venstre betyder, at den er

venstreskæv. Histogrammet i Figur 6.11 er altså venstreskævt, mens histogrammet i Figur 6.12 er højreskævt.

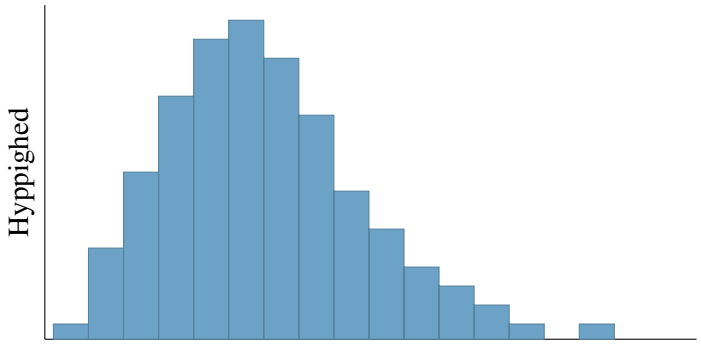
Definition 6.16 (Skævhed)

Skævhed måler asymmetrien i en datafordeling:

- **Positiv skævhed (højreskæv)**: Halen strækker sig mod højre (gennemsnit > median)
- **Negativ skævhed (venstreskæv)**: Halen strækker sig mod venstre (gennemsnit < median)
- **Symmetrisk**: Gennemsnit ≈ median

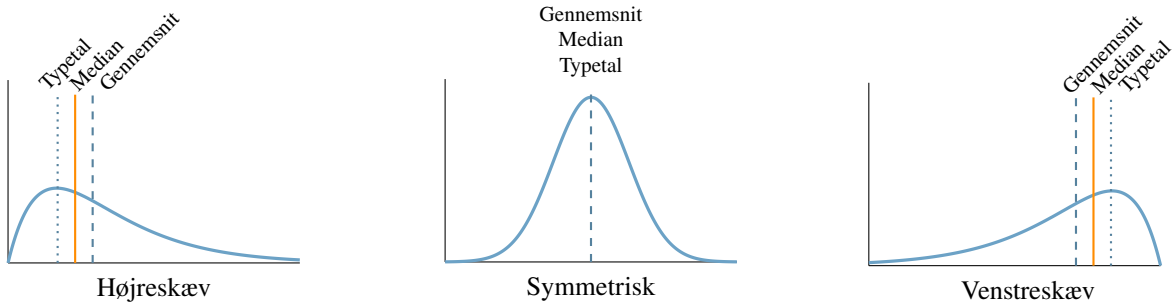


Figur 6.11: Et histogram over et datasæt, der er venstreskævt (negativt skævt). Fordelingens hale ligger til venstre.



Figur 6.12: Et histogram over et datasæt, der er højreskævt (positivt skævt). Fordelingens hale ligger til højre.

Figur 6.13 illustrerer almindelige mønstre for gennemsnit, median og typetal i unimodale skæve fordelinger. Disse rækkefølger er nyttige tommelfingerregler, ikke definitioner af skævhed og ikke generelle regler.



Figur 6.13: Sammenhængen mellem gennemsnit, median og typetal for positivt skæve, symmetriske og negativt skæve fordelinger.

Identifikation af outliers

Som nævnt i Afsnit 6.1 er outliers datapunkter, der adskiller sig markant fra resten af data. De kan opstå på grund af målefejl, indtastningsfejl eller reel variation i data. Det er vigtigt at finde outliers, fordi de kan forvrænge statistiske analyser.

En almindelig metode til at identificere outliers bruger kvartilafstanden (IQR):

- **Moderate outliers**: Datapunkter, der ligger uden for $1.5 \times IQR$-grænserne, men ikke uden for $3 \times IQR$-grænserne.
- **Ekstreme outliers**: Datapunkter, der er mindre end $Q_1 - 3 \times IQR$ eller større end $Q_3 + 3 \times IQR$.

Eksempel 6.16 Identifikation af outliers

Givet svartiderne: $\{120, 150, 180, 200, 220, 250, 280, 300, 350, 400\}$

- $Q_1 = 180$, $Q_3 = 300$, $IQR = 120$
- Nedre grænse: $Q_1 - 1.5 \times IQR = 180 - 180 = 0$
- Øvre grænse: $Q_3 + 1.5 \times IQR = 300 + 180 = 480$

Alle datapunkter ligger mellem 0 og 480, så der er ingen moderate outliers i dette datasæt.

Eksempel 6.17

Antag, at vi har følgende svartider (i millisekunder) for en webapplikation:

$\{120, 150, 180, 200, 220, 250, 280, 300, 350, 400, 1200\}$.

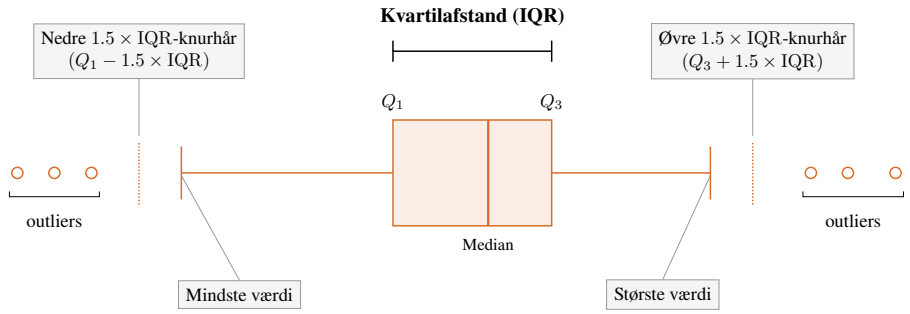
Lad os undersøge, om der er en øvre outlier:

- Sorterede data: $120, 150, 180, 200, 220, 250, 280, 300, 350, 400, 1200$
- $n = 11$
- Median (Q_2): 250 (6. værdi)
- Nederste halvdel: $120, 150, 180, 200, 220 \rightarrow Q_1 = 180$
- Øverste halvdel: $280, 300, 350, 400, 1200 \rightarrow Q_3 = 350$
- $IQR = Q_3 - Q_1 = 350 - 180 = 170$
- Øvre grænse: $Q_3 + 1.5 \times IQR = 350 + 1.5 \times 170 = 350 + 255 = 605$

Værdien 1200 er større end 605, så den er en øvre outlier.

Boksplot

Boksplot giver en kompakt opsummering af en datafordeling og fremhæver medianen, kvartilerne og outliers. Figur 6.14 viser et eksempel på et boksplot.



Figur 6.14: Et eksempel på et boksplot, der illustrerer medianen, kvartilerne, knurhårene og mulige outliers.

Definition 6.17 (Et boksplots bestanddele)

Et boksplot bygger på femtalsopsummeringen og kan også vise outliers:

- **Nedre knurhår**: Som regel den mindste observation inden for den nedre $1.5 \times IQR$-grænse
- **Q1 (første kvartil)**: 25.-percentilen
- **Median (Q2)**: 50.-percentilen
- **Q3 (tredje kvartil)**: 75.-percentilen
- **Øvre knurhår**: Som regel den største observation inden for den øvre $1.5 \times IQR$-grænse
- **Outliers**: Punkter, der ligger mere end 1.5 × IQR fra boksen

♣

Et boksplot giver en visuel opsummering af et datasæts fordeling, så det er let at sammenligne grupper og få øje på outliers. Sådan læser man et boksplot:

- **Boksens længde** viser spredningen af den midterste halvdel af data. En længere boks betyder mere variation i de midterste 50%.
- **Medianlinjens placering** og knurhårenes relative længder kan antyde asymmetri, men et boksplot alene fastlægger ikke retningen eller graden af skævhed.

- I mange unimodale skæve fordelinger trækkes gennemsnittet mod den længste hale. Derfor ser man ofte typetal $<$ median $<$ gennemsnit for højreskæve data og den omvendte rækkefølge for venstreskæve data. Det er almindelige mønstre, ikke garanterede identiteter.
- **Knurhårene** viser variationsbredden for hovedparten af data, når outliers er udeladt.
- **Outliers** fremhæver usædvanligt høje eller lave værdier, som kan kræve nærmere undersøgelse.

Boksplot er særligt nyttige til at sammenligne fordelinger på tværs af flere grupper eller datasæt, da de hurtigt afslører forskelle i medianer, spredning og forekomst af outliers.

Dette afslutter vores gennemgang af datavisualisering og opsummerende statistik og dermed vores gennemgang af sandsynlighedsregning og statistik i det hele taget.

Del III

Lineær algebra

Kapitel 7 Lineære ligninger i lineær algebra

I 1949 brugte Harvard-professor Wassily Leontief en af de tidligste computere, Mark II, til at løse et lineært ligningssystem, der modellerede den amerikanske økonomi. Han inddelte økonomien i 500 sektorer, fx kul, bilindustri og kommunikation, og beskrev med lineære ligninger, hvordan sektorerne påvirkede hinanden. På grund af hardwarebegrænsninger reducerede han problemet til 42 ligninger, som det tog Mark II 56 timer at løse. Arbejdet var en milepæl for anvendelsen af computere til beregninger og indbragte Leontief Nobelprisen i 1973.

Leontiefs arbejde viste, hvordan lineær algebra kombineret med computere kunne løse store problemer – en idé, der har endnu større betydning i dag inden for softwareudvikling. Lineær algebra er grundlæggende for mange moderne teknologier, især maskinlæring (ML) og kunstig intelligens (AI). Fra anbefalingssystemer til optimering af neurale netværk er lineær algebra kernen i de algoritmer, der driver nutidens intelligente systemer.

I ML og AI omdannes store datasæt til matricer, hvor teknikker fra lineær algebra som matrixfaktorisering, egenværdidekomposition og singulærværdidekomposition (SVD) bruges til at udlede indsigter og foretage forudsigelser. Neurale netværk, som er rygraden i AI, bygger på lineær algebra, når data sendes gennem lagene, og når vægtene justeres under træningen. Som softwareingeniør får du ved at mestre disse begreber i lineær algebra de redskaber, der skal til for at bygge skalerbare, intelligente systemer, som kan håndtere nutidens mest komplekse beregningsproblemer.

Med den eksponentielle vækst i data og regnekraft bliver lineær algebra stadig vigtigere i softwareudvikling, især inden for AI og ML. Det er et afgørende værktøj, der bygger bro mellem teoretisk matematik og virkelige anvendelser inden for teknologi.

7.1 Lineære ligningssystemer

Lineære ligninger og ligningssystemer udgør grundlaget for lineær algebra, et centralt område af matematikken med omfattende anvendelser inden for softwareudvikling. Fra computergrafik til algoritmer til maskinlæring er det afgørende at forstå, hvordan man modellerer og løser lineære ligningssystemer, for at kunne udvikle effektive softwareløsninger. Vi begynder med en definition:

Definition 7.1 (Lineære ligninger)

En lineær ligning i variablene $x_1, x_2, \ldots, x_n$ er en ligning, der kan skrives på formen

$$a_1x_1 + a_2x_2 + \cdots + a_nx_n = b$$

hvor b og koefficienterne $a_1, \ldots, a_n$ er konstanter.

♣

Lineære ligninger beskriver rette linjer, planer og hyperplaner i forskellige dimensioner, hvilket gør dem nyttige til at modellere sammenhænge i data og algoritmer.

Eksempel 7.1 En linje i det todimensionale rum givet ved $y = mx + b$ er en lineær ligning. Den kan omskrives, så den passer til vores standardform:

$$-mx + y = b$$

Her er $a_1 = -m$, $a_2 = 1$, og b er konstantleddet.

Eksempel 7.2 Den generelle ligning for en plan i det tredimensionale rum er

$$ax + by + cz = d$$

hvor a, b, c og d er konstanter. Ligningen er lineær i variablene x, y og z.

Definition 7.2 (Lineære ligningssystemer)

Et **lineært ligningssystem** (eller blot et **lineært system**) er en samling af en eller flere lineære ligninger i de samme variable. ♣

Eksempel 7.3 Se på følgende lineære ligningssystem i variablene x_1, x_2, x_3:

$$\begin{aligned} 2x_1 + 3x_2 + x_3 &= 3 \\ 7x_2 - 4x_3 &= 10 \\ x_3 &= 1 \end{aligned}$$

Systemet består af tre ligninger med tre ubekendte.

For at løse systemet bemærker vi først ud fra den tredje ligning, at $x_3 = 1$. Vi indsætter dette i den anden ligning og løser med hensyn til x_2:

$$\begin{aligned} 7x_2 - 4(1) &= 10 \\ 7x_2 &= 14 \\ x_2 &= 2 \end{aligned}$$

Dernæst indsætter vi $x_2 = 2$ og $x_3 = 1$ i den første ligning og løser med hensyn til x_1:

$$\begin{aligned} 2x_1 + 3(2) + 1 &= 3 \\ 2x_1 &= -4 \\ x_1 &= -2 \end{aligned}$$

Løsningen til systemet er altså $(x_1, x_2, x_3) = (-2, 2, 1)$.

Definition 7.3 (Løsninger til et lineært ligningssystem)

En **løsning** til et lineært ligningssystem i variablene $x_1, x_2, \ldots, x_n$ er en liste af tal $(s_1, s_2, \ldots, s_n)$, der opfylder alle systemets ligninger, når tallene indsættes på variablenes plads $x_1, x_2, \ldots, x_n$ i den rækkefølge. Mængden af alle mulige løsninger kaldes systemets **løsningsmængde**. To lineære ligningssystemer kaldes **ækvivalente**, hvis de har samme løsningsmængde. ♣

Et lineært ligningssystem har et af følgende udfald:

(i) **Ingen løsninger**, når ligningerne er inkonsistente.
(ii) **Præcis én løsning**, når der er ét entydigt sæt værdier, som opfylder alle ligningerne.
(iii) **Uendeligt mange løsninger**, når der er flere sæt værdier, som opfylder ligningerne.

Vi siger, at et system er **konsistent**, hvis det har enten én eller uendeligt mange løsninger. Vi siger, at et system er **inkonsistent**, hvis det ikke har nogen løsning. Vi formaliserer dette senere i kapitlet.

Bemærkning: At afgøre, om et system er konsistent, handler om løsningers *eksistens*. Hvis der findes løsninger, kan vi desuden undersøge, om løsningerne er *entydige*.

Ligningssystemer skrevet med matricer

Matricer giver en kompakt og effektiv måde at skrive og behandle lineære ligningssystemer på, hvilket især er en fordel i softwareanvendelser med store datasæt.

Definition 7.4

En **matrix** er et rektangulært skema af tal ordnet i rækker og søjler. Et lineært ligningssystems **koefficientmatrix** indeholder kun variablenes koefficienter, mens **totalmatrixen** har en ekstra søjle med konstanterne fra ligningernes højresider.

Eksempel 7.4 For det lineære ligningssystem:

$$\begin{aligned} 2x_1 + 3x_2 + x_3 &= 3 \\ 7x_2 - 4x_3 &= 10 \\ x_3 &= 1 \end{aligned}$$

er koefficientmatrixen:

$$\begin{bmatrix} 2 & 3 & 1 \\ 0 & 7 & -4 \\ 0 & 0 & 1 \end{bmatrix}$$

og totalmatrixen er:

$$\left[\begin{array}{ccc|c} 2 & 3 & 1 & 3 \\ 0 & 7 & -4 & 10 \\ 0 & 0 & 1 & 1 \end{array}\right]$$

Den lodrette streg mellem koefficientdelen og den tilføjede søjle er valgfri og bruges til visuelt at adskille koefficienterne fra konstanterne.

Definition 7.5

En matrix' **størrelse** er bestemt af antallet af rækker og søjler og skrives som *rækker* $\times$ *søjler*. For eksempel har koefficientmatrixen ovenfor størrelsen 3×3, og totalmatrixen har størrelsen 3×4.

Løsning af ligningssystemer med totalmatricer

Vi viser nu, hvordan man løser et lineært ligningssystem ved hjælp af totalmatrixnotation. Fremgangsmåden formaliseres i næste afsnit, men her giver vi et eksempel. Metoden gør det lettere at løse ligningssystemer, fordi vi kun skal arbejde med matricen.

Eksempel 7.5 Lad r_1 betegne den første række, r_2 den anden række og r_3 den tredje række.

$$\begin{bmatrix} 2 & 3 & 1 & 3 \\ 0 & 7 & -4 & 10 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

For at forenkle systemet udfører vi *elementære rækkeoperationer*. Konkret ændrer vi r_1 og r_2 således:

$$r_1 \mapsto r_1 - r_3,$$
$$r_2 \mapsto r_2 + 4r_3,$$

hvilket giver:

$$\begin{bmatrix} 2 & 3 & 0 & 2 \\ 0 & 7 & 0 & 14 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

Dernæst skalerer vi r_2 med $\frac{1}{7}$:

$$r_2 \mapsto \frac{1}{7} r_2, \quad \begin{bmatrix} 2 & 3 & 0 & 2 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

Nu eliminerer vi 3-tallet i den første række ved at udføre:

$$r_1 \mapsto r_1 - 3r_2, \quad \begin{bmatrix} 2 & 0 & 0 & -4 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

Til sidst skalerer vi r_1 med $\frac{1}{2}$ og får:

$$r_1 \mapsto \frac{1}{2} r_1, \quad \begin{bmatrix} 1 & 0 & 0 & -2 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

Nu kan vi igen tolke matricen som et lineært ligningssystem. Ud fra totalmatrixen får vi:

$$x_1 = -2,$$
$$x_2 = 2,$$
$$x_3 = 1.$$

Dette kan skrives kompakt som:

$$(x_1, x_2, x_3) = (-2, 2, 1).$$

Løsningen er altså den samme som den, vi fandt ved indsættelse.

Eksempel 7.6 Afgør, om følgende system er konsistent:

$$\begin{aligned} x_2 + 4x_3 &= 2 \\ x_1 - 3x_2 + 2x_3 &= 6 \\ x_1 - 2x_2 + 6x_3 &= 9 \end{aligned}$$

Løsning: Først skriver vi systemets totalmatrix:

$$\begin{bmatrix} 0 & 1 & 4 & 2 \\ 1 & -3 & 2 & 6 \\ 1 & -2 & 6 & 9 \end{bmatrix}$$

For at forenkle matricen ombytter vi r_1 og r_2:

$$r_1 \leftrightarrow r_2, \quad \begin{bmatrix} 1 & -3 & 2 & 6 \\ 0 & 1 & 4 & 2 \\ 1 & -2 & 6 & 9 \end{bmatrix}$$

Dernæst eliminerer vi 1-tallet i den første søjle af r_3 ved at udføre:

$$r_3 \mapsto r_3 - r_1, \quad \begin{bmatrix} 1 & -3 & 2 & 6 \\ 0 & 1 & 4 & 2 \\ 0 & 1 & 4 & 3 \end{bmatrix}$$

Derefter eliminerer vi 1-tallet i den anden søjle af r_3:

$$r_3 \mapsto r_3 - r_2, \quad \begin{bmatrix} 1 & -3 & 2 & 6 \\ 0 & 1 & 4 & 2 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Den tredje række svarer nu til ligningen:

$$0x_1 + 0x_2 + 0x_3 = 1$$

som kan reduceres til:

$$0 = 1$$

Det er en modstrid, så systemet er **inkonsistent** og har ingen løsning. ◀

Eksempel 7.7 Angiv en løsning til følgende system (hvis der findes en). Er den entydig?

$$x_1 + 2x_2 + 3x_3 = 4$$
$$3x_1 + 6x_2 + 9x_3 = 12$$

Løsning:

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 3 & 6 & 9 & 12 \end{bmatrix}$$

Vi bemærker, at r_2 er et multiplum af r_1:

$$r_2 = 3r_1$$

For at forenkle systemet udfører vi følgende elementære rækkeoperation, som fjerner overflødig information:

$$r_2 \mapsto r_2 - 3r_1,$$

hvilket giver:

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Den anden række består nu udelukkende af nuller, hvilket viser, at den ikke giver nogen ny information. Det betyder, at systemet er underbestemt og har uendeligt mange løsninger.

Vi kan udtrykke x_1 ud fra den første ligning:

$$x_1 = 4 - 2x_2 - 3x_3$$

Vi lader x_2 og x_3 være frie variable (parametre) og sætter:

$$x_2 = s, \quad x_3 = t, \quad \text{hvor } s, t \in \mathbb{R}$$

Ved indsættelse får vi:

$$x_1 = 4 - 2s - 3t,$$
$$x_2 = s,$$
$$x_3 = t.$$

Dette kan skrives kompakt som:

$$(x_1, x_2, x_3) = (4 - 2s - 3t,\ s,\ t), \quad \text{for alle } s, t \in \mathbb{R}.$$

Systemet har uendeligt mange løsninger, der er parametriseret ved s og t. Løsningen er derfor ikke entydig. ◀

Eksempel 7.8 Vælg h og k, så følgende system

$$x_1 - 3x_2 = 1$$
$$2x_1 + hx_2 = k$$

har

(i) en entydig løsning,
(ii) mange løsninger og
(iii) ingen løsning.

Løsning:

$$\begin{bmatrix} 1 & -3 & 1 \\ 2 & h & k \end{bmatrix}$$

For at forenkle systemet udfører vi en elementær rækkeoperation, som eliminerer x_1 fra den anden ligning. Konkret ændrer vi r_2 således:

$$r_2 \mapsto r_2 - 2r_1,$$

hvilket giver:

$$\begin{bmatrix} 1 & -3 & 1 \\ 0 & h+6 & k-2 \end{bmatrix}$$

Nu analyserer vi det resulterende system ud fra værdien af $h + 6$.

Tilfælde 1: $h + 6 \neq 0$

Når $h + 6 \neq 0$, kan vi løse den anden ligning med hensyn til x_2:

$$(h+6)x_2 = k - 2 \implies x_2 = \frac{k-2}{h+6}$$

Vi indsætter x_2 i den første ligning:

$$x_1 - 3x_2 = 1 \implies x_1 = 1 + 3x_2 = 1 + 3\left(\frac{k-2}{h+6}\right)$$

Vi får altså en entydig løsning for x_1 og x_2. **Systemet har en entydig løsning, når** $h + 6 \neq 0$**.**

Tilfælde 2: $h + 6 = 0$

Når $h + 6 = 0$, dvs. $h = -6$, bliver den anden ligning:

$$0x_2 = k - 2$$

som kan reduceres til:

$$0 = k - 2$$

Vi har to deltilfælde:

Deltilfælde 2a: $k - 2 = 0$

Hvis $k = 2$, bliver ligningen $0 = 0$, som altid er sand. Den anden ligning giver ingen ny information, så systemet reduceres til:

$$x_1 - 3x_2 = 1$$

Her er x_2 en fri variabel. Vi løser med hensyn til x_1:

$$x_1 = 1 + 3x_2$$

Systemet har uendeligt mange løsninger, når $h = -6$ og $k = 2$.

Deltilfælde 2b: $k - 2 \neq 0$

Hvis $k \neq 2$, bliver ligningen $0 = k - 2$, hvilket er en modstrid, da $0 \neq k - 2$. **Systemet har altså ingen løsning, når $h = -6$ og $k \neq 2$.**

Konklusion:

(i) **Ingen løsning**, når $h = -6$ og $k \neq 2$.
(ii) **En entydig løsning**, når $h \neq -6$.
(iii) **Uendeligt mange løsninger**, når $h = -6$ og $k = 2$.

◀

7.2 Elementære rækkeoperationer og echelonformer

I Eksempel 7.5–Eksempel 7.8 udførte vi en række operationer, der kaldes **elementære rækkeoperationer**. Det er grundlæggende omformninger, som forenkler ligningssystemer uden at ændre deres løsningsmængder.

De tre typer elementære rækkeoperationer er:

- **Erstatning:** Erstat en række med summen af den selv og et multiplum af en anden række.
- **Ombytning:** Byt om på to rækker.
- **Skalering:** Gang alle indgange i en række med en konstant forskellig fra nul.

To matricer kaldes **rækkeækvivalente**, hvis den ene kan omdannes til den anden ved en følge af elementære rækkeoperationer.

Bemærkning: To lineære ligningssystemer er ækvivalente (dvs. de har samme løsningsmængde), hvis deres totalmatricer er rækkeækvivalente. Elementære rækkeoperationer på en totalmatrix ændrer ikke systemets

løsningsmængde.

Som du måske har bemærket i eksemplerne, får de matricer, der fremkommer ved disse operationer, ofte et særligt mønster – mange nuller under bestemte indgange. Det er ikke et tilfælde, men et mål med processen. Vi kalder dette matricens **echelonform**. Ved at bringe en matrix på echelonform:

- gør vi det lettere at løse lineære ligningssystemer,
- forenkler vi matricen systematisk, så beregningerne bliver mindre komplicerede,
- afslører vi vigtige egenskaber ved systemet, fx om der findes løsninger og hvor mange, eller om bestemte ligninger er afhængige.

Definition 7.6 (Echelonformer)

En matrix er på **echelonform**, hvis den opfylder følgende betingelser:

1. Alle nulrækker står nederst.
2. Hver rækkes ledende indgang står i en søjle til højre for den ledende indgang i rækken ovenover.
3. Alle indgange i en søjle under en ledende indgang er nuller.

Ud over echelonform kan en matrix også være på **reduceret echelonform** (RREF, af engelsk *reduced row echelon form*), som har følgende egenskaber:

4. Den ledende indgang i hver række, der ikke er en nulrække, er 1.
5. Hvert ledende 1-tal er den eneste indgang forskellig fra nul i sin søjle.

♣

Vi siger, at en echelonmatrix U er en echelonform af matricen A, hvis U er rækkeækvivalent med A. Tilsvarende siger vi, at en reduceret echelonmatrix U er den reducerede echelonform af matricen A, hvis U er rækkeækvivalent med A.

Betydningen af at bringe totalmatrixen for et lineært ligningssystem på echelonform forklares af følgende sætning.

Sætning 7.1 (Eksistenssætningen)

Et lineært ligningssystem er konsistent, hvis og kun hvis en echelonform af totalmatrixen ikke har nogen række på formen

$$\left[\begin{array}{cccc} 0 & \dots & 0 & b \end{array}\right]$$

hvor b er forskellig fra nul.

♡

Vi så et eksempel på et inkonsistent system i Eksempel 7.6, fordi echelonformen af totalmatrixen har en række på formen $\left[\begin{array}{ccc|c} 0 & 0 & 0 & 1 \end{array}\right]$. Denne række viser, at systemet ikke har nogen løsning.

Eksempel 7.9 Totalmatrixen for det lineære ligningssystem, der var hovedeksemplet i det foregående afsnit,

$$\left[\begin{array}{cccc} 2 & 3 & 1 & 3 \\ 0 & 7 & -4 & 10 \\ 0 & 0 & 1 & 1 \end{array}\right],$$

er allerede på echelonform. Da den ikke har nogen række af den form, der er nævnt i sætningen, ved vi straks, at systemet er konsistent. De ledende indgange er 2, 7 og 1, og alle indgange under dem er nuller. Matricen er dog ikke på reduceret echelonform, fordi de ledende indgange ikke alle er 1.

Husk, at vi udførte en række rækkeoperationer på den foregående matrix og fik

$$\begin{bmatrix} 1 & 0 & 0 & -2 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

som er på reduceret echelonform. Det gjorde det let at se systemets løsninger, hvilket er den største fordel ved at bringe matricen på denne form. Det fører os til følgende vigtige sætning.

Sætning 7.2 (Entydighed af den reducerede echelonform)

Enhver matrix er rækkeækvivalent med én og kun én reduceret echelonmatrix. ♡

En pivotposition i en matrix A er en placering i A, der svarer til et ledende 1-tal i den reducerede echelonform af A. En pivotsøjle er en søjle i A, der indeholder en pivotposition. Bemærk, at man kan få en pivot ved at skalere den ledende indgang i en række, så den bliver 1. Pivotsøjlen er derfor den søjle, hvor den ledende indgang står.

Nu er vi klar til at præsentere algoritmen til løsning af et lineært ligningssystem ved hjælp af matricer.

Rækkereduktionsalgoritmen

Her beskriver vi en algoritme, der omdanner en vilkårlig matrix til en ækvivalent (reduceret) echelonmatrix. Algoritmen er grundlaget for at løse lineære ligningssystemer ved hjælp af matricer.

1. Begynd med den søjle længst til venstre, der ikke er en nulsøjle. Det er en pivotsøjle, og pivotpositionen står øverst.
2. Vælg en indgang forskellig fra nul i pivotsøjlen som pivot. Byt om nødvendigt rækker, så denne indgang flyttes til pivotpositionen.
3. Brug erstatningsoperationer til at skabe nuller på alle positioner under pivoten.
4. Anvend trin 1–3 på den delmatrix, der består af alle indgange under og til højre for pivotpositionen. Gentag processen, indtil der ikke er flere rækker forskellige fra nul at ændre. (Nu har vi nået en echelonform af matricen.)
5. Begynd med pivoten længst til højre, og arbejd opad og mod venstre: Skab nuller over hver pivot ved hjælp af rækkeoperationer. Hvis en pivot ikke er 1, gøres den til 1 ved en skalering. (Dette trin giver matricens reducerede echelonform.)

Løsninger til lineære ligningssystemer

Lad A være koefficientmatrixen for et lineært ligningssystem. Pivotsøjlerne i matricen svarer til det, vi kalder **basisvariable**. Søjlerne uden pivot svarer til det, vi kalder **frie variable**.

Eksempel 7.10 Antag, at totalmatrixen for et lineært ligningssystem er reduceret til følgende form:

$$\begin{bmatrix} \blacksquare & * & * & * & * \\ 0 & \blacksquare & * & * & * \\ 0 & 0 & 0 & \blacksquare & * \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

hvor $\blacksquare$ betegner et vilkårligt tal forskelligt fra nul, og $*$ betegner et vilkårligt tal (inklusive 0). Systemets basisvariable er x_1, x_2 og x_4. Den eneste frie variabel er x_3.

Sætning 7.3 (Entydighedssætningen)

Hvis et lineært ligningssystem er konsistent, indeholder løsningsmængden enten

(i) en entydig løsning, når der ikke er nogen frie variable, eller

(ii) uendeligt mange løsninger, når der er mindst én fri variabel.

♡

Eksempel 7.11 Antag, at følgende matrix er totalmatrixen for et lineært ligningssystem i variablene x_1, x_2 og x_3. Rækkereducér matricen til echelonform for at afgøre, om systemet er konsistent.

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}$$

Hvis systemet er konsistent, så find den reducerede echelonform, og skriv løsningsmængden med de frie variable som parametre.

Løsning: For at forenkle systemet udfører vi *elementære rækkeoperationer*.

Trin 1: Eliminér 5-tallet i r_2

Vi ændrer r_2 således:

$$r_2 \mapsto r_2 - 5r_1,$$

hvilket giver:

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 0 & -4 & -8 & -12 \\ 9 & 10 & 11 & 12 \end{bmatrix}$$

Trin 2: Eliminér 9-tallet i r_3

Vi ændrer r_3 således:

$$r_3 \mapsto r_3 - 9r_1,$$

hvilket giver:

$$\left[\begin{array}{ccc|c} 1 & 2 & 3 & 4 \\ 0 & -4 & -8 & -12 \\ 0 & -8 & -16 & -24 \end{array}\right]$$

Trin 3: Eliminér -8-tallet i r_3

Vi ændrer r_3 igen for at eliminere indgangen i den anden søjle:

$$r_3 \mapsto r_3 - 2r_2,$$

hvilket giver:

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 0 & -4 & -8 & -12 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Trin 4: Skalér r_2 for at få et ledende 1-tal

Vi skalerer r_2 med $-\frac{1}{4}$:

$$r_2 \mapsto -\frac{1}{4}r_2,$$

hvilket giver:

$$\left[\begin{array}{ccc|c} 1 & 2 & 3 & 4 \\ 0 & 1 & 2 & 3 \\ 0 & 0 & 0 & 0 \end{array}\right]$$

Trin 5: Eliminér 2-tallet i r_1

Vi ændrer r_1 således:

$$r_1 \mapsto r_1 - 2r_2,$$

hvilket giver:

$$\left[\begin{array}{ccc|c} 1 & 0 & -1 & -2 \\ 0 & 1 & 2 & 3 \\ 0 & 0 & 0 & 0 \end{array}\right]$$

Nu er matricen på *reduceret echelonform*.

Trin 6: Tolk matricen som et lineært ligningssystem

Ud fra totalmatrixen har vi:

$$\begin{aligned} r_1: &\quad x_1 - x_3 = -2, \\ r_2: &\quad x_2 + 2x_3 = 3, \\ r_3: &\quad 0 = 0. \end{aligned}$$

Da den tredje række svarer til ligningen $0 = 0$, som altid er sand, er systemet **konsistent**.

Trin 7: Udtryk løsningen ved hjælp af en fri variabel

$$\begin{aligned} x_1 &= -2 + x_3, \\ x_2 &= 3 - 2x_3, \\ x_3 &= x_3. \end{aligned}$$

◀

Kapitel 8 Vektorer og matricer

En **vektor** er et matematisk objekt, der har både en **størrelse** (længde) og en **retning**. I to eller tre dimensioner tegnes en vektor ofte som en pil fra ét punkt til et andet. For eksempel beskriver vektoren $\boldsymbol{v} = (x_1, x_2)$ i 2D-rummet en bevægelse fra origo $(0, 0)$ til punktet (x_1, x_2).

Vektorer bruges til at beskrive størrelser som hastighed, kraft og forskydning, der kræver både en størrelse og en retning for at være fuldt beskrevet. Vektorer kan lægges sammen, skaleres med et tal og opløses i komposanter.

Matricer er derimod rektangulære skemaer af tal ordnet i rækker og søjler. De bruges til at beskrive og løse lineære ligningssystemer, udføre transformationer og beskrive sammenhænge mellem mængder af vektorer.

8.1 Vektorer i $\mathbb{R}^2$ og $\mathbb{R}^n$

I lineær algebra skrives vektorer ofte som **søjlematricer**. For eksempel kan vektoren $\boldsymbol{v} = (x_1, x_2)$ skrives som en søjlematrix:

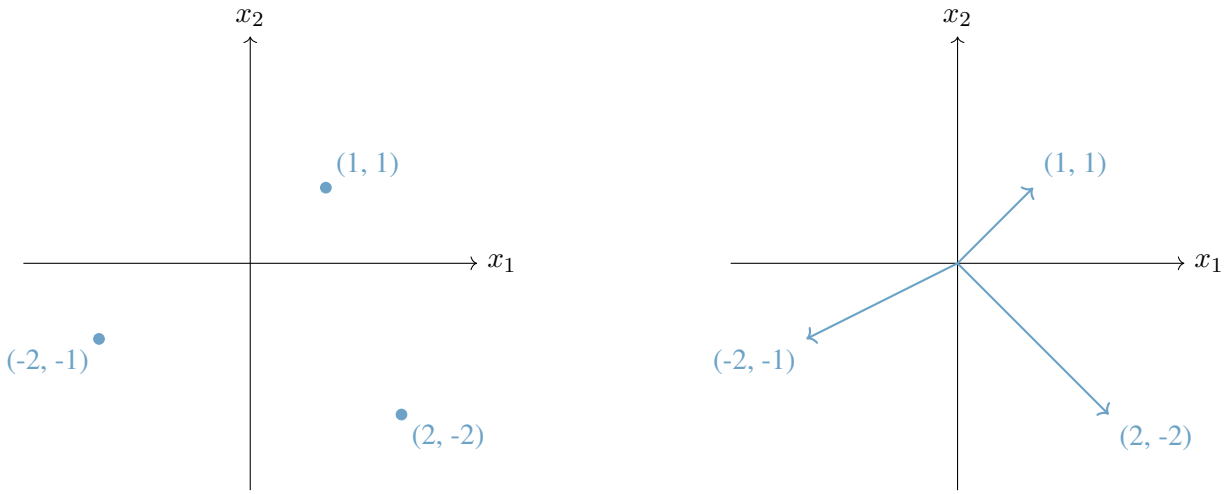
$$\boldsymbol{v} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

Vi siger, at to vektorer er **ens**, hvis og kun hvis deres tilsvarende indgange er ens.

Eksempel 8.1 Følgende er vektorer i $\mathbb{R}^2$ (dvs. planen bestående af ordnede par af reelle tal):

$$\boldsymbol{u} = \begin{bmatrix} 3 \\ 5 \end{bmatrix}, \quad \boldsymbol{v} = \begin{bmatrix} 5 \\ 3 \end{bmatrix}$$

De er ikke ens, fordi deres tilsvarende indgange ikke stemmer overens.



(a) Punkter i planen **(b)** Vektorer ud fra origo

Figur 8.1: Punkter og vektorer i planen

Summen af to vektorer $\boldsymbol{u}$ og $\boldsymbol{v}$ i $\mathbb{R}^2$, som skrives $\boldsymbol{u} + \boldsymbol{v}$, fås ved at lægge de tilsvarende indgange i $\boldsymbol{u}$ og $\boldsymbol{v}$ sammen. Givet et reelt tal c fås multiplummet af $\boldsymbol{u}$ med skalaren c, som skrives $c\boldsymbol{u}$, ved at gange hver indgang i $\boldsymbol{u}$ med c.

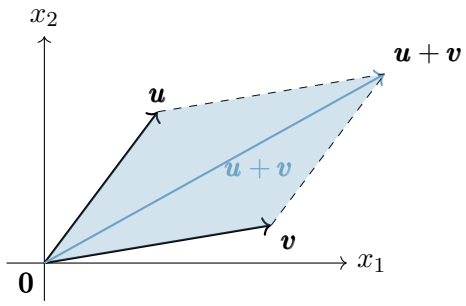
Eksempel 8.2 Hvis $\boldsymbol{u}$ og $\boldsymbol{v}$ er som i det foregående eksempel, så er

$$\boldsymbol{u}+\boldsymbol{v}=\begin{bmatrix}3\\5\end{bmatrix}+\begin{bmatrix}5\\3\end{bmatrix}=\begin{bmatrix}3+5\\5+3\end{bmatrix}=\begin{bmatrix}8\\8\end{bmatrix}, \text{ og}$$
$$6\boldsymbol{u}=6\begin{bmatrix}3\\5\end{bmatrix}=\begin{bmatrix}6\cdot 3\\6\cdot 5\end{bmatrix}=\begin{bmatrix}18\\30\end{bmatrix}$$

Bemærkning: Det er ofte en hjælp at identificere en vektor $\begin{bmatrix}a\\b\end{bmatrix}\in\mathbb{R}^2$ med et geometrisk punkt (a,b) i planen for at få et billede af det, vi arbejder med. Se Figur 8.1 for et eksempel.

Hvis vi opfatter $\boldsymbol{u}$ og $\boldsymbol{v}$ i $\mathbb{R}^2$ som punkter i planen, svarer $\boldsymbol{u}+\boldsymbol{v}$ til det fjerde hjørne i det parallelogram, hvis øvrige hjørner er $\mathbf{0}, \boldsymbol{u}$ og $\boldsymbol{v}$. Bemærk: Med $\mathbf{0}$ mener vi nulvektoren, dvs. vektoren, hvis indgange alle er nul. I $\mathbb{R}^2$ har vi $\mathbf{0}=\begin{bmatrix}0\\0\end{bmatrix}$.

Vi kalder dette *parallelogramreglen for addition*, og den er vist i Figur 8.2.



Figur 8.2: Parallelogramreglen for vektoraddition i $\mathbb{R}^2$.

Disse idéer kan generaliseres til rum af højere dimension. Mere præcist kan vi definere $\mathbb{R}^n$ på følgende måde.

Definition 8.1

For hvert positivt heltal n lader vi $\mathbb{R}^n$ betegne samlingen af ordnede n-tupler, hvor hver indgang ligger i $\mathbb{R}$. Vi skriver ofte disse elementer som $n\times 1$-matricer. Addition og multiplikation med skalar af vektorer i $\mathbb{R}^n$ defineres på samme måde som for $\mathbb{R}^2$. Det vil sige, at vi regner koordinat for koordinat. ♣

Eksempel 8.3 Hvis $u_1, u_2, \ldots, u_n \in \mathbb{R}$, så er

$$\boldsymbol{u}=\begin{bmatrix}u_1\\u_2\\\vdots\\u_n\end{bmatrix}\in\mathbb{R}^n.$$

Eksempel 8.4 Hvis $\boldsymbol{u}$ og $\boldsymbol{v}$ ligger i $\mathbb{R}^n$ (med indgange betegnet henholdsvis $u_1,\ldots,u_n$ og $v_1,\ldots,v_n$), og $c\in\mathbb{R}$,

så er

$$\boldsymbol{u}+\boldsymbol{v}=\begin{bmatrix}u_1\\u_2\\\vdots\\u_n\end{bmatrix}+\begin{bmatrix}v_1\\v_2\\\vdots\\v_n\end{bmatrix}=\begin{bmatrix}u_1+v_1\\u_2+v_2\\\vdots\\u_n+v_n\end{bmatrix},\text{ og }\quad c\boldsymbol{u}=c\begin{bmatrix}u_1\\u_2\\\vdots\\u_n\end{bmatrix}=\begin{bmatrix}cu_1\\cu_2\\\vdots\\cu_n\end{bmatrix}$$

Algebraiske egenskaber ved $\mathbb{R}^n$

Lad $\boldsymbol{u},\boldsymbol{v},\boldsymbol{w}\in\mathbb{R}^n$ og $c,d\in\mathbb{R}$. Så gælder følgende egenskaber:

1. **Den kommutative lov for addition:** $\boldsymbol{u}+\boldsymbol{v}=\boldsymbol{v}+\boldsymbol{u}$.
2. **Den associative lov for addition:** $(\boldsymbol{u}+\boldsymbol{v})+\boldsymbol{w}=\boldsymbol{u}+(\boldsymbol{v}+\boldsymbol{w})$.
3. **Neutralt element for addition:** Der findes en vektor $\mathbf{0}\in\mathbb{R}^n$, så $\boldsymbol{u}+\mathbf{0}=\boldsymbol{u}$ for alle $\boldsymbol{u}\in\mathbb{R}^n$.
4. **Modsat vektor:** For hver $\boldsymbol{u}\in\mathbb{R}^n$ findes en vektor $-\boldsymbol{u}\in\mathbb{R}^n$, så $\boldsymbol{u}+(-\boldsymbol{u})=\mathbf{0}$.
5. **Den distributive lov:** $c(\boldsymbol{u}+\boldsymbol{v})=c\boldsymbol{u}+c\boldsymbol{v}$ og $(c+d)\boldsymbol{u}=c\boldsymbol{u}+d\boldsymbol{u}$.
6. **Den associative lov for multiplikation med skalar:** $c(d\boldsymbol{u})=(cd)\boldsymbol{u}$.
7. **Neutralt element for multiplikation:** $1\boldsymbol{u}=\boldsymbol{u}$.

En **linearkombination** er en måde at kombinere vektorer på ved hjælp af multiplikation med skalar og addition. Givet en mængde af vektorer ganger vi hver vektor med en skalar og lægger derefter resultaterne sammen. Linearkombinationer hjælper os med at forstå, hvordan vektorer hænger sammen, og om en vektor kan udtrykkes ved hjælp af andre.

Definition 8.2 (Linearkombinationer)

Givet en mængde af vektorer $\boldsymbol{v}_1,\boldsymbol{v}_2,\ldots,\boldsymbol{v}_p\in\mathbb{R}^n$ og skalarer $c_1,c_2,\ldots,c_p\in\mathbb{R}$ kaldes vektoren $\boldsymbol{y}$ givet ved

$$\boldsymbol{y}=c_1\boldsymbol{v}_1+\cdots+c_p\boldsymbol{v}_p$$

en linearkombination af $\boldsymbol{v}_1,\boldsymbol{v}_2,\ldots,\boldsymbol{v}_p$ med vægtene $c_1,c_2,\ldots,c_p$.

♣

Eksempel 8.5 Lad $\boldsymbol{v}_1=\begin{bmatrix}1\\2\end{bmatrix}$ og $\boldsymbol{v}_2=\begin{bmatrix}1\\-1\end{bmatrix}$. Nogle linearkombinationer af $\boldsymbol{v}_1$ og $\boldsymbol{v}_2$ er

$$\begin{aligned}\mathbf{0}&=0\boldsymbol{v}_1+0\boldsymbol{v}_2\\\begin{bmatrix}3\\0\end{bmatrix}&=\boldsymbol{v}_1+2\boldsymbol{v}_2\\\begin{bmatrix}-5\\-1\end{bmatrix}&=-2\boldsymbol{v}_1-3\boldsymbol{v}_2,\text{ og}\\\begin{bmatrix}5\\1\end{bmatrix}&=2\boldsymbol{v}_1+3\boldsymbol{v}_2\end{aligned}$$

Vektorligninger og lineære ligningssystemer

Vi ønsker ofte at vide, om en vektor $\boldsymbol{b}$ kan dannes som en linearkombination af en anden mængde af vektorer $\boldsymbol{a}_1,\ldots,\boldsymbol{a}_n$. Fremgangsmåden til at afgøre det er givet ved følgende.

Brug af matricer til at bestemme linearkombinationer

En vektorligning

$$x_1\boldsymbol{a}_1 + x_2\boldsymbol{a}_2 + \cdots + x_n\boldsymbol{a}_n = \boldsymbol{b}$$

har samme løsningsmængde som det lineære ligningssystem, hvis totalmatrix er

$$\left[\begin{array}{cccc|c} \boldsymbol{a}_1 & \boldsymbol{a}_2 & \dots & \boldsymbol{a}_n & \boldsymbol{b} \end{array}\right].$$

Mere præcist: Hvis vektorerne $\boldsymbol{a}_i$ ligger i $\mathbb{R}^m$ med

$$\boldsymbol{a}_1 = \begin{bmatrix} a_{11} \\ a_{21} \\ \vdots \\ a_{m1} \end{bmatrix}, \quad \boldsymbol{a}_2 = \begin{bmatrix} a_{12} \\ a_{22} \\ \vdots \\ a_{m2} \end{bmatrix}, \dots, \quad \boldsymbol{a}_n = \begin{bmatrix} a_{1n} \\ a_{2n} \\ \vdots \\ a_{mn} \end{bmatrix}, \quad \text{og} \quad \boldsymbol{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}$$

så rækkereducerer man matricen

$$\left[\begin{array}{cccc|c} a_{11} & a_{12} & \dots & a_{1n} & b_1 \\ a_{21} & a_{22} & \dots & a_{2n} & b_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} & b_m \\ \uparrow & \uparrow & \dots & \uparrow & \uparrow \\ \boldsymbol{a}_1 & \boldsymbol{a}_2 & \dots & \boldsymbol{a}_n & \boldsymbol{b} \end{array}\right]$$

for at afgøre, om der findes et sæt vægte $x_1, \dots, x_n$, der passer.

Eksempel 8.6 Lad

$$\boldsymbol{a}_1 = \begin{bmatrix} 2 \\ -1 \\ 1 \end{bmatrix}, \quad \boldsymbol{a}_2 = \begin{bmatrix} 0 \\ 8 \\ -2 \end{bmatrix}, \quad \boldsymbol{a}_3 = \begin{bmatrix} 6 \\ 5 \\ 1 \end{bmatrix}, \quad \text{og } \boldsymbol{b} = \begin{bmatrix} 10 \\ 3 \\ 7 \end{bmatrix}$$

Vi afgør, om $\boldsymbol{b}$ er en linearkombination af $\boldsymbol{a}_1, \boldsymbol{a}_2, \boldsymbol{a}_3$, dvs. om der findes et sæt vægte x_1, x_2, x_3, så $x_1\boldsymbol{a}_1 + x_2\boldsymbol{a}_2 + x_3\boldsymbol{a}_3 = \boldsymbol{b}$. Ifølge ovenstående oversætter vi spørgsmålet til matrixform.

$$\left[\begin{array}{cccc} 2 & 0 & 6 & 10 \\ -1 & 8 & 5 & 3 \\ 1 & -2 & 1 & 7 \end{array}\right] \quad r_1 \leftrightarrow r_3 \quad \left[\begin{array}{cccc} 1 & -2 & 1 & 7 \\ -1 & 8 & 5 & 3 \\ 2 & 0 & 6 & 10 \end{array}\right]$$

$$\begin{array}{l} r_2 \mapsto r_2 + r_1 \\ r_3 \mapsto r_3 - 2r_1 \end{array} \quad \left[\begin{array}{cccc} 1 & -2 & 1 & 7 \\ 0 & 6 & 6 & 10 \\ 0 & 4 & 4 & -4 \end{array}\right]$$

$$r_3 \mapsto r_3 - \frac{4}{6}r_2 \quad \left[\begin{array}{cccc} 1 & -2 & 1 & 7 \\ 0 & 6 & 6 & 10 \\ 0 & 0 & 0 & \blacksquare \end{array}\right].$$

Da symbolet ■ betegner noget, der er forskelligt fra nul, ser vi, at der ikke er nogen løsning, dvs. at $\boldsymbol{b}$ ikke er en linearkombination af $\boldsymbol{a}_1, \boldsymbol{a}_2, \boldsymbol{a}_3$.

Eksempel 8.7 Lad

$$\boldsymbol{a}_1 = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}, \quad \boldsymbol{a}_2 = \begin{bmatrix} -4 \\ 6 \\ -4 \end{bmatrix}, \quad \boldsymbol{a}_3 = \begin{bmatrix} -6 \\ 7 \\ 5 \end{bmatrix}, \quad \text{og } \boldsymbol{b} = \begin{bmatrix} 11 \\ -5 \\ 9 \end{bmatrix}$$

Vi afgør, om $\boldsymbol{b}$ er en linearkombination af $\boldsymbol{a}_1, \boldsymbol{a}_2, \boldsymbol{a}_3$. Vi reducerer den tilhørende matrix til echelonform:

$$\begin{bmatrix} 1 & -4 & -6 & 11 \\ 0 & 6 & 7 & -5 \\ 1 & -4 & 5 & 9 \end{bmatrix} \quad r_3 \mapsto r_3 - r_1 \quad \begin{bmatrix} 1 & -4 & -6 & 11 \\ 0 & 6 & 7 & -5 \\ 0 & 0 & 11 & -2 \end{bmatrix}$$

og vi ser, at der findes en løsning. Nu finder vi de vægte x_1, x_2, x_3, der passer, ved at bestemme den reducerede echelonform.

$$r_3 \mapsto \frac{1}{11} r_3 \qquad \begin{bmatrix} 1 & -4 & -6 & 11 \\ 0 & 6 & 7 & -5 \\ 0 & 0 & 1 & -\frac{2}{11} \end{bmatrix}$$

$$\begin{matrix} r_2 \mapsto r_2 - 7r_3 \\ r_1 \mapsto r_1 + 6r_3 \end{matrix} \qquad \begin{bmatrix} 1 & -4 & 0 & \frac{109}{11} \\ 0 & 6 & 0 & -\frac{41}{11} \\ 0 & 0 & 1 & -\frac{2}{11} \end{bmatrix}$$

$$r_2 \mapsto \frac{1}{6} r_2 \qquad \begin{bmatrix} 1 & -4 & 0 & \frac{109}{11} \\ 0 & 1 & 0 & -\frac{41}{66} \\ 0 & 0 & 1 & -\frac{2}{11} \end{bmatrix}$$

$$r_1 \mapsto r_1 + 4r_2 \qquad \begin{bmatrix} 1 & 0 & 0 & \frac{245}{33} \\ 0 & 1 & 0 & -\frac{41}{66} \\ 0 & 0 & 1 & -\frac{2}{11} \end{bmatrix},$$

så $(x_1, x_2, x_3) = \left(\frac{245}{33}, -\frac{41}{66}, -\frac{2}{11}\right)$. Da der ikke er nogen frie variable, er dette den entydige løsning.

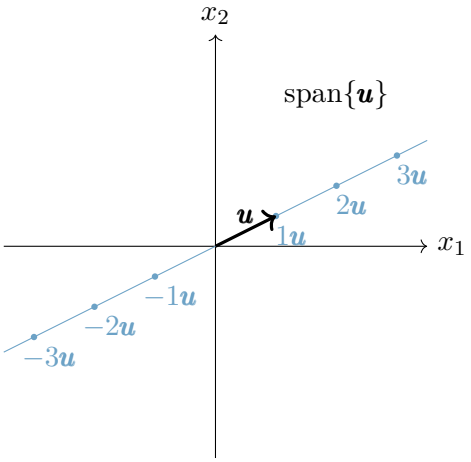
Udspændet (Span) af en mængde af vektorer er samlingen af alle mulige linearkombinationer af vektorerne. Det er med andre ord mængden af alle de vektorer, man kan nå ved at skalere og lægge de givne vektorer sammen (se eksempler i Figur 8.3). Udspændet giver os indsigt i, hvilket »rum« vektorerne dækker:

Definition 8.3 (Udspændet af en mængde af vektorer)

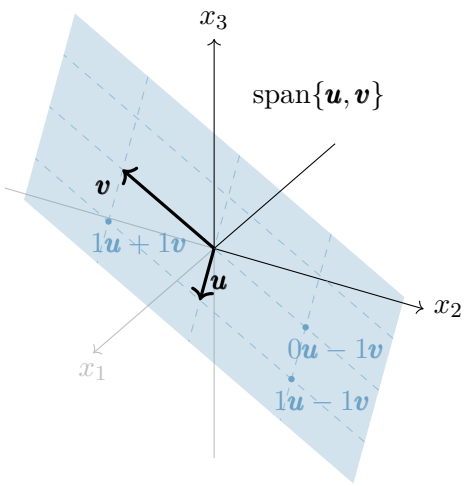
Hvis $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_p$ ligger i $\mathbb{R}^n$, betegnes mængden af alle linearkombinationer af $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_p$ med Span $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_p\}$, og den kaldes den delmængde af $\mathbb{R}^n$, der udspændes af $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_p$. Udspændet af $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_p$ består med andre ord af alle vektorer, der kan skrives på formen

$$c_1\boldsymbol{v}_1 + \cdots + c_p\boldsymbol{v}_p$$

hvor $c_1, \ldots, c_p$ er skalarer.



(a) Udspændet af én vektor i $\mathbb{R}^2$ er en linje gennem origo.



(b) Udspændet af to ikke-parallelle vektorer i $\mathbb{R}^3$ er en plan gennem origo.

Figur 8.3: To eksempler på den geometriske fortolkning af udspænd

8.2 Matrixligninger

I de foregående afsnit så vi på forskellige måder at beskrive lineære sammenhænge på. Vi så for eksempel på enkelte lineære ligninger som

$$2x_1 + 3x_2 = 5,$$

og på lineære ligningssystemer som

$$x_1 + 2x_2 = 4$$
$$3x_1 - x_2 = 2.$$

Vi skrev også disse systemer som vektorligninger, fx

$$x_1 \begin{bmatrix} 1 \\ 3 \end{bmatrix} + x_2 \begin{bmatrix} 2 \\ -1 \end{bmatrix} = \begin{bmatrix} 4 \\ 2 \end{bmatrix}.$$

Nu er det tid til at introducere en ny og stærk form: **matrixligningen**. Denne kompakte skrivemåde gør det muligt at håndtere lineære ligningssystemer effektivt med matrixnotation. På denne form bliver vores system til

$$A\boldsymbol{x} = \boldsymbol{b},$$

hvor A er en matrix, $\boldsymbol{x}$ er en vektor af variable, og $\boldsymbol{b}$ er resultatvektoren. Matrixligninger giver os en struktureret, algebraisk tilgang til at løse ligningssystemer.

Definition 8.4 (Matrixligning)

Hvis A er en $m \times n$-matrix med søjlerne $\boldsymbol{a}_1, \ldots, \boldsymbol{a}_n$, og hvis $\boldsymbol{x} \in \mathbb{R}^n$, så er produktet af A og $\boldsymbol{x}$, som skrives $A\boldsymbol{x}$, givet ved

$$A\boldsymbol{x} = \begin{bmatrix} \boldsymbol{a}_1 & \boldsymbol{a}_2 & \cdots & \boldsymbol{a}_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = x_1\boldsymbol{a}_1 + x_2\boldsymbol{a}_2 + \cdots + x_n\boldsymbol{a}_n$$

Eksempel 8.8

$$\begin{bmatrix} 4 & 1 & 2 \\ 8 & 0 & 3 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \\ 4 \end{bmatrix} = 2 \begin{bmatrix} 4 \\ 8 \end{bmatrix} + 1 \begin{bmatrix} 1 \\ 0 \end{bmatrix} + 4 \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$
$$= \begin{bmatrix} 8 \\ 16 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 8 \\ 12 \end{bmatrix}$$
$$= \begin{bmatrix} 17 \\ 28 \end{bmatrix}$$

Ved at bygge lidt videre på hovedresultatet fra det foregående afsnit får vi følgende sætning.

Sætning 8.1

Hvis A er en $m \times n$-matrix med søjlerne $\boldsymbol{a}_1, \ldots, \boldsymbol{a}_n \in \mathbb{R}^m$, og hvis $\boldsymbol{b} \in \mathbb{R}^m$, så har matrixligningen

$$A\boldsymbol{x} = \boldsymbol{b}$$

samme løsningsmængde som vektorligningen

$$x_1\boldsymbol{a}_1 + x_2\boldsymbol{a}_2 + \cdots + x_n\boldsymbol{a}_n = \boldsymbol{b}$$

som igen har samme løsningsmængde som det lineære ligningssystem med totalmatrixen

$$\begin{bmatrix} \boldsymbol{a}_1 & \boldsymbol{a}_2 & \ldots & \boldsymbol{a}_n & \boldsymbol{b} \end{bmatrix}.$$

♡

Sætningen er vigtig, fordi den binder de tre måder at beskrive og løse lineære ligningssystemer på sammen: matrixligninger, vektorligninger og ligningssystemer med totalmatricer. Den viser, at uanset hvilken form vi bruger, fører de alle til samme løsningsmængde.

Når matrixligningen $A\boldsymbol{x} = \boldsymbol{b}$ er ækvivalent med vektorligningen $x_1\boldsymbol{a}_1 + x_2\boldsymbol{a}_2 + \cdots + x_n\boldsymbol{a}_n = \boldsymbol{b}$, kan vi tolke det at løse en matrixligning som at finde den rette kombination af søjlerne i A, der giver $\boldsymbol{b}$. Denne sammenhæng gør det muligt at anskueliggøre problemet ved hjælp af vektorrum og linearkombinationer.

Sætningen viser desuden, at den samme proces afspejles i totalmatrixen, hvor rækkereduktion afslører løsningen gennem elementære rækkeoperationer. Uanset om vi angriber problemet algebraisk, geometrisk eller algoritmisk, har vi altså med den samme underliggende struktur at gøre. Dette samlende perspektiv forenkler vores tilgang til at løse ligningssystemer og giver fleksibilitet i, hvordan vi vælger at beskrive og behandle problemet.

Bemærkning: Ligningen $A\boldsymbol{x} = \boldsymbol{b}$ har en løsning, hvis og kun hvis $\boldsymbol{b}$ er en linearkombination af søjlerne i A.

Følgende sætning fortæller os, hvornår søjlevektorerne i en $m \times n$-matrix (søjlerne er vektorer i $\mathbb{R}^m$, da der er m rækker) kan bruges til at frembringe hele $\mathbb{R}^m$. Den opsummerer i bund og grund flere ting, vi allerede har set i forskellige sammenhænge.

Sætning 8.2

Lad A være en $m \times n$-matrix. Så er følgende udsagn ækvivalente:

(a) For hver $\boldsymbol{b} \in \mathbb{R}^m$ har ligningen $A\boldsymbol{x} = \boldsymbol{b}$ en løsning.

(b) Hver $\boldsymbol{b} \in \mathbb{R}^m$ er en linearkombination af søjlerne i A.

(c) Søjlerne i A udspænder $\mathbb{R}^m$.

(d) Matricen A har en pivotposition i hver række.

♡

Sætningen viser, at flere tilsyneladende forskellige idéer faktisk er ækvivalente. Om matrixligningen $A\boldsymbol{x} = \boldsymbol{b}$ har en løsning (udsagn (a)), afhænger af, om søjlerne i A kan danne en hvilken som helst vektor i $\mathbb{R}^m$ (udsagn (b)). Geometrisk betyder det, at søjlerne udspænder hele rummet $\mathbb{R}^m$ (udsagn (c)).

Endelig giver tilstedeværelsen af en pivotposition i hver række (udsagn (d)) en algebraisk betingelse, der garanterer, at søjlernes udspænd dækker $\mathbb{R}^m$, så $A\boldsymbol{x} = \boldsymbol{b}$ altid har en løsning.

Denne ækvivalens er et stærkt værktøj, fordi den forbinder løsninger, linearkombinationer og geometriske fortolkninger og samtidig giver en konkret metode (at undersøge pivotpositionerne) til at efterprøve disse egenskaber.

Række-vektor-reglen for beregning af $A\boldsymbol{x}$

Forudsat at produktet $A\boldsymbol{x}$ er defineret, er den i'te indgang i $A\boldsymbol{x}$ summen af produkterne af de tilsvarende indgange fra række i i A og vektoren $\boldsymbol{x}$. Med andre ord er den i'te indgang i $A\boldsymbol{x}$ prikproduktet af den vektor, der udgør den i'te række i A, og vektoren $\boldsymbol{x}$.

Reglen forenkler matrixmultiplikation ved at dele den op i velkendte prikprodukter. Der findes også en komplementær tolkning via søjlerne: Hvis søjlerne i A er $\boldsymbol{a}_1, \ldots, \boldsymbol{a}_n$ og $\boldsymbol{x} = (x_1, \ldots, x_n)^T$, så er

$$A\boldsymbol{x} = x_1\boldsymbol{a}_1 + \cdots + x_n\boldsymbol{a}_n.$$

$A\boldsymbol{x}$ er altså en linearkombination af *søjlerne* i A, mens hver enkelt indgang i produktet er et prikprodukt med en række i A.

Eksempel 8.9 Lad

$$A = \begin{bmatrix} 1 & -1 & 4 \\ 5 & 0 & 2 \end{bmatrix}, \text{ og } \boldsymbol{x} = \begin{bmatrix} 2 \\ 1 \\ 5 \end{bmatrix}$$

Så er

$$\begin{bmatrix} 1 & -1 & 4 \\ 5 & 0 & 2 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \\ 5 \end{bmatrix} = \begin{bmatrix} 1 \cdot 2 + (-1) \cdot 1 + 4 \cdot 5 \\ 5 \cdot 2 + 0 \cdot 1 + 2 \cdot 5 \end{bmatrix}$$
$$= \begin{bmatrix} 21 \\ 20 \end{bmatrix}$$

Bemærk, at antallet af søjler i A skal være lig med antallet af rækker i $\boldsymbol{x}$, da prikproduktet ellers ikke ville give mening!

Den næste sætning indfanger to vigtige egenskaber ved multiplikation af en matrix med en vektor: **distributivitet** og **multiplikation med skalar**. Egenskaberne svarer til de velkendte regneregler i algebra, men gælder nu for matricer og vektorer.

Sætning 8.3

Hvis A er en $m \times n$-matrix, $\boldsymbol{u}$ og $\boldsymbol{v}$ er vektorer i $\mathbb{R}^n$, og c er en skalar, så gælder:

(a) $A(\boldsymbol{u} + \boldsymbol{v}) = A\boldsymbol{u} + A\boldsymbol{v}$

(b) $A(c\boldsymbol{u}) = c(A\boldsymbol{u})$

♡

Egenskab (a) viser, at matrixmultiplikation er distributiv over vektoraddition. Det betyder, at det at gange A med summen af to vektorer giver det samme som at gange A med hver vektor for sig og derefter lægge resultaterne sammen. Egenskab (b) viser, at multiplikation med skalar kommuterer med matrixmultiplikation. At gange en vektor med en skalar først og derefter anvende matricen giver det samme resultat som først at anvende matricen og derefter gange den resulterende vektor med skalaren.

Eksempel 8.10 Lad

$$A = \begin{bmatrix} 1 & 5 & -2 & 0 \\ -3 & 1 & 9 & -5 \\ 4 & -8 & -1 & 7 \end{bmatrix}, \quad \boldsymbol{p} = \begin{bmatrix} 3 \\ -2 \\ 0 \\ -4 \end{bmatrix}, \quad \boldsymbol{b} = \begin{bmatrix} -7 \\ 9 \\ 0 \end{bmatrix}$$

Det kan vises, at $\boldsymbol{p}$ er en løsning til $A\boldsymbol{x} = \boldsymbol{b}$. Brug dette til at skrive $\boldsymbol{b}$ som en linearkombination af søjlerne i A.

Løsning: Da $\boldsymbol{p}$ er en løsning til $A\boldsymbol{x} = \boldsymbol{b}$, har vi:

$$A\boldsymbol{p} = \boldsymbol{b}.$$

Vi kan udtrykke $\boldsymbol{b}$ som en linearkombination af søjlerne i A ved hjælp af indgangene i $\boldsymbol{p}$. Lad $\boldsymbol{a}_1$, $\boldsymbol{a}_2$, $\boldsymbol{a}_3$ og $\boldsymbol{a}_4$ betegne søjlerne i A, så:

$$A = \left[\boldsymbol{a}_1 \mid \boldsymbol{a}_2 \mid \boldsymbol{a}_3 \mid \boldsymbol{a}_4\right].$$

Dermed er

$$\boldsymbol{b} = p_1\boldsymbol{a}_1 + p_2\boldsymbol{a}_2 + p_3\boldsymbol{a}_3 + p_4\boldsymbol{a}_4.$$

Nu beregner vi hvert led for sig.

Beregn $p_1\boldsymbol{a}_1$:

$$p_1\boldsymbol{a}_1 = 3\begin{bmatrix} 1 \\ -3 \\ 4 \end{bmatrix} = \begin{bmatrix} 3 \\ -9 \\ 12 \end{bmatrix}.$$

Beregn $p_2\boldsymbol{a}_2$:

$$p_2\boldsymbol{a}_2 = (-2)\begin{bmatrix} 5 \\ 1 \\ -8 \end{bmatrix} = \begin{bmatrix} -10 \\ -2 \\ 16 \end{bmatrix}.$$

Beregn $p_3\boldsymbol{a}_3$:

$$p_3\boldsymbol{a}_3 = 0\begin{bmatrix} -2 \\ 9 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}.$$

Beregn $p_4\boldsymbol{a}_4$:

$$p_4\boldsymbol{a}_4 = (-4)\begin{bmatrix} 0 \\ -5 \\ 7 \end{bmatrix} = \begin{bmatrix} 0 \\ 20 \\ -28 \end{bmatrix}.$$

Læg de beregnede vektorer sammen for at finde $\boldsymbol{b}$:

$$\boldsymbol{b} = p_1\boldsymbol{a}_1 + p_2\boldsymbol{a}_2 + p_3\boldsymbol{a}_3 + p_4\boldsymbol{a}_4 = \begin{bmatrix} 3 \\ -9 \\ 12 \end{bmatrix} + \begin{bmatrix} -10 \\ -2 \\ 16 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 20 \\ -28 \end{bmatrix}.$$

Nu beregner vi summen trin for trin.

Trin 1: Læg $p_1\boldsymbol{a}_1$ og $p_2\boldsymbol{a}_2$ sammen:

$$\begin{bmatrix} 3 \\ -9 \\ 12 \end{bmatrix} + \begin{bmatrix} -10 \\ -2 \\ 16 \end{bmatrix} = \begin{bmatrix} -7 \\ -11 \\ 28 \end{bmatrix}.$$

Trin 2: Læg $p_3\boldsymbol{a}_3$ (som er nul) til resultatet:

$$\begin{bmatrix} -7 \\ -11 \\ 28 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} -7 \\ -11 \\ 28 \end{bmatrix}.$$

Trin 3: Læg $p_4\boldsymbol{a}_4$ til resultatet:

$$\begin{bmatrix} -7 \\ -11 \\ 28 \end{bmatrix} + \begin{bmatrix} 0 \\ 20 \\ -28 \end{bmatrix} = \begin{bmatrix} -7 \\ 9 \\ 0 \end{bmatrix}.$$

Endeligt resultat:

$$\boldsymbol{b} = \begin{bmatrix} -7 \\ 9 \\ 0 \end{bmatrix}.$$

Konklusion:

Vi har udtrykt $\boldsymbol{b}$ som en linearkombination af søjlerne i A ved hjælp af indgangene i $\boldsymbol{p}$:

$$\boldsymbol{b} = 3\boldsymbol{a}_1 - 2\boldsymbol{a}_2 + 0\boldsymbol{a}_3 - 4\boldsymbol{a}_4.$$

◀

8.3 Løsningsmængder for lineære ligningssystemer

Løsningsmængder for lineære ligningssystemer spiller en central rolle i lineær algebra og dukker op igen i mange sammenhænge i faget. I dette afsnit bruger vi vektornotation til at give klare, eksplicitte og geometriske beskrivelser af disse løsningsmængder.

Et lineært ligningssystem kaldes *homogent*, hvis det kan skrives på formen

$$A\boldsymbol{x} = \boldsymbol{0}$$

hvor A er en $m \times n$-matrix, og $\boldsymbol{0}$ er nulvektoren i $\mathbb{R}^m$. Et sådant system har altid den *trivielle løsning*, nemlig $\boldsymbol{x} = \boldsymbol{0}$ (nulvektoren i $\mathbb{R}^n$, ikke $\mathbb{R}^m$).

Da homogene systemer altid har den trivielle løsning, er det interessante spørgsmål, om de har *ikke-trivielle løsninger*.

Bemærkning: Den homogene ligning $A\boldsymbol{x} = \boldsymbol{0}$ har en ikke-triviel løsning, hvis og kun hvis ligningen har mindst én fri variabel.

Eksempel 8.11 Afgør, om følgende homogene system har en ikke-triviel løsning. Hvis det har, så beskriv

løsningsmængden ved hjælp af den eller de frie variable.

$$\begin{aligned} 2x_1 + x_2 - 3x_3 &= 0 \\ x_1 - x_2 + x_3 &= 0 \\ -2x_1 + 5x_2 - 7x_3 &= 0 \end{aligned}$$

Løsning: Vi lader A være systemets koefficientmatrix og reducerer totalmatrixen $\left[\begin{array}{cc} A & 0 \end{array}\right]$ til echelonform (bemærk, at vi sætter den anden ligning først for at gøre beregningen lidt enklere):

$$\begin{bmatrix} 1 & -1 & 1 & 0 \\ 2 & 1 & -3 & 0 \\ -2 & 5 & -7 & 0 \end{bmatrix} \sim \begin{bmatrix} 1 & -1 & 1 & 0 \\ 0 & 3 & -5 & 0 \\ 0 & 3 & -5 & 0 \end{bmatrix} \sim \begin{bmatrix} 1 & -1 & 1 & 0 \\ 0 & 3 & -5 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

så vi ser, at x_3 er en fri variabel, og at der derfor findes ikke-trivielle løsninger. Nu bestemmer vi den reducerede echelonform:

$$\begin{bmatrix} 1 & -1 & 1 & 0 \\ 0 & 3 & -5 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \sim \begin{bmatrix} 1 & -1 & 1 & 0 \\ 0 & 1 & -\frac{5}{3} & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \sim \begin{bmatrix} 1 & 0 & -\frac{2}{3} & 0 \\ 0 & 1 & -\frac{5}{3} & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Det giver os ligningerne

$$\begin{aligned} x_1 - \frac{2}{3}x_3 = 0 \quad &\Longrightarrow x_1 = \frac{2}{3}x_3 \\ x_2 - \frac{5}{3}x_3 = 0 \quad &\Longrightarrow x_2 = \frac{5}{3}x_3 \end{aligned}$$

På vektorform skrives løsningen $\boldsymbol{x}$ til ligningen $A\boldsymbol{x} = 0$ som

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} \frac{2}{3}x_3 \\ \frac{5}{3}x_3 \\ x_3 \end{bmatrix} = x_3 \begin{bmatrix} \frac{2}{3} \\ \frac{5}{3} \\ 1 \end{bmatrix}$$

Da x_3 kan være hvad som helst, beskriver denne løsningsmængde geometrisk den linje i $\mathbb{R}^3$, der går fra origo gennem punktet $\left(\frac{2}{3}, \frac{5}{3}, 1\right)$. ◀

Parameterfremstilling på vektorform

Når en løsningsmængde er beskrevet eksplicit med vektorer (som i det foregående eksempel), siger vi, at løsningen er givet som en **parameterfremstilling på vektorform**.

Eksempel 8.12 Beskriv alle løsninger til den homogene ligning

$$x_1 - 2x_2 - 5x_3 = 0$$

Løsning:

Hvis vi skriver systemet som en matrix, ser vi, at x_2 og x_3 er frie variable, og at x_1 er en basisvariabel med $x_1 = 2x_2 + 5x_3$. Den fuldstændige løsning er derfor

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 2x_2 + 5x_3 \\ x_2 \\ x_3 \end{bmatrix} = x_2 \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix} + x_3 \begin{bmatrix} 5 \\ 0 \\ 1 \end{bmatrix}$$

hvor den yderste højre del af ligningen er løsningsmængdens **parameterfremstilling på vektorform**. I dette tilfælde er hele løsningsmængden en plan i $\mathbb{R}^3$, som indeholder de to linjer fra $(0,0,0)$ til $(2,1,0)$ og fra $(0,0,0)$ til $(5,0,1)$. ◀

Når et *inhomogent* lineært ligningssystem har mange løsninger, kan den fuldstændige løsning skrives som en parameterfremstilling på vektorform, der består af én vektor plus vilkårlige linearkombinationer af vektorer, som opfylder det tilhørende homogene system.

Eksempel 8.13 Vi ser igen på det homogene system fra begyndelsen af dette afsnit, men denne gang er det inhomogent (dvs. højresiden består ikke udelukkende af nuller):

$$\begin{aligned} 2x_1 + x_2 - 3x_3 &= 2 \\ x_1 - x_2 + x_3 &= 1 \\ -2x_1 + 5x_2 - 7x_3 &= -2 \end{aligned}$$

Som før udfører vi rækkeoperationer på totalmatrixen $\left[\begin{array}{cc} A & \boldsymbol{b} \end{array}\right]$, hvor A er den samme koefficientmatrix og

$$\boldsymbol{b} = \begin{bmatrix} 2 \\ 1 \\ -2 \end{bmatrix}$$

og vi får, at

$$\left[\begin{array}{cccc} 1 & -1 & 1 & 1 \\ 2 & 1 & -3 & 2 \\ -2 & 5 & -7 & -2 \end{array}\right] \sim \left[\begin{array}{cccc} 1 & 0 & -\frac{2}{3} & 1 \\ 0 & 1 & -\frac{5}{3} & 0 \\ 0 & 0 & 0 & 0 \end{array}\right]$$

Dermed er

$$\begin{aligned} x_1 &= 1 + \frac{2}{3}x_3, \text{ og} \\ x_2 &= \frac{5}{3}x_3 \end{aligned}$$

Dette kan skrives som

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 + \frac{2}{3}x_3 \\ \frac{5}{3}x_3 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + x_3 \begin{bmatrix} \frac{2}{3} \\ \frac{5}{3} \\ 1 \end{bmatrix}$$

som har den form, vi påstod. Det er ligningen for linjen gennem punktet $(1,0,0)$, der er parallel med linjen fra $(0,0,0)$ gennem $\left(\frac{2}{3}, \frac{5}{3}, 1\right)$.

Vi opsummerer den generelle situation med følgende sætning. Konklusionen kan måske bedst opfattes som en forskydning af alle de homogene løsninger med vektoren $\boldsymbol{p}$.

Sætning 8.4

Antag, at $A\boldsymbol{x} = \boldsymbol{b}$ er konsistent for et givet $\boldsymbol{b}$, og lad $\boldsymbol{p}$ være en løsning. Så er løsningsmængden for $A\boldsymbol{x} = \boldsymbol{b}$ mængden af alle vektorer på formen

$$\boldsymbol{w} = \boldsymbol{p} + \boldsymbol{v}_h,$$

hvor $\boldsymbol{v}_h$ er en vilkårlig løsning til den homogene ligning $A\boldsymbol{x} = \boldsymbol{0}$. ♡

Eksempel 8.14 Skriv den fuldstændige løsning til

$$x_1 - 2x_2 - 5x_3 = 3$$

som en parameterfremstilling på vektorform. Hvordan ser denne løsningsmængde geometrisk ud sammenlignet med løsningsmængden for ligningen $x_1 - 2x_2 - 5x_3 = 0$, som vi så tidligere?

Løsning: Vi løser ligningen:

$$x_1 - 2x_2 - 5x_3 = 3$$

For at udtrykke den fuldstændige løsning løser vi med hensyn til x_1 udtrykt ved x_2 og x_3:

$$x_1 = 3 + 2x_2 + 5x_3,$$
$$x_2 = x_2,$$
$$x_3 = x_3.$$

Vi kan skrive løsningsmængden som en *parameterfremstilling på vektorform*:

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \\ 0 \end{bmatrix} + x_2 \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix} + x_3 \begin{bmatrix} 5 \\ 0 \\ 1 \end{bmatrix}, \quad \text{for alle } x_2, x_3 \in \mathbb{R}.$$

Geometrisk fortolkning: Løsningsmængden er en plan i $\mathbb{R}^3$. Planen er parallel med den plan, der er bestemt af den homogene ligning:

$$x_1 - 2x_2 - 5x_3 = 0$$

Den homogene løsningsmængde kan skrives som:

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = x_2 \begin{bmatrix} 2 \\ 1 \\ 0 \end{bmatrix} + x_3 \begin{bmatrix} 5 \\ 0 \\ 1 \end{bmatrix}, \quad \text{for alle } x_2, x_3 \in \mathbb{R}.$$

Når vi sammenligner de to løsningsmængder, ser vi, at den oprindelige løsning er den homogene løsning forskudt med vektoren $\begin{bmatrix} 3 \\ 0 \\ 0 \end{bmatrix}$. Geometrisk betyder det, at planen bestemt af $x_1 - 2x_2 - 5x_3 = 3$ er parallel med, men ikke den samme som, planen bestemt af $x_1 - 2x_2 - 5x_3 = 0$. De to planer er forskudt 3 enheder i forhold til hinanden langs x_1-aksen. Den fuldstændige løsning til ligningen $x_1 - 2x_2 - 5x_3 = 3$ er en plan i $\mathbb{R}^3$, der er parallel med løsningsmængden for den homogene ligning $x_1 - 2x_2 - 5x_3 = 0$, men forskudt væk fra origo. ◀

Eksempel 8.15 Find en parameterfremstilling for linjen gennem $\boldsymbol{a} = \begin{bmatrix} 3 \\ -2 \end{bmatrix}$, der også er parallel med $\boldsymbol{b} = \begin{bmatrix} -7 \\ 6 \end{bmatrix}$.

Løsning: Linjen gennem $\boldsymbol{a}$ parallel med $\boldsymbol{b}$ er givet ved ligningen $\boldsymbol{r} = \boldsymbol{a} + t\boldsymbol{b}$, hvor t er en parameter. Ved at indsætte de givne vektorer får vi

$$\begin{aligned} \boldsymbol{r} &= \begin{bmatrix} 3 \\ -2 \end{bmatrix} + t \begin{bmatrix} -7 \\ 6 \end{bmatrix} \\ &= \begin{bmatrix} 3 - 7t \\ -2 + 6t \end{bmatrix} \end{aligned}$$

Dette er en parameterfremstilling for linjen gennem $\boldsymbol{a}$, der er parallel med $\boldsymbol{b}$. ◀

8.4 Lineær uafhængighed

Vi afslutter dette afsnit med at introducere et af de vigtigste begreber i lineær algebra: *lineær uafhængighed*. Begrebet er grundlæggende for at forstå vektorers struktur og deres indbyrdes sammenhænge i vektorrum.

Definition 8.5

En indekseret mængde af vektorer $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_p\} \subset \mathbb{R}^n$ kaldes **lineært uafhængig**, hvis vektorligningen

$$x_1\boldsymbol{v}_1 + \cdots + x_p\boldsymbol{v}_p = \boldsymbol{0}$$

kun har den trivielle løsning, dvs. hvis den eneste løsning er $(x_1, \ldots, x_p) = (0, \ldots, 0)$. Tilsvarende kaldes mængden $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_p\}$ **lineært afhængig**, hvis der findes vægte $c_1, \ldots, c_p$, som ikke alle er nul, så

$$c_1\boldsymbol{v}_1 + \cdots + c_p\boldsymbol{v}_p = \boldsymbol{0}$$

En sådan ligning kalder vi en *lineær afhængighedsrelation*, når vægtene ikke alle er nul. ♣

Bemærkning: En mængde af vektorer kan ikke være både lineært uafhængig og lineært afhængig, men den skal være det ene af de to!

At afgøre, om en mængde af vektorer er lineært uafhængig, svarer til at løse matrixligningen

$$A\boldsymbol{x} = 0,$$

hvor søjlerne i A er de givne vektorer. Mængden af vektorer er lineært uafhængig, hvis og kun hvis den eneste løsning er $\boldsymbol{x} = \boldsymbol{0}$. Ellers findes der en lineær afhængighedsrelation.

Eksempel 8.16 Afgør, om følgende vektorer i $\mathbb{R}^3$ er lineært uafhængige:

$$\boldsymbol{v}_1 = \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix}, \quad \boldsymbol{v}_2 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \quad \boldsymbol{v}_3 = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$$

Løsning: Vi begynder med at reducere den tilhørende totalmatrix til echelonform:

$$\begin{bmatrix} 2 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 2 & 1 & 1 & 0 \end{bmatrix} \sim \begin{bmatrix} 1 & 1 & 1 & 0 \\ 2 & 1 & 0 & 0 \\ 2 & 1 & 1 & 0 \end{bmatrix} \sim \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & -1 & -2 & 0 \\ 0 & -1 & -1 & 0 \end{bmatrix} \sim \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & -1 & -2 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

hvoraf vi ser, at der findes en løsning. Nu vil vi vide, om løsningen er den trivielle løsning $(0, 0, 0)$. Normalt ville vi fortsætte med rækkeoperationer, indtil vi når den reducerede echelonform, men vi kan gøre det smartere. Bemærk først og fremmest, at der ikke er nogen »dårlige« rækker (så der findes mindst én løsning), hvilket vi også forventer, da et homogent system altid har mindst den trivielle løsning. Da der ikke er nogen frie variable, ser vi, at der er præcis én løsning. Det fortæller os, at løsningen må være

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Mængden af vektorer $\{\boldsymbol{v}_1, \boldsymbol{v}_2, \boldsymbol{v}_3\}$ er altså lineært uafhængig. ◀

Generelt er det meget nyttigt at kunne afgøre, om en mængde af vektorer er lineært uafhængig, så det er godt at have nogle sætninger, der hurtigt klarer problemet i visse specialtilfælde.

Sætning 8.5

(a) Hvis en mængde af vektorer indeholder nulvektoren, er mængden lineært afhængig.

(b) Hvis en mængde af vektorer indeholder et skalarmultiplum af en anden vektor i mængden, er mængden lineært afhængig.

(c) Hvis en mængde af vektorer indeholder flere vektorer, end der er indgange i hver vektor, er mængden lineært afhængig.

♡

Sætningen giver hurtige test for lineær afhængighed. Mere generelt er en mængde lineært afhængig, hvis en af dens vektorer er en linearkombination af de andre. Vi kan formalisere denne idé i følgende korollar.

Korollar 8.1

En indekseret mængde $S = \{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_p\}$ af to eller flere vektorer er lineært afhængig, hvis og kun hvis mindst én af vektorerne i S er en linearkombination af de andre. Hvis S er lineært afhængig, og $\boldsymbol{v}_1 \neq \boldsymbol{0}$, så er en af vektorerne $\boldsymbol{v}_j$ (med $1 < j \leq p$) en linearkombination af de foregående vektorer $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_{j-1}$. ♡

Bemærkning: Korollaret fortæller os, at hvis en mængde af vektorer er lineært afhængig, er mindst én af vektorerne overflødig, da den kan udtrykkes som en linearkombination af de andre. Det er denne overflødighed, der giver den lineære afhængighed. Det betyder ikke, at hver vektor i S er en linearkombination af de andre, men kun at *mindst én* vektor i en lineært afhængig mængde er en linearkombination af andre.

Eksempel 8.17 Lad

$$\boldsymbol{u} = \begin{bmatrix} 2 \\ 1 \\ 1 \end{bmatrix}, \quad \boldsymbol{v} = \begin{bmatrix} 4 \\ 3 \\ 5 \end{bmatrix}, \quad \boldsymbol{w} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \quad \boldsymbol{z} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

(a) Er nogen af parrene af disse vektorer (fx $\{\boldsymbol{u}, \boldsymbol{v}\}$) lineært afhængige? Begrund dit svar.

(b) Fortæller svaret på del (a) os, at $\{\boldsymbol{u}, \boldsymbol{v}, \boldsymbol{w}, \boldsymbol{z}\}$ er lineært uafhængig?

(c) Er $\{\boldsymbol{u}, \boldsymbol{v}, \boldsymbol{w}, \boldsymbol{z}\}$ lineært afhængig? Du bør kunne besvare spørgsmålet uden at regne.

Løsning:

(a) For at afgøre, om nogen af parrene af disse vektorer er lineært afhængige, undersøger vi, om den ene vektor er et skalarmultiplum af den anden.

Undersøgelse af parret $\{\boldsymbol{u}, \boldsymbol{v}\}$: Antag, at der findes en skalar k, så:

$$\boldsymbol{v} = k\boldsymbol{u}.$$

Vi beregner k ud fra den første komposant:

$$k = \frac{v_1}{u_1} = \frac{4}{2} = 2.$$

Vi kontrollerer med den anden komposant:

$$v_2 = ku_2 \implies 3 = 2 \times 1 \implies 3 = 2.$$

Da $3 \neq 2$, er $\boldsymbol{v}$ ikke et skalarmultiplum af $\boldsymbol{u}$.

Undersøgelse af parret $\{\boldsymbol{u}, \boldsymbol{w}\}$: Antag, at der findes en skalar k, så:

$$\boldsymbol{u} = k\boldsymbol{w}.$$

Vi beregner k ud fra den første komposant:

$$k = \frac{u_1}{w_1} = \frac{2}{1} = 2.$$

Vi kontrollerer med den anden komposant:

$$u_2 = kw_2 \implies 1 = 2 \times 1 \implies 1 = 2.$$

Da $1 \neq 2$, er $\boldsymbol{u}$ ikke et skalarmultiplum af $\boldsymbol{w}$.

Undersøgelse af parret $\{\boldsymbol{w}, \boldsymbol{z}\}$: Da $z_1 = 0$ og $w_1 = 1$, fører antagelsen $\boldsymbol{w} = k\boldsymbol{z}$ til:

$$w_1 = kz_1 \implies 1 = k \times 0 \implies 1 = 0,$$

hvilket er en modstrid.

(b) Nej, at intet par er lineært afhængigt, betyder ikke, at hele mængden er lineært uafhængig. Lineær uafhængighed kræver, at den eneste løsning til:

$$c_1\boldsymbol{u} + c_2\boldsymbol{v} + c_3\boldsymbol{w} + c_4\boldsymbol{z} = \boldsymbol{0}$$

er $c_1 = c_2 = c_3 = c_4 = 0$.

(c) Ja, mængden er lineært afhængig. I $\mathbb{R}^3$ er enhver mængde med mere end tre vektorer nødvendigvis lineært afhængig, fordi det største antal lineært uafhængige vektorer i $\mathbb{R}^3$ er tre.

Konklusion:

(a) Intet par af disse vektorer er lineært afhængigt.

(b) Svaret på del (a) garanterer ikke, at hele mængden er lineært uafhængig.

(c) Mængden $\{\boldsymbol{u}, \boldsymbol{v}, \boldsymbol{w}, \boldsymbol{z}\}$ er lineært afhængig, uden at der er brug for yderligere beregninger.

◀

Kapitel 9 Matrixalgebra

I de foregående kapitler har vi set på begreber som lineære ligninger, echelonformer, rækkereduktion, lineær uafhængighed, vektorligninger og matrixligninger. For at løfte vores problemløsningsevner til næste niveau skal vi nu dykke ned i matrixalgebra. Matrixoperationer gør det muligt at håndtere komplekse systemer mere effektivt og bygger videre på det fundament, vi har lagt. Værktøjerne og teknikkerne i dette kapitel gør det lettere at arbejde med flere matricer og fører os frem mod dybere indsigt, herunder centrale idéer som sætningen om invertible matricer.

9.1 Regning med matricer

Regneoperationer på matricer som addition, subtraktion, skalering og multiplikation gør det muligt at kombinere og bearbejde matricer på enkle måder. Disse operationer er byggestenene i mere komplekse matrixmanipulationer, fx løsning af ligningssystemer og bestemmelse af inverse matricer.

Addition og multiplikation med skalar

Vi begynder med definitionen af matrixaddition.

Definition 9.1 (Matrixaddition)

Lad A og B være $m \times n$-matricer. **Summen** af A og B, betegnet $A + B$, er den $m \times n$-matrix, hvis indgange fås ved at addere de tilsvarende indgange i A og B.
For matricerne

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}, \quad B = \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ b_{m1} & b_{m2} & \cdots & b_{mn} \end{bmatrix}$$

som begge har dimensionen $m \times n$, er summen $A + B$ altså defineret som

$$A + B = \begin{bmatrix} a_{11} + b_{11} & a_{12} + b_{12} & \cdots & a_{1n} + b_{1n} \\ a_{21} + b_{21} & a_{22} + b_{22} & \cdots & a_{2n} + b_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{m1} + b_{m1} & a_{m2} + b_{m2} & \cdots & a_{mn} + b_{mn} \end{bmatrix}$$

♣

Bemærkning: Tilsvarende gælder for subtraktion af matricer, selv om det ikke nævnes eksplicit her.

Dernæst kommer definitionen af multiplikation af en matrix med en skalar.

Definition 9.2 (Multiplikation med skalar)

Lad A være en $m \times n$-matrix og c en skalar. **Produktet** af c og A, betegnet cA, er den $m \times n$-matrix, hvis indgange fås ved at multiplicere hver indgang i A med c. Produktet er altså defineret som

$$cA = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} = \begin{bmatrix} ca_{11} & ca_{12} & \cdots & ca_{1n} \\ ca_{21} & ca_{22} & \cdots & ca_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ ca_{m1} & ca_{m2} & \cdots & ca_{mn} \end{bmatrix}$$

Eksempel 9.1 Givet matricerne

$$A = \begin{bmatrix} -5 & 2 & 0 \\ 7 & -3 & 4 \\ -1 & 3 & 2 \end{bmatrix}, \quad B = \begin{bmatrix} 0 & -1 & 8 \\ 6 & -14 & 2 \\ 9 & 5 & 1 \end{bmatrix}$$

skal du bestemme summen $A + B$ og produktet $3A$.

Løsning: Summen $A + B$ fås ved at addere de tilsvarende indgange i A og B:

$$\begin{aligned} A + B &= \begin{bmatrix} -5 & 2 & 0 \\ 7 & -3 & 4 \\ -1 & 3 & 2 \end{bmatrix} + \begin{bmatrix} 0 & -1 & 8 \\ 6 & -14 & 2 \\ 9 & 5 & 1 \end{bmatrix} \\ &= \begin{bmatrix} (-5)+(0) & (2)+(-1) & (0)+(8) \\ (7)+(6) & (-3)+(-14) & (4)+(2) \\ (-1)+(9) & (3)+(5) & (2)+(1) \end{bmatrix} \\ &= \begin{bmatrix} -5 & 1 & 8 \\ 13 & -17 & 6 \\ 8 & 8 & 3 \end{bmatrix} \end{aligned}$$

Produktet $3A$ fås ved at multiplicere hver indgang i A med 3:

$$\begin{aligned} 3A &= 3\begin{bmatrix} -5 & 2 & 0 \\ 7 & -3 & 4 \\ -1 & 3 & 2 \end{bmatrix} \\ &= \begin{bmatrix} 3(-5) & 3(2) & 3(0) \\ 3(7) & 3(-3) & 3(4) \\ 3(-1) & 3(3) & 3(2) \end{bmatrix} \\ &= \begin{bmatrix} -15 & 6 & 0 \\ 21 & -9 & 12 \\ -3 & 9 & 6 \end{bmatrix} \end{aligned}$$

Eksempel 9.2 Bestem $3A - 2B$ for A og B nedenfor.

$$A = \begin{bmatrix} 1 & -2 & 5 \\ 0 & -3 & 9 \\ 4 & -6 & 7 \end{bmatrix}, B = \begin{bmatrix} 5 & 0 & -11 \\ 3 & -5 & 1 \\ -1 & -9 & 0 \end{bmatrix}$$

Løsning: Vi beregner:

$$\begin{aligned} 3A - 2B &= \begin{bmatrix} 3 & -6 & 15 \\ 0 & -9 & 27 \\ 12 & -18 & 21 \end{bmatrix} - \begin{bmatrix} 10 & 0 & -22 \\ 6 & -10 & 2 \\ -2 & -18 & 0 \end{bmatrix} \\ &= \begin{bmatrix} -7 & -6 & 37 \\ -6 & 1 & 25 \\ 14 & 0 & 21 \end{bmatrix} \end{aligned}$$

◀

Før vi går videre til matrixmultiplikation, skal vi formulere nogle grundlæggende algebraiske egenskaber ved matrixaddition og multiplikation med skalar.

Sætning 9.1

Lad A, B, C være matricer af samme størrelse, og lad α, β være skalarer. Så gælder

(a) $A + B = B + A$
(b) $(A + B) + C = A + (B + C)$
(c) $A + 0 = A$
(d) $\alpha(A + B) = \alpha A + \alpha B$
(e) $(\alpha + \beta)A = \alpha A + \beta A$
(f) $\alpha(\beta A) = (\alpha\beta)A$

♡

Matrixmultiplikation

Matrixmultiplikation er lidt mere kompliceret end addition og multiplikation med skalar. Produktet af to matricer er kun defineret, når antallet af søjler i den første matrix er lig med antallet af rækker i den anden matrix. Produktet af to matricer A og B er en ny matrix C, hvis indgange bestemmes af prikproduktet af rækkerne i A og søjlerne i B. Bemærk, at $\boldsymbol{x}$ er en søjlevektor.

Lad B være en $n \times p$-matrix og A en $m \times n$-matrix. Hvis $\boldsymbol{x} \in \mathbb{R}^p$, giver multiplikation af B med $\boldsymbol{x}$ en ny vektor $B\boldsymbol{x}$ i $\mathbb{R}^n$. Når vi har dette resultat, kan vi multiplicere det videre med A og får $A(B\boldsymbol{x})$, som er en vektor i $\mathbb{R}^m$.

For enhver vektor $\boldsymbol{x}$ i $\mathbb{R}^p$ giver denne proces altså en tilsvarende vektor i $\mathbb{R}^m$. Denne operation i to trin – først multiplikation med B og derefter med A – kaldes sammensætningen af A og B og skrives normalt som AB. Vi har derfor:

$$(AB)\boldsymbol{x} = A(B\boldsymbol{x})$$

For at beregne den matrix, der fremkommer ved denne sammensætning, multiplicerer vi A direkte med B efter reglerne for matrixmultiplikation. Resultatet er en matrix $C = AB$, hvor hver indgang i C bestemmes af samspillet mellem rækkerne i A og søjlerne i B.

Grundlæggende er C den matrix, der udtrykker den samlede virkning af at anvende både B og A på en vilkårlig vektor $\boldsymbol{x} \in \mathbb{R}^p$, uden at vi behøver at dele det op i mellemtrin.

Definition 9.3

For $A \in \mathbb{R}^{m\times n}$ og $B \in \mathbb{R}^{n\times p}$, med $B = \begin{bmatrix} \boldsymbol{b_1} & \boldsymbol{b_2} & \cdots & \boldsymbol{b_p} \end{bmatrix}$, definerer vi produktet AB ved formlen

$$AB = \begin{bmatrix} A\boldsymbol{b_1} & A\boldsymbol{b_2} & \cdots & A\boldsymbol{b_p} \end{bmatrix}$$

Produktet AB er kun defineret, når antallet af søjler i A er lig med antallet af rækker i B. Følgende diagram er nyttigt at huske det med:

$$(m \times n) \cdot (n \times p) \to m \times p$$

Diagrammet viser, at antallet af søjler i den første matrix skal være lig med antallet af rækker i den anden matrix (den blå pil). Resultatet er en ny matrix med lige så mange rækker som den første matrix og lige så mange søjler som den anden matrix (den røde pil).

Eksempel 9.3 Beregn AB og BA for A og B nedenfor.

$$A = \begin{bmatrix} 1 & 2 & -2 \\ 1 & 1 & -3 \end{bmatrix}, \quad B = \begin{bmatrix} -4 & 2 & 4 & -4 \\ -1 & -5 & -3 & 3 \\ -4 & -4 & -3 & -1 \end{bmatrix}$$

Løsning: Først $AB = \begin{bmatrix} A\boldsymbol{b_1} & A\boldsymbol{b_2} & A\boldsymbol{b_3} & A\boldsymbol{b_4} \end{bmatrix}$:

$$AB = \begin{bmatrix} 1 & 2 & -2 \\ 1 & 1 & -3 \end{bmatrix} \begin{bmatrix} -4 & 2 & 4 & -4 \\ -1 & -5 & -3 & 3 \\ -4 & -4 & -3 & -1 \end{bmatrix}$$

$$= \begin{bmatrix} \boxed{2} \\ \boxed{7} \end{bmatrix} \begin{matrix} \text{Række 1 i } A \text{ og søjle 1 i } B \\ \text{Række 2 i } A \text{ og søjle 1 i } B \end{matrix}$$

$$= \begin{bmatrix} 2 & \boxed{0} \\ 7 & \boxed{9} \end{bmatrix} \begin{matrix} \text{Række 1 i } A \text{ og søjle 2 i } B \\ \text{Række 2 i } A \text{ og søjle 2 i } B \end{matrix}$$

$$= \begin{bmatrix} 2 & 0 & \boxed{4} \\ 7 & 9 & \boxed{10} \end{bmatrix} \begin{matrix} \text{Række 1 i } A \text{ og søjle 3 i } B \\ \text{Række 2 i } A \text{ og søjle 3 i } B \end{matrix}$$

$$= \begin{bmatrix} 2 & 0 & 4 & \boxed{4} \\ 7 & 9 & 10 & \boxed{2} \end{bmatrix} \begin{matrix} \text{Række 1 i } A \text{ og søjle 4 i } B \\ \text{Række 2 i } A \text{ og søjle 4 i } B \end{matrix}$$

$$= \begin{bmatrix} 2 & 0 & 4 & 4 \\ 7 & 9 & 10 & 2 \end{bmatrix}$$

Derimod er BA ikke defineret! B har 4 søjler, og A har 2 rækker. Antallet af søjler i B er altså ikke lig med antallet af rækker i A. ◀

Eksempel 9.4 Beregn matricen AB, hvor

$$A = \begin{bmatrix} 1 & 2 \\ 2 & 3 \\ 3 & 4 \end{bmatrix}, \quad \text{og } B = \begin{bmatrix} 2 & -1 \\ 3 & 1 \end{bmatrix}$$

Løsning: Ifølge definitionen af multiplikation ovenfor er

$$\begin{aligned} AB &= \left[A \begin{bmatrix} 2 \\ 3 \end{bmatrix} A \begin{bmatrix} -1 \\ 1 \end{bmatrix} \right] \\ &= \left[\begin{bmatrix} 1 & 2 \\ 2 & 3 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 2 & 3 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \end{bmatrix} \right] \\ &= \left[\begin{bmatrix} 1 \cdot 2 + 2 \cdot 3 \\ 2 \cdot 2 + 3 \cdot 3 \\ 3 \cdot 2 + 4 \cdot 3 \end{bmatrix} \begin{bmatrix} 1 \cdot (-1) + 2 \cdot 1 \\ 2 \cdot (-1) + 3 \cdot 1 \\ 3 \cdot (-1) + 4 \cdot 1 \end{bmatrix} \right] \\ &= \begin{bmatrix} 8 & 1 \\ 13 & 1 \\ 18 & 1 \end{bmatrix} \end{aligned}$$

◀

Eksempel 9.5 Hvis A er 3×5 og B er 5×2, hvilke størrelser har AB og BA så (forudsat at de er defineret)?

Løsning:

- Produktet AB er defineret, da antallet af søjler i A svarer til antallet af rækker i B (5). Den resulterende matrix AB er en 3×2-matrix.
- Produktet BA er ikke defineret, da antallet af søjler i B (2) ikke svarer til antallet af rækker i A (3).

◀

Det næste eksempel viser, at selv hvis både AB og BA er defineret, er de ikke nødvendigvis ens.

Eksempel 9.6 Beregn AB og BA for $A = \begin{bmatrix} -4 & 4 & 3 \\ 3 & -3 & -1 \\ -2 & -1 & 1 \end{bmatrix}$ og $B = \begin{bmatrix} -1 & -1 & 0 \\ -3 & 0 & -2 \\ -2 & 1 & -2 \end{bmatrix}$.

Løsning: Først AB:

$$
\begin{aligned}
AB &= \begin{bmatrix} -4 & 4 & 3 \\ 3 & -3 & -1 \\ -2 & -1 & 1 \end{bmatrix} \begin{bmatrix} -1 & -1 & 0 \\ -3 & 0 & -2 \\ -2 & 1 & -2 \end{bmatrix} \\
&= \begin{bmatrix} -14 \\ 8 \\ 3 \end{bmatrix} \\
&= \begin{bmatrix} -14 & 7 \\ 8 & -4 \\ 3 & 3 \end{bmatrix} \\
&= \begin{bmatrix} -14 & 7 & -14 \\ 8 & -4 & 8 \\ 3 & 3 & 0 \end{bmatrix}
\end{aligned}
$$

Dernæst BA:

$$
\begin{aligned}
BA &= \begin{bmatrix} -1 & -1 & 0 \\ -3 & 0 & -2 \\ -2 & 1 & -2 \end{bmatrix} \begin{bmatrix} -4 & 4 & 3 \\ 3 & -3 & -1 \\ -2 & -1 & 1 \end{bmatrix} \\
&= \begin{bmatrix} 1 \\ 16 \\ 15 \end{bmatrix} \\
&= \begin{bmatrix} 1 & -1 \\ 16 & -10 \\ 15 & -9 \end{bmatrix} \\
&= \begin{bmatrix} 1 & -1 & -2 \\ 16 & -10 & -11 \\ 15 & -9 & -9 \end{bmatrix}
\end{aligned}
$$

Vi ser, at $AB \neq BA$. ◀

I almindelig aritmetik er det neutrale element for multiplikation 1. I matrixalgebra er det neutrale element for multiplikation identitetsmatricen, som betegnes I. Identitetsmatricen er en *kvadratisk* matrix med 1-taller i diagonalen og 0-taller alle andre steder. Identitetsmatricens størrelse bestemmes af sammenhængen og fremgår som regel klart af den. For eksempel er I_2 en 2×2-identitetsmatrix, og I_3 er en 3×3-identitetsmatrix, og generelt er $I_n \in \mathbb{R}^{n \times n}$ en $n \times n$-identitetsmatrix:

$$
I_n = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \cdots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{bmatrix}
$$

Vi kan nu formulere følgende sætning.

Sætning 9.2

Lad A, B, C være matricer af passende dimensioner, og lad α være en skalar. Så gælder

(a) $A(BC) = (AB)C$

(b) $A(B + C) = AB + AC$

(c) $(B + C)A = BA + CA$

(d) $\alpha(AB) = (\alpha A)B = A(\alpha B)$

(e) Hvis A er en $m \times n$-matrix, så er $I_m A = A = AI_n$

♡

Vi afslutter dette afsnit med at se på den k'te potens af en matrix.

Definition 9.4

Lad A være en kvadratisk matrix, dvs. $A \in \mathbb{R}^{n\times n}$. Den k'te potens af A, betegnet A^k, er defineret som produktet af A med sig selv k gange. Det vil sige

$$A^k = \underbrace{AAA \cdots A}_{k \text{ gange}}$$

hvor A optræder k gange på højre side.

♣

Eksempel 9.7 Beregn A^3, hvis

$$A = \begin{bmatrix} -2 & 3 \\ 1 & 0 \end{bmatrix}$$

Løsning:

Beregn A^2:

$$A^2 = \begin{bmatrix} -2 & 3 \\ 1 & 0 \end{bmatrix}\begin{bmatrix} -2 & 3 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 7 & -6 \\ -2 & 3 \end{bmatrix}$$

Og derefter A^3:

$$A^3 = A^2A = \begin{bmatrix} 7 & -6 \\ -2 & 3 \end{bmatrix}\begin{bmatrix} -2 & 3 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} -20 & 21 \\ 7 & -6 \end{bmatrix}$$

Vi kunne også gøre sådan:

$$A^3 = AA^2 = \begin{bmatrix} -2 & 3 \\ 1 & 0 \end{bmatrix}\begin{bmatrix} 7 & -6 \\ -2 & 3 \end{bmatrix} = \begin{bmatrix} -20 & 21 \\ 7 & -6 \end{bmatrix}$$

◀

9.2 Den transponerede matrix

Vi begynder med definitionen af den transponerede af en matrix.

Definition 9.5

For en matrix $A \in \mathbb{R}^{m\times n}$ er den transponerede af A matricen A^T, hvis i'te søjle er den i'te række i A.

Hvis A er $m \times n$, så er A^T $n \times m$. Hvis for eksempel

$$A = \begin{bmatrix} 0 & -1 & 8 & -7 & -4 \\ -4 & 6 & -10 & -9 & 6 \\ 9 & 5 & -2 & -3 & 5 \\ -8 & 8 & 4 & 7 & 7 \end{bmatrix}$$

så er

$$A^T = \begin{bmatrix} 0 & -4 & 9 & -8 \\ -1 & 6 & 5 & 8 \\ 8 & -10 & -2 & 4 \\ -7 & -9 & -3 & 7 \\ -4 & 6 & 5 & 7 \end{bmatrix}$$

Eksempel 9.8 Beregn $(AB)^T$ og B^TA^T, hvis

$$A = \begin{bmatrix} -2 & 1 & 0 \\ 3 & -1 & -3 \end{bmatrix}, \quad B = \begin{bmatrix} -2 & 1 & 2 \\ -1 & -2 & 0 \\ 0 & 0 & -1 \end{bmatrix}.$$

Løsning: Først beregner vi $(AB)^T$:

$$\begin{aligned} AB &= \begin{bmatrix} -2 & 1 & 0 \\ 3 & -1 & -3 \end{bmatrix} \begin{bmatrix} -2 & 1 & 2 \\ -1 & -2 & 0 \\ 0 & 0 & -1 \end{bmatrix} \\ &= \begin{bmatrix} 3 & -4 & -4 \\ -5 & 5 & 9 \end{bmatrix} \end{aligned}$$

og derefter $(AB)^T$:

$$\begin{aligned} (AB)^T &= \begin{bmatrix} 3 & -4 & -4 \\ -5 & 5 & 9 \end{bmatrix}^T \\ &= \begin{bmatrix} 3 & -5 \\ -4 & 5 \\ -4 & 9 \end{bmatrix} \end{aligned}$$

Dernæst beregner vi B^TA^T:

$$\begin{aligned} B^TA^T &= \begin{bmatrix} -2 & -1 & 0 \\ 1 & -2 & 0 \\ 2 & 0 & -1 \end{bmatrix} \begin{bmatrix} -2 & 3 \\ 1 & -1 \\ 0 & -3 \end{bmatrix} \\ &= \begin{bmatrix} 3 & -5 \\ -4 & 5 \\ -4 & 9 \end{bmatrix} \end{aligned}$$

Vi ser, at $(AB)^T = B^T A^T$. ◀

Følgende sætning opsummerer egenskaberne ved den transponerede af en matrix.

Sætning 9.3

Lad A og B være matricer af passende dimensioner, og lad α være en skalar. Så gælder

(a) $(A^T)^T = A$
(b) $(A + B)^T = A^T + B^T$
(c) $(\alpha A)^T = \alpha A^T$
(d) $(AB)^T = B^T A^T$

♡

En konsekvens af egenskab (d) er, at

$$(A_1 A_2 \dots A_k)^T = A_k^T A_{k-1}^T \cdots A_2^T A_1^T$$

og som et specialtilfælde

$$\left(A^k\right)^T = \left(A^T\right)^k$$

9.3 Invertible matricer

Den inverse af en kvadratisk matrix $A \in \mathbb{R}^{n\times n}$ udvider begrebet den reciprokke værdi af et reelt tal $a \in \mathbb{R}$, der er forskelligt fra nul. Mere præcist er den inverse af et tal $a \in \mathbb{R}$, der er forskelligt fra nul, det entydige tal $c \in \mathbb{R}$, som opfylder $ac = ca = 1$. Den inverse af $a \neq 0$, som typisk skrives $a^{-1} = \frac{1}{a}$, gør det muligt at løse ligningen $ax = b$:

$$ax = b \Rightarrow a^{-1}ax = a^{-1}b \Rightarrow x = a^{-1}b.$$

Dette begreb kan udvides til kvadratiske matricer, hvor den inverse af en matrix $A \in \mathbb{R}^{n\times n}$ er en matrix $C \in \mathbb{R}^{n\times n}$, som opfylder $AC = CA = I_n$, hvor I_n er identitetsmatricen af størrelse $n \times n$. Den inverse af en matrix betegnes A^{-1}. Vi kan nu definere en invertibel matrix.

Definition 9.6

En kvadratisk matrix $A \in \mathbb{R}^{n\times n}$ er invertibel (eller **ikke-singulær**), hvis der findes en matrix $C \in \mathbb{R}^{n\times n}$, så $AC = CA = I_n$. Matricen C kaldes den inverse af A og betegnes A^{-1}. Der gælder altså $A^{-1}A = AA^{-1} = I_n$.

♣

Eksempel 9.9 Vis, at C er den inverse af A, hvor A og C er givet nedenfor.

$$A = \begin{bmatrix} 1 & -3 & 0 \\ -1 & 2 & -2 \\ -2 & 6 & 1 \end{bmatrix}, \quad C = \begin{bmatrix} -14 & -3 & -6 \\ -5 & -1 & -2 \\ 2 & 0 & 1 \end{bmatrix}$$

Løsning: Vi skal vise, at $AC = CA = I_3$. Først beregner vi AC:

$$AC = \begin{bmatrix} 1 & -3 & 0 \\ -1 & 2 & -2 \\ -2 & 6 & 1 \end{bmatrix} \begin{bmatrix} -14 & -3 & -6 \\ -5 & -1 & -2 \\ 2 & 0 & 1 \end{bmatrix}$$
$$= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = I_3$$

Dernæst beregner vi CA:

$$CA = \begin{bmatrix} -14 & -3 & -6 \\ -5 & -1 & -2 \\ 2 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & -3 & 0 \\ -1 & 2 & -2 \\ -2 & 6 & 1 \end{bmatrix}$$
$$= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = I_3$$

Vi ser, at $C = A^{-1}$ er den inverse af A. ◀

Følgende sætning opsummerer sammenhængen mellem den inverse matrix og henholdsvis matrixmultiplikation og den transponerede matrix.

Sætning 9.4

Lad A og B være invertible $n \times n$-matricer. Så gælder:

(a) A^{-1} er invertibel, og
$$\left(A^{-1}\right)^{-1} = A$$

(b) Produktet AB er invertibelt, og
$$(AB)^{-1} = B^{-1}A^{-1}$$

(c) Den transponerede af A er også invertibel, dvs. A^T er invertibel, og
$$\left(A^T\right)^{-1} = \left(A^{-1}\right)^T$$

♡

Vi ser nu på, hvordan vi kan bestemme den inverse af en matrix.

Bestemmelse af determinanten og den inverse af en 2×2-matrix

Den inverse af en matrix er ikke altid let at bestemme. For en 2×2-matrix kan vi dog bruge en formel til at finde den inverse.

Sætning 9.5

Lad $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$. Hvis $ad - bc \neq 0$, er den inverse af A givet ved

$$A^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

♡

Bemærkning: Størrelsen $ad - bc$ kaldes determinanten af A, og vi skriver

$$\det(A) = |A| = ad - bc$$

Determinanten af en 2×2-matrix er en skalar størrelse, der giver information om matricen. Hvis determinanten er nul, er matricen ikke invertibel. Hvis determinanten er forskellig fra nul, er matricen invertibel. De eksplicitte formler $\det(A) = ad - bc$ og den tilhørende formel for den inverse, som er givet her, gælder specifikt for 2×2-matricer; determinanter og inverse af større kvadratiske matricer defineres ved hjælp af mere generelle metoder.

Eksempel 9.10 Vi lader

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

og beregner A^{-1}. Ifølge sætningen har vi

$$\begin{aligned} A^{-1} &= \frac{1}{1 \cdot 4 - 2 \cdot 3} \begin{bmatrix} 4 & -2 \\ -3 & 1 \end{bmatrix} \\ &= -\frac{1}{2} \begin{bmatrix} 4 & -2 \\ -3 & 1 \end{bmatrix} \\ &= \begin{bmatrix} -2 & 1 \\ \frac{3}{2} & -\frac{1}{2} \end{bmatrix} \end{aligned}$$

Nu bekræfter vi, at $AA^{-1} = I_2$:

$$\begin{aligned} AA^{-1} &= \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} -2 & 1 \\ \frac{3}{2} & -\frac{1}{2} \end{bmatrix} \\ &= \begin{bmatrix} 1(-2) + 2\left(\frac{3}{2}\right) & 1(1) + 2\left(-\frac{1}{2}\right) \\ 3(-2) + 4\left(\frac{3}{2}\right) & 3(1) + 4\left(-\frac{1}{2}\right) \end{bmatrix} \\ &= \begin{bmatrix} -2 + 3 & 1 - 1 \\ -6 + 6 & 3 - 2 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \end{aligned}$$

Eksempel 9.11 Bestem den inverse af $A = \begin{bmatrix} 1 & 3 \\ -1 & -2 \end{bmatrix}$, hvis den findes.

Løsning: Først beregner vi determinanten af A:

$$\det(A) = 1(-2) - 3(-1) = (-2) + 3 = 1$$

Da determinanten er forskellig fra nul, findes den inverse af A. Ifølge formlen har vi

$$A^{-1} = \frac{1}{1} \begin{bmatrix} -2 & -3 \\ 1 & 1 \end{bmatrix}$$

Vi kan bekræfte, at $AA^{-1} = I_2$:

$$AA^{-1} = \begin{bmatrix} 1 & 3 \\ -1 & -2 \end{bmatrix} \begin{bmatrix} -2 & -3 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I_2$$

◀

For større matricer skal vi bruge andre metoder til at bestemme den inverse.

Bestemmelse af den inverse af en $n \times n$-matrix

En **elementærmatrix** er en matrix, der fremkommer ved at udføre én enkelt elementær rækkeoperation (erstatning, ombytning eller skalering) på en identitetsmatrix. For eksempel er matricen

$$E_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -2 & 0 & 1 \end{bmatrix}$$

en elementærmatrix, da den fremkommer ud fra I_3 ved den ene elementære rækkeoperation $r_3 \mapsto r_3 - 2r_1$.

Eksempel 9.12

Lad A være en generel 3×3-matrix

$$A = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$$

og lad E_1, E_2 og E_3 være elementærmatricerne

$$E_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -2 & 0 & 1 \end{bmatrix}, \quad E_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \quad \text{og} \quad E_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Bemærk så, at E_1 svarer til $r_3 \mapsto r_3 - 2r_1$, E_2 svarer til $r_2 \longleftrightarrow r_3$, og E_3 svarer til $r_2 \mapsto 3r_2$. Vi har også følgende produkter:

$$E_1A = \begin{bmatrix} a & b & c \\ d & e & f \\ g-2a & h-2b & i-2c \end{bmatrix}, \quad E_2A = \begin{bmatrix} a & b & c \\ g & h & i \\ d & e & f \end{bmatrix}, \quad \text{og} \quad E_3A = \begin{bmatrix} a & b & c \\ 3d & 3e & 3f \\ g & h & i \end{bmatrix}.$$

Bemærk, at E_1A er den matrix, der fremkommer ved at udføre rækkeoperationen $r_3 \mapsto r_3 - 2r_1$ på A. Generelt er det at multiplicere A fra venstre med en elementærmatrix det samme som at udføre den tilsvarende rækkeoperation på A. Vi kan også repræsentere en række af rækkeoperationer ved at multiplicere flere elementærmatricer. For eksempel svarer

$$E_2E_1A = \begin{bmatrix} a & b & c \\ g-2a & h-2b & i-2c \\ d & e & f \end{bmatrix}$$

til at udføre $r_3 \mapsto r_3 - 2r_1$ efterfulgt af $r_2 \longleftrightarrow r_3$ på matricen A. Som vi tidligere har set, kan rækkeoperationer gøres om. Heraf følger, at elementærmatricer også er invertible. Det fører til følgende sætning.

Sætning 9.6

En $n \times n$-matrix A er invertibel, hvis og kun hvis A er rækkeækvivalent med I_n. I så fald vil enhver følge af elementære rækkeoperationer, der reducerer A til I_n, også omdanne I_n til A^{-1}. ♡

Denne sætning fører til en praktisk algoritme til at bestemme den inverse af en $n \times n$-matrix, forudsat at den inverse findes.

Algoritme til bestemmelse af den inverse af en $n \times n$-matrix

Lad A være en $n \times n$-matrix. Følg disse trin for at bestemme den inverse af A:

1. Opstil totalmatricen $[A|I_n]$.
2. Udfør rækkeoperationer på $[A|I_n]$ for at reducere A til I_n.
3. Matricen i højre side af totalmatricen er A^{-1}.

Hvis A ikke er invertibel, vil algoritmen ikke kunne reducere A til I_n.

Eksempel 9.13 Bestem den inverse af $A = \begin{bmatrix} 1 & 0 & 3 \\ 1 & 1 & 0 \\ -2 & 0 & -7 \end{bmatrix}$, hvis den findes.

Løsning: Opstil totalmatricen $\begin{bmatrix} A & I_3 \end{bmatrix}$, og rækkereducér:

$$\begin{bmatrix} 1 & 0 & 3 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ -2 & 0 & -7 & 0 & 0 & 1 \end{bmatrix} \quad \begin{matrix} r_2 \mapsto -r_1 + r_2 \\ r_3 \mapsto 2r_1 + r_3 \end{matrix} \quad \begin{bmatrix} 1 & 0 & 3 & 1 & 0 & 0 \\ 0 & 1 & -3 & -1 & 1 & 0 \\ 0 & 0 & -1 & 2 & 0 & 1 \end{bmatrix}$$

$$r_3 \mapsto -r_3 \quad \begin{bmatrix} 1 & 0 & 3 & 1 & 0 & 0 \\ 0 & 1 & -3 & -1 & 1 & 0 \\ 0 & 0 & 1 & -2 & 0 & -1 \end{bmatrix}$$

$$\begin{matrix} r_2 \mapsto 3r_3 + r_2 \\ r_1 \mapsto -3r_3 + r_1 \end{matrix} \quad \begin{bmatrix} 1 & 0 & 0 & 7 & 0 & 3 \\ 0 & 1 & 0 & -7 & 1 & -3 \\ 0 & 0 & 1 & -2 & 0 & -1 \end{bmatrix}$$

Altså er $\operatorname{rref} A = I_3$, hvilket bekræfter, at A er invertibel. Den inverse er

$$A^{-1} = \begin{bmatrix} 7 & 0 & 3 \\ -7 & 1 & -3 \\ -2 & 0 & -1 \end{bmatrix}$$

Kontrol:

$$AA^{-1} = \begin{bmatrix} 1 & 0 & 3 \\ 1 & 1 & 0 \\ -2 & 0 & -7 \end{bmatrix} \begin{bmatrix} 7 & 0 & 3 \\ -7 & 1 & -3 \\ -2 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

◀

Eksempel 9.14 Bestem den inverse af $A = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & -2 \\ -2 & 0 & -2 \end{bmatrix}$, hvis den findes.

Løsning: Opstil totalmatricen $\begin{bmatrix} A & I_3 \end{bmatrix}$, og rækkereducér:

$$\begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & -2 & 0 & 1 & 0 \\ -2 & 0 & -2 & 0 & 0 & 1 \end{bmatrix} \quad \begin{matrix} r_2 \mapsto -r_1 + r_2 \\ r_3 \mapsto 2r_1 + r_3 \end{matrix} \quad \begin{bmatrix} 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & -3 & -1 & 1 & 0 \\ 0 & 0 & 0 & 2 & 0 & 1 \end{bmatrix}$$

Vi behøver ikke at gå videre, da rref(A) ikke er I_3. Derfor er A ikke invertibel. ◀

Eksempel 9.15 Bestem den inverse af matricen

$$A = \begin{bmatrix} 1 & 2 & 1 \\ 4 & 5 & 3 \\ 0 & 0 & 2 \end{bmatrix}$$

hvis den findes.

Løsning: (Af hensyn til kortheden udelader vi trinene i rækkereduktionen.)

$$\begin{bmatrix} 1 & 2 & 1 & 1 & 0 & 0 \\ 4 & 5 & 3 & 0 & 1 & 0 \\ 0 & 0 & 2 & 0 & 0 & 1 \end{bmatrix} \sim \begin{bmatrix} 1 & 2 & 1 & 1 & 0 & 0 \\ 0 & -3 & -1 & -4 & 1 & 0 \\ 0 & 0 & 2 & 0 & 0 & 1 \end{bmatrix}$$
$$\sim \begin{bmatrix} 1 & 2 & 1 & 1 & 0 & 0 \\ 0 & -3 & -1 & -4 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & \frac{1}{2} \end{bmatrix}$$
$$\sim \begin{bmatrix} 1 & 2 & 0 & 1 & 0 & -\frac{1}{2} \\ 0 & -3 & 0 & -4 & 1 & \frac{1}{2} \\ 0 & 0 & 1 & 0 & 0 & \frac{1}{2} \end{bmatrix}$$
$$\sim \begin{bmatrix} 1 & 2 & 0 & 1 & 0 & -\frac{1}{2} \\ 0 & 1 & 0 & \frac{4}{3} & -\frac{1}{3} & -\frac{1}{6} \\ 0 & 0 & 1 & 0 & 0 & \frac{1}{2} \end{bmatrix}$$
$$\sim \begin{bmatrix} 1 & 0 & 0 & -\frac{5}{3} & \frac{2}{3} & -\frac{1}{6} \\ 0 & 1 & 0 & \frac{4}{3} & -\frac{1}{3} & -\frac{1}{6} \\ 0 & 0 & 1 & 0 & 0 & \frac{1}{2} \end{bmatrix}.$$

Dermed er

$$A^{-1} = \begin{bmatrix} -\frac{5}{3} & \frac{2}{3} & -\frac{1}{6} \\ \frac{4}{3} & -\frac{1}{3} & -\frac{1}{6} \\ 0 & 0 & \frac{1}{2} \end{bmatrix}.$$

◀

Med metoden til at bestemme inverse matricer på plads har vi nu et stærkt værktøj til at angribe lineære ligningssystemer. Vi skal se, hvordan inverse matricer kan bruges til at løse ligningssystemer effektivt, og hvordan de giver en ligetil måde at finde løsninger på, når en matrix er invertibel.

Løsning af matrixligninger med inverse matricer

Ligesom vi kan løse en ligning ved hjælp af den reciprokke værdi af et tal forskelligt fra nul, som vi så tidligere, kan vi gøre noget tilsvarende med matricer. Når en matrix A er invertibel, kan vi bruge dens inverse til at løse matrixligninger, hvori A indgår. Det bringer os til sætningen nedenfor, som viser, hvordan vi kan finde løsninger til ligninger som $A\boldsymbol{x} = \boldsymbol{b}$, når A har en invers.

Sætning 9.7

Lad A være en invertibel matrix. Så har ligningen $A\boldsymbol{x} = \boldsymbol{b}$ en entydig løsning givet ved $\boldsymbol{x} = A^{-1}\boldsymbol{b}$. ♡

Eksempel 9.16 Vi kan bruge den inverse fra Eksempel 9.10 til at løse det lineære ligningssystem

$$\begin{aligned} x_1 + 2x_2 &= 5 \\ 3x_1 + 4x_2 &= 6. \end{aligned}$$

Vi opfatter det på matrixform som $A\boldsymbol{x} = \boldsymbol{b}$, hvor

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad \boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad \text{og} \quad \boldsymbol{b} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

Sætningen fortæller os, at $\boldsymbol{x} = A^{-1}\boldsymbol{b}$ er en løsning, dvs. at

$$\begin{aligned} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} &= \begin{bmatrix} -2 & 1 \\ \frac{3}{2} & -\frac{1}{2} \end{bmatrix} \begin{bmatrix} 5 \\ 6 \end{bmatrix} \\ &= \begin{bmatrix} -2(5) + 1(6) \\ \frac{3}{2}(5) - \frac{1}{2}(6) \end{bmatrix} \\ &= \begin{bmatrix} -4 \\ \frac{9}{2} \end{bmatrix} \end{aligned}$$

Eksempel 9.17 Brug resultatet fra Eksempel 9.9 til at løse det lineære ligningssystem $A\boldsymbol{x} = \boldsymbol{b}$, hvor

$$A = \begin{bmatrix} 1 & -3 & 0 \\ -1 & 2 & -2 \\ -2 & 6 & 1 \end{bmatrix}, \quad \boldsymbol{b} = \begin{bmatrix} 1 \\ -3 \\ -1 \end{bmatrix}$$

Løsning: Vi viste i Eksempel 9.9, at

$$A^{-1} = \begin{bmatrix} -14 & -3 & -6 \\ -5 & -1 & -2 \\ 2 & 0 & 1 \end{bmatrix}$$

Den entydige løsning til det lineære ligningssystem $A\boldsymbol{x} = \boldsymbol{b}$ er derfor

$$A^{-1}\boldsymbol{b} = \begin{bmatrix} -14 & -3 & -6 \\ -5 & -1 & -2 \\ 2 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ -3 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$$

Kontrol:

$$\begin{bmatrix} 1 & -3 & 0 \\ -1 & 2 & -2 \\ -2 & 6 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ -3 \\ -1 \end{bmatrix}$$

◀

Vi afslutter dette kapitel med en af de vigtigste sætninger i lineær algebra: sætningen om invertible matricer.

9.4 Sætningen om invertible matricer

Sætningen om invertible matricer giver en omfattende liste over ækvivalente udsagn om, hvornår en kvadratisk matrix er invertibel, og den gør det muligt at afgøre sandhedsværdien af ét udsagn ved at undersøge sandhedsværdien af et andet.

Sætning 9.8

Lad A være en $n \times n$-matrix. Følgende udsagn er ækvivalente:

(a) A er invertibel.
(b) A er rækkeækvivalent med I_n.
(c) A har n pivotpositioner (dvs. én for hver række og søjle).
(d) Ligningen $A\boldsymbol{x} = \mathbf{0}$ har kun den trivielle løsning.
(e) Søjlerne i A er lineært uafhængige.
(f) Ligningen $A\boldsymbol{x} = \boldsymbol{b}$ har en entydig løsning for hvert $\boldsymbol{b} \in \mathbb{R}^n$.
(g) Søjlerne i A udspænder $\mathbb{R}^n$.
(h) $\det(A) \neq 0$.
(i) Der findes en $n \times n$-matrix C, så $CA = I$.
(j) Der findes en $n \times n$-matrix D, så $AD = I$.
(k) A^T er invertibel.

♡

Bemærk, hvordan sætningen forbinder en matrix' egenskaber med dens invertibilitet og dermed opsummerer hovedelementerne i vores gennemgang af lineære ligningssystemer, lineær uafhængighed, vektorer og matricer.

Eksempel 9.18 Afgør, om følgende matrix er invertibel:

$$A = \begin{bmatrix} 2 & 2 & 2 \\ 1 & 3 & 1 \\ 4 & 4 & 6 \end{bmatrix}$$

Løsning: Ved at udføre rækkeoperationer får vi

$$A \sim \begin{bmatrix} 1 & 3 & 1 \\ 2 & 2 & 2 \\ 4 & 4 & 6 \end{bmatrix} \sim \begin{bmatrix} 1 & 3 & 1 \\ 0 & -4 & 0 \\ 0 & -8 & 2 \end{bmatrix} \sim \begin{bmatrix} 1 & 3 & 1 \\ 0 & -4 & 0 \\ 0 & 0 & 2 \end{bmatrix}$$

som har 3 pivoter, så ifølge (c) er A invertibel. ◀

Dette afslutter vores gennemgang af matricer.

Del IV

Differentialregning

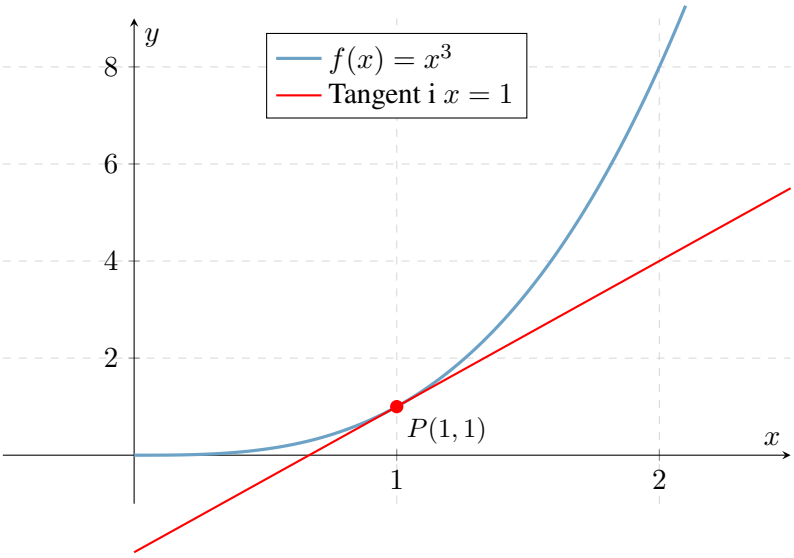
Kapitel 10 Differentialregning

Både i praksis og i moderne ingeniørarbejde møder vi ofte situationer, hvor én størrelse ændrer sig som følge af en anden. En bils brændstofforbrug afspejler, hvordan den tilbagelagte afstand afhænger af den mængde brændstof, der bruges. En sprinters hastighed beskriver, hvordan positionen varierer med tiden. I landbruget ændrer udbyttet sig med mængden af gødning, og i økonomi reagerer efterspørgslen på prisen. Studiet af sådanne sammenhænge er kernen i *differentialregning*, den matematiske disciplin, der handler om at forstå, hvordan størrelser varierer i forhold til hinanden.

Det begreb, der indfanger denne variation, er *ændringshastigheden*.

- Ændringen i afstand i forhold til tid er *hastighed*.
- Ændringen i fortjeneste i forhold til pris er *marginal fortjeneste*.
- Ændringen i tab i forhold til modelparametre er *gradienten*.

Når en ændringshastighed er konstant, er sammenhængen mellem størrelserne lineær, og grafen er en ret linje. Når ændringshastigheden varierer, bliver grafen krum, og hældningen afhænger af det konkrete punkt, hvor vi måler. Det fører naturligt til idéen om en *tangent*, som lokalt approksimerer en kurve i et enkelt punkt.



Figur 10.1: Tangenten til grafen for $f(x) = x^3$ i punktet $P(1, 1)$.

Bemærkning: For en ret linje er hældningen konstant. For en krum graf varierer hældningen fra punkt til punkt, og vi taler derfor om den *momentane ændringshastighed*, dvs. tangentens hældning i et bestemt punkt.

Det samme princip gælder for beregningsmæssige og datadrevne systemer. Et programs køretid ændrer sig med inputstørrelsen, en models nøjagtighed varierer med antallet af træningsepoker, og et netværks gennemløb afhænger af den tilgængelige båndbredde. Inden for maskinlæring minder træningen af en model om en vandrer, der leder efter det laveste punkt i et stort, tågedækket landskab. »Højden« repræsenterer modellens fejl, og for at komme videre må man gå nedad. Hvilken retning der er den rigtige blandt millioner af mulige, afhænger af, at vi kender den lokale hældning – den *afledte* – som udtrykker den momentane ændringshastighed.

Differentialregning giver os værktøjerne til præcist at beskrive, beregne og fortolke sådanne ændringer. Den

omsætter intuitive forestillinger som at være hurtigere, stejlere eller mere effektiv til eksakte matematiske udtryk. Dette kapitel introducerer de grundlæggende idéer i differentialregning. Vi begynder med begreberne *grænseværdi* og *kontinuitet*, som formaliserer forestillingen om at nærme sig et punkt uendeligt tæt. På det grundlag udvikler vi definitionen og fortolkningen af den *afledte* som et mål for momentan ændring og ser på anvendelser i sammenhænge, der spænder fra fysisk bevægelse til optimering af moderne algoritmer.

10.1 Grænseværdier og kontinuitet: grundlaget for differentialregning

Før vi kan måle en momentan ændringshastighed, må vi først opbygge en formel forståelse af, hvad det vil sige at komme »uendeligt tæt« på et punkt uden nødvendigvis at være i punktet. Det er begrebet **grænseværdi**. Med udgangspunkt i grænseværdier giver **kontinuitet** os et sprog til at beskrive funktioner, der er forudsigelige og velopdragne, uden pludselige spring eller brud.

Grænseværdier

En grænseværdi beskriver den værdi, som en funktion »nærmer sig«, når dens input kommer tættere og tættere på et bestemt punkt. Idéen er grundlæggende ikke kun i differentialregning, men også i softwareudvikling, hvor vi ofte analyserer systemers opførsel, når forholdene nærmer sig en bestemt tilstand.

Når vi for eksempel analyserer en algoritmes køretid, spørger vi, hvad der sker med udførelsestiden, når inputstørrelsen 'n' går mod uendelig. Inden for netværk kan vi modellere det teoretiske maksimale gennemløb som en grænseværdi, når latensen går mod nul.

Definition 10.1 (Grænseværdi)

Lad $f(x)$ være en funktion, der er defineret i nærheden af et punkt c. Vi siger, at **grænseværdien** af $f(x)$, når x går mod c, er L, og skriver

$$\lim_{x \to c} f(x) = L$$

hvis vi kan gøre værdien af $f(x)$ vilkårligt tæt på L ved at vælge et x, der er tilstrækkeligt tæt på c, men forskelligt fra c.

I mange simple tilfælde er grænseværdien blot funktionens værdi i punktet. For $f(x) = x^2$ er grænseværdien, når x går mod 3, for eksempel simpelthen $3^2 = 9$. Grænseværdiernes egentlige styrke viser sig dog i tilfælde, hvor funktionen ikke er defineret i det punkt, vi interesserer os for.

Eksempel 10.1 En grænseværdi i et punkt, hvor funktionen ikke er defineret

Se på funktionen $f(x) = \frac{\sin(x)}{x}$. Funktionen er grundlæggende i signalbehandling og kaldes sinc-funktionen. Bemærk, at $f(0)$ ikke er defineret, fordi det giver division med nul. Vi kan dog spørge, hvilken værdi funktionen nærmer sig, når x kommer meget tæt på 0.

Ved at indsætte værdier meget tæt på 0 kan vi se en tendens:

$$f(0.1) = \frac{\sin(0.1)}{0.1} \approx 0.9983$$
$$f(0.01) = \frac{\sin(0.01)}{0.01} \approx 0.99998$$
$$f(-0.01) = \frac{\sin(-0.01)}{-0.01} \approx 0.99998$$

Funktionen ser ud til at nærme sig 1. I differentialregning kan det bevises formelt, at:

$$\lim_{x \to 0} \frac{\sin(x)}{x} = 1$$

Evnen til at analysere opførslen i et problematisk punkt er afgørende for at kunne lave robuste numeriske algoritmer.

Eksempel 10.2 Asymptotisk analyse med grænseværdier

Antag, at du vil undersøge, om funktionen $f(n) = 5n^2 + 4n + 7$ er $\mathcal{O}(n^2)$, når n bliver stor, sådan som man ofte gør i studiet af algoritmers kompleksitet. En måde at formalisere det på er at undersøge grænseværdien:

$$\lim_{n \to \infty} \frac{f(n)}{n^2} = \lim_{n \to \infty} \frac{5n^2 + 4n + 7}{n^2} = \lim_{n \to \infty} \left(5 + \frac{4}{n} + \frac{7}{n^2}\right)$$

Når n bliver meget stor, går leddene $\frac{4}{n}$ og $\frac{7}{n^2}$ begge mod nul, så hele udtrykket nærmer sig 5. Da grænseværdien er en endelig konstant, bekræfter det, at $f(n)$ vokser med samme hastighed som n^2. Denne type grænseværdiberegning er kernen i asymptotisk analyse og giver en formel begrundelse for påstande om algoritmers køretid udtrykt med store $\mathcal{O}$.

Kontinuitet

Kontinuitet formaliserer idéen om, at en funktion er »velopdragen« eller forudsigelig. Intuitivt er en funktion kontinuert, hvis du kan tegne dens graf uden at løfte blyanten fra papiret. I software antager vi ofte, at vores systemer er kontinuerte: En lille ændring i input bør give en lille, forudsigelig ændring i output og ikke et pludseligt, drastisk spring.

Definition 10.2 (Kontinuitet)

En funktion f er **kontinuert** i et punkt c, hvis følgende tre betingelser er opfyldt:

1. $f(c)$ er defineret.
2. $\lim_{x \to c} f(x)$ eksisterer.
3. $\lim_{x \to c} f(x) = f(c)$.

En funktion er kontinuert på et interval, hvis den er kontinuert i alle punkter i intervallet. ♣

Diskontinuiteter repræsenterer ofte vigtige hændelser i softwaresystemer, fx tilstandsskift eller tærskler, der udløser en handling.

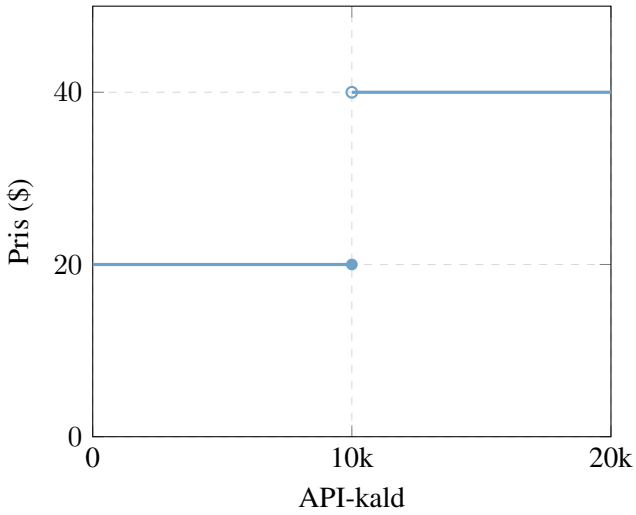
Eksempel 10.3 Springdiskontinuitet i en SaaS-prismodel

Se på en cloudtjeneste, der tager betaling efter antallet af API-kald pr. måned. Prismodellen er trinvis: \$20 for op til 10 000 kald og \$40 for ethvert forbrug over 10 000 kald. Omkostningsfunktionen $C(x)$, hvor x er antallet

af kald, har en **springdiskontinuitet** i $x = 10\,000$.

$$C(x) = \begin{cases} 20 & \text{hvis } x \leq 10000 \\ 40 & \text{hvis } x > 10000 \end{cases}$$

Som vist i Figur 10.2 springer prisen pludselig ved tærsklen. Det er afgørende at forstå diskontinuiteter, når man modellerer systemer med tærskelbaseret logik, lige fra regler for automatisk skalering til faktureringssystemer.



Figur 10.2: Omkostningsfunktionen har en springdiskontinuitet ved 10 000 API-kald, hvor pristrinnet skifter.

10.2 Den afledte: et mål for ændringshastigheden

Vi taler ofte om gennemsnitlige ændringshastigheder. Hvis en dataoverførsel tager 10 sekunder og flytter 500 MB, er den gennemsnitlige overførselshastighed for eksempel 50 MB/s. Men gennemsnittet fortæller intet om udsvingene under overførslen. Den **afledte** er det værktøj, der lader os gå fra den gennemsnitlige hastighed til den **momentane ændringshastighed** på et hvilket som helst bestemt tidspunkt.

Vi kan altså sige, at differentialregning handler om at bestemme, hvordan én størrelse ændrer sig i forhold til en anden. Geometrisk er den afledte af en funktion i et punkt hældningen af tangenten til funktionens graf i punktet. For at finde en tangents hældning approksimerer vi den med sekanter gennem to nærliggende punkter på kurven. Når punkterne rykker tættere på hinanden, nærmer sekantens hældning sig tangentens hældning. Som vist i Figur 10.3 er tangenten grænsen for sekanter gennem to punkter på kurven, når punkterne kommer uendeligt tæt på hinanden.

Når punktet Q i Figur 10.3 bevæger sig langs kurven mod P (fra Q_1 til Q_2 til P), nærmer sekanten PQ's hældning sig hældningen af tangenten til $f(x) = x^3$ i P. Bemærk, at tangenten kun rører kurven i P, mens sekanten går gennem to punkter.

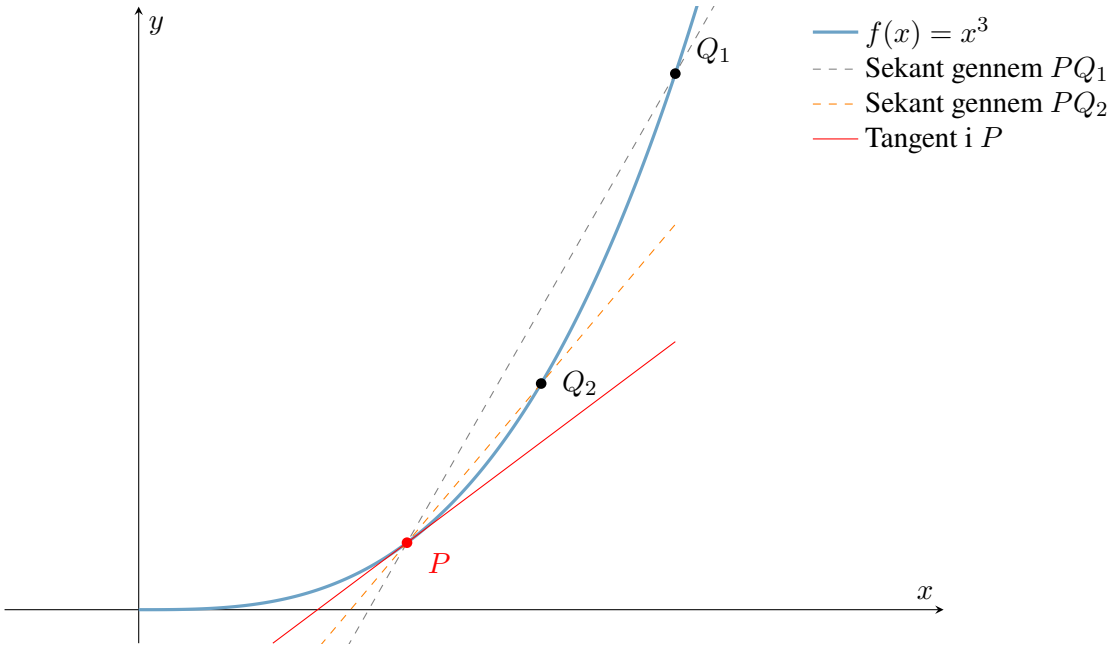
Det fører til den formelle definition af den afledte.

Definition 10.3 (Den afledte)

Den **afledte** af en funktion f med hensyn til x, betegnet $f'(x)$, er funktionen

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

forudsat at grænseværdien eksisterer. Hvis $f'(x)$ eksisterer, siger vi, at f er **differentiabel** i x. ♣



Figur 10.3: Tangenten som grænse for sekanter.

Udtrykket $\frac{f(x+h)-f(x)}{h}$ er **differenskvotienten**, som er den gennemsnitlige ændringshastighed på intervallet fra x til $x+h$. Den afledte er grænseværdien af denne gennemsnitlige hastighed, når intervallets længde h går mod nul.

Eksempel 10.4 Den afledte ud fra definitionen

Lad os finde den afledte af $f(x) = x^2$ ved hjælp af definitionen med grænseværdi.

Løsning: Vi begynder med differenskvotienten og reducerer:

$$\frac{f(x+h)-f(x)}{h} = \frac{(x+h)^2 - x^2}{h} = \frac{x^2 + 2xh + h^2 - x^2}{h} = \frac{2xh + h^2}{h} = 2x + h$$

Til sidst tager vi grænseværdien for $h \to 0$:

$$f'(x) = \lim_{h \to 0} (2x + h) = 2x$$

Den afledte af $f(x) = x^2$ er altså $f'(x) = 2x$. Den angiver parablens hældning i ethvert punkt x. ◀

Notation

Den afledte af f kan skrives på flere ækvivalente måder:

$$f'(x), \quad \frac{df}{dx}, \quad \frac{dy}{dx}, \quad Df(x), \quad \frac{d}{dx}f(x).$$

10.3 Differentiationsregler

Differentialregning giver os en metode til at finde den eksakte afledte af en funktion direkte ud fra dens forskrift uden at skulle bruge grafer eller numeriske skøn. I praksis sker det ved hjælp af en række enkle regler, som gør det muligt at beregne den afledte af næsten enhver funktion, vi sandsynligvis vil møde. I dette afsnit introducerer vi reglerne, forklarer deres betydning og viser, hvordan de anvendes i praksis.

Konstantreglen og potensreglen

De måske simpleste funktioner i matematikken er de konstante funktioner og funktionerne på formen x^n.

Sætning 10.1 (Konstantreglen og potensreglen)

1. **Konstantreglen:** Hvis c er en konstant, så er $\frac{d}{dx}(c) = 0$,
2. **Potensreglen:** For ethvert reelt n gælder $\frac{d}{dx}x^n = nx^{n-1}$.

♡

Eksempel 10.5

- Hvis $f(x) = x^7$, så er $f'(x) = 7x^6$,
- Hvis $y = x^{-0.5}$, så er $\frac{dy}{dx} = -0.5x^{-1.5}$,
- $\frac{d}{dx}x^{-3} = -3x^{-4}$,
- Hvis $g(x) = 3.2$, så er $g'(x) = 0$,
- Hvis $f(t) = \sqrt{t} = t^{\frac{1}{2}}$, så er $f'(t) = \frac{1}{2}t^{-\frac{1}{2}} = \frac{1}{2\sqrt{t}}$,
- Hvis $h(u) = -13.29$, så er $h'(u) = 0$,
- $\frac{d}{dx}x^3 = 3x^2$, $\frac{d}{dx}x^{-2} = -2x^{-3}$.

I eksemplerne ovenfor har vi brugt konstantreglen og potensreglen til at finde de afledte af flere simple funktioner. Det er dog vigtigt at huske, hvad disse beregninger faktisk repræsenterer. Den afledte angiver hældningen af tangenten til en funktions graf i et givet punkt.

Hvis for eksempel $f(x) = x^2$, så er $f'(x) = 2x$. For at finde hældningen af tangenten til grafen for x^2 i $x = 1$ indsætter vi $x = 1$ i den afledte og får $f'(1) = 2 \times 1 = 2$. Tilsvarende er tangentens hældning i $x = -0.5$ lig med $f'(-0.5) = 2 \times (-0.5) = -1$.

Resultaterne er illustreret i Figur 10.4, hvor tangenterne i $x = 1$ og $x = -0.5$ har hældningerne henholdsvis 2 og -1.

Eksempel 10.6

Lad $f(x) = (x^2 + 1)^3$. Så er

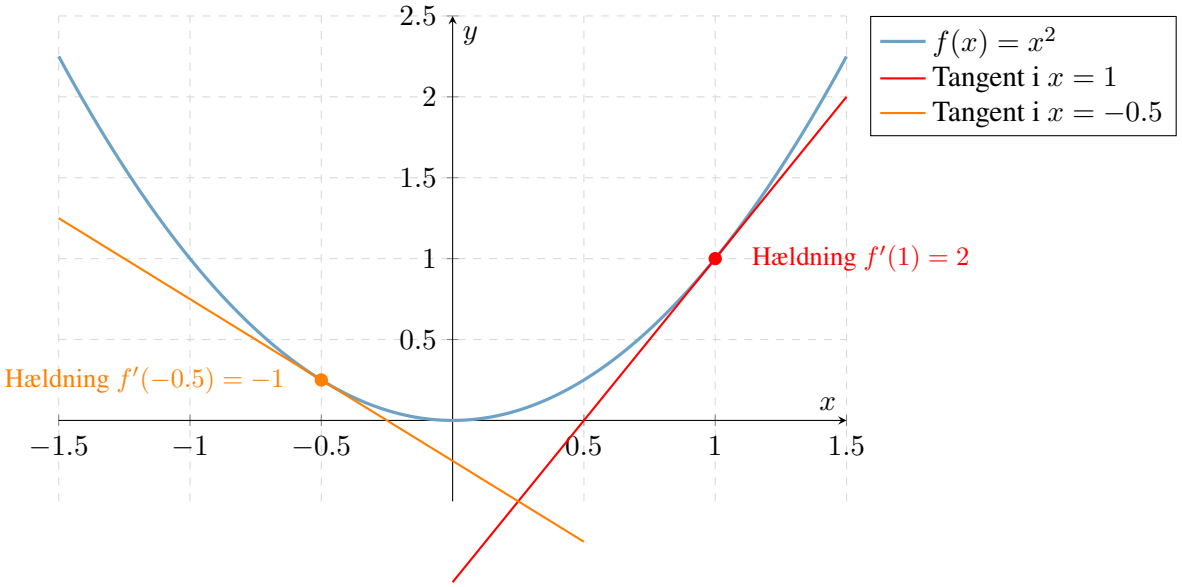
$$f'(x) = 3(x^2 + 1)^2 \cdot 2x = 6x(x^2 + 1)^2.$$

Eksempel 10.7

Find hældningen af tangenten til grafen for funktionen $g(t) = t^4$ i det punkt på grafen, hvor $t = -2$.

Løsning:

Den afledte er $g'(t) = 4t^3$, så tangentens hældning i $t = -2$ er $g'(-2) = 4 \times (-2)^3 = -32$. ◀



Figur 10.4: Tangenterne til grafen for $f(x) = x^2$ har forskellige hældninger i forskellige punkter.

Eksempel 10.8

Find ligningen for tangenten til grafen for $y = f(x) = x^{\frac{1}{2}}$ i punktet $x = 4$.

Løsning:

$f(4) = 4^{\frac{1}{2}} = \sqrt{4} = 2$, så koordinaterne til punktet på grafen er $(4, 2)$. Den afledte er $f'(x) = \frac{1}{2\sqrt{x}}$, så $f'(4) = \frac{1}{4}$.

Enhver ikke-lodret linje har en ligning på formen $y = mx + b$, hvor m er hældningen, og b er skæringen med andenaksen.

Her er hældningen $\frac{1}{4}$, så $m = \frac{1}{4}$, og ligningen er $y = \frac{x}{4} + b$. Da linjen går gennem punktet $(4, 2)$, ved vi, at $y = 2$, når $x = 4$.

Ved indsættelse får vi $2 = \frac{4}{4} + b$, så $b = 1$. Ligningen er derfor $y = \frac{x}{4} + 1$. ◀

Vi ved nu, hvordan man differentierer enhver funktion, der er en potens af variablen. Eksempler er funktioner som x^3 og $t^{-1.3}$. Du vil møde funktioner, der ved første øjekast ikke ser ud til at være en potens af variablen, men som kan omskrives til denne form. Et af de simpleste eksempler er funktionen

$$f(t) = \sqrt{t}$$

som også kan skrives på formen

$$f(t) = t^{\frac{1}{2}}.$$

Den afledte er så

$$f'(t) = \frac{t^{-\frac{1}{2}}}{2} = \frac{1}{2\sqrt{t}}$$

Tilsvarende gælder, at hvis

$$h(s) = \frac{1}{s} = s^{-1}$$

så er

$$h'(s) = -s^{-2} = -\frac{1}{s^2}$$

Eksempel 10.9

Hvis $f(x) = \frac{1}{\sqrt[3]{x}} = x^{-\frac{1}{3}}$, så er $f'(x) = -\frac{1}{3}x^{-\frac{4}{3}}$.

Eksempel 10.10

Hvis $y = \frac{1}{x\sqrt{x}} = x^{-\frac{3}{2}}$, så er $\frac{dy}{dx} = -\frac{3}{2}x^{-\frac{5}{2}}$.

Skalarreglen og linearitet

Indtil videre ved vi, hvordan man differentierer potenser af den uafhængige variabel. Mange af de funktioner, der optræder i anvendelser, er bygget op af sådanne potenser på enkle måder. Funktionen $3x^2$ er for eksempel blot et skalart multiplum af x^2, men hverken konstantreglen eller potensreglen fortæller os, hvordan vi differentierer $3x^2$. Reglerne forklarer heller ikke, hvordan man differentierer udtryk som $x^2 + x^3$ eller $x^2 - x^3$.

Sætning 10.2 (Skalarreglen og linearitetsreglen)

3. **Skalarreglen:** $\frac{d}{dx}[c \cdot f(x)] = c \cdot f'(x)$.
4. **Linearitetsreglen:** $\frac{d}{dx}[f(x) \pm g(x)] = f'(x) \pm g'(x)$.

♡

Skalarreglen og linearitetsreglen udfylder dette hul. De beskriver, hvordan man differentierer funktioner, der fremkommer ved at gange en funktion med en konstant og ved at lægge funktioner sammen eller trække dem fra hinanden. Med disse regler kan vi udvide vores differentiationsteknikker fra simple potenser til en bred klasse af funktioner, der er bygget op af dem.

Eksempel 10.11

- Hvis $f(x) = 3x^2$, så er $f'(x) = 3 \times \frac{d}{dx}x^2 = 6x$.
- Hvis $g(t) = 3t^2 + 2t^{-2}$, så er $g'(t) = \frac{d}{dt}3t^2 + \frac{d}{dt}2t^{-2} = 6t - 4t^{-3}$.
- Hvis $y = \frac{3}{\sqrt{x}} - 2x\sqrt[3]{x} = 3x^{-\frac{1}{2}} - 2x^{\frac{4}{3}}$, så er $\frac{dy}{dx} = -\frac{3}{2}x^{-\frac{3}{2}} - \frac{8}{3}x^{\frac{1}{3}}$.
- Hvis $y = -0.3x^{-0.4}$, så er $\frac{dy}{dx} = 0.12x^{-1.4}$.
- $\frac{d}{dx}2x^{0.3} = 0.6x^{-0.7}$.

Advarsel. Selv om linearitetsreglen siger, at $\frac{d}{dx}(f(x) \pm g(x)) = f'(x) \pm g'(x)$, gælder denne egenskab *ikke* for produkter eller kvotienter. Generelt er

$$\frac{d}{dx}(f(x)g(x)) \neq f'(x)g'(x), \qquad \frac{d}{dx}\left(\frac{f(x)}{g(x)}\right) \neq \frac{f'(x)}{g'(x)}.$$

For at differentiere $f(x)g(x)$ eller $\frac{f(x)}{g(x)}$ kan vi ikke blot differentiere faktorerne og derefter gange eller dividere resultaterne. De korrekte teknikker er *produktreglen* og *kvotientreglen*, som udvikles i næste afsnit.

Produktreglen og kvotientreglen

En anden almindelig måde at kombinere funktioner på er at gange dem med hinanden og derved danne et *produkt*. Produktreglen giver metoden til at differentiere funktioner, der er opbygget på denne måde.

Kvotientreglen gør det derimod muligt at differentiere funktioner, der fremkommer ved at dividere én funktion med en anden, dvs. ved at danne kvotienter af funktioner. Kvotientreglen giver metoden til at differentiere funktioner, der er opbygget på denne måde.

Sætning 10.3 (Produktreglen og kvotientreglen)

5. **Produktreglen:** Hvis $f(x)$ og $g(x)$ er differentiable, så gælder
$$\frac{d}{dx}(f(x)g(x)) = f'(x)g(x) + f(x)g'(x).$$
6. **Kvotientreglen:** Hvis $f(x)$ og $g(x)$ er differentiable, og $g(x) \neq 0$, så gælder
$$\frac{d}{dx}\left(\frac{f(x)}{g(x)}\right) = \frac{f'(x)g(x) - f(x)g'(x)}{[g(x)]^2}.$$

♡

Eksempel 10.12 Produktreglen

- Hvis $y = (x+2)\left(x^2+3\right)$, så er $y' = (x+2)2x + 1\left(x^2+3\right)$.
- Hvis $f(x) = \sqrt{x}\left(x^3 - 3x^2 + 7\right)$, så er $f'(x) = \sqrt{x}\left(3x^2 - 6x\right) + \frac{1}{2}x^{-\frac{1}{2}}\left(x^3 - 3x^2 + 7\right)$.
- Hvis $z = \left(t^2+3\right)\left(\sqrt{t} + t^3\right)$, så er $\frac{dz}{dt} = \left(t^2+3\right)\left(\frac{1}{2}t^{-\frac{1}{2}} + 3t^2\right) + 2t\left(\sqrt{t} + t^3\right)$.

Eksempel 10.13 Kvotientreglen

- Hvis $y = \frac{2x^2+3x}{x^3+1}$, så er $\frac{dy}{dx} = \frac{\left(x^3+1\right)(4x+3) - \left(2x^2+3x\right)3x^2}{\left(x^3+1\right)^2}$.
- Hvis $g(t) = \frac{t^2+3t+1}{\sqrt{t}+1}$, så er $g'(t) = \frac{(\sqrt{t}+1)(2t+3) - \left(t^2+3t+1\right)\left(\frac{1}{2}t^{-\frac{1}{2}}\right)}{(\sqrt{t}+1)^2}$.

På grund af minustegnet i tælleren (dvs. øverst i brøken) er det i kvotientreglen vigtigt at skrive leddene i tælleren i den rigtige rækkefølge. Det er en hyppig kilde til fejl, så vær omhyggelig. Find din egen måde at huske den rigtige rækkefølge af leddene på.

Kædereglen

Kædereglen er nok den vigtigste differentiationsregel, især inden for maskinlæring. Den fortæller os, hvordan vi finder den afledte af en sammensat funktion – en funktion, der er indlejret i en anden. I software sammensætter vi hele tiden funktioner, og kædereglen er det matematiske værktøj til at analysere, hvordan ændringer forplanter sig gennem disse sammensætninger.

Sætning 10.4 (Kædereglen)

1. Hvis $h(x) = f(g(x))$ er en sammensat funktion, er dens afledte lig med den afledte af den ydre funktion (udregnet i den indre funktion) ganget med den afledte af den indre funktion.
$$h'(x) = f'(g(x)) \cdot g'(x).$$

♡

Eksempel 10.14

Find den afledte af $h(x) = (x^3 + 2x)^5$.

Løsning: Der er tale om en sammensætning, hvor den ydre funktion er $f(u) = u^5$, og den indre funktion er $g(x) = x^3 + 2x$. Deres afledte er $f'(u) = 5u^4$ og $g'(x) = 3x^2 + 2$. Vi anvender kædereglen:

$$\begin{aligned} h'(x) &= f'(g(x)) \cdot g'(x) \\ &= 5(x^3 + 2x)^4 \cdot (3x^2 + 2) \end{aligned}$$

◀

Eksempel 10.15

Differentiér $(3x^2 - 5)^3$.

Løsning: Første skridt er altid at se, at udtrykket er en sammensat funktion, og derefter at opdele det i dets ydre og indre del. I dette eksempel er den ydre funktion $(\,\cdot\,)^3$ med den afledte $3(\,\cdot\,)^2$, og den indre funktion er $3x^2 - 5$ med den afledte $6x$. Reglen for sammensatte funktioner giver derfor

$$\frac{d}{dx}(3x^2 - 5)^3 = 3(3x^2 - 5)^2 \times 6x = 18x(3x^2 - 5)^2.$$

Alternativt kan vi indføre substitutionen $u = 3x^2 - 5$ og skrive $y = u^3$. Så er

$$\frac{dy}{dx} = \frac{dy}{du} \times \frac{du}{dx} = 3u^2 \times 6x = 18x(3x^2 - 5)^2.$$

◀

Eksempel 10.16

Find $\frac{dy}{dx}$, hvis $y = \sqrt{x^2 + 1}$.

Løsning: Den ydre funktion er $\sqrt{\cdot}$ med den afledte $\frac{1}{2\sqrt{\cdot}}$, og den indre funktion er $x^2 + 1$ med den afledte $2x$. Kædereglen giver derfor

$$\frac{dy}{dx} = \frac{1}{2\sqrt{x^2 + 1}} \times 2x = \frac{x}{\sqrt{x^2 + 1}}.$$

◀

Kædereglen er den centrale mekanisme bag **backpropagation**-algoritmen, som bruges til at træne neurale netværk. En fejl i et netværks output er en sammensætning af mange indlejrede funktioner (lagene). Backpropagation bruger kædereglen gentagne gange til at beregne den afledte af fejlen med hensyn til hver vægt i netværket og fortæller os dermed, hvordan hver vægt skal justeres for at forbedre modellen.

Eksempel 10.17 En opgave med flere regler

Find den afledte af $h(x) = \frac{x^2}{(3x+1)^4}$.

Løsning: Opgaven kræver kvotientreglen, og nævneren kræver kædereglen. Lad $f(x) = x^2$ og $g(x) = (3x+1)^4$. Så er $f'(x) = 2x$. For at finde $g'(x)$ bruger vi kædereglen: $g'(x) = 4(3x + 1)^3 \cdot 3 = 12(3x + 1)^3$. Nu anvender

vi kvotientreglen:

$$\begin{aligned} h'(x) &= \frac{f'(x)g(x) - f(x)g'(x)}{[g(x)]^2} \\ &= \frac{(2x)(3x+1)^4 - (x^2)(12(3x+1)^3)}{((3x+1)^4)^2} \\ &= \frac{(2x)(3x+1) - 12x^2}{(3x+1)^5} \quad \text{(efter at have sat } (3x+1)^3 \text{ uden for parentes og forkortet)} \\ &= \frac{6x^2 + 2x - 12x^2}{(3x+1)^5} = \frac{2x - 6x^2}{(3x+1)^5} \end{aligned}$$

◀

Oversigt over differentiationsreglerne

Differentiationsreglerne er samlet i følgende sætning.

Sætning 10.5 (Differentiationsregler)

1. **Konstantreglen:** Hvis c er en konstant, så er $\frac{d}{dx}c = 0$.
2. **Potensreglen:** For ethvert reelt n gælder $\frac{d}{dx}x^n = nx^{n-1}$.

   $$\frac{d}{dx}x^n = nx^{n-1}.$$
3. **Skalarreglen:** Hvis c er en konstant, så er $\frac{d}{dx}[c \cdot f(x)] = c \cdot f'(x)$.
4. **Linearitet:** For differentiable f, g og konstanter a, b gælder $\frac{d}{dx}(af + bg) = af' + bg'$.
5. **Produktreglen:** Hvis f, g er differentiable, så er $\frac{d}{dx}(fg) = f'g + fg'$.
6. **Kvotientreglen:** Hvis f, g er differentiable, og $g(x) \neq 0$, så er $\frac{d}{dx}\left(\frac{f}{g}\right) = \frac{f'g - fg'}{g^2}$.
7. **Kædereglen:** Hvis f, g er differentiable, så er $\frac{d}{dx}(f(g(x))) = f'(g(x))g'(x)$.

♡

Reglerne dækker ikke alle funktioner. Den afledte af e^x er for eksempel e^x, mens den afledte af $\ln x$ er $\frac{1}{x}$, og ingen af dem passer ind i de mønstre, vi har beskrevet indtil nu. De mest almindelige differentiable funktioner, der falder uden for de tidligere regler, er anført nedenfor.

- $\frac{d}{dx}e^x = e^x$
- $\frac{d}{dx}\ln x = \frac{1}{x}$
- $\frac{d}{dx}\sin x = \cos x$
- $\frac{d}{dx}\cos x = -\sin x$

Eksempel 10.18

Differentiér $\ln(x^2 + 3x + 1)$.

Løsning: Vi løser opgaven ved hjælp af kædereglen og vores viden om den afledte af $\ln x$.

$$\begin{aligned}
\frac{d}{dx}\ln\left(x^2+3x+1\right) &= \frac{d}{dx}\left(\ln u\right) \quad \left(\text{ hvor } u = x^2+3x+1\right) \\
&= \frac{d}{du}\left(\ln u\right) \times \frac{du}{dx} \quad (\text{ ifølge kædereglen }) \\
&= \frac{1}{u} \times \frac{du}{dx} \\
&= \frac{1}{x^2+3x+1} \times \frac{d}{dx}\left(x^2+3x+1\right) \\
&= \frac{1}{x^2+3x+1} \times (2x+3) \\
&= \frac{2x+3}{x^2+3x+1}
\end{aligned}$$

◀

Eksempel 10.19

Find $\frac{d}{dx}\left(e^{3x^2}\right)$.

Løsning: Det er en anvendelse af kædereglen sammen med vores viden om den afledte af e^x.

$$\begin{aligned}
\frac{d}{dx}\left(e^{3x^2}\right) &= \frac{de^u}{dx} \quad \text{hvor } u = 3x^2 \\
&= \frac{de^u}{du} \times \frac{du}{dx} \quad \text{ifølge kædereglen} \\
&= e^u \times \frac{du}{dx} \\
&= e^{3x^2} \times \frac{d}{dx}\left(3x^2\right) \\
&= 6xe^{3x^2}
\end{aligned}$$

◀

Eksempel 10.20

Find $\frac{d}{dx}\left(e^{x^2+2x}\right)$.

Løsning: Igen bruger vi vores viden om den afledte af e^x sammen med kædereglen.

$$\begin{aligned}
\frac{d}{dx}\left(e^{x^2+2x}\right) &= \frac{de^u}{dx} \quad \left(\text{ hvor } u = x^2+2x\right) \\
&= e^u \times \frac{du}{dx} \quad (\text{ifølge kædereglen}) \\
&= e^{x^2+2x} \times \frac{d}{dx}\left(x^2+2x\right) \\
&= (2x+2)\, e^{x^2+2x}
\end{aligned}$$

◀

Eksempel 10.21

Differentiér $\ln\left(2x^3+5x^2-3\right)$.

Løsning: Vi løser opgaven ved hjælp af kædereglen og vores viden om den afledte af $\ln x$.

$$\begin{aligned}
\frac{d}{dx}\ln\left(2x^3+5x^2-3\right) &= \frac{d\ln u}{dx} \quad \text{(hvor } u=\left(2x^3+5x^2-3\right)\text{)} \\
&= \frac{d\ln u}{du}\times\frac{du}{dx} \quad \text{(ifølge kædereglen)} \\
&= \frac{1}{u}\times\frac{du}{dx} \\
&= \frac{1}{2x^3+5x^2-3}\times\frac{d}{dx}\left(2x^3+5x^2-3\right) \\
&= \frac{1}{2x^3+5x^2-3}\times\left(6x^2+10x\right) \\
&= \frac{6x^2+10x}{2x^3+5x^2-3}
\end{aligned}$$

◀

Der findes to genveje til at differentiere funktioner med eksponentialfunktioner og logaritmer. De fire eksempler ovenfor gav

$$\begin{aligned}
\frac{d}{dx}\left(\ln\left(x^2+3x+1\right)\right) &= \frac{2x+3}{x^2+3x+1} \\
\frac{d}{dx}\left(e^{3x^2}\right) &= 6xe^{3x^2} \\
\frac{d}{dx}\left(e^{x^2+2x}\right) &= (2x+2)\,e^{x^2+2x} \\
\frac{d}{dx}\left(\ln\left(2x^3+5x^2-3\right)\right) &= \frac{6x^2+10x}{2x^3+5x^2-3}
\end{aligned}$$

Eksemplerne antyder de generelle regler

$$\begin{aligned}
\frac{d}{dx}\left(e^{f(x)}\right) &= f'(x)e^{f(x)} \\
\frac{d}{dx}(\ln f(x)) &= \frac{f'(x)}{f(x)}
\end{aligned}$$

Reglerne følger af kædereglen og af, at $\frac{d}{dx}e^x = e^x$ og $\frac{d}{dx}\ln x = \frac{1}{x}$ for $x > 0$. De kan gøre differentiationen hurtigere, men hvis du glemmer dem, kan de genskabes ved hjælp af kædereglen som i eksemplerne ovenfor.

10.4 Anvendelser af afledte

Matematikkens udvikling hører til blandt den menneskelige tænknings største bedrifter, og fremkomsten af infinitesimalregningen – både differential- og integralregning – markerer et vendepunkt i den historie. Især differentialregning har utallige praktiske anvendelser inden for naturvidenskab og ingeniørfag, alt for mange til at opregne. Dens betydning afspejles i, at stort set alle kvantitative discipliner bygger på dens metoder.

I elementær matematik bruges differentialregning primært på to områder: til at analysere og skitsere kurver og til at løse optimeringsproblemer. I dette afsnit giver vi en kort introduktion til, hvordan differentialregning anvendes til optimering.

Stationære punkter – idéen bag optimering

Lad os vende tilbage til vores metafor med bjergkæden. Forestil dig, at du står et sted mellem tinder og dale, måske endda med lukkede øjne eller indhyllet i tåge. Mens du vandrer rundt, bemærker du, at når du går op ad bakke eller ned ad bakke, ved du, at du ikke er på den højeste top eller i den laveste dal – du er enten stadig på vej op eller stadig på vej ned.

Men på toppen af en tinde er der et kort, plant stykke, hvor hældningen under dine fødder er nul. I bunden af en dal er der også et plant sted. Hvis du befinder dig på helt flad grund – i det mindste et øjeblik – har du et fingerpeg: Du er måske på en top, i en dal eller måske ved et blidt vendepunkt langs en bjergryg. I en bjergkædes store terræn er det kun på disse plane steder, hvor hældningen forsvinder, at du overhovedet kan være i et ekstremumspunkt: et maksimum eller et minimum.

Det motiverer et vigtigt skridt i optimering: Indre punkter, hvor en differentiabel funktion har et lokalt maksimum eller minimum, skal have afledt lig med nul. Disse steder kaldes stationære punkter. Et fuldstændigt optimeringsproblem kan også kræve, at man undersøger endepunkter og punkter, hvor den afledte ikke eksisterer.

Definition 10.4

For en funktion $y = f(x)$ kaldes de punkter på grafen, hvor grafen har hældningen nul, for stationære punkter. Stationære punkter er med andre ord der, hvor $f'(x) = 0$.

♣

For at finde en funktions stationære punkter differentierer vi, sætter den afledte lig med nul og løser ligningen.

Eksempel 10.22

Find de stationære punkter for funktionen $f(x) = 2x^3 + 3x^2 - 12x + 17$.

Løsning:

$f'(x) = 6x^2 + 6x - 12$. Vi sætter $f'(x) = 0$ og løser:

$$\begin{aligned} 6x^2 + 6x - 12 &= 0 \\ x^2 + x - 2 &= 0 \\ (x-1)(x+2) &= 0 \\ x &= 1, -2 \end{aligned}$$

Det giver de værdier af x, hvor funktionen f er stationær. De tilhørende funktionsværdier findes ved at indsætte 1 og -2 i funktionen.

De er $f(1) = 2 \times 1^3 + 3 \times 1^2 - 12 \times 1 + 17 = 10$ og $f(-2) = 2 \times (-2)^3 + 3 \times (-2)^2 - 12 \times (-2) + 17 = 37$. De stationære punkter er derfor $(1, 10)$ og $(-2, 37)$. ◀

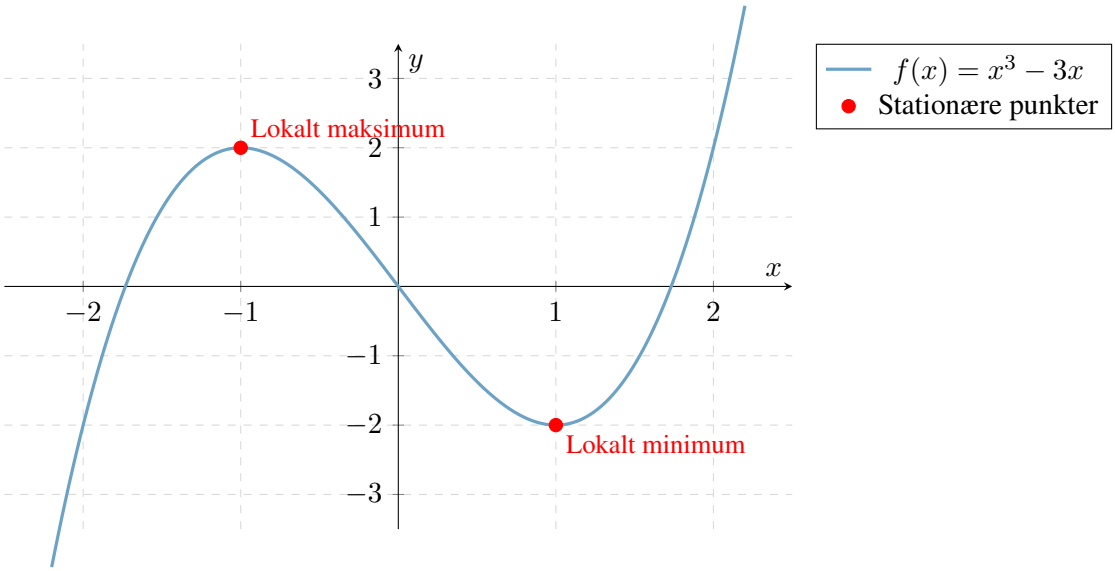
Eksempel 10.23

Find de stationære punkter for funktionen $g(t) = e^{t^2}$.

Løsning: Ved at differentiere og sætte den afledte lig med nul får vi ligningen $g'(t) = 2te^{t^2} = 0$. Da e^{t^2} aldrig er nul, er den eneste løsning til ligningen der, hvor $2t = 0$, dvs. $t = 0$. Ved indsættelse i forskriften for g får vi funktionsværdien $g(0) = e^{0^2} = 1$. Det stationære punkt er altså $(0, 1)$. ◀

Typer af stationære punkter

Hvis vi vender tilbage til metaforen med bjergkæden, kan stationære punkter opfattes som de særlige steder, hvor jorden et øjeblik føles helt plan under dine fødder. Nogle gange sker det helt oppe på toppen af en bakke – hvor du, uanset hvilken retning du rækker ud i, kun mærker skråninger nedad foran dig. Det kaldes et lokalt maksimum: toppen i dine umiddelbare omgivelser, men ikke nødvendigvis den højeste tinde i hele bjergkæden. Andre gange ligger det plane sted i bunden af en dal – alt omkring dig skråner opad. Det kaldes et lokalt minimum: det laveste sted i dit lokale nærområde, selv om der findes dybere dale andre steder. Betegnelsen »lokal« understreger, at vi taler om det højeste eller laveste punkt i nærheden, ikke globalt. Du vil måske også høre betegnelserne »relativt maksimum« og »relativt minimum« brugt om dem. Figur 10.5 viser en funktion som et landskab med både en bakketop (lokalt maksimum) og en dalbund (lokalt minimum). Bemærk, at hældningen er nul i hvert af disse punkter – præcis som på et plant stykke i bjergene.



Figur 10.5: En kurve med et lokalt maksimum i $x = -1$ og et lokalt minimum i $x = 1$.

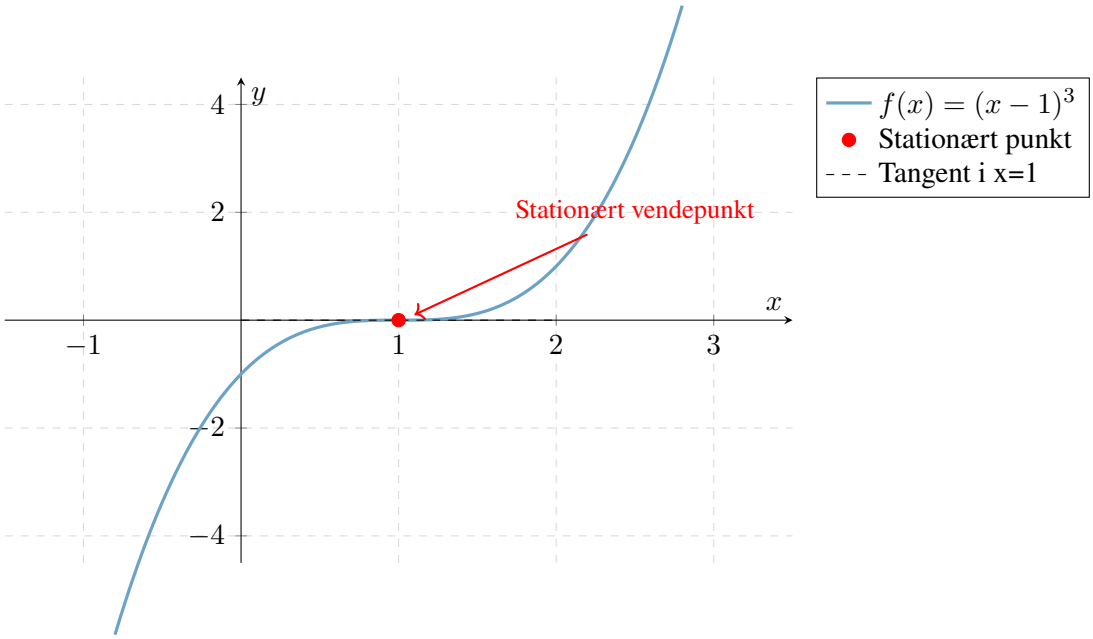
Lokale maksima og lokale minima er ikke de eneste typer af stationære punkter. Der findes en tredje slags. Figur 10.6 viser et stationært punkt, der hverken er et lokalt maksimum eller et lokalt minimum. Denne type stationært punkt kaldes et stationært vendepunkt eller blot et *vendepunkt*.

Første afledtes test

Lad os vende tilbage til Eksempel 10.22. For funktionen $f(x) = 2x^3 + 3x^2 - 12x + 17$ fandt vi stationære punkter i $(1, 10)$ og $(-2, 37)$. Det naturlige spørgsmål er: Hvilken type stationære punkter er det? Uden en skitse af grafen er det ikke umiddelbart klart, om de svarer til lokale maksima, lokale minima eller stationære vendepunkter. En nøjagtig graf ville besvare spørgsmålet, men den kan kræve en betydelig indsats. I stedet søger vi en metode, der gør det muligt at klassificere et stationært punkt uden at tegne hele funktionen.

Der findes flere fremgangsmåder, men i dette afsnit fokuserer vi på én teknik, kendt som *første afledtes test*. Idéen er at undersøge funktionens opførsel umiddelbart til venstre og umiddelbart til højre for det stationære punkt.

For at forstå princippet kan du forestille dig en person, der står et sted i et landskab, hvor jorden er plan. Uden at kunne se omgivelserne vil personen afgøre, om stedet er toppen af en bakke, bunden af en dal eller ingen af

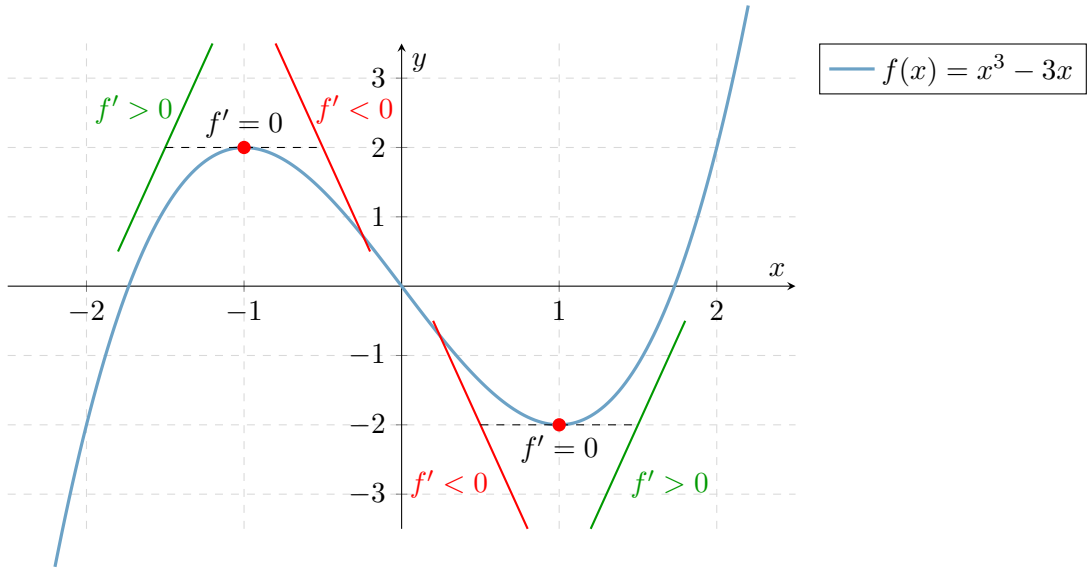


Figur 10.6: En funktion med et stationært vendepunkt i $x = 1$. Tangenten i punktet er vandret, men punktet er hverken et lokalt maksimum eller et lokalt minimum.

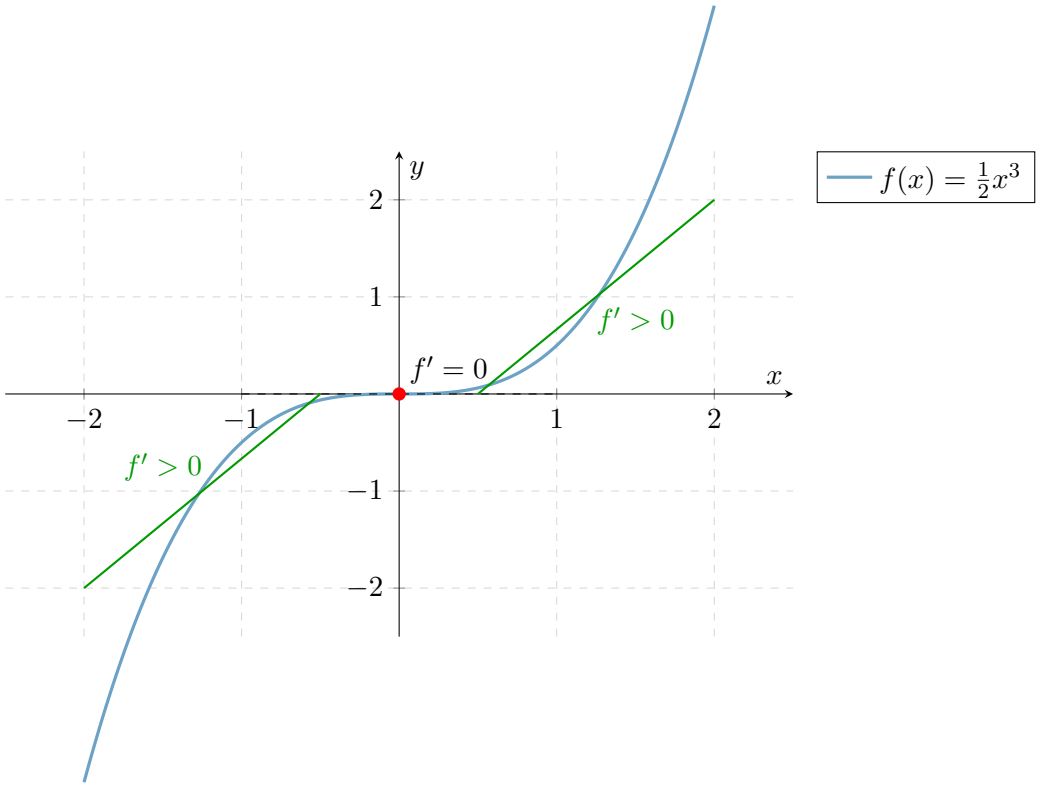
delene. En måde at afgøre det på er at tage et lille skridt tilbage og mærke hældningen og derefter tage et lille skridt frem og mærke hældningen igen.

Hvis jorden skråner opad bagude og nedad forude, er det plane punkt toppen af en bakke, svarende til et lokalt maksimum. Hvis jorden skråner nedad bagude og opad forude, er punktet bunden af en dal, svarende til et lokalt minimum. Hvis hældningen har samme fortegn på begge sider, er punktet ikke et lokalt ekstremum; der kræves yderligere analyse for at afgøre, om det er et vendepunkt.

Denne tankegang indfanger essensen af første afledtes test: Ved at analysere fortegnet for $f'(x)$ umiddelbart på hver side af et stationært punkt kan vi bestemme punktets karakter uden at være afhængige af en graf.



Figur 10.7: Illustration af første afledtes test. I et lokalt maksimum skifter den afledte fra positiv til negativ. I et lokalt minimum skifter den fra negativ til positiv.



Figur 10.8: Illustration af et stationært vendepunkt. Den afledte er nul i $x = 0$, men positiv på begge sider, så punktet er hverken et lokalt maksimum eller et lokalt minimum.

Vi kan sammenfatte disse overvejelser i følgende sætning:

Sætning 10.6 (Første afledtes test)

Lad c være et kritisk punkt for en kontinuert funktion f.

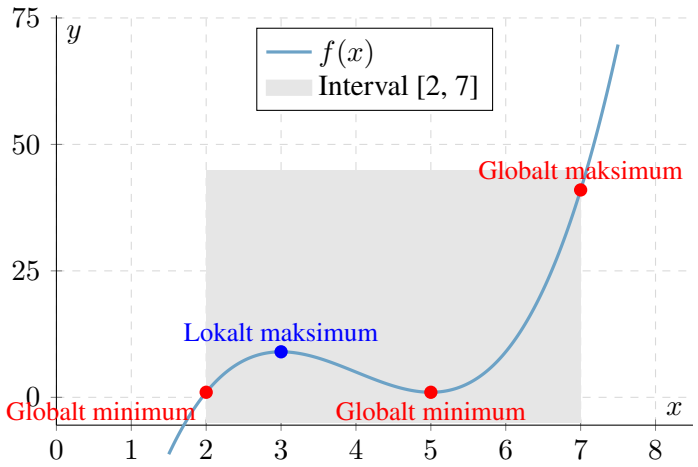
- Hvis $f'(x)$ skifter fra negativ til positiv i c, har f et **lokalt minimum** i c.
- Hvis $f'(x)$ skifter fra positiv til negativ i c, har f et **lokalt maksimum** i c.
- Hvis $f'(x)$ ikke skifter fortegn i c, har f intet lokalt ekstremum i c. Første afledtes test kan ikke alene afgøre, om c er et vendepunkt.

♡

Testen er illustreret i Figur 10.7 og Figur 10.8.

Optimering

Nu hvor vi har lagt fundamentet, kan vi gå i gang med nogle optimeringsproblemer. Når vi vil maksimere en funktion $f(x)$ på et bestemt interval for x, er målet at finde den største værdi, som $f(x)$ antager på intervallet. Det er vigtigt at være opmærksom på, at denne maksimumsværdi ikke altid ligger i et stationært punkt. Se på Figur 10.9, hvor vi søger funktionens største og mindste værdi på intervallet $2 \leq x \leq 7$. På intervallet har funktionen to stationære punkter – det ene svarer til et lokalt maksimum, det andet til et lokalt minimum. Funktionens største værdi på intervallet antages dog faktisk i endepunktet $x = 7$, som ikke er et stationært punkt. Funktionen antager denne værdi, simpelthen fordi større x-værdier uden for intervallet ikke tages i betragtning. Mindsteværdien antages derimod både i endepunktet $x = 2$ og i det stationære punkt $x = 5$. Med denne forståelse kan vi nu forklare de systematiske trin til at finde en funktions maksimum eller minimum i et givet område.



Figur 10.9: På det begrænsede interval $[2, 7]$ ligger det globale maksimum i et endepunkt ($x = 7$) og ikke i det lokale maksimum. Det globale minimum antages både i endepunktet $x = 2$ og i det indre stationære punkt $x = 5$.

Placeringen af maksima og minima

En funktion $f(x)$ har ikke nødvendigvis en største eller mindste værdi i et bestemt område af x-værdier. Men hvis de findes, skal maksimums- og minimumsværdien ligge et af tre steder:

1. I endepunkterne (hvis de findes) af det område, vi betragter.
2. Inde i området i et stationært punkt.
3. Inde i området i et punkt, hvor den afledte ikke eksisterer.

Bemærkning:

1. Det er let at finde tilfælde, hvor en funktion ikke antager et maksimum eller minimum inden for et bestemt område. Funktionen $f(x) = x$ har for eksempel hverken maksimum eller minimum på intervallet $-\infty < x < \infty$; grafen vokser ubegrænset, når x vokser. Med hensyn til punkt 1 ovenfor skal du bemærke, at området $-\infty < x < \infty$ faktisk ikke har nogen endepunkter. Et andet eksempel er området $x \geq 1$, som kun har ét endepunkt, nemlig $x = 1$.
2. Med hensyn til punkt 3 behandler denne tekst ikke situationer, hvor den afledte ikke eksisterer i visse punkter. Husk dog, at sådanne punkter kan forekomme, og at et maksimum eller minimum kan ligge netop dér. Flere detaljer findes i litteratur om videregående infinitesimalregning.

Nu hvor vi ved præcis, hvor maksima og minima kan ligge, kan vi give en fremgangsmåde til at finde dem.

Fremgangsmåde til at finde en funktions største eller mindste værdi.

1. Find endepunkterne for det område, vi betragter (hvis der er nogen).
2. Find alle stationære punkter i området.
3. Find alle punkter i området, hvor den afledte ikke eksisterer.
4. Indsæt hvert af disse punkter i funktionen, og se, hvilket der giver den største (eller mindste) funktionsværdi.

Eksempel 10.24

Find den mindste og den største værdi af funktionen $f(x) = x^2e^x$ for $-4 \leq x \leq 1$.

Løsning: Vi følger fremgangsmåden ovenfor. Endepunkterne er -4 og 1. Ved differentiation får vi $f'(x) = x^2e^x + 2xe^x = x(x+2)e^x$. Ved at sætte $f'(x) = 0$ og løse ligningen får vi stationære punkter i $x = 0$ og $x = -2$. Der er ingen punkter, hvor den afledte ikke eksisterer. Den største og den mindste værdi findes derfor i et af punkterne $x = -4, -2, 0, 1$. Ved indsættelse får vi $f(-4) \approx 0.29$, $f(-2) \approx 0.54$, $f(0) = 0$ og $f(1) = e \approx 2.7$. Den største værdi antages derfor i $x = 1$ og er lig med e, og den mindste værdi antages i $x = 0$ og er 0. ◀

Eksempel 10.25

Find den største og den mindste værdi af funktionen $g(t) = \frac{1}{3}t^3 - t + 2$ for $0 \leq t \leq 3$.

Løsning: Endepunkterne er $t = 0$ og $t = 3$. Ved at differentiere og sætte lig med nul får vi $g'(t) = t^2 - 1 = (t-1)(t+1) = 0$, så de stationære punkter ligger i $t = -1, 1$. Da -1 ikke ligger i området, er de mulige placeringer af maksimum og minimum $t = 0, 1, 3$. Ved indsættelse i g får vi $g(0) = 2$, $g(1) = \frac{4}{3}$ og $g(3) = 8$. Den største værdi er derfor $g(3) = 8$, og den mindste værdi er $g(1) = \frac{4}{3}$. ◀

Eksempel 10.26

Antag, at du tuner en maskinlæringsmodel og vil finde den læringsrate α (mellem 0 og 1), der minimerer tabsfunktionen $L(\alpha) = (\alpha - 0.3)^2 + 1$. Hvilken værdi af α giver det mindste tab?

Løsning: Vi anvender fremgangsmåden til at finde globale ekstrema på det lukkede interval $[0, 1]$. Den funktion, der skal minimeres, er tabsfunktionen $L(\alpha) = (\alpha - 0.3)^2 + 1$.

1. **Find endepunkterne:** Det område, vi betragter, er $0 \leq \alpha \leq 1$, så endepunkterne er $\alpha = 0$ og $\alpha = 1$.
2. **Find de stationære punkter:** Først finder vi den afledte af tabsfunktionen med hensyn til α:
   $$\frac{dL}{d\alpha} = \frac{d}{d\alpha}\left[(\alpha - 0.3)^2 + 1\right] = 2(\alpha - 0.3)$$
   Dernæst sætter vi den afledte lig med nul for at finde de stationære punkter:
   $$2(\alpha - 0.3) = 0 \implies \alpha = 0.3$$
   Det eneste stationære punkt er $\alpha = 0.3$, som ligger i vores interval $[0, 1]$.
3. **Find punkter, hvor den afledte ikke er defineret:** Den afledte $2(\alpha - 0.3)$ er en simpel lineær funktion og er defineret for alle værdier af α. Der er ingen sådanne punkter.
4. **Sammenlign værdierne i kandidatpunkterne:** Vi udregner nu tabsfunktionen $L(\alpha)$ i alle de kandidatpunkter, vi har fundet: endepunkterne og det stationære punkt.
   - I det venstre endepunkt, $\alpha = 0$:
     $$L(0) = (0 - 0.3)^2 + 1 = 0.09 + 1 = 1.09$$
   - I det stationære punkt, $\alpha = 0.3$:
     $$L(0.3) = (0.3 - 0.3)^2 + 1 = 0 + 1 = 1.00$$
   - I det højre endepunkt, $\alpha = 1$:
     $$L(1) = (1 - 0.3)^2 + 1 = (0.7)^2 + 1 = 0.49 + 1 = 1.49$$

Ved at sammenligne værdierne ser vi, at det mindste tab er 1.00, som antages i det stationære punkt.

Konklusion: Det mindste tab opnås, når læringsraten er $\alpha = 0.3$. ◀

10.5 Den anden afledte: krumning og vendepunkter

Den første afledte, $f'(x)$, fortæller os om en funktions hældning eller ændringshastighed. Den **anden afledte**, betegnet $f''(x)$, er den afledte af den første afledte. Den måler, hvordan selve hældningen ændrer sig. Det giver en dybere indsigt i formen af en funktions graf.

Definition 10.5 (Den anden afledte)

Den **anden afledte** af en funktion f, betegnet $f''(x)$, er funktionen

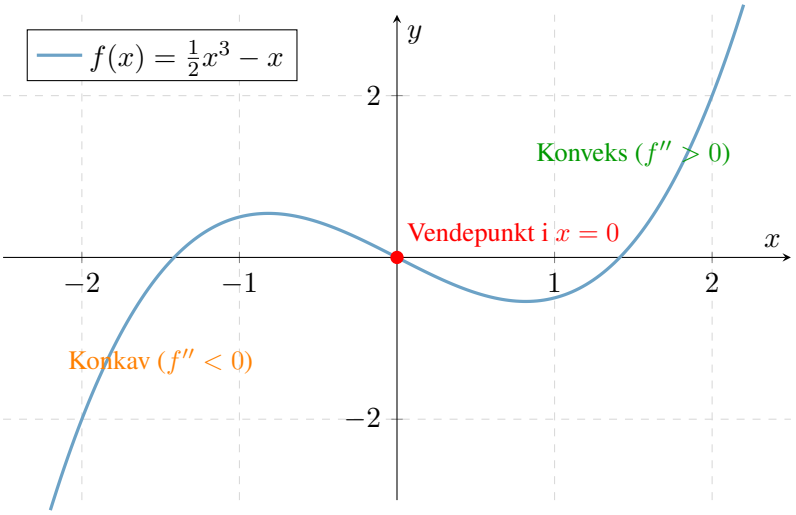
$$f''(x) = \frac{d}{dx}\left(f'(x)\right)$$

forudsat at f' er differentiabel i x.

Den vigtigste geometriske fortolkning af den anden afledte er **krumning**.

- Hvis $f''(x) > 0$ på et interval, er hældningen $f'(x)$ voksende. Grafen bøjer opad som en kop, der holder på vand. Vi siger, at funktionen er **konveks** (krummer opad).
- Hvis $f''(x) < 0$ på et interval, er hældningen $f'(x)$ aftagende. Grafen bøjer nedad som en kop, der spilder vand. Vi siger, at funktionen er **konkav** (krummer nedad).

Et punkt på grafen, hvor krumningen skifter, kaldes et **vendepunkt**. Det sker typisk der, hvor $f''(x) = 0$.



Figur 10.10: En funktion med et område, hvor den er konkav (krummer nedad), efterfulgt af et område, hvor den er konveks (krummer opad). Overgangen sker i vendepunktet.

Den anden afledte giver os også et stærkt værktøj til at klassificere stationære punkter.

Sætning 10.7 (Anden afledtes test)

Lad c være et stationært punkt for f (dvs. $f'(c) = 0$).

- Hvis $f''(c) > 0$, har f et **lokalt minimum** i c.
- Hvis $f''(c) < 0$, har f et **lokalt maksimum** i c.
- Hvis $f''(c) = 0$, giver testen ingen konklusion.

Kapitel 11 Funktioner af flere variable og gradienter

I det foregående kapitel så vi på den afledede som et værktøj til at måle ændringshastigheden for en funktion af én variabel. Geometrisk svarede det til at finde hældningen af en kurve i et punkt. Mange systemer i den virkelige verden afhænger imidlertid af flere faktorer. Et flys brændstoføkonomi afhænger både af dets hastighed og dets flyvehøjde. En maskinlæringsmodels ydeevne er en funktion af tusinder eller endda millioner af parametre. For at kunne håndtere disse komplekse sammenhænge må vi udvide vores forståelse af differentialregning til flere dimensioner.

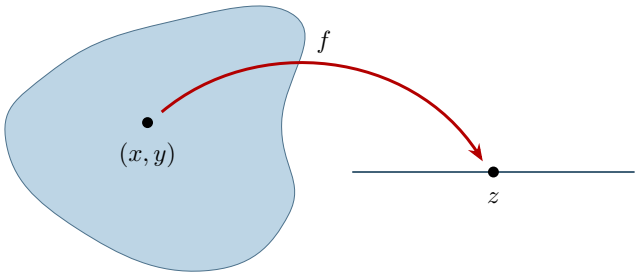
Det fører os fra den enkle bane langs en kurve til det vidtstrakte terræn på en flade. Forestil dig igen vores vandrer, men nu stående i en bjergkæde. »Hældningen« er ikke længere et enkelt tal; den afhænger helt af, hvilken retning vandreren vælger at vende. Peger man lige op ad bjerget, finder man den stejleste vej, mens man får en helt vandret vej, hvis man vender sig langs bjergets højdekurve. Det matematiske værktøj, der indfanger denne flerdimensionale hældning, er **gradienten**.

11.1 Funktioner af flere variable

Vi udvider nu vores perspektiv fra funktioner af én variabel, $f(x)$, til funktioner af to eller flere variable. For enkelhedens skyld fokuserer vi på funktioner af to variable, som skrives $z = f(x, y)$. Den væsentligste forskel er altså, at vi i stedet for at afbilde værdier af én variabel over i værdier af en anden variabel afbilder ordnede par af variable over i en anden variabel.

Definition 11.1 (Funktion af to variable)

En *funktion af to variable* er en regel, der til hvert ordnet par (x, y) i en mængde $D \subseteq \mathbb{R}^2$ knytter et entydigt reelt tal z, som vi skriver $z = f(x, y)$. Mængden D kaldes *definitionsmængden* for f. *Værdimængden* for f er mængden af alle reelle tal z, for hvilke der findes mindst ét $(x, y) \in D$, så $f(x, y) = z$, hvilket er illustreret i Figur 11.1.

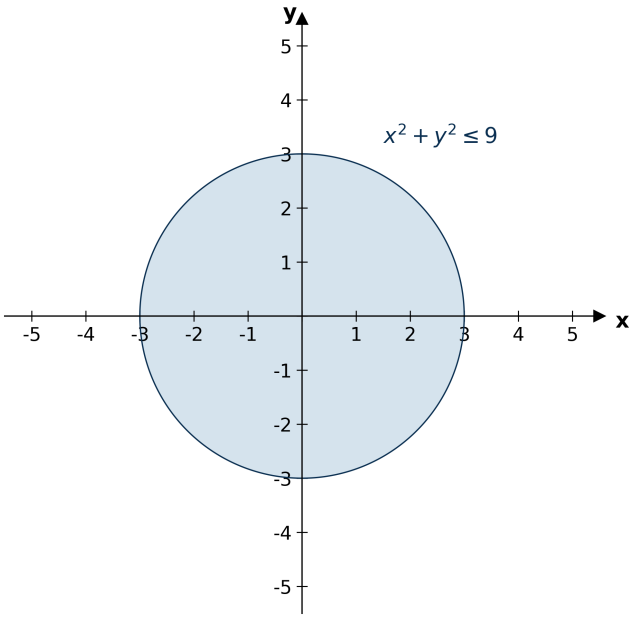


Figur 11.1: Definitionsmængden for en funktion af to variable består af ordnede par (x, y).

Eksempel 11.1 Bestem definitionsmængden og værdimængden for funktionen $f(x, y) = \sqrt{9 - x^2 - y^2}$.

Løsning: For at bestemme definitionsmængden kræver vi, at udtrykket under kvadratroden er ikke-negativt:

$$9 - x^2 - y^2 \geq 0.$$



Figur 11.2: Definitionsmængden for funktionen $f(x, y) = \sqrt{9 - x^2 - y^2}$ er en lukket cirkelskive med radius 3.

Omskrivning giver uligheden

$$x^2 + y^2 \leq 9.$$

Definitionsmængden for f er derfor den lukkede cirkelskive med radius 3 og centrum i origo, som er illustreret i Figur 11.2, og som eksplicit er givet ved

$$\mathrm{Dom}(f) = \{(x, y) \in \mathbb{R}^2 \mid x^2 + y^2 \leq 9\}.$$

For at bestemme værdimængden bemærker vi, at vi for ethvert punkt (x, y) i definitionsmængden har

$$0 \leq x^2 + y^2 \leq 9,$$

så

$$0 \leq 9 - x^2 - y^2 \leq 9.$$

Ved at tage kvadratroden (og huske, at kvadratroden altid er ikke-negativ) får vi

$$0 \leq f(x, y) = \sqrt{9 - x^2 - y^2} \leq 3.$$

Begge endepunkter i dette interval antages:

- $f(x, y) = 3$ i centrum $(0, 0)$,
- $f(x, y) = 0$ på randcirklen $x^2 + y^2 = 9$.

Værdimængden for f er altså det lukkede interval

$$\mathrm{Ran}(f) = \{z \in \mathbb{R} \mid 0 \leq z \leq 3\} = [0, 3].$$

◀

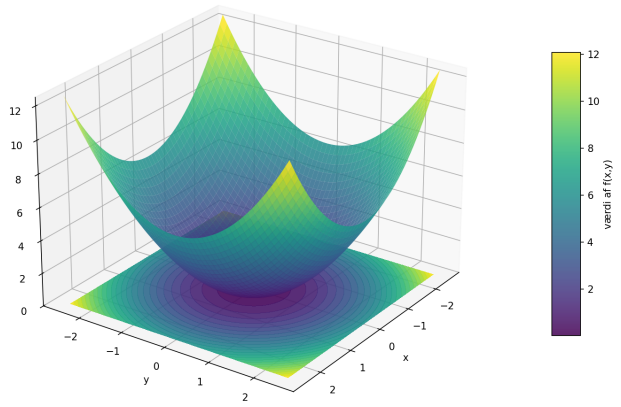
Mens grafen for $f(x)$ er en kurve i en 2D-plan, er grafen for $f(x, y)$ en **flade** i 3D-rummet. Når vi tegner grafen for en funktion af to variable $z = f(x, y)$, tildeles hvert punkt (x, y) i definitionsmængden en højde z, hvilket

giver et ordnet tripel (x, y, z). Samlingen af alle sådanne tripler danner en *flade* i det tredimensionale rum. Man kan forestille sig, at xy-planen ligger vandret, og at værdien af z afsættes lodret over (eller under) hvert punkt (x, y).

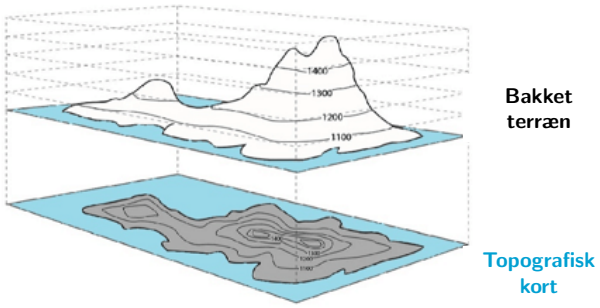
Definition 11.2 (Niveaukurve og konturkort)

En **niveaukurve** for en funktion $f(x, y)$ er mængden af alle punkter (x, y) i inputplanen, hvor funktionen har en konstant værdi, dvs. $f(x, y) = c$ for en konstant c.
En samling af niveaukurver for forskellige værdier af c udgør et **konturkort**.

Figur 11.3: Konturkort: (a) et 3D-plot af en flade med niveaukurver projiceret ned på xy-planen og (b) et topografisk kort med niveaukurver.



(a) Fladen $z = x^2 + y^2$ med niveaukurver projiceret ned på xy-planen. Hver niveaukurve svarer til en konstant funktionsværdi og illustrerer, hvordan niveaukurver opstår som vandrette snit gennem fladen.



(b) Et topografisk kort over et bakket terræn.

På et geografisk kort repræsenterer disse kurver linjer med konstant højde over havet. På vores funktionsflade repræsenterer de kurver med konstant »højde« z. Figur 11.3a viser fladen $z = x^2 + y^2$ med niveaukurver projiceret ned på xy-planen. Hver niveaukurve svarer til en konstant funktionsværdi og illustrerer, hvordan niveaukurver opstår som vandrette snit gennem fladen. Figur 11.3b viser et topografisk kort over et bakket terræn. Niveaukurver er ikke begrænset til simple funktioner; de kan også repræsentere langt mere komplicerede flader.

11.2 Partielle afledede og gradienten

Hvordan måler vi ændringshastigheden for en funktion af flere variable? Da hældningen afhænger af retningen, kan vi begynde med de simpleste retninger: parallelt med koordinatakserne. Det fører til begrebet **partiel afledet**.

Partielle afledede

For en funktion $z = f(x, y)$ undersøger vi, hvordan fladen ændrer sig, ved at lade den ene variabel variere, mens den anden holdes fast. De ændringshastigheder, vi får, er de *partielle afledede* af f. Den partielle afledede med hensyn til x beregnes ved at betragte y som en konstant og differentiere med hensyn til x; den afledede

med hensyn til y findes på tilsvarende måde. Disse to størrelser, $\partial z/\partial x$ og $\partial z/\partial y$, beskriver, hvor stejlt fladen stiger eller falder i koordinatretningerne.

Geometrisk repræsenterer $\frac{\partial f}{\partial x}$ i et punkt (a, b) fladens hældning i x-aksens retning, og $\frac{\partial f}{\partial y}$ repræsenterer fladens hældning i y-aksens retning.

Definition 11.3 (Partielle afledede)

Den **partielle afledede** af $f(x, y)$ med hensyn til x, betegnet $\frac{\partial f}{\partial x}$ eller f_x, er

$$\frac{\partial f}{\partial x} = \lim_{h \to 0} \frac{f(x+h, y) - f(x, y)}{h}$$

Den **partielle afledede** med hensyn til y, betegnet $\frac{\partial f}{\partial y}$ eller f_y, er

$$\frac{\partial f}{\partial y} = \lim_{h \to 0} \frac{f(x, y+h) - f(x, y)}{h}$$

♣

Eksempel 11.2 Se på funktionen

$$f(x, y) = x^2 + 2y^2.$$

Vi beregner dens partielle afledede ved at differentiere med hensyn til den ene variabel, mens den anden holdes konstant.

Partiel afledet med hensyn til x.

Betragt y som en konstant, og differentiér som ved funktioner af én variabel:

$$\frac{\partial}{\partial x}(x^2 + 2y^2) = 2x.$$

Geometrisk svarer det til at skære fladen $z = f(x, y)$ med en plan $y =$ konstant, hvilket giver parablen $z = x^2 +$ konstant. Værdien $2x$ er hældningen af denne parabel i det valgte punkt.

Partiel afledet med hensyn til y.

Betragt nu x som konstant:

$$\frac{\partial}{\partial y}(x^2 + 2y^2) = 4y.$$

Dette repræsenterer hældningen af det tværsnit, man får ved at skære fladen med en plan $x =$ konstant.

Eksempel 11.3 Bestem de partielle afledede af $f(x, y) = 3x^2 + 2xy^3 + 5y$.

Løsning:

For at bestemme $\frac{\partial f}{\partial x}$ betragter vi y som en konstant:

$$\frac{\partial f}{\partial x} = \frac{\partial}{\partial x}(3x^2) + \frac{\partial}{\partial x}(2xy^3) + \frac{\partial}{\partial x}(5y) = 6x + 2y^3 + 0 = 6x + 2y^3$$

For at bestemme $\frac{\partial f}{\partial y}$ betragter vi x som en konstant:

$$\frac{\partial f}{\partial y} = \frac{\partial}{\partial y}(3x^2) + \frac{\partial}{\partial y}(2xy^3) + \frac{\partial}{\partial y}(5y) = 0 + 2x(3y^2) + 5 = 6xy^2 + 5$$

◀

Gradientvektoren

De partielle afledede giver os ændringshastigheden i to bestemte retninger. **Gradienten** samler denne information i én vektor, der peger i retningen med den *stejleste* ændringshastighed.[1] Ordet »gradient« betegner i matematik en vektor, der angiver både retningen og stejlheden af stigningen for en funktion af flere variable:

$$\nabla f(x,y) = \left\langle \frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right\rangle = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$$

Symbolet ∇ kaldes **nabla**- eller **del**-operatoren.

Definition 11.4 (Gradienten)

Gradienten af en funktion $f(x,y)$, betegnet ∇f, er den vektorfunktion, der er defineret ved:

$$\nabla f(x,y) = \left\langle \frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right\rangle = \frac{\partial f}{\partial x}\mathbf{i} + \frac{\partial f}{\partial y}\mathbf{j}$$

Gradienten er det flerdimensionale sidestykke til den første afledede. Den har to afgørende geometriske egenskaber:

1. **Retning for stejleste stigning:** I et punkt, hvor $\nabla f(x,y) \neq \mathbf{0}$, angiver enhedsvektoren $\nabla f / \|\nabla f\|$ den retning, hvor f vokser hurtigst.
2. **Vinkelret på niveaukurver:** I et regulært punkt, hvor $\nabla f(x,y) \neq \mathbf{0}$, er gradienten ortogonal (vinkelret) på niveaukurven gennem (x,y).

Begge egenskaber er yderst nyttige. Den første fortæller os, at gradienten peger i retningen for stejleste stigning, mens den anden forklarer, at denne retning er vinkelret på niveaukurverne. For at komme hurtigst muligt op ad bakke skal man derfor gå i gradientens retning; for at komme hurtigst muligt ned ad bakke skal man gå i den modsatte retning, dvs. langs den negative gradient.

For fuldt ud at forstå gradientens geometriske betydning er det nyttigt at se den fra to forskellige perspektiver, som vist i Figur 11.4. Figuren viser den samme funktion, $f(x,y) = 3x^2 + 2xy^3 + 5y$, på to måder: som en tredimensional flade og som et todimensionalt konturkort.

Til venstre, i Figur 11.4a, ser vi funktionen som et landskab. Fladens højde i ethvert punkt (x,y) svarer til værdien af $z = f(x,y)$. Under fladen har vi projiceret dens konturkort sammen med dens gradientfelt, som er vist med røde pile. Man kan visuelt følge, hvordan fladens stejlhed i et punkt hænger sammen med pilen lige under det.

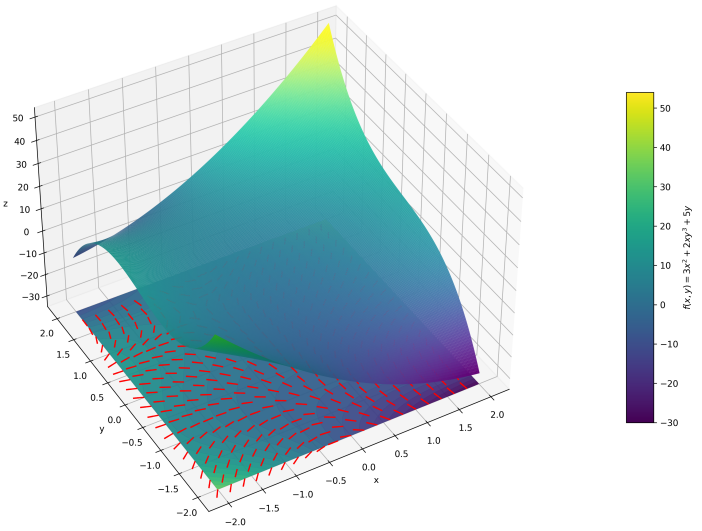
Til højre, i Figur 11.4b, har vi det perspektiv, vi oftest arbejder med: et billede af xy-planen set ovenfra. Det er det almindelige konturkort, hvor hver kurve forbinder punkter med samme »højde«. Gradientvektorerne er lagt oven på dette kort.

Ved at sammenligne de to billeder kan vi bekræfte gradientens to grundlæggende egenskaber:

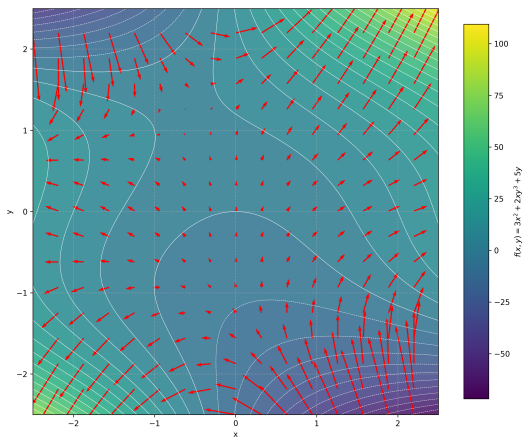
1. **Retning for stejleste stigning:** Se på en vilkårlig rød pil på 2D-kortet. Bemærk, at den altid peger fra et område med lavere værdier (lysere farver) mod et område med højere værdier (mørkere farver).

[1] Denne egenskab bevises formelt ved hjælp af begrebet *retningsafledet*, som måler ændringshastigheden i en vilkårlig retning.

Figur 11.4: To perspektiver på funktionen $f(x, y) = 3x^2 + 2xy^3 + 5y$. 3D-billedet (a) viser selve fladen, mens 2D-billedet (b) viser det konturkort, vi bruger i analysen. De røde gradientvektorer peger i retningen for stejleste stigning og er altid vinkelrette på niveaukurverne.



(a) Et 3D-billede af fladen $z = f(x, y)$ med dens gradientfelt projiceret nedenunder.



(b) Et 2D-billede set ovenfra, der viser konturkortet og gradientfeltet.

Pilene viser retningen »op ad bakke«. Det svarer direkte til den stejleste vej, man kan tage op ad fladen i 3D-billedet.

2. **Vinkelret på niveaukurver:** Se på sammenhængen mellem de røde pile og de hvide niveaukurver i Figur 11.4b. I hvert punkt er gradientvektoren præcis ortogonal (vinkelret) på den niveaukurve, der går gennem punktet. Det giver intuitivt mening: For at bestige en bakke mest effektivt skal man gå vinkelret på de kurver, der har samme højde.

Denne sammenligning side om side bygger bro mellem den fysiske virkelighed i funktionens form og de matematiske værktøjer, vi bruger til at analysere den.

11.3 Anvendelser af gradienten

Gradienten er et af de mest grundlæggende begreber i anvendt matematik med vidtrækkende betydning i fysik, ingeniørvidenskab og datalogi, og den er hjørnestenen i den seneste udvikling inden for maskinlæring og kunstig intelligens.

Optimering og gradientnedstigning

Husk vores vandrer, der prøver at finde det laveste punkt i en tåget dal. Den mest effektive strategi er altid at gå i retningen for stejleste *fald*. Ud fra gradientens egenskaber ved vi, at denne retning er præcis modsat gradientvektoren, $-\nabla f$.

Denne enkle idé er grundlaget for algoritmen **gradientnedstigning**, som er central for træningen af moderne maskinlæringsmodeller. Vandrerens »højde« er modellens fejl (eller tabsfunktion), og »positionen« er mængden af modelparametre. Algoritmen fungerer således:

1. Start i et tilfældigt punkt (et tilfældigt sæt parametre).
2. Beregn tabsfunktionens gradient i dette punkt.
3. Tag et lille skridt i den negative gradients retning, $-\nabla f$.
4. Gentag, indtil et valgt stopkriterium er opfyldt, fx en lille gradientnorm eller en tilstrækkelig lille ændring i tabet.

Denne iterative proces følger lokale nedstigningsretninger og kan finde brugbare parameterværdier, selv når en funktion har millioner af variable. En lille gradient garanterer ikke i sig selv et lokalt minimum, og for en generel ikke-konveks tabsfunktion kan metoden nærme sig et saddelpunkt eller et lokalt minimum, der ikke er globalt.

Eksempel 11.4

En simpel tabsfunktion er givet ved $L(w_1, w_2) = w_1^2 + 2w_2^2$. Hvis vi befinder os i punktet $(w_1, w_2) = (2, 1)$, i hvilken retning skal vi så bevæge os for at mindske tabet hurtigst?

Løsning: Vi skal finde den negative gradients retning, $-\nabla L$. Først beregner vi gradienten:

$$\nabla L = \left\langle \frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2} \right\rangle = \langle 2w_1, 4w_2 \rangle$$

Dernæst evaluerer vi gradienten i punktet $(2, 1)$:

$$\nabla L(2, 1) = \langle 2(2), 4(1) \rangle = \langle 4, 4 \rangle$$

Den negative gradient er en vektor i retningen for stejleste fald:

$$-\nabla L(2, 1) = \langle -4, -4 \rangle$$

Den tilsvarende enhedsretning er $\langle -1/\sqrt{2}, -1/\sqrt{2} \rangle$. Et skridt i gradientnedstigning er proportionalt med $\langle -4, -4 \rangle$, og skridtlængden styres af læringsraten. ◀

11.4 Anvendelse: Træning af en model med gradientnedstigning

Vi har set, at gradienten, ∇f, peger i retningen for stejleste stigning, og at dens negative, $-\nabla f$, peger i retningen for stejleste fald. Dette princip er ikke blot en geometrisk kuriositet; det er motoren bag træningen af de fleste moderne maskinlæringsmodeller. Vi gennemgår nu et komplet, praktisk eksempel: at finde den optimale linje, der passer til et datasæt, ved hjælp af **lineær regression** og algoritmen **gradientnedstigning**.

Eksempel 11.5 Optimering af en simpel lineær regressionsmodel

Løsning: Forestil dig, at vi har et datasæt bestående af n punkter (x_i, y_i). For eksempel kunne x_i være størrelsen af et hus og y_i dets pris. Vores mål er at finde den rette linje, der passer bedst til disse data. Denne proces kaldes lineær regression.

Trin 1: Definér modellen

En ret linje er bestemt ved sit konstantled (β_0) og sin hældning (β_1). For ethvert givet input x_i forudsiger vores model en værdi, som vi kalder $\hat{y}_i$, ifølge ligningen:

$$\hat{y}_i = \beta_0 + \beta_1 x_i$$

Vores opgave er at finde de optimale værdier af parametrene β_0 og β_1.

Trin 2: Definér tabsfunktionen

Hvordan ved vi, om en linje »passer godt«? Vi måler dens fejl. For hvert datapunkt er fejlen forskellen mellem den faktiske værdi (y_i) og den forudsagte værdi ($\hat{y}_i$). For at sikre, at fejlene ikke ophæver hinanden, og for at straffe store fejl hårdere, kvadrerer vi disse forskelle. Derefter tager vi gennemsnittet af den kvadrerede fejl over alle n datapunkter. Det giver os tabsfunktionen **middelkvadratfejlen** (MSE, af engelsk *mean squared error*), L:

$$L(\beta_0, \beta_1) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 = \frac{1}{n}\sum_{i=1}^{n}(y_i - \beta_0 - \beta_1 x_i)^2$$

Dette er den centrale ligning. Bemærk, at tabet L ikke er en funktion af x eller y (som er faste datapunkter), men en funktion af modellens parametre, β_0 og β_1. Det er en konveks andengradsflade: typisk skålformet, selv om den kan have en flad retning, hvis dataene ikke bestemmer begge parametre entydigt. Vores mål er at finde et globalt minimum for denne flade.

Trin 3: Beregn tabsfunktionens gradient

For at finde bunden af tabsfladen med gradientnedstigning skal vi kende retningen for stejleste fald i ethvert punkt. Det kræver, at vi beregner gradienten af L med hensyn til dens variable, β_0 og β_1. Gradienten er vektoren af partielle afledede:

$$\nabla L = \left\langle \frac{\partial L}{\partial \beta_0}, \frac{\partial L}{\partial \beta_1} \right\rangle$$

Vi finder hver partiel afledet ved hjælp af differentiationsreglerne, især kædereglen.

For $\frac{\partial L}{\partial \beta_0}$ betragter vi β_1 som en konstant og differentierer:

$$\frac{\partial L}{\partial \beta_0} = \frac{1}{n}\sum_{i=1}^{n} 2(y_i - \beta_0 - \beta_1 x_i)\cdot(-1) = -\frac{2}{n}\sum_{i=1}^{n}(y_i - \beta_0 - \beta_1 x_i)$$

For $\frac{\partial L}{\partial \beta_1}$ betragter vi β_0 som en konstant og differentierer:

$$\frac{\partial L}{\partial \beta_1} = \frac{1}{n}\sum_{i=1}^{n} 2(y_i - \beta_0 - \beta_1 x_i)\cdot(-x_i) = -\frac{2}{n}\sum_{i=1}^{n}(y_i - \beta_0 - \beta_1 x_i)x_i$$

Disse to udtryk giver os komponenterne i gradientvektoren ∇L for alle givne værdier af β_0 og β_1.

Trin 4: Algoritmen gradientnedstigning

Nu har vi alle brikkerne. Algoritmen gradientnedstigning er en iterativ proces:

1. Start med et første gæt på β_0 og β_1 (fx tilfældige værdier).
2. Beregn gradienten ∇L ud fra de aktuelle værdier af β_0 og β_1.
3. Opdatér parametrene ved at tage et lille skridt i den *negative* gradients retning.

Dette opdateringstrin er kernen i algoritmen. Vi indfører en lille positiv konstant η (eta), kaldet **læringsraten**,

som styrer størrelsen af vores skridt. Opdateringsreglerne er:

$$\beta_0 \leftarrow \beta_0 - \eta \frac{\partial L}{\partial \beta_0}$$

$$\beta_1 \leftarrow \beta_1 - \eta \frac{\partial L}{\partial \beta_1}$$

Vi gentager denne proces mange gange. Med en passende læringsrate mindsker opdateringerne tabsfunktionen L og bevæger parametrene β_0 og β_1 mod et minimum på denne konvekse tabsflade.

Med en mere kompakt vektornotation, hvor $\boldsymbol{\beta}$ er vektoren $\langle \beta_0, \beta_1 \rangle$, kan hele opdateringsreglen skrives som:

$$\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} - \eta \nabla L$$

Dette ene udtryk indfanger på elegant vis processen med at optimere en maskinlæringsmodel. Det er en direkte og stærk anvendelse af gradienten og viser, hvordan et grundlæggende begreb fra differentialregningen bruges til at løse komplekse problemer fra den virkelige verden. ◀

I dette kapitel har vi udvidet begrebet den afledede til flere variable. Vi begyndte med at se på funktioner af flere variable og partielle afledede og til sidst på gradientvektoren. Gradienten, som er den »fulde« afledede for funktioner af flere variable, beskriver ikke blot ændringshastigheden i alle retninger, men giver også det teoretiske grundlag for effektive optimeringsalgoritmer, der driver moderne teknologi. Derefter anvendte vi gradienten på problemet med at finde den optimale linje, der passer til et datasæt, ved hjælp af lineær regression og algoritmen gradientnedstigning. Dette afslutter kapitlet og dermed denne del af bogen Matematik for softwareingeniører.

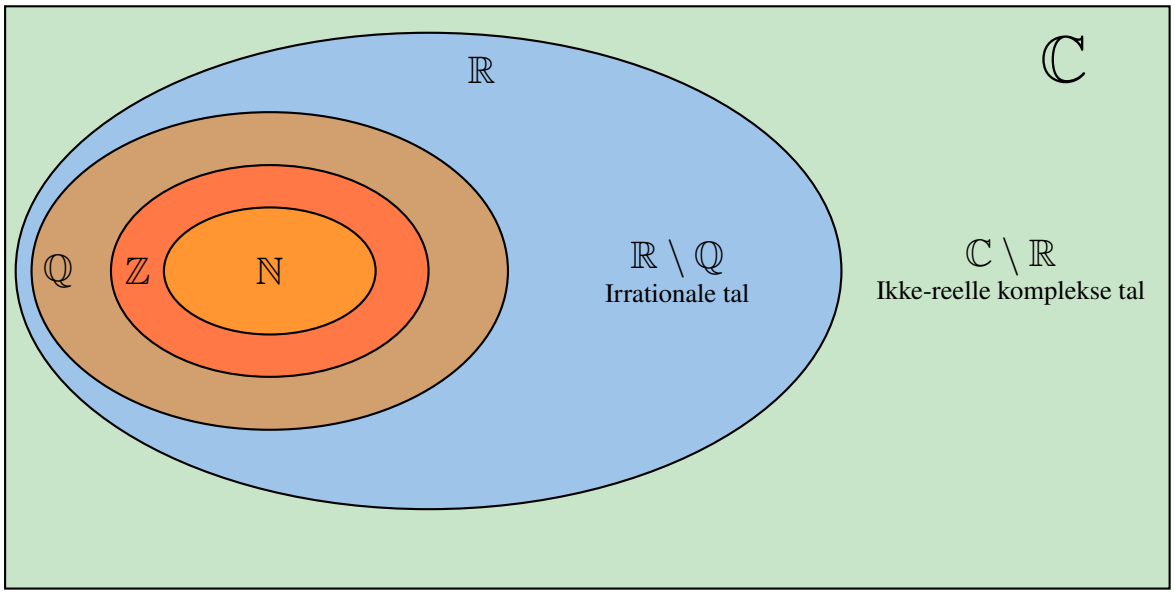
Del V

Supplerende emner

Kapitel 12 Grundlæggende begreber i talteori

Tal er fundamentet for matematikken. De repræsenterer mængder, målinger og sammenhænge i både abstrakt og konkret form. Studiet af tal går tusinder af år tilbage, og tidlige civilisationer som babylonierne, egypterne og grækerne lagde grunden til den moderne talteori. Disse oldtidskulturer udviklede den grundlæggende aritmetik, herunder addition, subtraktion, multiplikation og division, som der stadig undervises i i dag.

Talteori, som ofte kaldes »matematikkens dronning«, er en gren af matematikken, der beskæftiger sig med de hele tal og sammenhængene mellem dem. Den omfatter en lang række emner, herunder primtal, delelighed, modulær aritmetik og talsystemers egenskaber. Talteorien har en rig historie med betydelige bidrag fra matematikere som Euklid, der omkring 300 f.v.t. beviste, at der findes uendeligt mange primtal, og Pierre de Fermat, der er kendt for Fermats sidste sætning – et problem, som matematikere sled med i over 350 år, indtil det endelig blev løst af Andrew Wiles i 1994.



Figur 12.1: Venn-diagram over tal

Gennem historien har talteorien udviklet sig fra en rent teoretisk disciplin til et felt med praktiske anvendelser i moderne teknologi. Kryptografi, videnskaben om sikker kommunikation, bygger for eksempel i høj grad på talteori, især på primtals egenskaber og modulær aritmetik. De algoritmer, der beskytter vores onlinetransaktioner og digitale kommunikation, er bygget på talteoriens principper.

I dette kapitel udforsker vi forskellige aspekter af tal og talteori. Vi begynder med det grundlæggende om hele tal, primtal og talsystemer og bevæger os gradvist videre til mere komplekse emner. Ved at forstå tallenes grundlæggende egenskaber får vi indsigt i de matematiske strukturer, som den digitale verden og meget mere hviler på.

Bemærkning: Mange tal indgår i mere end én mængde. Tabel 12.1 giver et overblik over navne, egenskaber og symboler for de vigtigste taltyper.

Bemærkning: I Figur 12.1 udgør de rationale og de irrationale tal en partition af de reelle tal. Ethvert komplekst tal har formen $a + bi$ med $a, b \in \mathbb{R}$. De reelle tal er netop dem med $b = 0$; komplekse tal med $b \neq 0$ er

Tabel 12.1: Taltyper og deres egenskaber

Naturlige tal	$\mathbb{N}$	De ikke-negative hele tal (den konvention, der bruges i denne bog).	$0, 1, 2, \ldots$
Hele tal	$\mathbb{Z}$	Alle positive og negative hele tal.	$\{\ldots, -2, -1, 0, 1, 2, \ldots\}$
Rationale tal	$\mathbb{Q}$	Alle reelle tal, der kan skrives som en brøk, $\frac{p}{q}$, hvor p og q er hele tal, og $q \neq 0$. Alle hele tal er rationale tal, da 1 er et helt tal forskelligt fra nul.	$\frac{1}{5}, \frac{5}{1}(=5), \frac{2}{3}, \frac{3}{2}, \frac{0}{3}(=0)$
Irrationale tal	$\mathbb{R} \setminus \mathbb{Q}$	Alle reelle tal, der ikke kan skrives som en brøk, hvis tæller og nævner er hele tal.	$\pi, \sqrt{2}, \sqrt{3}$
Reelle tal	$\mathbb{R}$	Omfatter alle tal på tallinjen.	$\frac{1}{5}, \sqrt{\frac{1}{5}}, 0, -2$
Rene imaginære tal	$i\mathbb{R}$	Tal på formen bi, hvor b er reel, og $i^2 = -1$.	$3i = \sqrt{-9},\ -5i = -\sqrt{-25},$ $3\sqrt{2}i = \sqrt{-18}$
Komplekse tal	$\mathbb{C}$	Alle tal på formen $a + bi$, hvor $a, b \in \mathbb{R}$, og $i^2 = -1$.	$1 + 2i, 1, i, -3i, 0, -5 + i$

ikke-reelle. Rene imaginære tal har $a = 0$ og udgør kun én delmængde af de komplekse tal, ikke alle ikke-reelle komplekse tal.

12.1 Taltyper

Lad os nu fokusere på de delmængder af de reelle tal, der står i tabellen: naturlige tal, ikke-negative hele tal (på engelsk *whole numbers*), hele tal, rationale tal, irrationale tal og reelle tal. Overblikket ovenfor placerer disse mængder i det større system af komplekse tal.

Naturlige tal

Vi begynder med de naturlige tal. Vi skelner mellem de ikke-negative hele tal: $0, 1, 2, 3, \ldots$ og tælletallene (på engelsk *counting numbers*): $1, 2, 3, \ldots$. Disse tal bruges primært til at tælle med. Naturlige tal opfattes ofte som eksakte værdier (fx har en bil 4 dæk, og en edderkop har 8 ben). I nogle sammenhænge kan de dog bruges som tilnærmelser (fx var der cirka 1000 mennesker i mængden).

En af de naturlige tals centrale egenskaber er, at de er *lukkede* under visse operationer, såsom addition og multiplikation. Det betyder, at hvis du tager to vilkårlige naturlige tal og lægger dem sammen eller ganger dem, vil resultatet altid være et naturligt tal. Hvis a og b for eksempel er naturlige tal, er både $a + b$ og $a \times b$ også naturlige tal.

Der er meget lidt enighed om, hvorvidt symbolet $\mathbb{N}$ omfatter 0. Derfor betegnes mængden af ikke-negative hele tal (som omfatter 0) ofte $\mathbb{W}$ (af engelsk »whole numbers«).

Hele tal

De hele tal er talteoriens grundlæggende byggesten. De består af de ikke-negative hele tal og deres negative modstykker. Formelt omfatter de hele tal tal som $\ldots, -3, -2, -1, 0, 1, 2, 3, \ldots$ Grundlæggende regneoperationer

som addition og multiplikation følger bestemte regler:

Kommutative, associative og distributive love

- Kommutativitet: $a + b = b + a$
- Associativitet: $(a + b) + c = a + (b + c)$
- Den distributive lov: $a \times (b + c) = a \times b + a \times c$

De hele tal er en grundlæggende talmængde i matematikken og betegnes med symbolet $\mathbb{Z}$. Mængden omfatter alle positive og negative hele tal samt nul. De hele tal udvider de naturlige tal ved at inddrage deres additive inverse, så subtraktion fuldt ud kan udføres inden for mængden.

Additivt neutralt element: Tallet 0 kaldes det additive neutrale element. Det skyldes, at det for ethvert helt tal a gælder, at værdien af a ikke ændres, når man lægger nul til. Matematisk udtrykkes denne egenskab som $0 + a = a + 0 = a$. Det neutrale element er afgørende, fordi det sikrer, at mængden af hele tal forbliver stabil under addition.

Multiplikativt neutralt element: Tilsvarende fungerer tallet 1 som det multiplikative neutrale element. For ethvert helt tal a gælder, at multiplikation med én lader værdien af a være uændret: $1 \times a = a \times 1 = a$. Denne egenskab ligger til grund for de hele tals stabilitet under multiplikation og bevarer mængdens integritet.

Additiv invers: Til ethvert helt tal a findes en tilhørende additiv invers, som betegnes $-a$. Den additive inverse er defineret sådan, at når a og $-a$ lægges sammen, er resultatet det additive neutrale element, nul: $a + (-a) = 0$. Denne egenskab gør det muligt at udføre subtraktion inden for mængden af hele tal, da subtraktion kan opfattes som addition af en additiv invers.

Lukkethed

Addition: Mængden af hele tal er lukket under addition. Det betyder, at hvis du tager to vilkårlige hele tal og lægger dem sammen, vil summen altid være et helt tal. Hvis a og b for eksempel er hele tal, er $a + b$ også et helt tal. Denne lukkethed sikrer, at mængden af hele tal er stabil og fuldstændig under addition. Uanset hvor mange gange du lægger hele tal sammen, vil resultatet altså forblive inden for mængden af hele tal.
Multiplikation: Mængden af hele tal er også lukket under multiplikation. Hvis du ganger to vilkårlige hele tal, vil produktet altid være et helt tal. Hvis a og b for eksempel er hele tal, er $a \times b$ også et helt tal. Denne egenskab garanterer, at multiplikation ligesom addition ikke giver resultater uden for mængden af hele tal, og bevarer derved mængdens integritet under multiplikation.

Der er universel enighed om, at definitionen af et helt tal er klar og præcis. Når du er i tvivl, er det derfor tilrådeligt at omtale tal i denne mængde som »hele tal«. Når du specifikt kun vil omtale de positive hele tal, er det både præcist og professionelt eksplicit at skrive »positive hele tal«. Denne terminologi sikrer ikke blot klarhed, men viser også en sikker forståelse af matematiske konventioner.

Husk, at nul hverken er positivt eller negativt. Når man taler om delmængder af de hele tal, skal man derfor tage nøje hensyn til dette:

- **Hele tal:** $\mathbb{Z} = \{\ldots, -4, -3, -2, -1, 0, 1, 2, 3, 4, \ldots\}$
- **Negative hele tal:** $\mathbb{Z}^- = \{\ldots, -4, -3, -2, -1\}$

- **Positive hele tal:** $\mathbb{Z}^+ = \{1, 2, 3, 4, \ldots\}$
- **Ikke-negative hele tal:** $\mathbb{Z}_0^+ = \mathbb{Z}_{\geq 0} = \{0, 1, 2, 3, 4, \ldots\}$
- **Ikke-positive hele tal:** $\mathbb{Z}_0^- = \mathbb{Z}_{\leq 0} = \{\ldots, -4, -3, -2, -1, 0\}$

Denne notation bør være i overensstemmelse med gængse matematiske konventioner.

Rationale tal

Rationale tal, som betegnes med symbolet $\mathbb{Q}$, er tal, der kan skrives som kvotienten af to hele tal, hvor tælleren er et vilkårligt helt tal, og nævneren er et helt tal forskelligt fra nul. Formelt kan et rationalt tal skrives som $\frac{a}{b}$, hvor $a \in \mathbb{Z}$ og $b \in \mathbb{Z} \setminus \{0\}$. Denne talmængde er grundlæggende i matematikken, da den giver mulighed for at repræsentere brøker og forhold og dermed for en lang række regneoperationer. De rationale tal er således en generalisering af de almindelige brøker, som vi så i Kapitel 0. De kan repræsentere ethvert tal, der kan skrives som et endeligt eller periodisk decimaltal.

En af de rationale tals centrale egenskaber er, at ethvert rationalt tal forskelligt fra nul har en *multiplikativ invers*. Den multiplikative inverse til et rationalt tal $\frac{a}{b}$ er det rationale tal $\frac{b}{a}$, forudsat at $a \neq 0$. Betydningen af den multiplikative inverse ligger i, at når et tal ganges med sin inverse, er resultatet det *multiplikative neutrale element*, som er 1. For ethvert rationalt tal $\frac{a}{b}$ betegnes den inverse med andre ord $\left(\frac{a}{b}\right)^{-1} = \frac{b}{a}$, og vi har:

$$\frac{a}{b} \times \frac{b}{a} = \frac{ab}{ba} = 1$$

Denne egenskab sikrer, at de rationale tal er lukkede under multiplikation og division (bortset fra division med nul), hvilket gør dem til en robust og alsidig talmængde til forskellige matematiske operationer.

De rationale tal ligger tæt på tallinjen. Det betyder, at der mellem to vilkårlige rationale tal findes et andet rationalt tal. Denne egenskab gør mængden af rationale tal særligt vigtig i studiet af de reelle tal, da den gør det muligt at tilnærme irrationale tal med en vilkårlig ønsket nøjagtighed.

Irrationale tal

Irrationale tal, som betegnes med symbolet $\mathbb{R} \setminus \mathbb{Q}$, er reelle tal, der ikke kan skrives som kvotienten af to hele tal. I modsætning til rationale tal, som har en periodisk eller endelig decimalfremstilling, er decimalfremstillingen af et irrationalt tal hverken periodisk eller endelig. Denne egenskab gør irrationale tal fundamentalt forskellige fra rationale tal, da de ikke kan repræsenteres præcist som brøker.

Nogle af de mest kendte eksempler på irrationale tal er:

- **Pi** (π): π er måske det mest kendte irrationale tal og er forholdet mellem en cirkels omkreds og dens diameter. Værdien er cirka $3.14159\ldots$, men decimalfremstillingen fortsætter uendeligt uden at gentage sig. π spiller en afgørende rolle i geometri, trigonometri og infinitesimalregning.
- **Det gyldne snit** (ϕ): Det gyldne snit, $\phi \approx 1.61803\ldots$, er et irrationalt tal, der ofte optræder i naturen, kunsten og arkitekturen. Det er defineret som den positive løsning til ligningen $x^2 - x - 1 = 0$, og det er grænseværdien for forholdet mellem på hinanden følgende Fibonaccital.
- **Kvadratroden af 2** ($\sqrt{2}$): Kvadratroden af 2, cirka $1.41421\ldots$, er længden af diagonalen i et kvadrat med sidelængden 1. Tallet har historisk betydning, fordi oldtidens grækere, især pythagoræerne, med opdagelsen af det afslørede, at der findes tal, som ikke kan skrives som forholdet mellem to hele tal. Pythagoras' sætning siger, at i en retvinklet trekant er kvadratet på hypotenusen lig med summen af

kvadraterne på de to andre sider. For en retvinklet trekant, hvor begge kateter har længden 1, er hypotenusen $\sqrt{2}$, hvilket viser, at $\sqrt{2}$ optræder naturligt som en længde.

- **Eulers tal** (e): Tallet $e \approx 2.71828\ldots$ er grundtallet for den naturlige logaritme og en grundlæggende konstant i matematikken, især i infinitesimalregning og kompleks analyse. Tallet e opstår naturligt i forskellige vækstprocesser, såsom renters rente og befolkningsvækst.

Irrationale tal udfylder hullerne mellem de rationale tal på tallinjen og gør dermed de reelle tal til en kontinuert mængde. Ligesom de rationale tal har de dog også vigtige egenskaber. Selv om irrationale tal for eksempel ikke har en simpel brøkfremstilling, er de alligevel uundværlige til at repræsentere de længder, arealer og rumfang, som ikke kan beskrives med rationale tal alene.

Reelle tal

Reelle tal, som betegnes med symbolet $\mathbb{R}$, danner grundlaget for det meste af den matematiske analyse og er uundværlige til at beskrive kontinuerte størrelser. Mængden af reelle tal består af både de rationale tal ($\mathbb{Q}$) og de irrationale tal ($\mathbb{R} \setminus \mathbb{Q}$) og omfatter dermed alle tal, der kan placeres på tallinjen. Selv om både rationale og irrationale tal er en del af det reelle talsystem, adskiller deres egenskaber sig markant:

Ligheder og forskelle mellem rationale og irrationale tal

- **Repræsentation:** Rationale tal kan repræsenteres som brøker, mens irrationale tal ikke kan. Denne forskel gør irrationale tal mere komplicerede at håndtere, især i regneoperationer.
- **Decimalfremstilling:** Decimalfremstillingen af rationale tal er enten endelig eller periodisk, mens decimalfremstillingen af irrationale tal er uendelig og ikke-periodisk.
- **Lukkethed:** De rationale tal er lukkede under addition, subtraktion, multiplikation og division (bortset fra division med nul). De irrationale tal er ikke lukkede under disse operationer; for eksempel kan summen eller produktet af to irrationale tal undertiden være rationalt.
- **Tæthed:** De rationale tal ligger tæt på den reelle tallinje. Det betyder, at der mellem to vilkårlige rationale tal findes et andet rationalt tal. De irrationale tal ligger også tæt, men på en komplementær måde: De udfylder de »huller«, som de rationale tal efterlader, og sikrer, at den reelle tallinje er kontinuert uden afbrydelser.

Den reelle tallinje er en kontinuert, ubrudt linje, der strækker sig uendeligt i begge retninger. Hvert punkt på linjen svarer til ét bestemt reelt tal, uanset om det er rationalt eller irrationalt. Det er denne kontinuitet, der gør det muligt at bruge de reelle tal til at modellere kontinuerte fænomener i naturen, såsom tid, afstand og temperatur.

12.2 Egenskaber ved hele tal og modulær aritmetik

I kapitel 0 introducerede vi begrebet faktor. En faktor er en del af en størrelse, som regel et helt tal eller et polynomium, der ganget med andre faktorer giver hele størrelsen.

I talteori er en faktor i et tal n det samme som en **divisor** i n. Mere præcist er en divisor i et helt tal n et helt tal d, der går op i n, så resultatet bliver et andet helt tal.

En positiv **ægte divisor** er en positiv divisor i et tal n, bortset fra n selv. Tilsvarende er en **ægte faktor** i et

positivt helt tal n en faktor i n, der er forskellig fra 1 og n.

Faktorerne i 12 er for eksempel:

$$1, 2, 3, 4, 6, 12$$

I denne liste er 2, 3, 4 og 6 ægte faktorer i 12. Det betyder, at de er mindre end 12 selv, men større end 1.

Et primtal er et naturligt tal større end 1, som ikke har andre positive divisorer end 1 og sig selv. Et primtal kan med andre ord ikke dannes ved at gange to mindre naturlige tal. De første primtal er:

$$2, 3, 5, 7, 11, 13, 17, 19, 23, 29, \ldots$$

Bemærk, at 2 er det eneste lige primtal, fordi ethvert andet lige tal kan divideres med 2 og dermed er *sammensat*. Et **sammensat tal** er i modsætning til et primtal et naturligt tal større end 1, som går op med mindst ét andet positivt helt tal end 1 og sig selv. For eksempel er 6 et sammensat tal, fordi det kan skrives som $6 = 2 \times 3$.

Primfaktoropløsning

Primfaktoropløsning er processen, hvor et sammensat tal opløses i et produkt af dets primfaktorer. For ethvert sammensat tal er denne opløsning entydig, bortset fra faktorernes rækkefølge. Se for eksempel på tallet 60:

$$60 = 2 \times 2 \times 3 \times 5 = 2^2 \times 3 \times 5$$

Her er 2, 3 og 5 primfaktorer i 60, og $2^2 \times 3 \times 5$ er den **kanoniske faktorisering** af 60 – normalt kalder vi den blot primfaktoropløsningen.

Sætning 12.1 (Aritmetikkens fundamentalsætning)

Ethvert helt tal større end 1 kan skrives entydigt som et produkt af primtal, bortset fra faktorernes rækkefølge. ♡

Denne sætning understreger primtallenes betydning som »byggesten« for alle naturlige tal.

Eksempel 12.1

$$84 = 2^2 \times 3 \times 7$$

Uanset hvordan du faktoriserer 84, ender du altid med de samme primtal (2, 3 og 7), selv om rækkefølgen kan variere.

Primfaktoropløsningen giver en entydig fremstilling af tallet ved hjælp af dets primfaktorer.

Største fælles divisor (GCD)

Den største fælles divisor (GCD, af engelsk *greatest common divisor*) af to eller flere hele tal, som også kaldes *største fælles faktor* (på engelsk *highest common factor*, HCF, eller *greatest common factor*, GCF), er det største positive hele tal, der går op i hvert af de givne tal uden at efterlade en rest. For at bestemme GCD bruger man almindeligvis primfaktoropløsningen af de involverede tal. GCD findes derefter ved at gange de fælles primfaktorer med de mindste eksponenter fra tallenes primfaktoropløsninger.

Se for eksempel på, hvordan man finder GCD af 24 og 36:

$$24 = 2^3 \times 3, \quad 36 = 2^2 \times 3^2$$

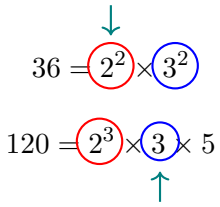
De fælles primfaktorer er 2 og 3. Dermed er GCD:

$$\text{GCD} = 2^2 \times 3 = 12$$

Eksempel 12.2 Find GCD af 36 og 120

Løsning:

Pilene viser de tal, vi har valgt.



Vi ser, at 2 og 3 er fælles faktorer, og vælger dem med de mindste eksponenter:

$$GCD = 2^2 \times 3 = 12$$

◀

Mindste fælles multiplum (LCM)

Det mindste fælles multiplum (LCM, af engelsk *least common multiple*) af to eller flere hele tal er det mindste positive hele tal, som hvert af de givne tal går op i. LCM kan også bestemmes ved hjælp af tallenes primfaktoropløsning.

Se for eksempel på, hvordan man finder LCM af 24 og 36:

$$24 = 2^3 \times 3, \quad 36 = 2^2 \times 3^2$$

LCM fås ved at tage den højeste potens af hver primfaktor, der optræder i opløsningen af et af tallene:

$$\text{LCM} = 2^3 \times 3^2 = 72$$

Eksempel 12.3 Find LCM af 12 og 18.

Løsning:

$$12 = 2^2 \times 3$$

$$18 = 2 \times 3^2$$

De højeste potenser, der optræder i en af opløsningerne, er 2^2 og 3^2, så

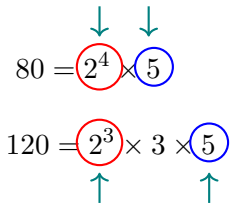
$$LCM = 2^2 \times 3^2 = 36$$

◀

Eksempel 12.4 Find LCM af 80 og 120

Løsning:

Pilene viser de tal, vi har valgt.



Til LCM vælger vi den største eksponent for hvert primtal, der optræder i en af opløsningerne:

$$LCM = 2^4 \times 3 \times 5 = 240$$

◀

Sammenhængen mellem GCD og LCM

En vigtig sammenhæng mellem GCD og LCM af to tal a og b er givet ved formlen:

$$\text{GCD}(a, b) \times \text{LCM}(a, b) = |ab|$$

Med tallene 24 og 36 får vi for eksempel:

$$12 \times 72 = 24 \times 36 = 864$$

Denne sammenhæng er et stærkt værktøj til at løse problemer inden for delelighed, talteori og algebra.

Studiet af primtal og faktorer er grundlæggende i matematikken og giver vigtige værktøjer til at forstå tallenes struktur. Begreberne primfaktoropløsning, aritmetikkens fundamentalsætning og beregning af GCD og LCM er ikke kun afgørende i teoretisk matematik, men også i praktiske anvendelser som kryptografi, kodningsteori og analyse af algoritmer.

Divisorer og rester

Division med hele tal kan være vanskelig, da resultatet ikke altid er et helt tal. Ofte giver en division en rest. For eksempel er

$$9 = 2 \times 4 + 1.$$

Her giver division af 9 med 4 en *rest* på 1.

Generelt gælder det for hele tal $a \neq 0$ og b, at der findes entydigt bestemte hele tal q og r, så

$$b = qa + r, \qquad 0 \leq r < |a|.$$

Det hele tal q er kvotienten, og r er resten. Hvis $r = 0$, går a op i b, hvilket skrives $a \mid b$. For eksempel gælder $2 \mid 128$ og $7 \mid 49$, men $3 \nmid 4$.

I denne bog bruges *faktor* og *divisor* i flæng. En positiv ægte divisor i et positivt helt tal b er en positiv divisor, der er forskellig fra b selv. Et primtal har derfor præcis to positive divisorer, 1 og sig selv, og ingen ægte divisor større end 1.

Modulær aritmetik

Operatoren `mod`, som man ofte støder på i programmering, giver resten efter division. For eksempel:

1. $25 \bmod 4 = 1$, fordi $25 \div 4 = 6$ med en rest på 1.
2. $19 \bmod 5 = 4$, da $19 = 3 \times 5 + 4$.
3. $24 \bmod 5 = 4$.
4. $99 \bmod 11 = 0$.

Selv om der i programmeringssprog er nogle konventionsafhængige detaljer for negative operander, fokuserer vi på positive hele tal. Kongruensnotation udtrykker, at to hele tal har samme rest: $24 \equiv 4 \pmod 5$ og $21 \equiv 0 \pmod 7$. Det er forskelligt fra restoperationen, som skrives $24 \bmod 5 = 4$ og $21 \bmod 7 = 0$.

Modulær aritmetik kaldes undertiden uraritmetik. Forestil dig et 24-timersur: 09:00 betyder klokken 9 om morgenen, mens 21:00 betyder klokken 9 om aftenen. Hvis en rejse starter kl. 07:00 og varer 25 timer, er ankomsttiden kl. 08:00 næste dag. Matematisk kan det udtrykkes som $7 + 25 = 32$ og $32 \bmod 24 = 8$. Vi starter altså ved 7 og bevæger os 25 timer frem på urskiven, så vi lander på 8.

Kapitel 13 Boolesk algebra

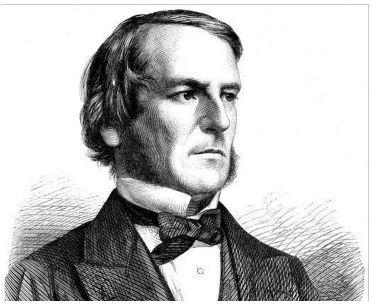
I kapitel 2 udforskede vi det binære, det decimale, det oktale og det hexadecimale talsystem og lærte at omregne mellem dem og at regne binært. Totalssystemet er med sit grundtal 2 særligt vigtigt i computere, fordi digitale systemer repræsenterer og behandler information ved hjælp af to logiske tilstande.

Når vi går fra at arbejde med binære tal til at forstå **boolesk algebra**, tager vi det næste skridt i forståelsen af, hvordan computere udfører logiske operationer med binære værdier. Talsystemerne gav os en forståelse af, hvordan tal repræsenteres og behandles. Boolesk algebra gør det muligt at forstå, hvordan beslutninger og logiske processer udføres i digitale systemer. Kort sagt giver boolesk algebra de regler, der styrer logiske beslutninger i det binære system, på samme måde som aritmetikken fungerer i talsystemerne.

13.1 Introduktion til boolesk algebra

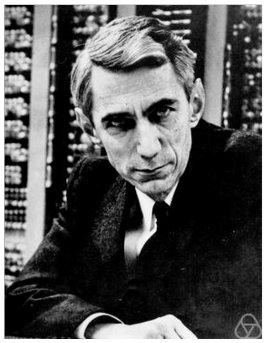
Boolesk algebra er en matematisk ramme, der arbejder med binære værdier – typisk repræsenteret som 0 og 1 – og viser os, hvilke slags operationer der kan udføres på bit. Systemet blev først introduceret af **George Boole** i midten af det 19. århundrede. Det danner grundlaget for moderne digital logik og er uundværligt inden for datalogi og elektroteknik.

George Boole (1815-1864)



Figur 13.1: Udviklede logikkens regler.

Claude Shannon (1916-2001)



Figur 13.2: Viste, hvordan boolesk algebra kan bruges til at konstruere computerkredsløb.

Selv om Boole lagde grunden, var det først i 1930'erne, at **Claude Shannon**, der bredt anses for informationsteoriens fader, viste, hvordan boolesk algebra kunne anvendes til at konstruere elektroniske kredsløb. I sit banebrydende speciale viste Shannon, at boolesk algebra kunne bruges til at modellere og forenkle de komplekse arrangementer af kontakter og relæer, som findes i elektriske kredsløb. Denne erkendelse slog bro mellem abstrakt matematisk logik og praktisk ingeniørarbejde og lagde grunden til de digitale kredsløb, der bruges i moderne computere. Shannons speciale vandt Alfred Noble-prisen i 1939.

Relevans for datalogi

Boolesk algebra spiller en central rolle i datalogi, især i design og funktion af digitale systemer som computere, mobile enheder og indlejrede systemer. Grundlæggende bruges boolesk algebra til at modellere og designe

opførslen af **digitale kredsløb**. De er bygget af elektroniske komponenter med to tilstande: tændt eller slukket, hvilket svarer direkte til de binære værdier 1 og 0.

Kernen i disse digitale kredsløb er **transistorer**. En transistor er en halvlederkomponent, der kan styre elektrisk strøm og derfor kan bruges som en elektronisk kontakt. Moderne processorer indeholder milliarder af forbundne transistorer.



Figur 13.3: En transistor.

Transistorerne kan forbindes på en sådan måde, at de gemmer information i binær form – hvert 0 eller 1 repræsenterer en enkelt bit information. Desuden kan de bruges til at udføre logiske operationer på disse bit.

Enhver beregning eller beslutning, som en computer foretager, involverer en række logiske operationer – såsom `AND`, `OR` og `NOT` – der beskrives og optimeres ved hjælp af booleske udtryk. Disse logiske operationer er grundlæggende for styringen af datastrømme og beslutninger i software- og hardwaresystemer. For eksempel:

- I **processorer** bruges boolesk algebra til at skabe **logiske gates** og kredsløb, der står for databehandling, regneoperationer og beslutninger.
- I **software** bygger betingede sætninger som `IF` og `ELSE` på boolesk logik og afgør, hvilke veje et program følger.

Ud over digitale kredsløb ligger boolesk algebra også til grund for centrale begreber inden for **logikprogrammering**, **databaseforespørgsler** og **søgealgoritmer**, hvor betingelser skal evalueres som `TRUE` eller `FALSE`.

På det fysiske niveau fungerer kredsløbene i digitale enheder ved at behandle binære input og frembringe et binært output. Hvert af disse kredsløb kan modelleres med booleske funktioner, som er udtryk, der beskriver, hvordan inputværdierne behandles for at frembringe et output. En simpel `AND`-gate giver for eksempel kun outputtet 1, hvis begge dens input er 1, hvilket svarer direkte til den booleske operation $x \wedge y$.

Boolesk algebra er ikke kun en måde at forenkle logiske ræsonnementer på, men også et værktøj til at optimere design af digitale kredsløb. Ved at reducere booleske udtryk kan designere mindske antallet af nødvendige logiske gates. Det giver mere effektive kredsløb, der bruger mindre strøm og fylder mindre på en chip.

13.2 Grundlæggende operationer på bit

De booleske operatorer definerer de grundlæggende regler for behandling af binære værdier (0 og 1). De tre primære operationer i boolesk algebra er:

Primære booleske operationer

- **Komplementering** (betegnes `NOT`, fx $\overline{0}$)
- **Boolesk sum** (betegnes `OR`, fx $1 + 0$)
- **Boolesk produkt** (betegnes `AND`, fx $1 \cdot 0$)

Komplementet af en boolesk værdi er den modsatte tilstand. Det betegnes med en streg over variablen. For eksempel:

$$\overline{0} = 1, \quad \overline{1} = 0$$

Komplementeringen vender værdien: Hvis input er 0, er output 1, og omvendt.

Den booleske sum, eller `OR`, er en logisk operation, der giver 1, hvis mindst én af operanderne er 1. Resultaterne er som følger:

$$1 + 1 = 1, \quad 1 + 0 = 1, \quad 0 + 1 = 1, \quad 0 + 0 = 0$$

Denne operation svarer til addition, men følger andre regler for binær logik.

Det booleske produkt, eller `AND`, giver kun 1, hvis begge operander er 1. Resultaterne er:

$$1 \cdot 1 = 1, \quad 1 \cdot 0 = 0, \quad 0 \cdot 1 = 0, \quad 0 \cdot 0 = 0$$

I nogle tilfælde kan multiplikationstegnet (`AND`) udelades for overskuelighedens skyld, på samme måde som det udelades ved multiplikation i algebra.

Vi kan opsummere overvejelserne om de grundlæggende booleske operatorer i følgende tabel:

`OR`	`AND`	`NOT`
$0 + 0 = 0$	$0 \cdot 0 = 0$	$\overline{0} = 1$
$0 + 1 = 1$	$0 \cdot 1 = 0$	$\overline{1} = 0$
$1 + 0 = 1$	$1 \cdot 0 = 0$	
$1 + 1 = 1$	$1 \cdot 1 = 1$	

Tabel 13.1: Grundlæggende booleske operatorer.

Bemærkning: En **boolesk variabel** kan kun antage to mulige værdier: 0 (`FALSE`) eller 1 (`TRUE`). Disse variable repræsenterer tilstandene i logiske kredsløb. Givet $x = 1$ og $y = 0$ kan du for eksempel anvende booleske operationer:

$$x + y = 1, \quad x \cdot y = 0, \quad \overline{x} = 0$$

Vi bruger typisk x, y og z som vores booleske variable.

Operatorpræcedens

Når udtryk med flere booleske operatorer skal evalueres, anvendes følgende rækkefølge:

1. Komplementer (`NOT`) evalueres først.
2. Booleske produkter (`AND`) beregnes derefter.
3. Booleske summer (`OR`) evalueres til sidst.

For eksempel betyder $x \cdot y + z$ altså $(x \cdot y) + z$. I de fleste tilfælde skriver vi dog parenteserne for at undgå misforståelser. Bemærk også, at parenteser kan bruges til at tilsidesætte den normale præcedens, så bestemte operationer evalueres før andre.

Se på udtrykket:

$$\overline{x} \cdot y + z$$

Ifølge præcedensreglerne evalueres komplementet (`NOT`) af x først, derefter det booleske produkt (`AND`) af $\overline{x}$ og y og til sidst den booleske sum (`OR`) med z. Det kan skrives eksplicit som:

$$(\overline{x} \cdot y) + z$$

Se nu på et andet udtryk:

$$\overline{x \cdot (y + z)}$$

Her tilsidesætter parenteserne den normale præcedens. Den booleske sum (`OR`) af y og z evalueres først, derefter det booleske produkt (`AND`) med x og til sidst komplementet (`NOT`) af hele resultatet.

Eksempel 13.1 Find værdien af $1 \cdot 0 + \overline{(0 + 1)}$.

Løsning: Ud fra definitionerne af komplementering, sum og produkt og rækkefølgen af operationerne følger det, at

$$\begin{aligned} 1 \cdot 0 + \overline{(0 + 1)} &= 0 + \overline{1} \\ &= 0 + 0 \\ &= 0 \end{aligned}$$

◀

Det viser sig, at de booleske operatorer er tæt forbundet med de logiske operatorer.

Logiske operatorer

Komplementet, den booleske sum og det booleske produkt svarer til henholdsvis de logiske operatorer $\neg$, $\vee$ og $\wedge$. I denne sammenhæng svarer 0 til `FALSE` ($\mathbf{F}$), og 1 svarer til `TRUE` ($\mathbf{T}$).

Ligheder i boolesk algebra kan oversættes direkte til ækvivalenser mellem sammensatte udsagn i logik. Tilsvarende kan logiske ækvivalenser oversættes til ligheder i boolesk algebra. Disse oversættelser gør det muligt at bevæge sig gnidningsløst mellem boolesk algebra og udsagnslogik.

Eksempel 13.2 Oversæt $1 \cdot 0 + \overline{(0 + 1)} = 0$, den lighed, vi fandt i Eksempel 13.1, til en logisk ækvivalens.

Løsning: $(\mathbf{T} \wedge \mathbf{F}) \vee \neg(\mathbf{F} \vee \mathbf{T}) \equiv \mathbf{F}$ ◀

Da logik er et selvstændigt fagområde, går vi ikke videre med det her.[1]

[1] Symbolet »≡« betegner logisk ækvivalens: To udsagn er logisk ækvivalente, når de har samme sandhedsværdi for enhver tildeling af værdier til deres variable. Ækvivalente udsagn behøver ikke at have samme syntaktiske struktur. I boolesk algebra angiver »=« tilsvarende, at to booleske udtryk definerer den samme funktion for alle input.

13.3 Booleske funktioner

En **boolesk funktion** afbilder booleske input over i et boolesk output. Se på den booleske funktion $F(x, y, z)$:

$$F(x, y, z) = (x \cdot \overline{y}) + z$$

Funktionen giver resultatet af boolesk `OR` mellem to led: $x \cdot \overline{y}$ (`AND` af x og komplementet af y) og z. Nedenfor er funktionen vist i en tabel, som angiver værdien af F for alle kombinationer af inputværdier:

x	y	z	$F(x, y, z)$
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

Tabel 13.2: Sandhedstabel for $F(x, y, z) = (x \cdot \overline{y}) + z$.

Eksempel 13.3 Se på den booleske funktion, der er defineret ved:

$$F(x, y, z) = xy + \overline{z}$$

Opstil sandhedstabellen for funktionen.

Løsning: Funktionen er vist i tabellen nedenfor, som angiver værdien af F for alle kombinationer af inputværdierne x, y og z:

x	y	z	xy	$\overline{z}$	$F(x, y, z)$
0	0	0	0	1	1
0	0	1	0	0	0
0	1	0	0	1	1
0	1	1	0	0	0
1	0	0	0	1	1
1	0	1	0	0	0
1	1	0	1	1	1
1	1	1	1	0	1

Tabel 13.3: Sandhedstabel for den booleske funktion $F(x, y, z) = xy + \overline{z}$.

Søjlerne xy og $\overline{z}$ er »hjælpesøjler«, der hjælper med at beregne F, selv om de ikke er strengt nødvendige. De

opdeler udtrykket $F(x, y, z)$ i mindre dele, så det bliver lettere at beregne det endelige resultat.

◀

Reduktion af et boolesk udtryk

Tabellen nedenfor viser output for funktionerne $F(x, y) = (x\overline{y} + xy) \cdot y$ og $G(x, y) = xy$ for alle værdier af x og y:

x	y	xy	$\overline{y}$	$x\overline{y}$	$xy + x\overline{y}$	$F(x, y)$	$G(x, y)$
0	0	0	1	0	0	0	0
0	1	0	0	0	0	0	0
1	0	0	1	1	1	0	0
1	1	1	0	0	1	1	1

Tabel 13.4: Sammenligning af $F(x, y) = (x\overline{y} + xy) \cdot y$ og $G(x, y) = xy$.

Funktionerne giver samme output for alle værdier af x og y. De to funktioner er altså ækvivalente!

Booleske udtryk kan ofte reduceres betydeligt. I det følgende afsnit ser vi nærmere på nogle regler og teknikker til at opnå denne reduktion.

Booleske identiteter

Booleske identiteter er grundlæggende regler, der definerer sammenhængene og operationerne mellem booleske variable. Identiteterne gør det muligt at reducere og omskrive booleske udtryk. Ved at anvende dem kan komplekse booleske udtryk reduceres til simplere former, så de bliver lettere at evaluere eller implementere i digitale kredsløb. Uanset om der er tale om de kommutative, associative eller distributive love eller De Morgans love, er identiteterne byggestenene til effektiv logisk manipulation og hjælper os til at forstå booleske funktioners opførsel på et dybere niveau.

Vi begynder med at se på de mest grundlæggende af disse identiteter:

Grundlæggende booleske egenskaber

- **Dobbelt komplement:**
  $\overline{\overline{x}} = x$
- **Idempotenslove:**
  $x + x = x, \quad x \cdot x = x$
- **Identitetslove:**
  $x + 0 = x, \quad x \cdot 1 = x$
- **Dominanslove:**
  $x + 1 = 1, \quad x \cdot 0 = 0$
- **Enhedsegenskaben:**
  $\overline{x} + x = 1$
- **Nulegenskaben:**
  $\overline{x} \cdot x = 0$

Disse identiteter udgør de mest grundlæggende egenskaber ved boolesk algebra og handler om, hvordan `OR`, `AND` og de neutrale elementer (0 og 1) opfører sig.

Eksempel 13.4 Vis, at $(1 \cdot 1) + (\overline{0 \cdot 1} + 0) = 1$.

Løsning: For at vise, at $(1 \cdot 1) + (\overline{0 \cdot 1} + 0) = 1$, deler vi udtrykket op og reducerer det trin for trin ved hjælp af booleske identiteter.

- **Trin 1:** Reducér den første del $(1 \cdot 1)$ Ved hjælp af idempotensloven $(x \cdot x = x)$ eller ved blot at beregne resultatet:
  $$1 \cdot 1 = 1$$
  Udtrykket bliver altså:
  $$1 + (\overline{0 \cdot 1} + 0)$$
- **Trin 2:** Reducér den anden del $(\overline{0 \cdot 1} + 0)$ Reducér først $0 \cdot 1$ ved hjælp af dominansloven $(0 \cdot x = 0)$:
  $$0 \cdot 1 = 0$$
  Udtrykket bliver altså:
  $$1 + (\overline{0} + 0)$$
- **Trin 3:** Reducér komplementet $\overline{0}$ Ved hjælp af komplementloven $(\overline{0} = 1)$:
  $$\overline{0} = 1$$
  Dermed bliver udtrykket:
  $$1 + (1 + 0)$$
- **Trin 4:** Reducér det indre udtryk $(1 + 0)$
  Ved hjælp af identitetsloven $(x + 0 = x)$:
  $$1 + 0 = 1$$
  Udtrykket bliver nu:
  $$1 + 1$$
- **Trin 5:** Reducér $1 + 1$
  Ved hjælp af idempotensloven $(x + x = x)$ eller dominansloven:
  $$1 + 1 = 1$$
  Udtrykket er sandt.

Dermed har vi vist, at:

$$(1 \cdot 1) + (\overline{0 \cdot 1} + 0) = 1$$

◀

Dernæst ser vi på nogle andre vigtige love:

Strukturelle booleske love

- **Kommutative love:**
  $x + y = y + x, \quad x \cdot y = y \cdot x$
- **Absorptionslove og beslægtede forenklingsregler:**
  $x + (x \cdot y) = x, \quad x \cdot (x + y) = x, \quad \bar{x} \cdot \bar{y} + x = \bar{y} + x$
- **De Morgans love:**
  $\overline{x \cdot y} = \overline{x} + \overline{y}, \quad \overline{x + y} = \overline{x} \cdot \overline{y}$
- **Associative love:**
  $x + (y + z) = (x + y) + z, \quad x \cdot (y \cdot z) = (x \cdot y) \cdot z$
- **Den distributive lov:**
  $x \cdot (y + z) = (x \cdot y) + (x \cdot z), \quad x + (y \cdot z) = (x + y) \cdot (x + z)$

Sammen med de grundlæggende identiteter gør de strukturelle love det muligt at reducere mere komplekse udtryk.

Eksempel 13.5 Vis, at $(x \cdot y) + \overline{(x + z) \cdot \overline{y}} = y + \overline{x} \cdot \overline{z}$

Løsning:

Bemærk, at vi ikke eksplicit nævner brugen af de **kommutative love**.

- **Trin 1:** Anvend **De Morgans lov** på komplementet
  $\overline{(x + z) \cdot \overline{y}}$
  Udtrykket bliver:
  $(x \cdot y) + \overline{(x + z)} + \overline{\overline{y}}$
- **Trin 2:** Brug **loven om dobbelt komplement**
  $\overline{\overline{y}}$
  Udtrykket bliver:
  $(x \cdot y) + \overline{(x + z)} + y$
- **Trin 3:** Anvend **De Morgans lov** på komplementet
  $\overline{(x + z)}$
  Udtrykket bliver:
  $(x \cdot y) + \overline{x} \cdot \overline{z} + y$
- **Trin 4:** Anvend **den distributive lov** baglæns
  $(x \cdot y) + \overline{x} \cdot \overline{z} + y = y(x + 1) + \overline{x} \cdot \overline{z}$
- **Trin 5:** Brug **dominansloven** til at reducere $x + 1$:
  Udtrykket reduceres altså til:
  $y + \overline{x} \cdot \overline{z}$

Dermed har vi vist, at:

$$(x \cdot y) + \overline{(x + z) \cdot \overline{y}} = y + \overline{x} \cdot \overline{z}$$

◀

Ud over de grundlæggende operatorer har vi andre almindeligt anvendte operatorer i boolesk algebra:

- NOT AND (NAND): Dette er komplementet til AND-operationen. Den giver sand, medmindre begge input er sande. I boolesk algebra betegnes NAND med symbolet $\uparrow$ og kan skrives som:
$$x \uparrow y = \overline{x \cdot y}$$

- NOT OR (NOR): Operatoren NOR er komplementet til OR-operationen. Den giver kun sand, når begge input er falske. Den betegnes med symbolet $\downarrow$ og kan udtrykkes som:
$$x \downarrow y = \overline{x + y}$$

- Eksklusiv OR (XOR): Operatoren XOR giver sand, hvis præcis ét af input er sandt, men ikke begge. Det er særligt nyttigt i regneoperationer som binær addition. Den betegnes med $\oplus$, og det booleske udtryk for XOR er:
$$x \oplus y = (x \cdot \overline{y}) + (\overline{x} \cdot y)$$

Vi opsummerer de supplerende operatorer i følgende tabel:

NAND	NOR	XOR
$0 \uparrow 0 = 1$	$0 \downarrow 0 = 1$	$0 \oplus 0 = 0$
$0 \uparrow 1 = 1$	$0 \downarrow 1 = 0$	$0 \oplus 1 = 1$
$1 \uparrow 0 = 1$	$1 \downarrow 0 = 0$	$1 \oplus 0 = 1$
$1 \uparrow 1 = 0$	$1 \downarrow 1 = 0$	$1 \oplus 1 = 0$

Tabel 13.5: Supplerende booleske operatorer.

Her er alle operatorerne samlet i én tabel:

OR	AND	NOT	NAND	NOR	XOR
$0 + 0 = 0$	$0 \cdot 0 = 0$	$\overline{0} = 1$	$0 \uparrow 0 = 1$	$0 \downarrow 0 = 1$	$0 \oplus 0 = 0$
$0 + 1 = 1$	$0 \cdot 1 = 0$	$\overline{1} = 0$	$0 \uparrow 1 = 1$	$0 \downarrow 1 = 0$	$0 \oplus 1 = 1$
$1 + 0 = 1$	$1 \cdot 0 = 0$		$1 \uparrow 0 = 1$	$1 \downarrow 0 = 0$	$1 \oplus 0 = 1$
$1 + 1 = 1$	$1 \cdot 1 = 1$		$1 \uparrow 1 = 0$	$1 \downarrow 1 = 0$	$1 \oplus 1 = 0$

Tabel 13.6: Grundlæggende og supplerende booleske operatorer.

Nu kan vi angive de sidste og mere avancerede booleske identiteter:

Avancerede booleske identiteter

- **Eksklusiv OR (XOR):**
$$x \oplus y = (x \cdot \overline{y}) + (\overline{x} \cdot y)$$
- **Eksklusiv NOR (XNOR):**
$$\overline{x \oplus y} = (x \cdot y) + (\overline{x} \cdot \overline{y})$$
- **Den forsvindende modsætning:**
$$x + \overline{x} \cdot y = x + y$$
- **FOIL-loven (af engelsk *First, Outer, Inner, Last*: første, yderste, inderste, sidste):**
$$(x + y) \cdot (z + w) = (x \cdot z) + (x \cdot w) + (y \cdot z) + (y \cdot w)$$

13.4 Logiske gates

Logiske gates er de grundlæggende byggesten i digitale kredsløb og styrer den elektriske strøm, der løber ind i og ud af kredsløbene. Logiske gates modtager binære input og frembringer output og regulerer derved elektriske signaler. Disse gates er opbygget af transistorer og indgår også i større systemer som regneenheder, multipleksere og mikroprocessorer og muliggør vigtige elektroniske logiske operationer.

Typer af logiske gates

Der findes forskellige logiske gates, som fungerer forskelligt afhængigt af, hvilken type binært input de modtager. I boolesk algebra svarer disse gates til grundlæggende logiske operationer som `AND`, `OR` og `NOT`. Nedenfor ses de mest almindelige typer logiske gates, som findes i de fleste elektroniske kredsløb:

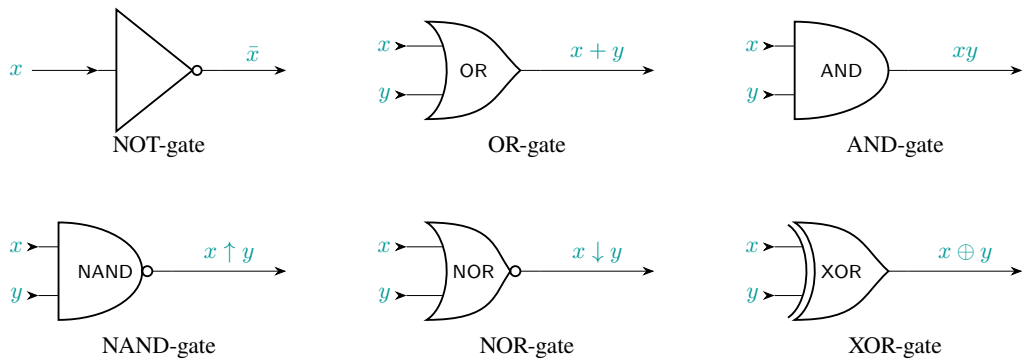
- `AND`
- `OR`
- `NOT`
- `NOR`
- `XOR`
- `NAND`

Om hver af disse logiske gates aktiveres, afhænger af, om et bestemt binært input, som gaten er designet til at genkende, er til stede eller ej. Hvis inputtet passer til gatens konfiguration, giver den logiske gate outputtet 1; hvis det ikke passer, er outputtet 0.

`AND`-gates
En `AND`-gate fungerer efter alt-eller-intet-princippet. For at aktivere gaten skal alle input være 1. Hvis blot ét input er 0, bliver outputtet 0.

`OR`-gates
`OR`-gates er mindre restriktive end `AND`-gates. Hvis mindst ét input er 1, bliver outputtet 1, uanset de øvrige input.

`NOT`-gates
En `NOT`-gate inverterer sit input. Den har ét input og ét output, og outputtet er altid det modsatte af inputtet. Hvis inputtet for eksempel er 1, bliver outputtet 0, og omvendt.



Figur 13.4: ANSI-symboler for de mest almindelige logiske gates. (Rosen, 2012).

`NOR`-gates
En `NOR`-gate kombinerer egenskaberne ved en `OR`-gate med en inverter. Outputtet fra en `NOR`-gate er kun 1, når

alle input er 0. Alle andre kombinationer af input giver outputtet 0.

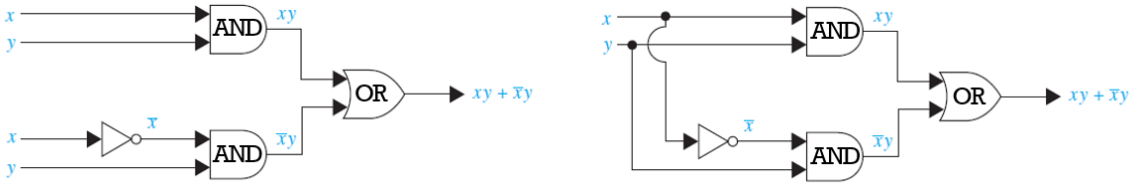
`XOR`-gates

En `XOR`-gate (eksklusiv OR) giver kun outputtet 1, når præcis ét af dens input er 1. Hvis begge input er ens, enten begge 0 eller begge 1, bliver outputtet 0. Den fungerer som en `OR`-gate, men med den ekstra betingelse, at begge input ikke samtidig må være 1, for at den giver 1.

`NAND`-gates

En `NAND`-gate er en kombination af en `AND`-gate og en `NOT`-gate. Outputtet er kun 0, hvis alle input er 1. I alle andre tilfælde er outputtet 1.

På samme måde som vi tidligere reducerede booleske udtryk til simplere former, kan vi opnå samme output med forskellige kredsløbsdesign, som det ses i figur 13.5.



Figur 13.5: To ækvivalente kredsløb af logiske gates. (Rosen, 2012).

Vi afslutter kapitlet med at se på idéen om **funktionel fuldstændighed**, også kendt som en **universel gate**. Begrebet henviser til en mængde af logiske gates, der kan kombineres, så de implementerer en vilkårlig boolesk funktion. En funktionelt fuldstændig mængde af gates, såsom `NAND` eller `NOR`, kan altså efterligne opførslen af alle de andre grundlæggende logiske gates, hvilket gør dem til alsidige værktøjer i design af digitale kredsløb.

Funktionel fuldstændighed og universelle gates

En operator eller en mængde af operatorer kaldes **funktionelt fuldstændig**, hvis den kan bruges til at udtrykke en vilkårlig boolesk funktion. Både `NAND`- og `NOR`-gates er funktionelt fuldstændige, eller **universelle gates**. Det betyder, at ethvert logisk udtryk kan konstrueres med kun `NAND` eller kun `NOR`.

Grunden til, at `NAND`-gaten (eller `NOR`-gaten) er funktionelt fuldstændig, er dens evne til at efterligne andre booleske operationer. Med `NAND`-operationer alene kan vi konstruere de grundlæggende operationer `NOT`, `AND` og `OR`, som er byggestenene i enhver boolesk funktion.

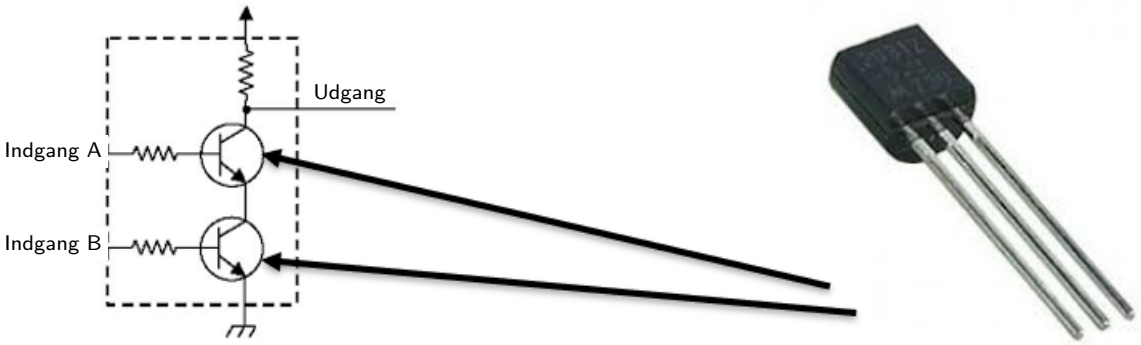
- `NOT` ($\overline{x}$): En `NOT`-gate kan konstrueres ved at forbinde begge input på en `NAND`-gate til den samme variabel: $\overline{x} = x \uparrow x$
- `AND` ($x \cdot y$): `AND`-operationen kan opnås ved først at anvende en `NAND`-gate og derefter sende resultatet gennem en `NOT` (som, vi har set, også kan konstrueres med `NAND`): $x \cdot y = \overline{x \uparrow y} = (x \uparrow y) \uparrow (x \uparrow y)$
- `OR` ($x + y$): `OR`-operationen kan dannes med flere `NAND`-gates ved at udnytte De Morgans love: $x + y = (x \uparrow x) \uparrow (y \uparrow y)$

Tabellen nedenfor viser, hvordan vi kan konstruere de grundlæggende booleske operationer med kun `NAND`-gates. Læg mærke til, hvordan de forskellige søjler viser transformationer af variablene x og y, hvor kombinationer af `NAND`-gates danner de logiske operationer `NOT`, `AND` og `OR`:

Input x	**Input** y	**NOT** $x \uparrow x$	**AND** $(x \uparrow y) \uparrow (x \uparrow y)$	**OR** $(x \uparrow x) \uparrow (y \uparrow y)$
0	0	1	0	0
0	1	1	0	1
1	0	0	0	1
1	1	0	1	1

Tabel 13.7: Konstruktion af grundlæggende booleske operatorer med `NAND`-gates.

Da vi har set, hvordan alle booleske funktioner kan bygges med kun `NAND`-gates, forstår vi nu, at alle logiske operationer i en computer kan implementeres med denne ene type gate. Det gør `NAND`-gaten utrolig vigtig i design af digitale kredsløb, da den giver mulighed for simplere og mere effektiv implementering i hardware. I moderne computersystemer bruges transistorer til at bygge `NAND`-gates, hvilket betyder, at enhver logisk operation i sidste ende kan repræsenteres ved en kombination af transistorer.



Figur 13.6: Logiske gates i computere.

Begrebet funktionel fuldstændighed er således ikke kun teoretisk; det er et stærkt princip, der gør det muligt for computeringeniører at designe fleksible og effektive kredsløb med et minimum af komponenter.

Kapitel 14 Tidskompleksitet

Når vi diskuterer en algoritmes effektivitet, vil vi gerne forstå, hvordan den skalerer med inputstørrelsen, uafhængigt af detaljer om bestemt hardware eller bestemte programmeringsmiljøer. *Teoretisk* tidskompleksitet, som almindeligvis udtrykkes med store $\mathcal{O}$-notation (med et stort $\mathcal{O}$, ikke et nul), muliggør denne abstraktion ved at fokusere på de dominerende led i algoritmens opførsel, når inputtet bliver stort. Notationen, som også kaldes Landaus symbol, blev indført af den tyske talteoretiker Edmund Landau. Her står $\mathcal{O}$ for »orden« (engelsk »order«) og indfanger en funktions vækstrate.

I dette kapitel introducerer vi de værktøjer, der skal til for at analysere algoritmers effektivitet med notationerne store $\mathcal{O}$, Omega (Ω) og Theta (Θ).[1]

14.1 Introduktion til kompleksitetsanalyse

En analyse af en algoritme kan for eksempel vise, at den tid, det tager at løse et problem af størrelse n, er givet ved $T(n) = 4n^2 - 2n + 2$. Ved at se bort fra konstanter – da de varierer med hardwaren – og fra langsommere voksende led forenkler vi dette til $T(n) = \mathcal{O}\left(n^2\right)$ og fremhæver dermed det dominerende led. Denne abstraktion hjælper os med at forstå kompleksiteten for store input uafhængigt af det konkrete miljø, og det gør store $\mathcal{O}$ til et stærkt værktøj til at sammenligne algoritmer.

De to centrale elementer i denne teoretiske abstraktion er:

1. **Vi ser bort fra multiplikative konstanter:** Konstanter afhænger ofte af faktorer som processorhastighed eller compileroptimeringer, hvilket gør dem upraktiske i en generel analyse.
2. **Vi fokuserer på den asymptotiske opførsel:** Vi koncentrerer os om algoritmens opførsel for store input, da små tilfælde typisk kører hurtigt på enhver moderne maskine.

I *praktisk* tidskompleksitet spiller hvert led derimod en rolle – også konstanterne. Når koden køres på bestemt hardware i et kendt miljø, kan de »ubetydelige« konstanter i store $\mathcal{O}$ blive betydelige, især for algoritmer, der anvendes på små og mellemstore datasæt. Store $\mathcal{O}$ er altså afgørende for analyse på et overordnet niveau og for sammenligning af algoritmer, men i praksis kræver kompleksiteten ofte, at man ser på hvert eneste led i kompleksitetsudtrykket. Det følgende eksempel illustrerer dette.

Algoritme 14.1: Lineær søgning i Python

```
def linear_search(arr, target):
    for i in range(len(arr)):
        if arr[i] == target:
            return i
    return -1
```

[1] I praksis bruges store $\mathcal{O}$ ofte uformelt som en paraplybetegnelse, når man taler om asymptotisk analyse i almindelighed, men teknisk set omfatter den ikke Θ eller Ω. Hver notation har sit eget specifikke anvendelsesområde, idet den afgrænser funktionens vækstrate fra forskellige perspektiver (øvre, nedre og stramme grænser), som vi snart skal se.

Algoritme 14.2: Lineær søgning i Java

```
public static int linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            return i;
        }
    }
    return -1;
}
```

Koden udfører en lineær søgning gennem en liste og kontrollerer hvert element, indtil den finder målet eller når listens slutning. Her er tidskompleksiteten $\mathcal{O}(n)$ i værste tilfælde, da alle elementer skal kontrolleres. For små lister kan forskellen være ubetydelig, men når inputstørrelsen vokser, bliver den lineære sammenhæng mellem inputstørrelse og køretid tydelig.

Når vi spørger »Hvor effektiv er en algoritme eller et stykke kode?«, skal vi være opmærksomme på, at effektivitet omfatter forskellige ressourcer, herunder:

- CPU-forbrug (tid)
- Hukommelsesforbrug
- Diskforbrug
- Netværksforbrug

Alle disse ressourcer er vigtige, men i dette kapitel fokuserer vi primært på tidskompleksitet, som vedrører CPU-forbruget.

1. **Ydeevne:** Dette angiver, hvor meget tid, hukommelse, diskplads eller andre ressourcer der faktisk bruges, når et program køres. Ud over selve koden påvirkes ydeevnen af faktorer som maskinen, compileren, kørselsmiljøet og andre implementeringsdetaljer.
2. **Kompleksitet:** Kompleksitet beskriver derimod, hvordan et programs eller en algoritmes ressourcebehov skalerer med størrelsen af det problem, der skal løses. Kompleksitetsanalyse fokuserer på de teoretiske aspekter af en algoritmes ydeevne og karakteriserer, hvordan ressourceforbruget vokser, når inputstørrelsen øges.

Kompleksitet påvirker ydeevnen, men ydeevnen alene giver ikke et fuldstændigt billede af en algoritmes effektivitet med hensyn til skalerbarhed. Det er afgørende at forstå begge dele for at kunne vurdere, hvor godt en algoritme vil klare sig i forskellige virkelige scenarier, især når inputstørrelserne vokser betydeligt.

14.2 Funktioners vækst

I Afsnit 14.1 bemærkede vi, at store $\mathcal{O}$-notation fokuserer på det dominerende led i en algoritmes tidskompleksitet og abstraherer konstanter og mindre led væk. Denne forenkling bygger på den idé, at det dominerende led bestemmer algoritmens kompleksitet, når inputstørrelserne bliver store. Se på følgende Java-kodestykke:

Algoritme 14.3: SumCalculator i Java

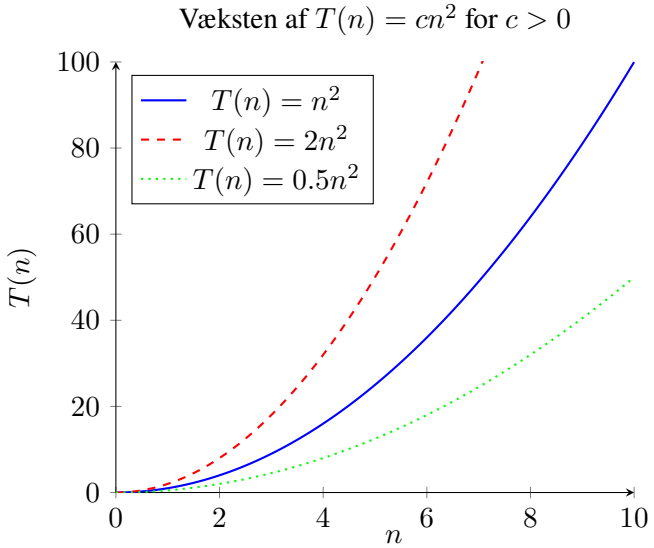
```
public static int f(int n) {
    int sum = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            sum += (i + j) / 6;
        }
    }
    return sum;
}
```

Koden ovenfor er en implementering af følgende udtryk:

$$\text{sum} = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \frac{i+j}{6}$$

Antag, at n er et ikkenegativt heltal, og lad $T(n)$ betegne den CPU-tid, en typisk bærbar computer bruger på at udføre funktionen[2] $f(n)$ med input n. $T(n)$ kan naturligvis variere på grund af faktorer, som vi ikke fuldt ud har kontrol over, såsom den specifikke Java-compiler, den bærbare computers hardwarespecifikationer, styresystemet og andre programmer, der kører samtidig. Vi kan dog være sikre på løkkernes struktur: Den ydre løkke kører n gange, og for hvert gennemløb af den ydre løkke kører den indre løkke også n gange, hvilket giver i alt n^2 iterationer.

Hver iteration kræver et omtrent konstant antal clockcyklusser, betegnet c, som kan variere fra system til system. I stedet for at sige, at »$T(n)$ er omtrent cn^2 nanosekunder for en konstant c, der varierer mellem systemer«, bruger vi store $\mathcal{O}$-notation og skriver $T(n) = \mathcal{O}(n^2)$. Notationen udtrykker, at CPU-tiden højst vokser kvadratisk med n. For at gøre det anskueligt kan vi se på en graf af $T(n)$, som generelt vil være parabelformet med den omtrentlige form cn^2, hvor $c > 0$ er en konstant.



Figur 14.1: Graf af $T(n) = cn^2$, der illustrerer kvadratisk vækst for forskellige værdier af c

[2] I Java er det teknisk set en metode, ikke en funktion. I Javas terminologi kaldes en funktion, der tilhører en klasse (funktionen ville formentlig tilhøre en klasse som `public class SumCalculator`), en metode. En funktion betegner generelt en selvstændig kodeblok, der kan kaldes uafhængigt, hvilket er typisk i sprog som C eller Python.

Store $\mathcal{O}$-notation gør det muligt at se bort fra konstanter, som ofte varierer og ligger uden for vores kontrol, men som ikke påvirker funktionens »familie«, dvs. den overordnede klassificering af dens vækstrate. For eksempel tilhører enhver mulig funktion $T(n)$ i dette tilfælde familien af andengradsfunktioner, som illustreret i figur 14.1. Desuden bliver led af lavere orden ubetydelige for store n, da de kan begrænses af et konstant multiplum af det dominerende led.

Hvis for eksempel $f(n) = 3.5n^2 + 4n + 36$, ser vi, at $f(n) \geq 3.5n^2$. Samtidig gælder

$$f(n) = 3.5n^2 + 4n + 36 \leq 3.5n^2 + 4n^2 + 36n^2 = 43.5n^2,$$

hvilket reelt »klemmer« $f(n)$ inde mellem to andengradsfunktioner og bekræfter, at dens vækst i det væsentlige er kvadratisk.

Grundlæggende funktioner

For at forstå, hvordan funktioner vokser, ordner vi nogle grundlæggende funktioner efter deres vækstrate. Vi leder efter relationer på formen $f(n) \leq g(n)$, men da vi kun interesserer os for opførslen for store input, gælder disse relationer kun over en vis nedre grænse k for n.

Lad os begynde med det grundlæggende: Enhver konstant funktion vokser langsommere end en lineær funktion, simpelthen fordi en konstant slet ikke vokser. For tilstrækkeligt store værdier vokser en lineær funktion langsommere end en andengradsfunktion, mens en tredjegradsfunktion vokser hurtigere end en andengradsfunktion.

Eksponentialfunktioner er velkendte for deres hurtige vækst. Tænk på historien om en virksomhed, der besluttede at belønne en medarbejder ved at fordoble vedkommendes bonus hver dag i en måned. Selv om det begyndte med et beskedent beløb, eskalerede den eksponentielle stigning hurtigt, og ved månedens udgang ville bonussen være astronomisk. Da virksomheden indså, at omkostningerne var uholdbare, ændrede den hurtigt aftalen til et fast beløb i stedet. Matematisk er effekten tydelig: $n^2 < 2^n$ for ethvert heltal $n \geq 5$. Mere generelt gælder $n^k < 2^n$ for enhver eksponent k, når n er tilstrækkeligt stor.

Endnu mere dramatisk er fakultetsvækst, repræsenteret ved $n!$, som overhaler eksponentiel vækst for store n. Begrundelsen er, at 2^n består af n faktorer 2, mens $n!$ har de første n heltal som faktorer, og de fleste af dem er større end 2. Derfor vokser $n!$ til sidst hurtigere end 2^n.

Sammenfattende er eksponentiel vækst hurtigere end enhver polynomiel vækst, og fakultetsvækst overgår dem begge. Med 1 som vores konstante funktion kan vi udtrykke disse relationer som:

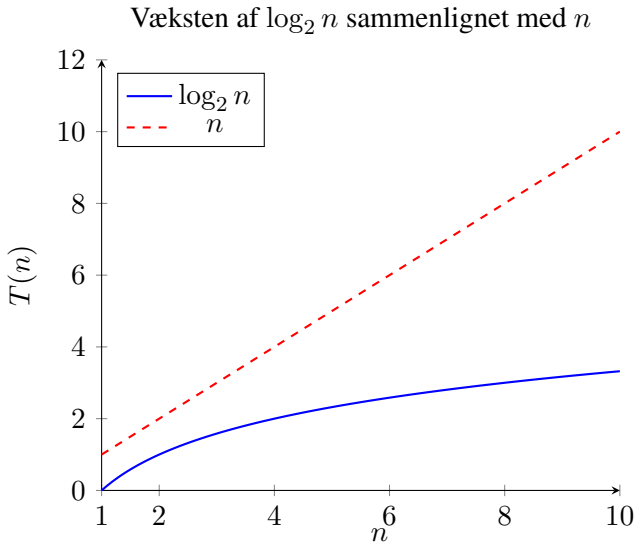
$$1 \prec n \prec n^2 \prec n^3 \ldots \prec 2^n \prec n!$$

Symbolet $\prec$ bruges her, fordi denne ordning ikke er en almindelig algebraisk ulighed og kun gælder, når n er tilstrækkeligt stor.

Ved design af computerprogrammer er kun de første få af disse vækstrater praktisk anvendelige. Tredjegradspolynomier kan allerede vokse for hurtigt for større input, og algoritmer med eksponentiel køretid er kun brugbare i meget små tilfælde. Eksponentielle algoritmer erstattes ofte af hurtigere, approksimative metoder som statistisk stikprøveudtagning.

Lad os nu se på funktioner med langsommere vækst – det er den type funktioner, vi stræber efter i effektive algoritmer. Algoritmer til søgning i store datasæt har ofte køretider, der er proportionale med $\log n$. Hvis vi

plotter $\log n$ for værdier større end 1, ses en langsom, men stabil vækst, som er meget langsommere end væksten af n selv.



Figur 14.2: Graf, der sammenligner vækstraterne for $\log_2 n$ og n

Sorteringsalgoritmer har derimod ofte køretider, der er proportionale med $n \log n$. For store n ser vi, at $1 < \log n < n$, hvilket fører til $n < n \log n < n^2$. Denne rækkefølge kan sammenfattes som:

$$1 \prec \log n \prec n \prec n \log n \prec n^2$$

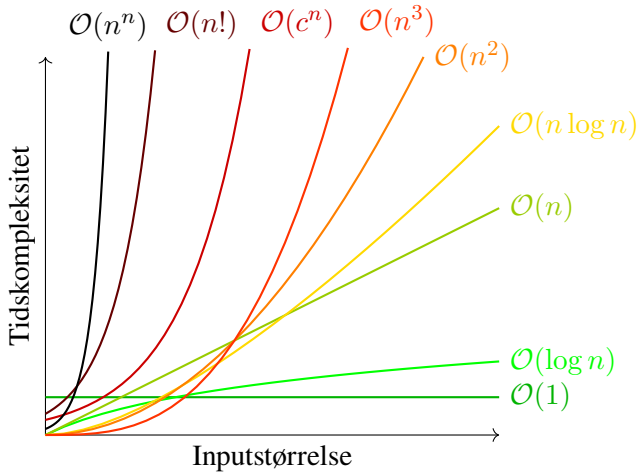
Bemærkning: Når vi skriver $\log n$, mener vi typisk logaritmen med grundtal 2, da den er den mest almindelige inden for datalogi og softwareudvikling. Logaritmens grundtal påvirker dog ikke vækstraten, da det kun tilføjer en konstant faktor. For eksempel vokser $\log_2 n$ og $\log_{10} n$ lige hurtigt og adskiller sig kun ved et konstant multiplum.

Kompleksitet	**Vækstrate**	**Eksempler på algoritmer**
$\mathcal{O}(1)$	Konstant	Opslag af et element i et array
$\mathcal{O}(\log n)$	Logaritmisk	Binær søgning
$\mathcal{O}(n)$	Lineær	Lineær søgning
$\mathcal{O}(n \log n)$	Linearitmisk (eller log-lineær)	MergeSort, HeapSort, QuickSort (forventet)
$\mathcal{O}(n^2)$	Kvadratisk	Bubble Sort, Selection Sort
$\mathcal{O}(n^3)$	Kubisk	Matrixmultiplikation
$\mathcal{O}(n^k)$	Polynomiel	Approksimationsalgoritmer i polynomiel tid
$\mathcal{O}(c^n)$	Eksponentiel	Subset sum, brute force til kombinatoriske problemer
$\mathcal{O}(n!)$	Fakultet	Brute force til den handelsrejsendes problem
$\mathcal{O}(n^n)$	Supereksponentiel	Rekursive algoritmer med høj forgreningsgrad

Tabel 14.1: Vækstrater for almindelige funktioner og eksempler på algoritmer

Tabel 14.1 sammenfatter vækstraterne for almindelige funktioner og giver eksempler på algoritmer med disse vækstrater, og figur 14.3 giver en visuel fremstilling af dem. Det er værdifuldt at lære den indbyrdes rækkefølge

af disse almindelige funktioner udenad, da de vil dukke op igen mange gange i dette kapitel og i andre sammenhænge inden for softwareudvikling.



Figur 14.3: Begrebsmæssig illustration af tidskompleksiteter

I næste afsnit introducerer vi notationerne store $\mathcal{O}$, Omega (Ω) og Theta (Θ), som beskriver henholdsvis øvre, nedre og stramme asymptotiske grænser. Notationerne kan anvendes på enhver valgt køretidsfunktion. Køretiderne i bedste, gennemsnitlige og værste tilfælde er separate funktioner, og hver af dem kan afgrænses med en hvilken som helst af de tre notationer.

14.3 Asymptotisk analyse

I algoritmeanalyse beskriver notationerne store $\mathcal{O}$, Omega (Ω) og Theta (Θ) asymptotiske grænser for en bestemt funktion, fx køretiden i værste tilfælde $T_{\text{worst}}(n)$ eller den forventede køretid $T_{\text{avg}}(n)$. Vi kan generelt følge denne systematiske fremgangsmåde:

1. **Identificér de dominerende operationer:** Fokusér på de operationer, der bidrager mest til køretiden, især dem, der vokser hurtigst, når inputstørrelsen øges.
2. **Se bort fra konstanter og led af lavere orden:** I store $\mathcal{O}$- og Θ-notation ignorerer vi konstante faktorer og led af lavere orden for at koncentrere os om det led, der bestemmer den samlede vækstrate. For Ω skal du sikre dig, at du fastlægger en nedre grænse, som stadig er repræsentativ for vækstraten.
3. **Udtryk kompleksiteten ved inputstørrelsen:** Skriv kompleksiteten ved hjælp af inputstørrelsen n. Hvis køretiden er proportional med kvadratet på inputstørrelsen, udtrykkes den som $\mathcal{O}(n^2)$, hvis man analyserer en øvre grænse, som $\Omega(n^2)$, hvis man analyserer en nedre grænse, eller som $\Theta(n^2)$, hvis man fastlægger en stram grænse.

Eksempel 14.1

Se på funktionen $f(n) = 4n^3 + 3n^2 + 2n + 10$. Lad os bestemme en asymptotisk øvre grænse med store $\mathcal{O}$-notation.

1. **Identificér de dominerende operationer:**
   - Se på hvert led i $f(n) = 4n^3 + 3n^2 + 2n + 10$. Leddene $4n^3, 3n^2, 2n$ og 10 bidrager forskelligt til køretiden. Når n vokser, vil leddet $4n^3$ dominere funktionens vækst, fordi n^3 vokser hurtigere end

n^2, n og konstante led. Derfor fokuserer vi på $4n^3$.

2. **Se bort fra konstanter:**
   - Ignorér den konstante koefficient 4 i $4n^3$, da store $\mathcal{O}$-notation kun tager hensyn til den asymptotiske vækstrate. Vi kan se bort fra leddene af lavere orden $3n^2, 2n$ og 10, da de ikke påvirker den asymptotiske opførsel, når n bliver meget stor.
3. **Udtryk kompleksiteten ved inputstørrelsen n:**
   - Når konstanterne og leddene af lavere orden er fjernet, står vi tilbage med n^3. Da $4n^3$ dominerer væksten, beskriver vi kompleksiteten ved n^3 med store $\mathcal{O}$-notation
     $$f(n) = \mathcal{O}\left(n^3\right)$$

Tilsvarende overvejelser gælder, når vi bestemmer nedre og stramme grænser. Vi introducerer nu de tre notationer mere indgående og begynder med Theta (Θ).

Theta-notation (Θ)

Theta-notation giver en *stram grænse*, dvs. at en funktions vækstrate er begrænset både opadtil og nedadtil af det samme asymptotiske udtryk. Med andre ord bruges Θ-notation, når den øvre og den nedre grænse for en algoritmes køretid er de samme, og den beskriver dermed algoritmens kompleksitet konsistent på tværs af forskellige inputstørrelser.

Matematisk gælder $T(n) = \Theta(f(n))$, hvis der findes konstanter $c_1, c_2 > 0$ og $k \geq 0$, så $c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n)$ for alle $n \geq k$.

Vi kan sammenfatte dette i følgende definition:

Definition 14.1 (Theta(Θ))

$f(n)$ er $\Theta(g(n))$, hvis der findes positive konstanter c_1, c_2 og k, så der for alle $n \geq k$ gælder

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

Det betyder, at $f(n)$ asymptotisk er begrænset både opadtil og nedadtil af $g(n)$, dvs. at $f(n)$ vokser lige så hurtigt som $g(n)$, når n bliver tilstrækkeligt stor.

Bemærk, at vi med »n bliver tilstrækkeligt stor« mener, at ovenstående uligheder ikke behøver at gælde for alle $n \geq 0$, men for alle $n \geq k$ for en konstant $k \geq 0$. Nogle gange angiver vi en præcis værdi af k, og andre gange bruger vi formuleringen »tilstrækkeligt stor«, når værdien af k ikke er vigtig.

Figur 14.4 illustrerer Θ-notation: Efter en passende tærskel k er $f(n)$ begrænset nedadtil og opadtil af positive konstante multipla af $g(n)$. Figuren viser én mulig tærskel, ikke nødvendigvis den mindste. Theta beskriver den asymptotiske vækst af den funktion f, der er valgt. Mere kortfattet:

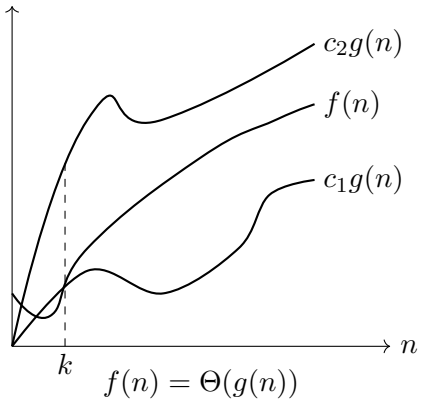
$$f(n) = \Theta(g(n)): \quad f \text{ vokser lige så hurtigt som } g$$

Eksempel 14.2

Fra diskussionen ovenfor har vi $f(n) = 3.5n^2 + 4n + 36 = \Theta\left(n^2\right)$, da

$$3.5n^2 \leq f(n) = 3.5n^2 + 4n + 36 \leq 3.5n^2 + 4n^2 + 36n^2 = 43.5n^2,$$

gælder for alle $n \geq 1$. Altså er $f(n) = \Theta\left(n^2\right)$ ifølge definition 14.3 med $c_1 = 3.5$ og $c_2 = 43.5$.



Figur 14.4: Illustration af Θ-notation, hvor $f(n)$ er afgrænset af $c_1g(n)$ og $c_2g(n)$

Følgende egenskaber gælder for Θ-notation:

- Hvis $f(n) = \Theta(g(n))$, så er $g(n) = \Theta(f(n))$.
- Hvis $f(n) = \Theta(g(n))$ og $g(n) = \Theta(h(n))$, så er $f(n) = \Theta(h(n))$.

Desuden giver følgende proposition en bekvem måde at vise, at to funktioner har samme vækstorden.

Proposition 14.1

Lad $f(n)$ og $g(n)$ være to ikkenegative funktioner, og antag, at der findes en konstant $c > 0$, så

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = c$$

Da er $f(n) = \Theta(g(n))$. Hvis grænseværdien

$$\lim_{n\to\infty} \frac{f(n)}{g(n)}$$

desuden eksisterer, men **ikke** er lig med en positiv konstant, så er $f(n) \neq \Theta(g(n))$.

Eksempel 14.3 Brug proposition 14.1 til at bevise, at hvis $f(n) = 3n^2 + 6n + 7$, så er $f(n) = \Theta\left(n^2\right)$. Med andre ord har $f(n)$ kvadratisk vækst.

Løsning: Lad $g(n) = n^2$. Ifølge proposition 14.1 skal vi beregne grænseværdien:

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \frac{3n^2 + 6n + 7}{n^2}.$$

Ved at dividere hvert led i tælleren med n^2 får vi:

$$\lim_{n\to\infty} \frac{3n^2}{n^2} + \frac{6n}{n^2} + \frac{7}{n^2} = \lim_{n\to\infty} \left(3 + \frac{6}{n} + \frac{7}{n^2}\right).$$

Når $n \to \infty$, går leddene $\frac{6}{n}$ og $\frac{7}{n^2}$ mod 0, så vi har:

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 3.$$

Da denne grænseværdi er en positiv konstant ($c = 3 > 0$), følger det af proposition 14.1, at $f(n) = \Theta(n^2)$.

Vi har dermed vist, at $f(n) = 3n^2 + 6n + 7$ har kvadratisk vækst, som ønsket. ◀

Øvre og nedre grænse

Antag, at vi definerer en funktion $f(n)$ således:

$$f(n) = \begin{cases} 2n & \text{hvis } n \text{ er lige} \\ 3n^2 & \text{hvis } n \text{ er ulige} \end{cases}$$

Hvad er vækstraten for $f(n)$? Vi kan desværre ikke blot kategorisere $f(n)$ som havende enten lineær eller kvadratisk vækst. Det skyldes, at grænseværdierne

$$\lim_{n\to\infty} \frac{f(n)}{n}$$

og

$$\lim_{n\to\infty} \frac{f(n)}{n^2}$$

ikke eksisterer, da $f(n)$ hele tiden veksler mellem lineær og kvadratisk opførsel og skifter fra kvadratisk vækst til lineær vækst og tilbage igen. Det bedste, vi kan gøre, er derfor at fastlægge øvre og nedre grænser for $f(n)$.

For store $\mathcal{O}$: $f(n) = \mathcal{O}(g(n))$, hvis $f(n)$ ikke vokser hurtigere end $g(n)$. Med andre ord gælder $f(n) \leq c \cdot g(n)$ for en konstant $c > 0$ og for alle $n \geq k$, hvor $k \geq 0$ er en konstant.

For store Omega: $f(n) = \Omega(g(n))$, hvis $f(n)$ vokser mindst lige så hurtigt som $g(n)$. Det vil sige, at $f(n) \geq c \cdot g(n)$ for en konstant $c > 0$ og for alle $n \geq k$, hvor $k \geq 0$ er en konstant.

Ud fra disse definitioner finder vi, at $f(n) = \mathcal{O}(n^2)$ og $f(n) = \Omega(n)$. Vi formaliserer disse overvejelser i de følgende afsnit.

Store $\mathcal{O}$-notation

Store $\mathcal{O}$-notation giver en *asymptotisk øvre grænse* for en funktions vækstrate. Den anvendes ofte på en køretidsfunktion for værste tilfælde, men store $\mathcal{O}$ betyder ikke i sig selv værste tilfælde.

Definition 14.2 (Store $\mathcal{O}$)

$f(n)$ er $\mathcal{O}(g(n))$, hvis der findes positive konstanter c og k, så der for alle $n \geq k$ gælder

$$f(n) \leq c \cdot g(n)$$

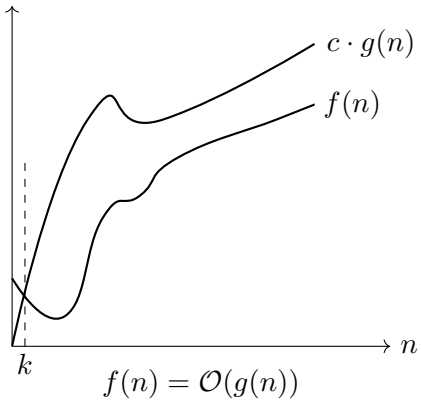
Det betyder, at $f(n)$ asymptotisk er begrænset opadtil af $g(n)$, dvs. at $f(n)$ højst vokser lige så hurtigt som $g(n)$, når n bliver tilstrækkeligt stor.

Udtrykket »n bliver tilstrækkeligt stor« betyder, at uligheden ikke behøver at gælde for alle $n \geq 0$, men for alle $n \geq k$, hvor k er en konstant. Ofte har vi ikke brug for den præcise værdi af k, så vi skriver »tilstrækkeligt stor«, når den præcise værdi er uden betydning.

Figur 14.5 illustrerer, at $f(n)$ fra et vist punkt er begrænset opadtil af $c\,g(n)$. Mere kortfattet:

$$f(n) = \mathcal{O}(g(n)): \quad f \text{ vokser højst lige så hurtigt som } g$$

Eksempel 14.4 Brug definition 14.3 til at vise, at $f(n) = 3n^2 + 6n + 7$ er $\mathcal{O}(n^2)$. Med andre ord har $f(n)$ højst kvadratisk vækst.



Figur 14.5: Illustration af $\mathcal{O}$-notation, hvor $f(n)$ er begrænset opadtil af $c \cdot g(n)$.

Løsning: Lad $g(n) = n^2$. Vi skal finde konstanter $c > 0$ og k, så $f(n) \leq c \cdot g(n)$ for alle $n \geq k$.

Vi kan skrive:

$$f(n) = 3n^2 + 6n + 7 \leq 3n^2 + 6n^2 + 7n^2 = 16n^2.$$

Her opfylder $c = 16$ og $k = 1$ uligheden, så vi konkluderer, at $f(n) = \mathcal{O}(n^2)$. ◀

Følgende egenskaber gælder for $\mathcal{O}$-notation:

- Hvis $f(n) = \mathcal{O}(g(n))$ og $g(n) = \mathcal{O}(h(n))$, så er $f(n) = \mathcal{O}(h(n))$.
- Konstante funktioner er $\mathcal{O}(1)$, dvs. at deres vækst er begrænset uanset inputstørrelsen.

Omega-notation (Ω)

Omega-notation giver en *asymptotisk nedre grænse* for vækstraten af en bestemt funktion. Den kan anvendes på køretidsfunktioner for bedste, gennemsnitlige eller værste tilfælde; Omega betyder ikke i sig selv bedste tilfælde.

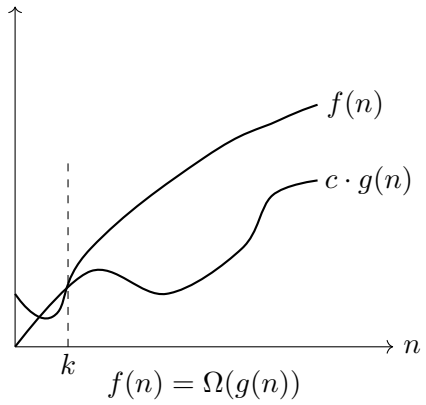
Definition 14.3 (Omega-notation)

$f(n)$ er $\Omega(g(n))$, hvis der findes positive konstanter c og k, så der for alle $n \geq k$ gælder

$$f(n) \geq c \cdot g(n)$$

Det betyder, at $f(n)$ asymptotisk er begrænset nedadtil af $g(n)$, dvs. at $f(n)$ vokser mindst lige så hurtigt som $g(n)$, når n bliver tilstrækkeligt stor.

Formuleringen »n bliver tilstrækkeligt stor« angiver, at ovenstående ulighed ikke behøver at gælde for alle $n \geq 0$, men kun for $n \geq k$ for en konstant k.



Figur 14.6: Illustration af Ω-notation, der viser, at $f(n)$ asymptotisk er begrænset nedadtil af $c \cdot g(n)$ for store værdier af n.

Figur 14.6 illustrerer, at $f(n)$ fra et vist punkt er begrænset nedadtil af $c\,g(n)$. Mere kortfattet:

$$f(n) = \Omega(g(n)): \quad f \text{ vokser mindst lige så hurtigt som } g$$

Eksempel 14.5 Brug definition 14.3 til at vise, at $f(n) = 3n^2 + 6n + 7$ er $\Omega(n^2)$. Med andre ord har $f(n)$ mindst kvadratisk vækst.

Løsning: Lad $g(n) = n^2$. Vi skal finde konstanter $c > 0$ og k, så $f(n) \geq c \cdot g(n)$ for alle $n \geq k$.

Vi kan skrive:

$$f(n) = 3n^2 + 6n + 7 \geq 3n^2.$$

Her opfylder $c = 3$ og $k = 1$ uligheden, så vi konkluderer, at $f(n) = \Omega(n^2)$. ◀

Følgende egenskaber gælder for Ω-notation:

- Hvis $f(n) = \Omega(g(n))$ og $g(n) = \Omega(h(n))$, så er $f(n) = \Omega(h(n))$.
- Positive konstante funktioner er $\Omega(1)$, dvs. at de har en nedre grænse, der ikke afhænger af inputstørrelsen.

Formelt gælder $T(n) = \Omega(g(n))$, hvis der findes konstanter $c > 0$ og $k \geq 0$, så $T(n) \geq c \cdot g(n)$ for alle $n \geq k$.

Hvis en sorteringsalgoritme for eksempel altid kræver mindst $n \log n$ sammenligninger, kan vi sige, at den har tidskompleksiteten $\Omega(n \log n)$.

Eksempel 14.6

Antag, at $f(n)$ er defineret som følger.

$$f(n) = \begin{cases} 2n^{2.1} \log^3 n & \text{hvis } n \bmod 3 = 0 \\ 10n^2 \log^{50} n & \text{hvis } n \bmod 3 = 1 \\ 6n^2 (\log(n^{80}))^{30} & \text{hvis } n \bmod 3 = 2 \end{cases}$$

Angiv en øvre grænse med store $\mathcal{O}$ og en nedre grænse med store Ω for $f(n)$.

Løsning: For at finde en øvre grænse med store $\mathcal{O}$ og en nedre grænse med store Ω for funktionen undersøger vi hvert tilfælde. Til den øvre grænse vælger vi det led, der vokser hurtigst, når $n \to \infty$, og til den nedre grænse det led, der vokser langsomst.

Bestem den øvre grænse med store $\mathcal{O}$

For at finde en øvre grænse med store $\mathcal{O}$ leder vi efter den gren, der har den højeste vækstrate. Leddet $n^{2.1} \log^3 n$ dominerer til sidst både $n^2 \log^{50} n$ og $n^2 (\log n)^{30}$.

En øvre grænse med store $\mathcal{O}$ for $f(n)$ er dermed:

$$f(n) = \mathcal{O}(n^{2.1} \log^3 n).$$

Bestem den nedre grænse med store Ω

For en nedre grænse med store Ω leder vi efter den gren, der har den laveste vækstrate. Da $\log(n^{80}) = 80 \log n$, vokser den tredje gren som et konstant multiplum af $n^2 (\log n)^{30}$, hvilket er langsommere end $n^2 (\log n)^{50}$.

En nedre grænse med store Ω for $f(n)$ er derfor:

$$f(n) = \Omega(n^2 \log^{30} n).$$

Sammenfattende har vi:

$$f(n) = \mathcal{O}(n^{2.1} \log^3 n) \quad \text{og} \quad f(n) = \Omega(n^2 \log^{30} n).$$

◀

Kort opsummeret:

- Store $\mathcal{O}$-notation giver en asymptotisk øvre grænse.
- Omega-notation (Ω) giver en asymptotisk nedre grænse.
- Theta-notation (Θ) giver en stram asymptotisk grænse.

Når vi forstår disse notationer, får vi et mere fuldstændigt billede af en algoritmes effektivitet i forskellige scenarier.

14.4 Videre analyse

Ikke alle funktioner kan uden videre klassificeres med notationerne store $\mathcal{O}$, Omega (Ω) og Theta (Θ). I nogle tilfælde er en funktions vækstrate ikke let at bestemme, eller funktionen passer ikke pænt ind i en af disse kategorier. I sådanne tilfælde kan vi bruge yderligere værktøjer til at analysere funktioners vækstrate mere præcist.

Lille o-notation

Lille o-notation giver en *streng øvre grænse* for en funktions vækstrate. Den beskriver en situation, hvor funktionen vokser langsommere end en anden funktion. Notationen er nyttig til at angive, at en algoritmes tidskompleksitet er strengt mindre end en given funktion.

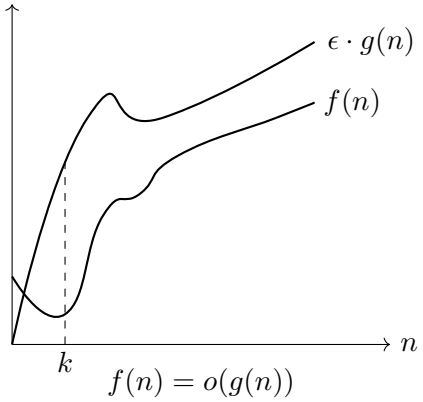
Definition 14.4 (Lille o)

$f(n)$ er $o(g(n))$, hvis der for enhver positiv konstant $\epsilon > 0$ findes en konstant $k \geq 0$, så der for alle $n \geq k$ gælder

$$f(n) < \epsilon \cdot g(n)$$

Det betyder, at $f(n)$ vokser strengt langsommere end $g(n)$, når n bliver tilstrækkeligt stor. ♣

Formuleringen »n bliver tilstrækkeligt stor« angiver, at ovenstående ulighed ikke behøver at gælde for alle $n \geq 0$, men kun for $n \geq k$ for en konstant k.



Figur 14.7: o-notation: $f(n)$ er strengt begrænset opadtil af $\epsilon \cdot g(n)$ for store n.

Figur 14.7 viser begrebet lille o-notation og illustrerer, at $f(n)$ er strengt begrænset opadtil af $\epsilon \cdot g(n)$. Notationen giver en streng øvre grænse for en algoritmes køretid og beskriver den asymptotiske vækstrate af algoritmens køretid på en mere præcis måde. Eller sagt mere kortfattet:

$$f(n) = o(g(n)): \quad f \text{ vokser strengt langsommere end } g$$

Eksempel 14.7 Brug definition 14.4 til at vise, at $f(n) = 3n^2 + 6n + 7$ er $o(n^3)$. Med andre ord vokser $f(n)$ strengt langsommere end n^3.

Løsning: Lad $g(n) = n^3$. Vi skal vise, at der for enhver positiv konstant $\epsilon > 0$ findes en konstant k, så $f(n) < \epsilon \cdot g(n)$ for alle $n \geq k$.

Vi kan skrive:

$$f(n) = 3n^2 + 6n + 7 < \epsilon \cdot n^3$$

for tilstrækkeligt store n. Ved at dividere begge sider med n^3 får vi:

$$\frac{f(n)}{n^3} = \frac{3n^2 + 6n + 7}{n^3} = \frac{3}{n} + \frac{6}{n^2} + \frac{7}{n^3}$$

Når $n \to \infty$, går højresiden mod 0, som er mindre end enhver positiv konstant ϵ. Derfor er $f(n) = o(n^3)$. ◀

Det fører os til følgende proposition

Proposition 14.2

Lad $f(n)$ og $g(n)$ være to ikkenegative funktioner, og antag, at

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$$

Da er $f(n) = o(g(n))$, dvs. at $f(n)$ vokser strengt langsommere end $g(n)$, når $n \to \infty$.

Eksempel 14.8 Brug proposition 14.2 til at bevise, at hvis $f(n) = 3n^2 + 6n + 7$, så er $f(n) = o(n^3)$. Med andre ord vokser $f(n)$ strengt langsommere end n^3.

Løsning:

Lad $g(n) = n^3$. Ifølge proposition 14.2 skal vi beregne grænseværdien:

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \frac{3n^2 + 6n + 7}{n^3}$$

Ved at dividere hvert led i tælleren med n^3 får vi:

$$\lim_{n\to\infty} \frac{3n^2}{n^3} + \frac{6n}{n^3} + \frac{7}{n^3} = \lim_{n\to\infty} \left(\frac{3}{n} + \frac{6}{n^2} + \frac{7}{n^3}\right)$$

Når $n \to \infty$, går leddene $\frac{3}{n}$, $\frac{6}{n^2}$ og $\frac{7}{n^3}$ mod 0, så vi har:

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$$

Da grænseværdien er 0, følger det af proposition 14.2, at $f(n) = o(n^3)$.

Vi har dermed vist, at $f(n) = 3n^2 + 6n + 7$ vokser strengt langsommere end n^3, som ønsket. ◀

Følgende egenskaber gælder for o-notation:

- Hvis $f(n) = o(g(n))$, så er $g(n) \neq o(f(n))$.
- Hvis $f(n) = o(g(n))$ og $g(n) = o(h(n))$, så er $f(n) = o(h(n))$.

Lille o-notation giver en mere præcis måde at beskrive funktioners vækstrate på, især når funktionen vokser strengt langsommere end en anden funktion. Notationen er nyttig til at angive, at en algoritmes tidskompleksitet er strengt mindre end en given funktion, og giver en mere detaljeret forståelse af algoritmens kompleksitet.

Lille ω-notation

Lille ω-notation giver en *streng nedre grænse* for en funktions vækstrate. Den beskriver en situation, hvor funktionen vokser hurtigere end en anden funktion. Notationen er nyttig til at angive, at en algoritmes tidskompleksitet er strengt større end en given funktion.

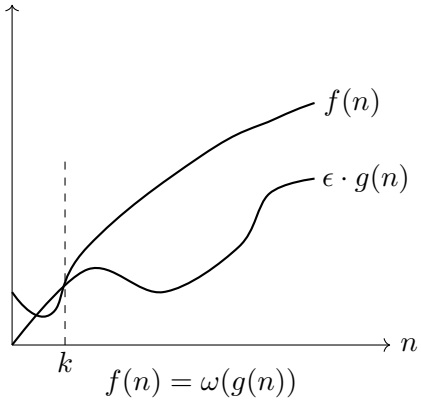
Definition 14.5 (Lille ω)

$f(n)$ er $\omega(g(n))$, hvis der for enhver positiv konstant $\epsilon > 0$ findes en konstant $k \geq 0$, så der for alle $n \geq k$ gælder

$$f(n) > \epsilon \cdot g(n)$$

Det betyder, at $f(n)$ vokser strengt hurtigere end $g(n)$, når n bliver tilstrækkeligt stor. ♣

Formuleringen »n bliver tilstrækkeligt stor« angiver, at ovenstående ulighed ikke behøver at gælde for alle $n \geq 0$, men kun for $n \geq k$ for en konstant k.



Figur 14.8: ω-notation: $f(n)$ er strengt begrænset nedadtil af $\epsilon \cdot g(n)$ for store n.

Figur 14.8 viser begrebet lille ω-notation og illustrerer, at $f(n)$ er strengt begrænset nedadtil af $\epsilon \cdot g(n)$. Notationen giver en streng nedre grænse for en algoritmes køretid og beskriver den asymptotiske vækstrate af algoritmens køretid på en mere præcis måde. Eller sagt mere kortfattet:

$$f(n) = \omega(g(n)): \quad f \text{ vokser strengt hurtigere end } g$$

Eksempel 14.9 Brug definition 14.5 til at vise, at $f(n) = n^3 + 6n + 7$ er $\omega(n^2)$. Med andre ord vokser $f(n)$ strengt hurtigere end n^2.

Løsning: Lad $g(n) = n^2$. Vi skal vise, at der for enhver positiv konstant $\epsilon > 0$ findes en konstant k, så $f(n) > \epsilon \cdot g(n)$ for alle $n \geq k$.

Vi kan skrive:

$$f(n) = n^3 + 6n + 7 > \epsilon \cdot n^2$$

for tilstrækkeligt store n. Ved at dividere begge sider med n^2 får vi:

$$\frac{f(n)}{n^2} = \frac{n^3 + 6n + 7}{n^2} = n + \frac{6}{n} + \frac{7}{n^2}$$

Når $n \to \infty$, går højresiden mod uendelig og overstiger derfor til sidst enhver positiv konstant ϵ. Altså er $f(n) = \omega(n^2)$. ◀

Det fører os til følgende proposition:

Proposition 14.3

Lad $f(n)$ og $g(n)$ være to ikkenegative funktioner, og antag, at

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = \infty$$

Da er $f(n) = \omega(g(n))$, dvs. at $f(n)$ vokser strengt hurtigere end $g(n)$, når $n \to \infty$. ♠

Eksempel 14.10 Brug proposition 14.3 til at bevise, at hvis $f(n) = n^3 + 6n + 7$, så er $f(n) = \omega(n^2)$. Med andre ord vokser $f(n)$ strengt hurtigere end n^2.

Løsning: Lad $g(n) = n^2$. Ifølge proposition 14.3 skal vi beregne grænseværdien:

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \lim_{n\to\infty} \frac{n^3 + 6n + 7}{n^2}$$

Ved at dividere hvert led i tælleren med n^2 får vi:

$$\lim_{n\to\infty} \frac{n^3}{n^2} + \frac{6n}{n^2} + \frac{7}{n^2} = \lim_{n\to\infty} \left(n + \frac{6}{n} + \frac{7}{n^2} \right)$$

Når $n \to \infty$, går leddet n mod ∞, så vi har:

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$$

Da grænseværdien er ∞, følger det af proposition 14.3, at $f(n) = \omega(n^2)$.

Vi har dermed vist, at $f(n) = n^3 + 6n + 7$ vokser strengt hurtigere end n^2, som ønsket. ◀

Følgende egenskaber gælder for ω-notation:

- Hvis $f(n) = \omega(g(n))$, så er $g(n) \neq \omega(f(n))$.
- Hvis $f(n) = \omega(g(n))$ og $g(n) = \omega(h(n))$, så er $f(n) = \omega(h(n))$.

Lille ω-notation giver en mere præcis måde at beskrive funktioners vækstrate på, især når funktionen vokser strengt hurtigere end en anden funktion. Notationen er nyttig til at angive, at en algoritmes tidskompleksitet er strengt større end en given funktion, og giver en mere detaljeret forståelse af algoritmens kompleksitet.

Brug af lille o og lille ω til at bestemme $\mathcal{O}$ og Ω

Notationerne lille o og lille ω kan bruges til at give mere præcise grænser, når vi bestemmer store $\mathcal{O}$ og Ω. De hjælper med at afgøre, om en funktion vokser strengt langsommere eller strengt hurtigere end en anden funktion, hvilket kan være nyttigt i asymptotisk analyse.

Eksempel 14.11 Se på funktionen $f(n) = n^2 + 3n + 2$. Vi vil bestemme store $\mathcal{O}$ og Ω.

1. **Store $\mathcal{O}$-notation:**
   For at bestemme store $\mathcal{O}$ ser vi på forholdet $\frac{f(n)}{g(n)}$, hvor $g(n) = n^2$. Ved hjælp af proposition 14.4 beregner vi $\limsup$[3]:
   $$\limsup_{n\to\infty} \frac{f(n)}{g(n)} = \limsup_{n\to\infty} \frac{n^2 + 3n + 2}{n^2} = \limsup_{n\to\infty} \left(1 + \frac{3}{n} + \frac{2}{n^2} \right).$$
   Når $n \to \infty$, går leddene $\frac{3}{n}$ og $\frac{2}{n^2}$ mod 0, så:
   $$\limsup_{n\to\infty} \frac{f(n)}{g(n)} = 1.$$
   Da denne værdi er endelig, konkluderer vi, at $f(n) = \mathcal{O}(n^2)$.
2. **Store Ω-notation:**

[3]Notationen $\limsup$ står for »limes superior« (øvre grænseværdi) og betragter den øvre grænse for værdierne af forholdet $\frac{f(n)}{g(n)}$, når n vokser. I modsætning til en almindelig grænseværdi, som ikke nødvendigvis eksisterer, hvis funktionen oscillerer, eksisterer $\limsup$ altid for reelle følger og indfanger den maksimale vækstopførsel i det lange løb.

For at bestemme store Ω beregner vi $\liminf$ af det samme forhold[4]:

$$\liminf_{n\to\infty} \frac{f(n)}{g(n)} = \liminf_{n\to\infty} \frac{n^2+3n+2}{n^2} = \liminf_{n\to\infty} \left(1 + \frac{3}{n} + \frac{2}{n^2}\right).$$

Igen går leddene $\frac{3}{n}$ og $\frac{2}{n^2}$ mod 0, når $n \to \infty$, så:

$$\liminf_{n\to\infty} \frac{f(n)}{g(n)} = 1.$$

Da denne værdi er positiv, konkluderer vi, at $f(n) = \Omega(n^2)$.

Sammenfattende har vi:

$$f(n) = \mathcal{O}(n^2) \quad \text{og} \quad f(n) = \Omega(n^2).$$

Ved at bruge notationerne lille o og lille ω kan vi give mere præcise grænser for store $\mathcal{O}$ og Ω, hvilket hjælper os til bedre at forstå funktioners vækstrater og algoritmers kompleksitet. Men måske endnu vigtigere kan vi bruge grænseværdibegrebet til at bestemme store $\mathcal{O}$ og Ω ud fra lille o og lille ω.

Det fører os til følgende propositioner:

Proposition 14.4

Lad $f(n)$ og $g(n)$ være to ikkenegative funktioner med $g(n) > 0$ for alle tilstrækkeligt store n. Hvis

$$\limsup_{n\to\infty} \frac{f(n)}{g(n)} < \infty$$

så er $f(n) = \mathcal{O}(g(n))$, dvs. at $f(n)$ asymptotisk højst vokser lige så hurtigt som $g(n)$.

Og desuden:

Proposition 14.5

Lad $f(n)$ og $g(n)$ være to ikkenegative funktioner med $g(n) > 0$ for alle tilstrækkeligt store n. Hvis

$$\liminf_{n\to\infty} \frac{f(n)}{g(n)} > 0$$

så er $f(n) = \Omega(g(n))$, dvs. at $f(n)$ asymptotisk vokser mindst lige så hurtigt som $g(n)$.

Disse propositioner giver en mere formel måde at bestemme store $\mathcal{O}$ og Ω ud fra lille o og lille ω. Ved at undersøge grænseværdierne af forholdet mellem funktioner kan vi afgøre, om en funktion højst eller mindst vokser lige så hurtigt som en anden funktion, hvilket giver en mere stringent analyse af vækstrater.

Eksempel 14.12 Brug proposition 14.5 til at bevise, at hvis $f(n) = 3n^2 + 5n + 7$, så er $f(n) = \Omega(n^2)$.

Løsning: Lad $g(n) = n^2$. Vi skal beregne den nedre grænseværdi (lim inf) af $\frac{f(n)}{g(n)}$, når n går mod uendelig:

$$\liminf_{n\to\infty} \frac{f(n)}{g(n)} = \liminf_{n\to\infty} \frac{3n^2+5n+7}{n^2}.$$

Ved at dividere hvert led i tælleren med n^2 har vi:

$$= \liminf_{n\to\infty} \left(3 + \frac{5}{n} + \frac{7}{n^2}\right).$$

[4]Notationen $\liminf$ står for »limes inferior« (nedre grænseværdi) og betragter den nedre grænse for værdierne af forholdet $\frac{f(n)}{g(n)}$, når n vokser. I modsætning til en almindelig grænseværdi, som ikke nødvendigvis eksisterer, hvis funktionen oscillerer, eksisterer $\liminf$ altid for reelle følger. Den indfanger den minimale vækstopførsel på lang sigt og fremhæver den laveste hastighed, hvormed $f(n)$ vokser i forhold til $g(n)$, når n bliver stor.

Når $n \to \infty$, går leddene $\frac{5}{n}$ og $\frac{7}{n^2}$ mod 0. Dermed får vi:

$$\liminf_{n\to\infty} \frac{f(n)}{g(n)} = 3.$$

Da denne grænseværdi er en positiv konstant $c = 3 > 0$, følger det af proposition 14.5, at $f(n) = \Omega(n^2)$. Eksemplet viser, at $f(n) = 3n^2 + 5n + 7$ har kvadratisk vækst i betydningen nedre grænse. ◀

Eksempel 14.13 Afgør, om $f(n) = 2n\sin(n) + 3n$ er $\mathcal{O}(n)$ og $\Omega(n)$, ved at bruge grænseværdierne som beskrevet i propositionerne 14.4 og 14.5.

Løsning: Lad $g(n) = n$. Vi undersøger opførslen af $\frac{f(n)}{g(n)}$, når $n \to \infty$.

Først udtrykker vi $\frac{f(n)}{g(n)}$ således:

$$\frac{f(n)}{g(n)} = \frac{2n\sin(n) + 3n}{n} = 2\sin(n) + 3.$$

Nu analyserer vi grænseværdiens opførsel, når $n \to \infty$:

$$\frac{f(n)}{g(n)} = 2\sin(n) + 3.$$

Da $\sin(n)$ svinger mellem -1 og 1, når n vokser, svinger udtrykket $2\sin(n)+3$ mellem 1 og 5. Den almindelige grænseværdi $\lim_{n\to\infty} \frac{f(n)}{g(n)}$ eksisterer derfor ikke på grund af denne svingning.

Vi kan dog beregne $\limsup$ og $\liminf$ af $\frac{f(n)}{g(n)}$:

$$\limsup_{n\to\infty} \frac{f(n)}{g(n)} = 5 \quad \text{og} \quad \liminf_{n\to\infty} \frac{f(n)}{g(n)} = 1.$$

Da $\limsup$ er endelig (5), konkluderer vi ifølge proposition 14.4, at $f(n) = \mathcal{O}(n)$. Da $\liminf$ er positiv (1), konkluderer vi tilsvarende ifølge proposition 14.5, at $f(n) = \Omega(n)$. ◀

Brug af grænseværdier til at bestemme asymptotiske notationer (en uformel fremgangsmåde)

En uformel, men praktisk fremgangsmåde til at bestemme store $\mathcal{O}$ og Ω består i at undersøge forholdet $\frac{f(n)}{g(n)}$, når $n \to \infty$. Vi har allerede set denne fremgangsmåde i propositionerne 14.1–14.5. Selv om metoden ikke er lige så stringent som formelle propositioner, giver den hurtig indsigt i vækstrater.

Følgende regler viser, hvordan grænseværdier kan anvendes på hver notation:

Uformel grænseværdimetode

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = L < \infty \quad \Rightarrow f(n) = \mathcal{O}(g(n))$$

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = L > 0 \quad \Rightarrow f(n) = \Omega(g(n))$$

Bemærk: Denne fremgangsmåde er en uformel retningslinje og ikke en streng matematisk definition. Der er tilfælde, hvor den grænseværdibaserede metode ikke kan anvendes, men den er ofte et bekvemt værktøj til at bestemme asymptotisk opførsel.

Reglerne fungerer således:

- **Store $\mathcal{O}$:** Hvis grænseværdien eksisterer og ikke er uendelig, er $f(n) = \mathcal{O}(g(n))$, dvs. at $f(n)$ ikke vokser hurtigere end $g(n)$.
- **Store Ω:** Hvis grænseværdien eksisterer og er forskellig fra nul, er $f(n) = \Omega(g(n))$, dvs. at $f(n)$ vokser mindst lige så hurtigt som $g(n)$.

Eksempel 14.14 Afgør ved hjælp af den uformelle grænseværdimetode, om $\sqrt{n} \cdot \log n = \mathcal{O}(n \log n)$.

Løsning: Vi skal beregne grænseværdien af forholdet $\frac{\sqrt{n} \cdot \log n}{n \log n}$, når $n \to \infty$. Ved at forenkle udtrykket får vi:

$$\lim_{n\to\infty} \frac{\sqrt{n} \cdot \log n}{n \log n} = \lim_{n\to\infty} \frac{\sqrt{n}}{n} = \lim_{n\to\infty} \frac{1}{\sqrt{n}} = 0.$$

Da grænseværdien er endelig, konkluderer vi, at $\sqrt{n} \cdot \log n = \mathcal{O}(n \log n)$. Vi bemærker også, at da grænseværdien er 0, kan vi slutte, at $\sqrt{n} \cdot \log n \neq \Omega(n \log n)$.

◀

Grænseværdimetoden er bekvem, men ikke endegyldig. For nogle funktioner (fx oscillerende funktioner) bør de formelle definitioner og propositioner anvendes direkte. Den grænseværdibaserede teknik er en nyttig tommelfingerregel, især når den asymptotiske vækst er tydelig.

Sammenligning af notationer

Vi afslutter kapitlet med at sammenligne de forskellige asymptotiske notationer og deres indbyrdes relationer:

1. $f(n) = \mathcal{O}(g(n))$ og $f(n) = \Omega(g(n)) \Leftrightarrow f(n) = \Theta(g(n))$
   Denne relation siger, at hvis $f(n)$ asymptotisk både er begrænset opadtil af $g(n)$ (store $\mathcal{O}$) og begrænset nedadtil af $g(n)$ (store Ω), så vokser $f(n)$ og $g(n)$ lige hurtigt, hvilket betegnes $\Theta(g(n))$. Med andre ord vokser $f(n)$ asymptotisk hverken hurtigere eller langsommere end $g(n)$, når $n \to \infty$.
2. $f(n) = o(g(n)) \Rightarrow f(n) = \mathcal{O}(g(n))$
   Hvis $f(n) = o(g(n))$, betyder det, at $f(n)$ vokser strengt langsommere end $g(n)$, når $n \to \infty$. Derfor er $f(n)$ asymptotisk begrænset opadtil af $g(n)$, men vokser ikke lige så hurtigt, hvilket begrunder, at $f(n) = \mathcal{O}(g(n))$.
3. $f(n) = \mathcal{O}(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$
   Denne ækvivalens viser den symmetriske sammenhæng mellem store $\mathcal{O}$ og store Ω. Hvis $f(n)$ asymptotisk er begrænset opadtil af $g(n)$ (dvs. at $f(n)$ ikke vokser hurtigere end $g(n)$), kan vi også sige, at $g(n)$ vokser mindst lige så hurtigt som $f(n)$ (dvs. $g(n) = \Omega(f(n))$).
4. $f(n) = \omega(g(n)) \Rightarrow f(n) = \Omega(g(n))$
   Når $f(n) = \omega(g(n))$, betyder det, at $f(n)$ vokser strengt hurtigere end $g(n)$, når $n \to \infty$. Da $f(n)$ asymptotisk er større, følger det, at $f(n)$ vokser mindst lige så hurtigt som $g(n)$, så $f(n) = \Omega(g(n))$ er sand.
5. $f(n) = o(g(n)) \Leftrightarrow g(n) = \omega(f(n))$
   Denne ækvivalens viser den symmetriske sammenhæng mellem lille o og lille ω. Hvis $f(n) = o(g(n))$, dvs. at $f(n)$ vokser strengt langsommere end $g(n)$, kan vi også sige, at $g(n)$ vokser strengt hurtigere end $f(n)$, eller $g(n) = \omega(f(n))$. Relationen fremhæver den strenge forskel i deres vækstrater, når $n \to \infty$.

I dette kapitel har vi undersøgt tidskompleksitet som et middel til at vurdere algoritmers effektivitet. Vi introducerede notationerne store $\mathcal{O}$, Omega (Ω) og Theta (Θ) som måder at fastlægge øvre, nedre og stramme grænser for vækstrater på, hvilket hjælper os med at forstå, hvordan krav til køretid og plads skalerer med inputstørrelsen.

Vi fremhævede, hvor vigtigt det er at se bort fra konstanter og led af lavere orden i asymptotisk analyse og i stedet fokusere på det dominerende led. Desuden undersøgte vi notationerne lille o og lille ω for strengere øvre og nedre grænser, som giver mere præcis indsigt i kompleksitet.

Til sidst sammenlignede vi notationerne og deres vigtigste relationer og byggede dermed et fundament for at vurdere algoritmers kompleksitet både teoretisk og praktisk – som forberedelse til den praktiske analyse af tidskompleksitet i næste kapitel.

Kapitel 15 Teknikker til kodeanalyse

I softwareudvikling er det afgørende at forstå kodens effektivitet, hvis man vil bygge skalerbare applikationer med hurtige svartider. Med udgangspunkt i principperne for asymptotisk notation går vi nu over til praktisk kodeanalyse – vi skiller koden ad og ser på, hvordan komponenter som løkker, betingelser og sekventielle sætninger bidrager til en algoritmes samlede kompleksitet.

Kodeanalyse kan groft inddeles i to kategorier:

1. **Statisk analyse:** Her undersøger vi koden uden at køre den. Vi fokuserer på at vurdere kodens struktur og operationer, især gennem kompleksitetsanalyse. Det er dette kapitels hovedemne.
2. **Dynamisk analyse:** Ved at køre koden på forskellige input måler vi de faktiske køretider og det faktiske hukommelsesforbrug i bestemte miljøer. Dynamisk analyse er værdifuld, men den ligger uden for dette kapitels rammer og bruges typisk til ydelsesprofilering.

Asymptotisk notation giver et overordnet billede af en algoritmes ydelse, idet den forudsiger algoritmens vækstrate, når inputstørrelsen vokser. Den teoretiske kompleksitet tager dog ikke højde for konstanter, mindre led eller maskinspecifikke faktorer. I praksis kan disse detaljer have stor betydning for køretiden, især i applikationer, der ofte udfører operationer på store datasæt.

Se på to algoritmer, der begge har køretiden $\Theta(n)$. De har samme asymptotiske vækstrate, men den ene algoritme kan have en konstant faktor, der er flere gange større end den andens, så den i praksis kører langsommere. For at forstå kompleksitet i praksis må vi analysere hver del af koden, vurdere antallet af operationer konkret og finde de steder, hvor koden kan optimeres.

Nedenfor ser du to algoritmer, der filtrerer de lige tal ud af et heltalsarray. Begge algoritmer har køretiden $\Theta(n)$, men den ene bruger en enkelt løkke, mens den anden foretager et overflødigt indledende gennemløb og derfor har en større konstant faktor i denne model.

Algoritme 15.1: Ét gennemløb til at filtrere lige tal

```
1  public List<Integer> filterEvenSinglePass(int[] arr) {
2      List<Integer> evenNumbers = new ArrayList<>();
3      for (int i = 0; i < arr.length; i++) {
4          if (arr[i] % 2 == 0) {
5              evenNumbers.add(arr[i]);
6          }
7      }
8      return evenNumbers;
9  }
```

Algoritme 15.2: Flere gennemløb til at filtrere lige tal

```
public List<Integer> filterEvenMultiplePasses(int[] arr) {
    List<Integer> evenNumbers = new ArrayList<>();

    // First pass: count even numbers
    int evenCount = 0;
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] % 2 == 0) {
            evenCount++;
        }
    }

    // Second pass: collect even numbers
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] % 2 == 0) {
            evenNumbers.add(arr[i]);
        }
    }

    return evenNumbers;
}
```

Algoritme 15.1 gennemløber arrayet én gang for at finde de lige tal. Undervejs tilføjer den hvert lige tal direkte til en resultatliste, så der kun er ét gennemløb på $\mathcal{O}(n)$.

Algoritme 15.2 foretager derimod to fulde gennemløb af arrayet. Det første gennemløb tæller antallet af lige elementer, selv om dette antal ikke bruges bagefter, og det andet gennemløb samler elementerne. Begge algoritmer har køretiden $\Theta(n)$, men det overflødige første gennemløb giver ekstra arbejde. Forholdet mellem dem er ikke blot to, fordi listeoperationer, sammenligninger og andre sætninger også bidrager til køretiden.

I praksis er Algoritme 15.1 hurtigere, fordi den løser opgaven i ét gennemløb, mens Algoritme 15.2 får en større konstant faktor ved at gennemløbe arrayet to gange. Eksemplet viser, hvordan forskellige implementeringer med samme kompleksitet, $\mathcal{O}(n)$, alligevel kan yde forskelligt i praksis på grund af forskellige konstante faktorer.

I dette kapitel analyserer vi systematisk kodens komponenter. Vi begynder med pseudokode, så vi kan se bort fra sprogspecifik syntaks. Derefter undersøger vi kompleksiteten af forskellige kodestrukturer, herunder løkker, betingelser og sekvenser. Til sidst ser vi på praktiske forhold og diskuterer, hvordan konstanter og led af lavere orden, som man ofte ser bort fra i asymptotisk notation, kan påvirke ydelsen i praksis.

15.1 Pseudokode

For at kunne analysere algoritmer effektivt uden at blive tynget af sprogspecifik syntaks bruger vi pseudokode – en forenklet, overordnet fremstilling af kode, der fremhæver en algoritmes logik og struktur. Med pseudokode kan vi beskrive og analysere algoritmer uafhængigt af et bestemt programmeringssprog og udelukkende fokusere på rækkefølgen af operationerne og deres effektivitet. Pseudokode fungerer som et bindeled mellem det konceptuelle algoritmedesign og implementeringen. Fordi sprogets detaljer er skåret væk, gør pseudokode det lettere at se en algoritmes centrale operationer og dermed lettere at vurdere kompleksiteten og optimere ydelsen, før algoritmen

implementeres i kode.

Pseudokode gør det også lettere at samarbejde, fordi teammedlemmer med forskellige sprogkompetencer kan forstå og bidrage til algoritmedesignet uden at kende en bestemt syntaks. Desuden er pseudokode nyttig til dokumentation af algoritmer, da den giver en kortfattet, men klar opsummering af de enkelte trin, som er lettere at læse end fuldt udfoldet kode.

Vi følger konventionerne fra Cormen m.fl., 2022, som består af et bestemt sæt formateringsregler, der sikrer klarhed og ensartethed:

1. **Løkkestrukturer**: Vi bruger `for` og `while` til at angive iterationer:
   - `for` $i = 1$ `to` n betegner en løkke, der øger i med 1 fra 1 til og med n.
   - `while` betingelse betegner en løkke, der fortsætter, så længe den angivne betingelse er sand.
2. **Tildeling og operationer**: Tildelingsoperatoren $=$ angiver, at en variabel tildeles en værdi, fx sætter $sum = 0$ `sum` til nul. De grundlæggende regneoperationer (`+, -, *, /`) skrives med den sædvanlige notation.
3. **Kald af procedurer og funktioner**: Procedurenavne skrives med kapitæler, fx SUM-ARRAY(arr), mens variabler og identifikatorer skrives med kursiv.
4. **Kommentarer**: Kommentarer vises med bordeauxrød farve og giver kontekstuel information om hvert trin.

Lad os som illustration se på en simpel algoritme, der beregner summen af elementerne i et array. Her bruger vi pseudokode til at beskrive algoritmen med fokus på logikken og de enkelte trin.

```
SUM-ARRAY(arr)
1  sum = 0                          // initialise sum to zero
2  for i = 0 to arr.length − 1
3      sum = sum + arr[i]           // Add the current element to sum
4  return sum                       // Return the final sum
```

I denne pseudokode:

- Proceduren SUM-ARRAY tager et array `arr` som input.
- Vi initialiserer en variabel `sum` til nul.
- En `for` -løkke gennemløber hvert element i arrayet og lægger det til `sum`.
- Til sidst returnerer funktionen den beregnede `sum`.

Pseudokoden indfanger algoritmens væsentlige trin uden syntaksspecifikke detaljer, så den er let at analysere og tilpasse. Her er et mere komplekst eksempel:

```
FIND-MAX-EVEN-SUM(arr)
1  maxSum = 0                    // Initialise max sum to zero
2  currentSum = 0                // Track the sum of the current subarray
3  for i = 0 to arr.length − 1
4      if arr[i] mod 2 == 0
5          currentSum = currentSum + arr[i]      // Add even number to current sum
6          maxSum = MAX(maxSum, currentSum)// Update if currentSum > maxSum
7      else
8          currentSum = 0        // Reset current sum if odd number is encountered
9  return maxSum                 // Return the maximum sum of any contiguous even subarray
```

Proceduren beregner den største sum af sammenhængende lige elementer i et array:

- Proceduren tager et inputarray `arr` og initialiserer `maxSum`, der holder styr på den største sum, og `currentSum`, der opsamler summen af det aktuelle sammenhængende delarray af lige tal.
- En enkelt `for` -løkke gennemløber `arr`:
- Hvis et element er lige, lægges det til `currentSum`, og `maxSum` opdateres, hvis `currentSum` overstiger den.
- Hvis et element er ulige, nulstilles `currentSum` til 0, fordi det sammenhængende delarray er afbrudt.

Her er yderligere to eksempler på pseudokode, et for *iterativ binær søgning* og et for *rekursiv binær søgning*. Pseudokoden for de to algoritmer ligner hinanden, men den rekursive version kalder sig selv via et funktionskald, mens den iterative version styrer søgningen med en løkke.

```
BINARY-SEARCH-ITERATIVE(arr, key)
 1  low = 0                       // Initialise low index
 2  high = arr.length − 1         // Initialise high index
 3  while low ≤ high
 4      mid = ⌊(low + high)/2⌋ // Calculate mid index
 5      if arr[mid] == key
 6          return mid            // Key found at mid index
 7      elseif arr[mid] < key
 8          low = mid + 1         // Update low index
 9      else
10          high = mid − 1        // Update high index
11  return −1                     // Key not found
```

Her er en trinvis forklaring af, hvad koden gør:

1. **Initialisering**:
   - `low` initialiseres til det første indeks i arrayet `arr`.
   - `high` initialiseres til det sidste indeks i arrayet `arr`.
2. **While-løkke**:
   - Løkken fortsætter, så længe `low` er mindre end eller lig med `high`.

3. **Beregning af midterindeks**:
   - `mid` beregnes som gennemsnittet af `low` og `high` rundet ned.
4. **Sammenligning**:
   - Hvis elementet på indeks `mid` er lig med `key`, returnerer funktionen `mid`, som angiver positionen af `key` i arrayet.
   - Hvis elementet på indeks `mid` er mindre end `key`, må `key` ligge i arrayets højre halvdel. Derfor opdateres `low` til `mid + 1`.
   - Hvis elementet på indeks `mid` er større end `key`, må `key` ligge i arrayets venstre halvdel. Derfor opdateres `high` til `mid - 1`.
5. **Nøglen findes ikke**:
   - Hvis løkken afsluttes uden at have fundet `key`, returnerer funktionen `-1`, hvilket angiver, at `key` ikke findes i arrayet.

Algoritmen søger effektivt efter en nøgle, `key`, i et sorteret array ved gentagne gange at halvere søgeintervallet. Tidskompleksiteten for binær søgning er $\mathcal{O}(\log n)$, så den er meget hurtigere end lineær søgning for store arrays. Algoritmen er **iterativ**, fordi den bruger en `while` -løkke til at udføre en kodeblok gentagne gange, indtil en bestemt betingelse er opfyldt. Her fortsætter løkken, så længe `low` er mindre end eller lig med `high`. I hver iteration af løkken udføres en sammenligning, og indeksene `low` og `high` opdateres tilsvarende. Denne iterative fremgangsmåde står i kontrast til en rekursiv fremgangsmåde, hvor funktionen kalder sig selv med opdaterede parametre, indtil basisbetingelsen er opfyldt.

```
BINARY-SEARCH-RECURSIVE(arr, key, low, high)
1  if low > high
2      return −1                    // Key not found
3  mid = ⌊(low + high)/2⌋           // Calculate mid index
4  if arr[mid] == key
5      return mid                   // Key found at mid index
6  elseif arr[mid] < key
7      // Search right half
8      return BINARY-SEARCH-RECURSIVE(arr, key, mid + 1, high)
9  else
10     // Search left half
11     return BINARY-SEARCH-RECURSIVE(arr, key, low, mid − 1)
```

Koden implementerer en rekursiv binær søgealgoritme. Funktionen BINARY-SEARCH-RECURSIVE tager et array `arr`, en nøgle `key`, der skal søges efter, og indeksene `low` og `high`, som afgrænser det aktuelle søgeområde.

1. **Basistilfælde**: Hvis `low` er større end `high`, returnerer funktionen -1, hvilket angiver, at nøglen ikke findes i arrayet.
2. **Beregning af midtpunkt**: Midtpunktet `mid` beregnes som gennemsnittet af `low` og `high` rundet ned.
3. **Sammenligning med nøglen**:
   - Hvis elementet på indeks `mid` er lig med `key`, returnerer funktionen `mid`, hvilket angiver, at nøglen er fundet på dette indeks.
   - Hvis elementet på indeks `mid` er mindre end `key`, søger funktionen rekursivt i arrayets højre halvdel

ved at opdatere `low` til `mid + 1`.

- Hvis elementet på indeks `mid` er større end `key`, søger funktionen rekursivt i arrayets venstre halvdel ved at opdatere `high` til `mid - 1`.

Algoritmen søger effektivt efter en nøgle, `key`, i et sorteret array ved gentagne gange at halvere søgeintervallet. Algoritmen er **rekursiv**, fordi den kalder sig selv med opdaterede parametre for at indsnævre søgeområdet. I hvert rekursivt kald beregner funktionen midtpunktet af det aktuelle område og foretager en sammenligning. Afhængigt af resultatet returnerer funktionen enten indekset `mid`, hvis nøglen er fundet, eller kalder sig selv rekursivt på enten venstre eller højre halvdel af det aktuelle interval, idet indekset `low` eller `high` justeres tilsvarende. Rekursionen fortsætter, indtil basisbetingelsen er opfyldt, dvs. indtil nøglen enten er fundet, eller det er fastslået, at den ikke findes. Denne rekursive fremgangsmåde står i kontrast til en iterativ fremgangsmåde, der udfører søgningen med en løkke i stedet for rekursive funktionskald.

15.2 Analyse af kodekonstruktioner

Ved konstruktionsanalyse deler vi koden op i dens grundlæggende komponenter, fx løkker, betingelser og sekvenser, for at forstå, hvordan hver af dem bidrager til den samlede kompleksitet. Ved at analysere disse konstruktioner kan vi finde flaskehalse, ineffektiviteter og steder i en algoritme, der kan optimeres. I dette afsnit undersøger vi kompleksiteten af forskellige kodestrukturer med fokus på løkker, betingelser og sekvenser, og vi ser på, hvordan de påvirker en algoritmes ydelse.

Basisoperationer

Den tid, en funktion eller procedure bruger, er proportional med det antal »basisoperationer«, den udfører. Eksempler på basisoperationer er:

1. **Regneoperation**: En enkelt regneoperation, fx addition (+), multiplikation (∗), subtraktion (-) eller division (/).
2. **Tildeling**: At tildele en variabel en værdi, fx $x = 0$.
3. **Test/sammenligning**: At evaluere en betingelse, fx $x == 0$ eller $i \leq n$.
4. **Funktionskald**: At kalde en funktion eller procedure, fx et kald af FIND-MAX-EVEN-SUM.
5. **Return-sætning**: At returnere en værdi fra en funktion, fx `return total`.
6. **Læseoperation**: At læse en værdi af en primitiv type (fx heltal, kommatal, tegn eller boolesk værdi) fra input eller hukommelsen.
7. **Skriveoperation**: At skrive en værdi af en primitiv type til output eller hukommelsen.

Disse basisoperationer er grundlaget for mere komplekse operationer. I den forenklede *enhedsomkostningsmodel*, som vi bruger i dette kapitel, tildeles hver operation på primitive værdier af fast størrelse en konstant omkostning. Antagelsen gælder ikke uændret for vilkårligt store heltal eller for operationer, hvis omkostning afhænger af, hvordan data er repræsenteret. Ved at tælle operationer i denne model kan vi vurdere en konstruktions køretid. I vores detaljerede optællinger medtager vi kun punkt 1–5 og ser bort fra særskilte omkostninger til læsning og skrivning.

Alle de følgende algoritmer er $\mathcal{O}(1)$-algoritmer, fordi de udfører et konstant antal basisoperationer:

```
INC(x)
1  return x + 1
```

```
MUL(x, y)
1  return x × y
```

```
FOO(x)
1  y = x × 77.3
2  return x/8.2
```

```
BAR(x, y)
1  z = x + y
2  w = x × y
3  q = (w^z) mod 870
4  return 9 × q
```

I det sidste eksempel udfører funktionen BAR en række regneoperationer, tildelinger og en modulo-operation. Hver af disse operationer regnes som en basisoperation, og funktionens tidskompleksitet er $\mathcal{O}(1)$, fordi den udfører et konstant antal basisoperationer. Her er en opdeling:

- **Linje 1:** $z = x + y$
  - **Regneoperation**: 1 (addition)
  - **Tildeling**: 1 (tildeling til z)

  Operationer i alt: 2
- **Linje 2:** $w = x \times y$
  - **Regneoperation**: 1 (multiplikation)
  - **Tildeling**: 1 (tildeling til w)

  Operationer i alt: 2
- **Linje 3:** $q = (w^z) \bmod 870$
  - **Regneoperationer**: 2 (én for potensopløftningen w^z og én for modulo 870)
  - **Tildeling**: 1 (tildeling til q)

  Operationer i alt: 3
- **Linje 4: return** $9 \times q$
  - **Regneoperation**: 1 (multiplikation med 9)
  - **Tildeling (implicit)**: 1 (returnering af resultatet)

  Operationer i alt: 2

Når vi lægger operationerne for hver linje sammen, får vi:

$$2 + 2 + 3 + 2 = 9$$

Det samlede antal basisoperationer er konstant (9 operationer), så dette er en $O(1)$-algoritme.

Sekventielle sætninger

Når en algoritme indeholder flere sætninger, der udføres efter hinanden, kalder man dem »sekventielle sætninger« eller »på hinanden følgende sætninger«. Den samlede tidskompleksitet af sekventielle sætninger afhænger af de enkelte sætningers kompleksitet. Mere præcist er det leddet af højeste orden blandt de enkelte sætninger, der dominerer den samlede kompleksitet.

Se på en sekvens af sætninger:

sætning 1;
sætning 2;
⋮
sætning k;

Den samlede tid for at udføre disse sætninger kan skrives som:

$$\text{Samlet tid} = \text{tid(sætning 1)} + \text{tid(sætning 2)} + \cdots + \text{tid(sætning } k\text{)}$$

Hvis hver sætning er »simpel«, dvs. kun består af basisoperationer, tager hver af sætningerne konstant tid, og den samlede kompleksitet bliver $\mathcal{O}(1)$. Det gælder, når antallet af operationer i hver sætning ikke afhænger af inputstørrelsen – det så vi i forrige afsnit.

I mere generelle tilfælde er det dog den højeste orden blandt de enkelte kompleksiteter, der bestemmer kompleksiteten af de sekventielle sætninger. Hvis fx én sætning har kompleksiteten $\mathcal{O}(n)$ og en anden har kompleksiteten $\mathcal{O}(n^2)$, er den samlede kompleksitet $\mathcal{O}(n^2)$, fordi leddet af højeste orden dominerer for tilstrækkeligt store inputstørrelser.

Som vi har set flere gange, bliver led af lavere orden ubetydelige i asymptotisk analyse, når inputstørrelsen vokser, og derfor medtages de ikke i den endelige angivelse af algoritmens kompleksitet. Når vi vurderer sekventielle sætninger, behøver vi altså kun at se på leddet med den højeste vækstrate for at vurdere den samlede tidskompleksitet.

Simple løkker

Løkker er blandt de vigtigste faktorer for en algoritmes kompleksitet, fordi de ofte bestemmer, hvor mange gange en bestemt operation udføres. En simpel løkke, der kører fra 1 til n, har tidskompleksiteten $\mathcal{O}(n)$. Det skyldes, at løkken udføres n gange, og hvis sekvensen af sætninger i løkken har kompleksiteten $\mathcal{O}(1)$, bliver løkkens samlede tidskompleksitet $n \times \mathcal{O}(1)$, hvilket kan reduceres til $\mathcal{O}(n)$.

Den generelle form for en løkke kan skrives som:

for i i $1, 2, \ldots, N$ løkke
  sekvens af sætninger
slut løkke;

Her udføres løkken N gange, hvilket betyder, at sekvensen af sætninger også udføres N gange. Hvis vi antager, at hver af disse sætninger er en basisoperation med kompleksiteten $\mathcal{O}(1)$, er løkkens samlede tidskompleksitet $\mathcal{O}(N)$.

Et illustrativt eksempel er en funktion, der udfører multiplikation uden at bruge den indbyggede multiplikationsoperator ($*$). I stedet bruger funktionen en løkke til gentagne gange at lægge den første operand til:

```
MUL2(x, y)
1  result = 0
2  for i = 0 to y − 1
3      result = result + x
4  return result
```

I denne funktion kører løkken y gange og udfører en addition i konstant tid i hver iteration. Funktionens kompleksitet er derfor $\mathcal{O}(y)$. Tidskompleksiteten afhænger her direkte af størrelsen af inputtet y, hvilket viser, hvordan løkkegrænserne direkte påvirker en algoritmes ydelse.

Funktioner med simple for-løkker, der gennemløber hele inputtet, har generelt kompleksiteten $\mathcal{O}(n)$, hvor n er inputstørrelsen. Det skyldes, at hvert element i inputtet behandles i rækkefølge, så der er en lineær sammenhæng mellem inputstørrelsen og antallet af udførte operationer.

Se på følgende:

```
FACTORIAL(n)
1  result = 1
2  for num = 1 to n
3      result = result × num
4  return result
```

Koden, der beregner fakultetet af et tal n, består af en enkelt løkke, der gennemløber værdierne fra 1 til n. I hver iteration udføres en multiplikation i konstant tid (result *= num). Løkken kører altså n gange, og hver iteration er en $\mathcal{O}(1)$-operation.

Den samlede tidskompleksitet for funktionen FACTORIAL er derfor $\mathcal{O}(n)$. Se nu på:

```
FACTORIAL2(n)
1  result = 1
2  count = 0
3  for num = 1 to n
4      result = result × num
5      count = count + 1
6  return result
```

Funktionen FACTORIAL2 minder meget om det forrige eksempel, hvor vi beregnede fakultetet af et tal n. Den indeholder en enkelt løkke, der gennemløber værdierne fra 1 til n. I hver iteration udføres to operationer i konstant tid: en multiplikation (result *= num) og en addition (count += 1). Da begge operationer er $\mathcal{O}(1)$, og løkken kører n gange, er den samlede tidskompleksitet stadig $\mathcal{O}(n)$.

Den ekstra tildeling (count += 1) ændrer ikke den *asymptotiske* kompleksitet, da det stadig er en operation i

konstant tid, der gentages n gange. Den samlede kompleksitet forbliver altså $\mathcal{O}(n)$. Den ekstra operation kan dog påvirke funktionens ydelse i praksis, fordi den øger antallet af basisoperationer, der udføres i hver iteration.

If-then-else-sætninger

For en `if-then-else`-struktur afhænger køretiden af omkostningen ved at evaluere betingelsen og af den gren, der udføres. Når vi analyserer det *værste tilfælde*, bruger vi den langsomste gren plus omkostningen ved betingelsen. For et bestemt input medtages kun omkostningen ved den gren, der faktisk udføres.

- **Hvis betingelsen er simpel**, fx en sammenligning af to tal, tager den konstant tid, $\mathcal{O}(1)$.
- **Hvis betingelsen er mere kompleks**, fx en test af, om et element findes i en stor datastruktur, afspejler tidskompleksiteten evalueringen af betingelsen.

Se på følgende pseudokode for en `if-then-else`-sætning:

```
1  if cond
2        block 1 (sequence of statements)
3  else
4        block 2 (sequence of statements)
5
```

Her udføres enten blok 1 eller blok 2. Tiden i værste tilfælde er derfor den langsomste af de to muligheder:

$$\max(\text{tid(blok 1)}, \text{tid(blok 2)})$$

Hvis blok 1 tager $\mathcal{O}(1)$, og blok 2 tager $\mathcal{O}(N)$, er den samlede tidskompleksitet for `if-then-else`-sætningen $\mathcal{O}(N)$.

Kompleksiteten af betingelser afhænger af selve betingelsen. Tiden til at evaluere en betingelse kan være konstant, lineær eller endnu værre, afhængigt af hvilken test der udføres.

Se på følgende eksempler, der illustrerer dette nærmere:

```
COUNT_TS(a_str)
1  count = 0
2  for each char in a_str
3        if char = 't'
4              count = count + 1
5  return count
```

I dette eksempel brugte vi en `if` -sætning til at teste, om ét tegn er lig med et andet, hvilket er en operation i konstant tid. Funktionen kører samlet set i lineær tid i forhold til længden af `a_str`, fordi testen i betingelsen er simpel. Hvis vi lader $n = |\texttt{a_str}|$, har funktionen tidskompleksiteten $\mathcal{O}(n)$.

Se nu på denne kode:

```
COUNT_SAME_LTRS(a_str, b_str)
1  count = 0
2  for each char in a_str
3      if char ∈ b_str
4          count = count + 1
5  return count
```

Koden ligner COUNT_TS, men testen i betingelsen, `char ∈ b_str`, er mere kompleks. Vi analyserer køretiden i værste tilfælde: En medlemskabstest kan blive nødt til at undersøge hvert tegn i `b_str`. Det sker, når `char` ikke findes, fordi søgningen ikke kan returnere `False`, før alle tegn er kontrolleret. Store $\mathcal{O}$ betegner en øvre grænse; valget om at undersøge det værste tilfælde er en separat modelleringsbeslutning.

Lad $n = |\texttt{a_str}|$ og $m = |\texttt{b_str}|$. `for` -løkken udfører $\mathcal{O}(n)$ iterationer, og hver iteration udfører en test i betingelsen på $\mathcal{O}(m)$. Da vi udfører en test på $\mathcal{O}(m)$ i alt $\mathcal{O}(n)$ gange, er funktionens samlede kompleksitet $\mathcal{O}(nm)$.

While-løkker

While-løkker er grundlæggende konstruktioner til at skabe iterative processer i algoritmer. I modsætning til **`for`** -løkker, der gennemløbes et forudbestemt antal gange, fortsætter en **`while`** -løkke, så længe en given betingelse er sand. Denne fleksibilitet gør **`while`** -løkker velegnede i situationer, hvor antallet af iterationer ikke kendes på forhånd, men afhænger af forhold under kørslen.

Strukturen af en **`while`** -løkke kan skrives som:

> så længe betingelsen er sand:
>
> > sekvens af sætninger

- Betingelsen kontrolleres før hver iteration. Hvis den er falsk fra starten, udføres løkkens krop slet ikke.
- Antallet af iterationer afhænger af, hvordan betingelsen udvikler sig og til sidst bliver falsk.

Tidskompleksiteten af en **`while`** -løkke afhænger af kompleksiteten af testen i betingelsen og af det antal iterationer, der skal til, før betingelsen bliver falsk.

Se på følgende eksempel:

```
FACTORIAL3(n)
1  result = 1
2  while n > 0
3      result = result × n
4      n = n − 1
5  return result
```

I dette eksempel fortsætter løkken, så længe `n > 0`. Da `n` formindskes med 1 i hver iteration, kører løkken n gange, hvilket giver tidskompleksiteten $\mathcal{O}(n)$.

Se på et andet eksempel, der bruger en `while` -løkke til at opdele en streng:

```
CHAR-SPLIT(a_str)
1  result = [] // initialise an empty list
2  index = 0
3  while len(a_str) ≠ len(result)
4      append(result, a_str[index])
5      index = index + 1
6  return result
```

I denne funktion er `len(a_str)` og `len(result)` operationer i konstant tid, og det samme gælder indeksering i strengen og tilføjelse til listen. While-løkken kører, indtil længden af `result` svarer til længden af `a_str`, hvilket giver n iterationer, hvor n er inputstrengens længde. Funktionens samlede tidskompleksitet er derfor $\mathcal{O}(n)$.

Se på dette eksempel:

```
SUM-UNTIL-THRESHOLD(arr, threshold)
1  sum = 0
2  i = 0
3  while sum < threshold and i < arr.length
4      sum = sum + arr[i]
5      i = i + 1
6  return sum
```

Her kører løkken, indtil betingelsen `sum < threshold` ikke længere er sand, eller indtil alle elementer i arrayet `arr` er behandlet. Køretiden afhænger både af arrayets indhold og af værdien af `threshold`. I værste tilfælde undersøger løkken hvert element, så køretiden i værste tilfælde er $\Theta(n)$, hvor n er arrayets længde.

Generelt er `while`-løkker stærke konstruktioner til at iterere over betingelser, der ikke kendes på forhånd. Nøglen til at analysere deres kompleksitet er at forstå, hvordan betingelsen udvikler sig i hver iteration, og at vurdere, hvor mange iterationer der skal til, før løkken stopper. En grundig analyse kræver, at man forstår både betingelsens udvikling og omkostningen ved hver iteration.

Indlejrede løkker

Indlejrede løkker er en vigtig konstruktion i algoritmedesign, hvor én løkke kører inde i en anden. Denne type struktur har stor betydning for en algoritmes tidskompleksitet, fordi antallene af iterationer ganges med hinanden.

Se på den generelle form for en indlejret løkke:

for $i = 1$ til n :

sekvens af sætninger

for $j = 1$ til m :

sekvens af sætninger

Ved indlejrede løkker gennemløbes den indre løkke fuldt ud for hver iteration af den ydre løkke. Det betyder, at det samlede antal iterationer er produktet af de enkelte løkkers antal iterationer. Hvis den ydre løkke kører n gange og den indre løkke m gange, udføres den indre løkkes krop i alt $n \times m$ gange. Hvis både m og n er proportionale med inputstørrelsen, er den indlejrede løkkes tidskompleksitet $\mathcal{O}(n \times m)$.

Hvis både den indre og den ydre løkke gennemløber det samme interval, fx fra 1 til n, er den samlede tidskompleksitet for strukturen af indlejrede løkker $\mathcal{O}(n^2)$. Det er ofte tilfældet, når man udfører operationer på en todimensional struktur som en matrix, hvor hver række og søjle skal gennemløbes.

For eksempel:

```
SUM-MATRIX(matrix)
1  sum = 0
2  for i = 0 to matrix.rows − 1
3      for j = 0 to matrix.cols − 1
4          sum = sum + matrix[i][j]
5  return sum
```

I denne pseudokode gennemløber den ydre løkke hver række i matricen, mens den indre løkke gennemløber hver søjle. Hvis matricen har n rækker og m søjler, er tidskompleksiteten af den indlejrede løkke $\mathcal{O}(n \times m)$. For en kvadratisk matrix, hvor $n = m$, bliver kompleksiteten $\mathcal{O}(n^2)$.

I nogle tilfælde afhænger antallet af iterationer i den indre løkke af den ydre løkkes indeks. Et almindeligt eksempel er en trekantet løkkestruktur, hvor den indre løkke kører færre gange, efterhånden som den ydre løkke skrider frem.

Se på følgende pseudokode:

```
TRIANGULAR-SUM(n)
1  sum = 0
2  for i = 1 to n
3      for j = i to n
4          sum = sum + (i + j)
5  return sum
```

Her starter den indre løkke ved den aktuelle værdi af i og kører til n. Det betyder, at den indre løkke i første iteration kører n gange, i anden iteration $n - 1$ gange og så videre, indtil den kun kører én gang. Det samlede

antal iterationer for de indlejrede løkker kan beregnes som (se Bilag B):

$$\sum_{i=1}^{n}(n-i+1) = \frac{n(n+1)}{2}$$

Tidskompleksiteten af denne indlejrede løkke er altså $\mathcal{O}(n^2)$. Den trekantede struktur reducerer det samlede antal iterationer sammenlignet med en fuld $\mathcal{O}(n^2)$-løkke, men kompleksiteten er stadig kvadratisk.

Det samme princip gælder for mere end to indlejrede løkker. Hvis der er k indlejrede løkker, der hver kører n gange, er den samlede tidskompleksitet $\mathcal{O}(n^k)$.

Når man analyserer indlejrede løkker, er det afgørende at forstå hver løkkes interval og grænser. Hvis grænserne er dynamiske og afhænger af værdien af andre variabler, kan kompleksiteten være sværere at bestemme, og det kan kræve en grundigere analyse af, hvordan grænserne ændrer sig under udførelsen.

Indlejrede løkker bruges ofte til problemer med flerdimensionale datastrukturer eller til at sammenligne hvert element i en samling med alle andre elementer (fx i sorteringsalgoritmer som BUBBLE SORT, der har tidskompleksiteten $\mathcal{O}(n^2)$ på grund af sin struktur med indlejrede løkker).

Rekursion

Denne bog og de kurser, den understøtter, behandler ikke rekursion i dybden. Vi omtaler kort rekursion her i forbindelse med konstruktionsanalyse for fuldstændighedens skyld. Vi har allerede set rekursion i forbindelse med binær søgning. Rekursion er et stærkt værktøj i algoritmedesign, hvor funktioner kalder sig selv og løser problemer ved at opdele dem i mindre delproblemer. Rekursive algoritmer er særligt effektive til problemer med en naturlig hierarkisk eller gentagen struktur, fx gennemløb af træer, del og hersk-algoritmer eller beregning af fakulteter.

En central del af enhver rekursiv funktion er **basistilfældet** – den betingelse, hvor funktionen holder op med at kalde sig selv, så man undgår uendelig rekursion.

Se på den generelle struktur af en rekursiv funktion:

- **Basistilfælde**: Den stopbetingelse, der afslutter rekursionen.
- **Rekursivt tilfælde**: Den del af funktionen, der kalder sig selv med et ændret input og dermed nærmer sig basistilfældet.

Et klassisk eksempel på en rekursiv funktion er beregningen af et fakultet:

$$\texttt{factorial}(n) = \begin{cases} 1 & \text{hvis } n = 0 \\ n \times \texttt{factorial}(n-1) & \text{hvis } n > 0 \end{cases}$$

I pseudokode kan fakultetsfunktionen skrives som:

```
FACTORIAL4(n)
1  if n == 0
2      return 1
3  return n × FACTORIAL(n − 1)
```

I denne pseudokode:

- **Basistilfældet** indtræffer, når $n == 0$, og returnerer 1.
- **Det rekursive tilfælde** ganger n med resultatet af FACTORIAL4$(n-1)$.

Tidskompleksiteten af en rekursiv funktion afhænger af, hvor mange gange funktionen kalder sig selv, og hvor meget arbejde der udføres på hvert rekursionsniveau. For fakultetsfunktionen gælder:

- Funktionen foretager ét rekursivt kald i hvert trin og formindsker n med 1 hver gang.
- Rekursionsdybden er n, og hvert kald udfører $\mathcal{O}(1)$ arbejde (en multiplikation).

Fakultetsfunktionens tidskompleksitet er altså $\mathcal{O}(n)$.

Til at bestemme tidskompleksiteten af en rekursiv algoritme kan vi bruge rekursionsligninger. En rekursionsligning udtrykker en rekursiv funktions tidskompleksitet ved hjælp af dens delproblemer. Rekursionsligningen for fakultetsfunktionen kan fx skrives som:

$$T(n) = T(n-1) + \mathcal{O}(1)$$

Det betyder, at tiden til at beregne `factorial`(n) er lig med tiden til at beregne `factorial`$(n-1)$ plus en konstant mængde arbejde, hvilket giver kompleksiteten $\mathcal{O}(n)$.

Et andet velkendt eksempel på en rekursiv funktion er Fibonacci-følgen:

$$\texttt{fibonacci}(n) = \begin{cases} 0 & \text{hvis } n = 0 \\ 1 & \text{hvis } n = 1 \\ \texttt{fibonacci}(n-1) + \texttt{fibonacci}(n-2) & \text{hvis } n > 1 \end{cases}$$

Den tilsvarende pseudokode er:

```
NAIVE-FIBONACCI(n)
1  if n == 0
2      return 0
3  if n == 1
4      return 1
5  return FIBONACCI(n − 1) + FIBONACCI(n − 2)
```

I dette eksempel:

- Funktionen kalder sig selv to gange i det rekursive tilfælde, hvilket skaber en forgrenet struktur.
- **Tidskompleksiteten** er $\mathcal{O}(2^n)$, som vokser eksponentielt, fordi de samme delproblemer løses mange gange. Det viser, hvor ineffektiv den naive rekursive implementering af Fibonacci er.

For at gøre rekursive funktioner, der løser overlappende delproblemer som i Fibonacci-følgen, mere effektive kan man bruge **memoisering**. Memoisering går ud på at gemme resultaterne af delproblemer for at undgå overflødige beregninger.

Hvis man fx anvender memoisering på Fibonacci-funktionen, ændres tidskompleksiteten fra $\mathcal{O}(2^n)$ til $\mathcal{O}(n)$, fordi hvert delproblem kun løses én gang.

Rekursion kan ofte erstattes af iteration, især for problemer, hvor rekursionsdybden kan blive et problem. Fakultetsfunktionen kan fx omskrives iterativt, som vi så tidligere. Her er den iterative version af fakultetsfunktionen:

```
FACTORIAL-ITERATIVE(n)
1  result = 1
2  for i = 1 to n
3      result = result × i
4  return result
```

Den iterative fremgangsmåde har samme tidskompleksitet, $\mathcal{O}(n)$, som den rekursive version, men undgår omkostningerne ved rekursive kald og er ikke begrænset af systemets stakstørrelse.

Halerekursion er en særlig form for rekursion, hvor det rekursive kald er den sidste operation i funktionen. I nogle sprog kan compileren optimere halerekursive funktioner til iteration, så de ikke bruger ekstra plads på stakken. Se på følgende halerekursive fakultetsfunktion:

```
FACTORIAL-TAIL(n, accumulator)
1  if n == 0
2      return accumulator
3  return FACTORIAL-TAIL(n − 1, accumulator × n)
```

Her står det rekursive kald i **haleposition**, hvilket betyder, at der ikke skal foretages yderligere beregninger, efter at kaldet har returneret. Halerekursion mindsker risikoen for *stakoverløb* (på engelsk *stack overflow*) og bliver ofte optimeret til en løkke af compileren.

Rekursion er et alsidigt værktøj i algoritmedesign, der giver elegante løsninger på komplekse problemer ved at opdele dem i mindre og mere håndterbare delproblemer. Den indebærer dog afvejninger med hensyn til plads- og tidskompleksitet, især ved dyb eller eksponentiel rekursion.

Analyse af rekursive algoritmer kræver, at man forstår de rekursionsligninger, der beskriver deres opførsel, identificerer basistilfældene og vurderer, hvor mange gange funktionen bliver kaldt. Rekursion kan give klar og kortfattet kode, men ofte er en iterativ fremgangsmåde eller optimeringsteknikker som memoisering og optimering af halekald nødvendige for at forbedre effektiviteten.

15.3 Analyse af løkker

I forrige afsnit har vi allerede diskuteret løkkers tidskompleksitet. I dette afsnit analyserer vi løkker mere detaljeret, herunder indlejrede løkker, løkker med variable grænser og løkker med betingede udgange.

Når der i denne bog bedes om en klassifikation med store $\mathcal{O}$, menes der normalt den mest informative standardgrænse opadtil blandt de almindelige vækstklasser, der er i spil. Selv om $\mathcal{O}(n^2)$ er en gyldig øvre grænse for $f(n) = 4n + 3$, er $\mathcal{O}(n)$ altså mere informativ. Strengt taget udtrykker store $\mathcal{O}$ alene ikke nødvendigvis, at grænsen er stram; $f(n) = \Theta(n)$ angiver både en øvre og en nedre asymptotisk grænse, der passer sammen.

Teknikker til praktisk analyse af løkker

Nu hvor vi har set de forskellige typer løkker og deres generelle bidrag til kompleksiteten, er det tid til at gå i dybden med praktiske teknikker til at analysere disse konstruktioner i virkelige situationer. Dette afsnit giver

konkrete metoder, som læseren kan bruge til systematisk at vurdere løkkerne i sine algoritmer, med fokus på både de teoretiske og de praktiske sider af analysen.

Bestemmelse af løkkegrænser

For at bestemme en løkkes tidskompleksitet begynder man med at fastlægge grænserne for gennemløbene. Det indebærer, at man identificerer startpunktet, slutpunktet og mønstret for, hvordan løkkevariablerne ændres. Løkkegrænserne fastlægger den øvre grænse for antallet af iterationer, så vi kan udlede det værste tilfælde for, hvor mange gange en løkke udføres.

Se fx på en løkke af formen:

for $i = a$ til b med skridt c :

Antallet af iterationer i denne løkke kan beregnes som:

$$\left(\frac{b-a}{c}\right)+1$$

hvor a er startværdien, b er slutværdien, og c er skridtlængden. En analyse af disse grænser hjælper med at bestemme løkkens samlede kompleksitet.

De følgende eksempler illustrerer begrebet.

Eksempel 15.1

```
LOOP-CONSTANT-INCREMENT(n)
1  for i = 1 to n
2      // Perform some O(1) operation
```

I dette eksempel kører løkken fra a til b med skridtlængden 1. Antallet af iterationer er:

$$\left(\frac{b-a}{1}\right)+1=(b-a+1)=(n-1+1)=n$$

I dette eksempel starter i ved 1 og øges med 1 indtil n. Løkken udføres n gange, hvilket giver tidskompleksiteten $\mathcal{O}(n)$.

Eksemplet ovenfor viser direkte, hvordan formlen for antallet af iterationer bruges, når skridtlængden er konstant. Her er $a=1$, $b=n$ og $c=1$, hvilket giver n iterationer.

Eksempel 15.2

```
LOOP-GEOMETRIC-GROWTH(n)
1  i = 1
2  while i ≤ n
3      // Perform some O(1) operation
4      i = 2 × i
```

I dette eksempel starter i ved 1 og fordobles i hver iteration, indtil den overstiger n. Antallet af iterationer kan bestemmes således:

Værdien af i udvikler sig som:

$$1, 2, 4, 8, \ldots, 2^k$$

hvor $2^k \leq n$. For at bestemme antallet af iterationer løser vi:

$$2^k \leq n \implies k \leq \log_2(n)$$

Antallet af iterationer er altså cirka $\log_2(n)$, hvilket giver tidskompleksiteten:

$$\mathcal{O}(\log n)$$

I dette eksempel er ændringsmønstret multiplikativt i stedet for additivt. Selv om den tidligere formel kun gælder direkte for additive skridt, gælder princippet om at analysere løkkegrænserne for at bestemme antallet af iterationer stadig. Her bestemmer vi, hvor mange gange vi kan fordoble i, før den overstiger n, hvilket giver en logaritmisk kompleksitet.

Eksempel 15.3

```
LOOP-DECREMENT(n)
1  i = n
2  while i > 0
3      // Perform some O(1) operation
4      i = i − 3
```

I dette eksempel starter i ved n og formindskes med 3 hver gang, indtil den bliver mindre end eller lig med 0. Antallet af iterationer er givet ved:

$$i : n, (n-3), (n-6), \ldots$$

Løkken fortsætter, indtil $i \leq 0$. For at bestemme antallet af iterationer skal vi se på, hvor mange gange vi kan trække 3 fra n:

$$\frac{n}{3}$$

Antallet af iterationer er altså cirka:

$$\left(\frac{n}{3}\right)$$

Løkkens samlede tidskompleksitet er:

$$\mathcal{O}(n)$$

I dette eksempel er skridtlængden -3. Med den generelle formel fra tidligere har vi $a = n$, $b = 0$ og $c = -3$. Det giver et antal iterationer på cirka $n/3$. Princippet om at fastlægge grænserne og bruge formlen gælder altså direkte her og viser, hvordan formindskelsen påvirker antallet af iterationer.

Kompleksitet gennem summation

I algoritmer med flere løkker efter hinanden er den samlede kompleksitet summen af de enkelte løkkers kompleksitet. Når løkker er indlejrede, er kompleksiteten derimod produktet af de enkelte kompleksiteter. To løkker efter hinanden, der hver har kompleksiteten $\mathcal{O}(n)$, giver fx stadig $\mathcal{O}(n)$, mens en indlejret struktur giver $\mathcal{O}(n^2)$. Det er afgørende at forstå denne forskel.

Ved indlejrede løkker er det ofte nødvendigt at bestemme det samlede antal operationer ved hjælp af summation. Hvis den indre løkke afhænger af den ydre løkkes værdi, er det nyttigt at opskrive den sum, der angiver antallet af iterationer for alle løkkerne. For eksempel:

for $i = 1$ til n :

  for $j = 1$ til i :

Det samlede antal iterationer er givet ved summen (se Bilag B):

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \mathcal{O}(n^2)$$

Denne type analyse er særligt nyttig for indlejrede løkker med afhængige intervaller. De følgende eksempler illustrerer begrebet.

Eksempel 15.4

```
LINEAR-SUM-LARGE-CONSTANT(n)
1  for i = 1 to n
2      for j = 1 to 10^9
3          // Perform some O(1) operation
```

I dette eksempel kører den ydre løkke fra 1 til n, mens den indre løkke kører et konstant antal gange (fra 1 til 10^9). Det samlede antal iterationer er:

$$\sum_{i=1}^{n} 10^9 = 10^9 \times n = \mathcal{O}(n)$$

Selv om den indre løkke har et meget stort konstant antal iterationer, er den samlede kompleksitet stadig lineær i n, fordi den konstante faktor ikke påvirker den asymptotiske vækst.

Eksempel 15.5

```
QUADRATIC-SUM(n)
1  for i = 1 to n
2      for j = 1 to i
3          // Perform some O(1) operation
```

I dette eksempel kører den ydre løkke fra 1 til n, og den indre løkke kører fra 1 til i. Det samlede antal iterationer

er:

$$\sum_{i=1}^{n}\sum_{j=1}^{i} 1 = \sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \mathcal{O}(n^2)$$

I dette udtryk gælder:

- Den ydre sum, $\sum_{i=1}^{n}$, svarer til den ydre løkkes iterationer over i.
- Den indre sum, $\sum_{j=1}^{i}$, svarer til den indre løkkes iterationer for hvert fast i.

Tallet 1 i den indre sum svarer til, at der udføres en operation i konstant tid i hver iteration af den indre løkke. Udtrykket tæller hver iteration af den indre løkke enkeltvis for alle par af i og j, der opfylder $1 \leq j \leq i$. Dobbeltsummen svarer til $\sum_{i=1}^{n} i$, fordi den indre løkke for hvert i kører præcis i gange. For at se, hvorfor den enkelte sum svarer til dobbeltsummen, kan vi prøve at implementere den enkelte sum:

```
SUM-UP-TO(n)
1  total = 0
2  for i = 1 to n
3      total = total + i
4  return total
```

I denne kode:

- Den ydre løkke kører fra 1 til n.
- I hver iteration lægges den aktuelle værdi af i til `total`.
- Når løkken er færdig, indeholder `total` summen $1 + 2 + \cdots + n$, som netop er $\sum_{i=1}^{n} i$.

Summen $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$ vokser proportionalt med n^2, men tidskompleksiteten af funktionen SUM-UP-TO(N) er stadig lineær, $\mathcal{O}(n)$, fordi den udfører en operation i konstant tid i hver af sine n iterationer. Det viser, at en algoritmes output kan have en talværdi, der vokser kvadratisk med n, mens dens tidskompleksitet afhænger af antallet af operationer, som vokser lineært.

Eksempel 15.6

```
LOGARITHMIC-GROWTH(n)
1  for i = 1 to n
2      j = 1
3      while j < n
4          // Perform some O(1) operation
5          j = 2 × j
```

I dette eksempel kører den ydre løkke fra 1 til n, mens den indre løkke starter ved 1 og fordobler j, indtil den når eller overstiger n. Det samlede antal iterationer af den indre løkke over alle iterationer af den ydre løkke er:

$$\sum_{i=1}^{n} \lceil \log_2 n \rceil = n \lceil \log_2 n \rceil = \mathcal{O}(n \log n)$$

Den samlede tidskompleksitet for denne indlejrede struktur er altså $\mathcal{O}(n \log n)$.

Kompleksitet i værste, bedste og gennemsnitligt tilfælde

Ikke alle løkker kører lige mange gange i alle situationer. Opførslen i bedste, værste og gennemsnitligt tilfælde kan variere afhængigt af den betingelse, der styrer, hvornår løkken stopper. Hvis en løkke fx afsluttes tidligt, fordi en betingelse er opfyldt, kan kompleksiteten i bedste tilfælde være betydeligt mindre end i værste tilfælde. Det er vigtigt at forstå disse tilfælde, og hvordan de påvirker køretiden, i virkelige applikationer, hvor inputtet kan variere.

Eksempel 15.7

```
FIND-TARGET(arr, target)
1  n = arr.length
2  for i = 1 to n
3      if arr[i] == target
4          return i // Return the index of the target if found
```

I dette eksempel er målet at finde et bestemt element (*target*) i et array af længde n. Hvor mange iterationer løkken udfører, afhænger af elementets position:

- **Kompleksitet i bedste tilfælde:** Elementet findes på den første position i arrayet. Løkken kører kun én gang, hvilket giver kompleksiteten $\boldsymbol{\Theta(1)}$.[1]
- **Kompleksitet i værste tilfælde:** Elementet findes ikke i arrayet eller står på den sidste position. Løkken kører n gange, hvilket giver kompleksiteten $\boldsymbol{\mathcal{O}(n)}$.
- **Kompleksitet i gennemsnitligt tilfælde:** I gennemsnit kan elementet stå et sted midt i arrayet. Hvis vi antager en ligefordeling, er det gennemsnitlige antal iterationer cirka $n/2$, hvilket giver kompleksiteten $\boldsymbol{\Theta(n)}$ i gennemsnitligt tilfælde.

Eksemplet viser, hvordan den samme løkke kan opføre sig forskelligt afhængigt af inputtet, så der er forskellige kompleksiteter i bedste, værste og gennemsnitligt tilfælde.

Eksempel 15.8

Se på den iterative binære søgealgoritme, som vi omtalte tidligere i kapitlet, anvendt på et sorteret array af længde n. Binær søgning er en del og hersk-algoritme, der virker ved gentagne gange at halvere søgeintervallet.

[1] Man kunne tro, at den nedre grænse er $\Omega(1)$. I bedste tilfælde, hvor elementet står på den første position, udfører algoritmen en konstant mængde arbejde. Tidskompleksiteten er både $\boldsymbol{\mathcal{O}(1)}$ (øvre grænse) og $\boldsymbol{\Omega(1)}$ (nedre grænse), hvilket betyder, at den er $\boldsymbol{\Theta(1)}$ (en stram grænse). Det er mere præcist at bruge $\boldsymbol{\Theta(1)}$, fordi det angiver, at tiden er begrænset af en konstant både opadtil og nedadtil.

```
BINARY-SEARCH-ITERATIVE(arr, key)
1  low = 0                          // Initialise low index
2  high = arr.length − 1            // Initialise high index
3  while low ≤ high
4      mid = ⌊(low + high)/2⌋       // Calculate mid index
5      if arr[mid] == key
6          return mid               // Key found at mid index
7      elseif arr[mid] < key
8          low = mid + 1            // Update low index
9      else
10         high = mid − 1           // Update high index
11 return −1                        // Key not found
```

- **Kompleksitet i bedste tilfælde:** I bedste tilfælde står det søgte element midt i arrayet og findes ved den allerførste sammenligning. Da det kun kræver én operation, er kompleksiteten $\boldsymbol{\Theta(1)}$. Vi bruger denne notation, fordi den giver en stram grænse – algoritmen bruger altid konstant tid i bedste tilfælde.
- **Kompleksitet i værste tilfælde:** I værste tilfælde bliver algoritmen ved med at halvere søgeintervallet, indtil der kun er én kandidat tilbage, eller intervallet er tomt. Antallet af iterationer vokser logaritmisk, så køretiden i værste tilfælde er $\boldsymbol{\Theta(\log n)}$.
- **Kompleksitet i gennemsnitligt tilfælde:** Betinget af, at søgningen lykkes, og at det søgte element med lige stor sandsynlighed står på hver position i arrayet, er det forventede antal sammenligninger proportionalt med $\log n$. Under denne model er den forventede køretid derfor $\boldsymbol{\Theta(\log n)}$. En model, der også tillader mislykkede søgninger, må angive deres sandsynlighed, men giver samme asymptotiske orden under almindelige antagelser om en fast sandsynlighed.

Eksempel 15.9

Se på den naive strengsøgningsalgoritme (*Naïve String Matching*) anvendt på en tekststreng T af længde n og en mønsterstreng P af længde m. Målet er at finde alle forekomster af P i T ved at teste for et match ved hver mulig forskydning.

```
NAIVE-STRING-MATCHING(T, P)
1  n = T.length
2  m = P.length
3  for s = 0 to n − m
4      i = 0
5      while i < m and T[s + i] == P[i]
6          i = i + 1
7      if i == m
8          report s // Pattern found at position s
```

- **Kompleksitet i bedste tilfælde:** I bedste tilfælde fejler den første tegnsammenligning ved hver forskyd-

ning. Algoritmen tester stadig alle $n-m+1$ forskydninger, så køretiden i bedste tilfælde er $\boldsymbol{\Theta(n-m+1)}$ (og dermed $\Theta(n)$, når $m \leq cn$ for et fast $c < 1$).

- **Kompleksitet i værste tilfælde:** I værste tilfælde indeholder teksten og mønstret gentagne sekvenser, der tvinger algoritmen til at foretage m sammenligninger ved stort set hver forskydning. Køretiden i værste tilfælde er derfor $\boldsymbol{\Theta((n-m+1)m)}$, som normalt skrives $\Theta(nm)$, når m ikke ligger så tæt på n, at det ændrer den første faktor.
- **Kompleksitet i gennemsnitligt tilfælde:** Under en model med uafhængige tegn over et fast alfabet med mindst to symboler er det forventede antal sammenligninger pr. forskydning begrænset af en konstant. Den forventede køretid er derfor $\boldsymbol{\Theta(n-m+1)}$, som ofte forenkles til $\Theta(n)$ under samme konvention som ovenfor.

Eksemplet viser også, at bedste, værste og gennemsnitligt tilfælde beskriver forskellige køretidsfunktioner, mens $\mathcal{O}$, Ω og Θ beskriver grænser for den funktion, der analyseres.

Eksempler på analyse af løkker

Vi giver nu en række eksempler, der illustrerer, hvordan man analyserer løkker.

Eksempel 15.10

Se på følgende algoritme:

```
LOOP1(n)
1  i = 1
2  while i ≤ n
3        i = 3 × i
```

I hver iteration af `while` -løkken ganges variablen i med 3. Vi vil bestemme, hvor mange gange løkken udføres, før i overstiger n.

Lad k være det antal iterationer, løkken udfører.

Til at begynde med er $i_0 = 1$.

Efter første iteration:

$$i_1 = 3 \times i_0 = 3 \times 1 = 3$$

Efter anden iteration:

$$i_2 = 3 \times i_1 = 3 \times 3 = 3^2 = 9$$

Efter k iterationer:

$$i_k = 3^k \times i_0 = 3^k$$

Løkken fortsætter, så længe $i_k \leq n$. Betingelsen for, at løkken stopper, er derfor:

$$3^k > n$$

Vi tager logaritmen med grundtal 3 på begge sider:

$$k > \log_3 n$$

Da k skal være et helt tal, er antallet af iterationer:

$$k = \lfloor \log_3 n \rfloor + 1$$

Konklusion: Løkken kører $\lfloor \log_3 n \rfloor + 1$ gange, så algoritmens tidskompleksitet er $\mathcal{O}(\log n)$.

Eksempel 15.11

Se på følgende algoritme:

```
LOOP2(n)
1  s = 1
2  for i = 1 to n
3      for j = 1 to n
4          s = s + 1
```

Vi har to indlejrede `for` -løkker:

- Den ydre løkke kører fra $i = 1$ til $i = n$ og udfører n iterationer.
- Den indre løkke kører fra $j = 1$ til $j = n$ og udfører ligeledes n iterationer for hver iteration af den ydre løkke.

I hver iteration af den indre løkke udføres en operation i konstant tid: $s = s + 1$.

Samlet antal iterationer:

- Den indre løkke kører n gange for hver af den ydre løkkes n iterationer.
- Det samlede antal gange, $s = s + 1$ udføres, er derfor:
  $$n \times n = n^2$$

Konklusion: Algoritmen udfører omkring n^2 operationer, så tidskompleksiteten er $\mathcal{O}(n^2)$.

Eksempel 15.12

Se på følgende algoritme:

```
LOOP3(n)
1  i = 1
2  while i ≤ n
3      i = 2 × i
```

I hver iteration af `while` -løkken ganges variablen i med 2. Vi skal bestemme, hvor mange gange løkken udføres, før i overstiger n.

Lad k være det antal iterationer, løkken udfører.

- **Initialisering:** $i_0 = 1$.
- **Opdatering i hver iteration:** Efter hver iteration opdateres i som:
  $$i_k = 2^k \times i_0 = 2^k$$
- **Stopbetingelse:** Løkken fortsætter, så længe $i_k \leq n$. Altså:
  $$2^k \leq n$$
- **Løsning med hensyn til k:** Vi tager logaritmen med grundtal 2 på begge sider:
  $$k \leq \log_2 n$$
- **Antal iterationer:** Da k skal være et helt tal, udføres løkken:
  $$k = \lfloor \log_2 n \rfloor + 1$$

Konklusion: Løkken kører $\lfloor \log_2 n \rfloor + 1$ gange, så algoritmens tidskompleksitet er $\mathcal{O}(\log n)$.

Eksempel 15.13

Se på følgende algoritme:

```
LOOP4(n)
1  i = 1
2  while i ≤ n × n
3        i = 3 × i
```

I hver iteration af `while` -løkken ganges variablen i med 3. Vi skal bestemme, hvor mange gange løkken udføres, før i overstiger n^2.

Lad k være det antal iterationer, løkken udfører.

- **Initialisering:** $i_0 = 1$.
- **Opdatering i hver iteration:** Efter hver iteration:
  $$i_k = 3^k \times i_0 = 3^k$$
- **Stopbetingelse:** Løkken fortsætter, så længe $i_k \leq n^2$. Altså:
  $$3^k \leq n^2$$
- **Løsning med hensyn til k:** Vi tager logaritmen med grundtal 3 på begge sider:
  $$k \leq \log_3 n^2$$
- **Reduktion:** Da $\log_3 n^2 = 2 \log_3 n$:
  $$k \leq 2 \log_3 n$$
- **Antal iterationer:** Løkken udføres:
  $$k = \lfloor 2 \log_3 n \rfloor + 1$$

Konklusion: Løkken kører $\lfloor 2 \log_3 n \rfloor + 1$ gange, så algoritmens tidskompleksitet er $\mathcal{O}(\log n)$.

Eksempel 15.14

Se på følgende algoritme:

```
LOOP5(n)
1  i = 1
2  while i ≤ n
3      j = 0
4      while j ≤ n
5          j = j + 1
6      i = 2 × i
```

Vi skal bestemme det samlede antal gange, den indre sætning $j = j + 1$ udføres.

- **Ydre løkke (`while` $i \leq n$):**
  - Variablen i starter ved 1 og fordobles hver gang: $i = 1, 2, 4, 8, \ldots$.
  - Antallet af iterationer i den ydre løkke er:
    $$k = \lfloor \log_2 n \rfloor + 1$$
  - Det skyldes, at løkken fortsætter, så længe $i \leq n$, og at i vokser eksponentielt.
- **Indre løkke (`while` $j \leq n$):**
  - For hver iteration af den ydre løkke initialiseres j til 0.
  - Den indre løkke øger j fra 0 til n og udføres $n + 1$ gange.
- **Operationer i alt:**
  - Den indre løkke kører $n + 1$ gange pr. iteration af den ydre løkke.
  - Det samlede antal gange, $j = j + 1$ udføres:
    $$(\lfloor \log_2 n \rfloor + 1) \times (n + 1)$$
  - Ved reduktion får vi:
    $$\mathcal{O}((\log n) \times n) = \mathcal{O}(n \log n)$$

Konklusion: Algoritmen udfører $(\lfloor \log_2 n \rfloor + 1) \times (n + 1)$ operationer, så tidskompleksiteten er $\boldsymbol{\mathcal{O}(n \log n)}$.

Eksempel 15.15

Se på følgende algoritme:

```
LOOP6(n)
1  i = 1
2  while i ≤ n
3      j = i
4      while j ≤ n
5          j = j + 1
6      i = 2 × i
```

Fuldstændig analyse af tidskompleksiteten:

Vi skal bestemme det samlede antal gange, den indre sætning $j = j + 1$ udføres.

- **Ydre løkke (`while` $i \leq n$):**
  - Variablen i starter ved 1 og fordobles hver gang: $i = 1, 2, 4, 8, \ldots$.

- Antallet af iterationer i den ydre løkke er:
$$k_{\max} = \lfloor \log_2 n \rfloor + 1$$
- Det skyldes, at løkken fortsætter, så længe $i \leq n$, og at i vokser eksponentielt.

- **Indre løkke (`while` $j \leq n$):**
  - For hver iteration af den ydre løkke initialiseres j til den aktuelle værdi af i.
  - Den indre løkke øger j fra i til n og udføres $n - i + 1$ gange.
- **Operationer i alt:**
  - Det samlede antal gange, $j = j + 1$ udføres, er:
$$\text{I alt} = \sum_{k=0}^{k_{\max}-1} (n - i_k + 1)$$
hvor $i_k = 2^k$.
  - Vi indsætter i_k:
$$\text{I alt} = \sum_{k=0}^{\lfloor \log_2 n \rfloor} (n - 2^k + 1)$$
  - Vi reducerer summen:
$$\text{I alt} = \sum_{k=0}^{\lfloor \log_2 n \rfloor} (n + 1 - 2^k)$$
$$= (\lfloor \log_2 n \rfloor + 1)(n + 1) - \sum_{k=0}^{\lfloor \log_2 n \rfloor} 2^k$$
  - Vi beregner summen af 2^k:
$$\sum_{k=0}^{\lfloor \log_2 n \rfloor} 2^k = 2^{\lfloor \log_2 n \rfloor + 1} - 1 \leq 2^{\log_2 n + 1} - 1 = 2n - 1$$
  - Derfor:
$$\text{I alt} = (\lfloor \log_2 n \rfloor + 1)(n + 1) - (2^{\lfloor \log_2 n \rfloor + 1} - 1)$$
$$\leq (\log_2 n + 1)(n + 1).$$
  - For store n dominerer leddet $(\log_2 n)(n + 1)$, så:
$$\text{I alt} = \mathcal{O}(n \log n)$$

Konklusion: Algoritmen udfører højst $(\log_2 n + 1)(n + 1)$ iterationer af den indre løkke, så tidskompleksiteten er $\boldsymbol{\mathcal{O}(n \log n)}$.

Forenklet analyse af tidskompleksiteten:

Vi kan forenkle analysen ved at undersøge den ydre og den indre løkkes opførsel hver for sig.

- **Ydre løkke (`while` $i \leq n$):**
  - Variablen i starter ved 1 og fordobles hver gang:
$$i = 1, 2, 4, 8, \ldots, n$$
  - Fordoblingen giver et logaritmisk antal iterationer:
$$\text{Antal iterationer i den ydre løkke} = \mathcal{O}(\log n)$$
- **Indre løkke (`while` $j \leq n$):**
  - For hver værdi af i starter j ved i og øges med 1, indtil $j > n$.
  - Antallet af iterationer i den indre løkke for et givet i er:
$$n - i + 1 \leq n$$
  - Det betyder, at den indre løkke kører cirka n gange for hver værdi af i.
  - Den indre løkke kører derfor $\mathcal{O}(n)$ gange for hver iteration af den ydre løkke.

- **Samlet kompleksitet:**
  - Den ydre løkke kører $\mathcal{O}(\log n)$ gange.
  - For hver iteration af den ydre løkke kører den indre løkke $\mathcal{O}(n)$ gange.
  - Det samlede antal operationer er derfor:
    $$\mathcal{O}(\log n) \times \mathcal{O}(n) = \mathcal{O}(n \log n)$$

15.4 Analyse af kode

Vi giver nu en række eksempler, der illustrerer, hvordan man analyserer kodestykker. Al koden er Java-kode, men principperne gælder for ethvert programmeringssprog.

Eksempel 15.16

Algoritme 15.3: Simpel funktion

```
1  public static int sumSquareDifference(int n) {
2      int sum = (n * (n + 1)) / 2;
3      int sumOfSquares = (n * (n + 1) * (2 * n + 1)) / 6;
4      return (sum * sum) - sumOfSquares;
5  }
```

Linje 2: `int sum = (n * (n + 1)) / 2;`

- **Tildelinger**:
  - Tildeling `sum` = ... : **1 tidsenhed**
- **Regneoperationer**:
  - Multiplikation `n * (n + 1)`: **1 tidsenhed**
  - Addition `n + 1`: **1 tidsenhed**
  - Division `(n * (n + 1)) / 2`: **1 tidsenhed**
- **I alt for linje 2**: **4 tidsenheder**

Linje 3: `int sumOfSquares = (n * (n + 1) * (2 * n + 1)) / 6;`

- **Tildelinger**:
  - Tildeling `sumOfSquares` = ... : **1 tidsenhed**
- **Regneoperationer**:
  - Multiplikation `n * (n + 1)`: **1 tidsenhed**
  - Addition `n + 1`: **1 tidsenhed**
  - Multiplikation `(n * (n + 1)) * (2 * n + 1)`: **1 tidsenhed**
  - Multiplikation `2 * n`: **1 tidsenhed**
  - Addition `2 * n + 1`: **1 tidsenhed**
  - Division `(...) / 6`: **1 tidsenhed**
- **I alt for linje 3**: **7 tidsenheder**

Linje 4: `return (sum * sum) - sumOfSquares;`

- **Regneoperationer**:
  - Multiplikation `sum * sum`: **1 tidsenhed**

    - Subtraktion `(sum * sum) - sumOfSquares`: **1 tidsenhed**
- **Returnering** (skrivning til output eller returværdi):
    - Returværdi: **1 tidsenhed**
- **I alt for linje 4**: **3 tidsenheder**

Opsummering af den samlede tid (T):

- **Linje 2**: 4 tidsenheder
- **Linje 3**: 7 tidsenheder
- **Linje 4**: 3 tidsenheder

Den samlede tid T for at udføre funktionen `sumSquareDifference` er altså:

$$T = 4 + 7 + 3 = 14 \text{ tidsenheder}$$

Med denne detaljerede analyse kan vi beregne tiden T præcist ved at tælle alle de enkelte operationer, og vi ser, at tidskompleksiteten er $\mathcal{O}(1)$.

Vi kunne også skrive analysen som kodekommentarer i funktionen for at dokumentere tidskompleksiteten af hver linje.

Algoritme 15.4: Simpel funktion

```
1       public static int sumSquareDifference(int n)
2       {
3           int sum = (n * (n + 1)) / 2;     // 4 time units
4           int sumOfSquares = (n * (n + 1) * (2 * n + 1)) / 6; // 7 time units
5           return (sum * sum) - sumOfSquares;  // 3 time units
6       } // Total time: T(n) = 4 + 7 + 3 = 14 time units so we get O(1)
```

Vi foretrækker den sidste metode, fordi den giver en klar og kortfattet måde at dokumentere funktionens eller metodens tidskompleksitet på direkte i koden.

Eksempel 15.17

I det følgende eksempel præsenterer vi en algoritme, der udfører iterativ binær søgning.

Algoritme 15.5: Iterativ binær søgning

```
1   public static int binarySearchIter(int[] a, int x) {
2       int low = 0, high = a.length - 1;
3
4       while (low <= high) {
5           int mid = (low + high) / 2;
6           if (x == a[mid]) {
7               return mid;
8           }
9           else if (x < a[mid]) {
10              high = mid - 1;
11          }
12          else {
13              low = mid + 1;
```

```
            }
        }
        return -1;
    }
```

Algoritme 15.6: Iterativ binær søgning

```
    public static int binarySearchIter(int[] a, int x) {
        int low = 0;
        int high = a.length - 1; // 2 time units (assignment: 1, subtraction: 1)

        // Loop runs log(n) times and makes 1 comparison per iteration
        while (low <= high) {
            int mid = (low + high) / 2; // For each iteration: 1 addition, 1 division, 1
                assignment (3 time units)

            if (x == a[mid]) { // Comparison to check if element is found (1 time unit per
                iteration)
                return mid; // 1 time unit (return)
            } else if (x < a[mid]) { // Comparison to decide which half to search next (1
                time unit per iteration)
                high = mid - 1; // For each iteration: 1 subtraction, 1 assignment (2 time
                    units)
            } else { // If neither condition is satisfied
                low = mid + 1; // For each iteration: 1 addition, 1 assignment (2 time units)
            }
        }
        return -1; // 1 time unit (return)
    }
    // Detailed Time Complexity Analysis
    // ---------------------------------
    // Initialisations before the loop: 2 time units (assignments and subtraction)
    // For each iteration of the while loop (which runs log(n) times):
    // - Loop condition comparison: 1 time unit
    // - Midpoint calculation: 3 time units
    // - Comparisons to check element: 1 time unit
    // - One of the assignments for updating bounds (either low or high): 2 time units
    // Total time per iteration: 1 (while comparison) + 3 (mid calculation) + 1 (comparison)
         + 2 (update bounds) = 7 time units per iteration
    // The loop runs approximately log(n) times.
    // T(n) = 2 (initialisation) + 7 * log(n) (loop iterations) + 1 (final return)
    //      = 3 + 7 * log(n)
    // Ignoring constants and lower-order terms, we get: T(n) = O(log n)
```

Når vi analyserer kode, er vi oftest interesseret i tidskompleksiteten udtrykt med store $\mathcal{O}$-notation. Den viser, hvordan algoritmen skalerer med inputstørrelsen. Den detaljerede analyse af de tidsenheder, der bruges på hver kodelinje, er nyttig til at forstå algoritmens effektivitet og finde flaskehalse. Det kan være nyttigt, når man optimerer kode for at opnå bedre ydelse. Også i det tilfælde behøver vi kun at fokusere på det dominerende led

i tidskompleksiteten, fordi det er det, der bestemmer algoritmens samlede vækstrate.

Vi analyserer desuden som regel, hvor tidsenhederne bruges i hver kodelinje. De følgende eksempler bruger denne fremgangsmåde.

Optimering af en algoritmes effektivitet

Det sidste afsnit i dette kapitel – og i bogen – giver et eksempel på, hvordan man optimerer en algoritme. Vi bruger eksemplet med at finde summen af alle mulige par i et array til at illustrere, hvordan man kan forbedre en algoritmes effektivitet. Algoritmen optimeres fra en naiv fremgangsmåde til stadig mere effektive løsninger. Algoritmen gør følgende:

- Algoritmen beregner **den samlede sum af alle mulige parvise summer** $\text{arr}[i] + \text{arr}[j]$ i arrayet `arr`.
- Summen løber over alle *ordnede par* af indeks (i, j). Både (i, j) og (j, i) medtages altså, og det samme gør par med $i = j$.

Eksempel 15.18 Her er en manuel illustration af summen af par:

Antag, at $\text{arr} = [1, 2, 3]$

Manuel beregning af alle parsummer:

$$\begin{aligned}\text{totalSum} &= (1+1) + (1+2) + (1+3) + \\ &\quad (2+1) + (2+2) + (2+3) + \\ &\quad (3+1) + (3+2) + (3+3) \\ &= 2+3+4+3+4+5+4+5+6 \\ &= 36\end{aligned}$$

Lad os nu se på den **naive fremgangsmåde** til at løse problemet, hvor vi gennemløber alle mulige par i arrayet:

Algoritme 15.7: Naiv sum af par med analyse

```
1   public static int naiveSumOfPairs(int[] arr) {
2       int n = arr.length; // 1 time unit
3       int totalSum = 0; // 1 time unit
4
5       // Outer loop over 'i' from 0 to n - 1
6       for (int i = 0; i < n; i++) { // 2n + 2
7           // Inner loop over 'j' from 0 to n - 1
8           for (int j = 0; j < n; j++) { // (2n + 2) * n
9               totalSum += arr[i] + arr[j]; // 3n^2
10          }
11      }
12      return totalSum; // 1 time unit
13  }
14  // Detailed Time Complexity Analysis
15  // --------------------------------
16  // Initializations: 2 time units (n and totalSum assignments)
17  // Outer loop: 2n + 2 time units
```

```
    // Inner loop 1: n * (2n + 2) time units
    // Inner loop body: 3n^2 time units
    // Return statement: 1 time unit
    //
    // Total Time: T(n) = 2 (initializations)
    //                  + 2n + 2
    //                  + 2n^2 + 2n
    //                  + 3n^2
    //                  + 1 (return)
    //                  = 5n^2 + 4n + 5
    // Dominating Term: 5n^2
    // Tight asymptotic complexity: Theta(n^2)
```

Algoritmen giver os en intuitiv løsning på problemet. Desværre er den meget ineffektiv for store input. Lad os nu se på en mere optimeret algoritme til samme problem. Analysen af den indre løkke bruger de summationsmetoder, der er samlet i Bilag B.

Algoritme 15.8: Optimeret sum af par med analyse

```
public static int optimizedSumOfPairs(int[] arr) {
    int n = arr.length;         // 1 time unit
    int totalSum = 0;           // 1 time unit

    // Outer loop over 'i' from 0 to n - 1
    for (int i = 0; i < n; i++) { // 2n + 2 time units
        // Inner loop over 'j' from i to n - 1
        for (int j = i; j < n; j++) { // (2(n - i) + 2) per i, summed over i
            if (i == j) {           // Comparison: (n(n + 1)/2) time units
                totalSum += arr[i] + arr[j]; // 3 time units, executed n times
            } else {
                totalSum += 2 * (arr[i] + arr[j]); // 4 time units, executed (n^2 - n)/2
                    times
            }
        }
    }
    return totalSum; // 1 time unit
}

// Detailed Time Complexity Analysis
// ---------------------------------
// Initializations: 2 time units (n and totalSum assignments)
// Outer loop: 2n + 2 time units (1 initialization, n + 1 comparisons, n increments)
// Inner loop control: Sum of (2(n - i) + 2) for i from 0 to n - 1
//     = Sum of (2n - 2i + 2) from i = 0 to n - 1
//     = 2n^2 + 2n - n(n - 1)
//     = n^2 + 3n time units
// Inner loop body (if condition):
//     - Comparisons: (n(n + 1)/2) time units
//     - if block (i == j): 3 time units, executed n times => 3n
```

```
    //     - else block (i != j): 4 time units, executed (n^2 - n)/2 times => 2n^2 - 2n
    //     Total inner loop body: n(n + 1)/2 + 3n + 2n^2 - 2n
    //     = (5n^2 + 3n)/2 time units
    // Return statement: 1 time unit
    //
    // Total Time: T(n) = 2 (initializations)
    //                  + 2n + 2 (outer loop)
    //                  + n^2 + 3n (inner loop control)
    //                  + (5n^2 + 3n)/2 (inner loop body)
    //                  + 1 (return)
    //                  = (7n^2 + 13n + 10)/2
    // Dominating Term: (7/2)n^2
    // Tight asymptotic complexity: Theta(n^2)
```

Denne algoritme er stadig $\Theta(n^2)$, men i vores enhedsomkostningsmodel er dens ledende led reduceret fra $5n^2$ til $\frac{7}{2}n^2$. Med de fuldstændige formler for antallet af operationer får vi ved $n = 10^5$:

1. NAIVESUMOFPAIRS ($5n^2 + 4n + 5$):
$$T(10^5) = 5(10^5)^2 + 4(10^5) + 5 = 50\,000\,400\,005$$
2. OPTIMIZEDSUMOFPAIRS ($\frac{7n^2+13n+10}{2}$):
$$T(10^5) = \frac{7(10^5)^2 + 13(10^5) + 10}{2} = 35\,000\,650\,005$$

$$\text{Forskel} = 50\,000\,400\,005 - 35\,000\,650\,005 = 14\,999\,750\,000$$

For $n = 10^5$ udfører den naive version altså cirka **15 milliarder flere operationer** i denne model. Begge algoritmer har ikke desto mindre samme stramme asymptotiske kompleksitet, $\Theta(n^2)$.

Den næste algoritme er endnu mere effektiv med køretiden $\Theta(n)$.

Algoritme 15.9: Effektiv sum af par med analyse

```
public static int efficientSumOfPairs(int[] arr) {
    int n = arr.length;         // 1
    int totalSum = 0;           // 1

    for (int i = 0; i < n; i++) { //2n + 2
        totalSum += 2 * arr[i]; // 3n
    }

    return n * totalSum;        // 2
}
    // Detailed Time Complexity Analysis
    // ---------------------------------
    // Initializations: 2 time units (n and totalSum assignments)
    // Loop: 2n + 2 time units (1 initialization, n + 1 comparisons, n increments)
    // Body: 3n time units (multiplication, addition and assignment, n iterations)
    // Return statement: 2 time units (multiplication and return)
    //
    // Total Time: T(n) = 2 (initializations)
    //                  + 2n + 2 (loop)
```

```
20        //                  + 3n (body)
21        //                  + 2 (return)
22        //          = 5n + 6
23        //
24        // Tight asymptotic complexity: Theta(n)
```

Denne algoritme er langt mere effektiv end de to foregående algoritmer. Dens køretid er $\Theta(n)$, hvilket asymptotisk er bedre end deres kvadratiske køretider. For inputstørrelsen $n = 10^5$ er antallet af operationer:

1. EFFICIENTSUMOFPAIRS ($5n + 6$):
   Antal operationer $= 5n + 6 = 5 \cdot 10^5 + 6 = 500\,006$
2. OPTIMIZEDSUMOFPAIRS ($\frac{7n^2+13n+10}{2}$):
   Antal operationer $= 35\,000\,650\,005$
3. Forskel $= 35\,000\,650\,005 - 500\,006 = 35\,000\,149\,999$

Det illustrerer gevinsten ved at erstatte en $\Theta(n^2)$-algoritme med en $\Theta(n)$-algoritme. Den følgende algoritme har også køretiden $\Theta(n)$, men bruger færre operationer i kapitlets model:

Algoritme 15.10: Hurtig sum af par med analyse

```
1     public static int fastSumOfPairs(int[] arr) {
2         int n = arr.length;            // 1 time unit
3         int sumOfArr = 0;              // 1 time unit
4
5         // Loop over 'i' from 0 to n - 1
6         for (int i = 0; i < n; i++) {  // 2n + 2 time units (1 initialization, n + 1
              comparisons, n increments)
7             sumOfArr += arr[i];        // 2n time units (addition and assignment per
                  iteration)
8         }
9
10        int totalSum = 2 * n * sumOfArr; // 3 time units (2 multiplications, 1 assignment)
11        return totalSum;               // 1 time unit
12    }
13    // Updated Detailed Time Complexity Analysis
14    // ----------------------------------------
15    // Initializations: 2 time units (n and sumOfArr assignments)
16    // Loop over 'i':
17    //     - Loop control: 2n + 2 time units (1 initialization, n + 1 comparisons, n
          increments)
18    //     - Loop body: 2n time units (sumOfArr += arr[i]; addition and assignment)
19    // Total for loop: (2n + 2) + 2n = 4n + 2 time units
20    // Computation of totalSum: 3 time units (2 multiplications, 1 assignment)
21    // Return statement: 1 time unit
22    //
23    // Total Time: T(n) = 2 (initializations)
24    //                  + 4n + 2 (loop over 'i')
25    //                  + 3 (compute totalSum)
26    //                  + 1 (return)
```

```
    //                  = 4n + 8
    // Dominating Term: 4n
    // Tight asymptotic complexity: Theta(n)
```

I kapitlets model for optælling af operationer bruger FASTSUMOFPAIRS $4n + 8$ operationer i stedet for $5n + 6$. Ved $n = 10^5$ er forskellen 99 998 operationer. Begge algoritmer har ikke desto mindre køretiden $\Theta(n)$, og den faktiske ydelse bør måles, hvis forskellen i den konstante faktor har betydning. Vi slutter med en sidste algoritme, der har samme asymptotiske kompleksitet og et lidt mindre antal operationer i denne model.

Algoritme 15.11: Ultraeffektiv sum af par med analyse

```
    public static int ultraEfficientSumOfPairs(int[] arr) {
        int n = arr.length;            // 1 time unit
        int sumOfArr = 0;              // 1 time unit

        // Loop over 'value' in 'arr'
        for (int value : arr) {        // 2n + 1 time units (n iterations)
            sumOfArr += value;         // 2n time units (addition and assignment per
                iteration)
        }

        int totalSum = 2 * n * sumOfArr; // 3 time units (2 multiplications, 1 assignment)
        return totalSum;               // 1 time unit
    }

    // Updated Detailed Time Complexity Analysis
    // -----------------------------------------
    // Initializations: 2 time units (n and sumOfArr assignments)
    // Loop over 'arr':
    //     - Loop control: 2n + 1 time units (1 initialization, n comparisons, n increments)
    //     - Loop body: 2n time units (sumOfArr += value; addition and assignment)
    // Total for loop: (2n + 1) + 2n = 4n + 1 time units
    // Computation of totalSum: 3 time units (2 multiplications, 1 assignment)
    // Return statement: 1 time unit
    //
    // Total Time: T(n) = 2 (initializations)
    //                  + (4n + 1) (loop over 'arr')
    //                  + 3 (compute totalSum)
    //                  + 1 (return)
    //                  = 4n + 7
    // Dominating Term: 4n
    // Tight asymptotic complexity: Theta(n)
```

De to har identisk asymptotisk kompleksitet, og forskellen i konstanten er ubetydelig.

Hvorfor ultraEfficientSumOfPairs er bedre:

1. **Læsbarhed:** Den udvidede `for-each`-løkke gennemløber elementerne direkte:
   `for (int value : arr) { sumOfArr += value; }`

Den er kortfattet, undgår indeksering og giver mindre risiko for fejl end en traditionel `for`-løkke:

```
for (int i = 0; i < n; i++) { sumOfArr += arr[i]; }
```

2. **Vedligeholdelse:** Udvidede løkker er lettere at ændre og tilpasse til forskellige datastrukturer.
3. **Moderne praksis:** Den følger nutidens Java-standarder og gør koden og hensigten tydeligere.

Selv om de to funktioner yder omtrent lige godt, foretrækkes ULTRAEFFICIENTSUMOFPAIRS på grund af sin læsbarhed, enkelhed og overensstemmelse med moderne programmeringspraksis, hvilket gør den til en mere elegant løsning.

Litteratur

Cormen, T., Leiserson, C., Rivest, R., & Stein, C. (2022). *Introduction to Algorithms* (4th). MIT Press.

Lay, D. (2003). *Linear Algebra and Its Applications*. Pearson Education.

Montgomery, D. (2013). *Applied Statistics and Probability for Engineers, 6th Edition*. John Wiley; Sons, Incorporated.

Pishro-Nik, H. (2014). *Introduction to Probability, Statistics, and Random Processes*. Kappa Research, LLC.

Rosen, K. H. (2012). *Discrete Mathematics and Its Applications* (7th). McGraw-Hill Education.

Ross, S. M. (2020). *Introduction to Probability and Statistics for Engineers and Scientists* (6th). Academic Press.

Thomas, C. *Introduction to Differential Calculus* [Accessed online]. Mathematics Learning Centre, University of Sydney. Accessed online. Sydney, NSW, 1997.

Bilag A: Vigtige begreber

Dette bilag er en samling af vigtige matematiske begreber, som ofte bruges i softwareudvikling. Indholdet i bilaget bygger på begreberne i denne bog.

Proposition A.1 (Regneregler for brøker)

For $a, b, c, m \in \mathbb{R}$, hvor $a, b, c, m \neq 0$ når det er nødvendigt, gælder følgende identiteter:

(1) $\frac{a}{b} \times m = \frac{am}{b}$

(2) $\frac{a}{b} \div m = \frac{a}{bm}$

(3) $m \div \frac{a}{b} = \frac{mb}{a}$

(4) $\frac{a}{b} \times \frac{c}{a} = \frac{c}{b}$

(5) $\frac{a}{b} \div \frac{c}{a} = \frac{a^2}{bc}$

(6) $\frac{a}{b} = \frac{ac}{bc}$

(7) $\frac{a}{b} + \frac{c}{a} = \frac{a^2 + bc}{ab}$

Proposition A.2 (Regneregler for rødder)

(1)	$\sqrt[n]{xy} = \sqrt[n]{x} \times \sqrt[n]{y}$	begge sider reelle; for lige n kræves $x, y \geq 0$
(2)	$\sqrt[n]{\frac{x}{y}} = \frac{\sqrt[n]{x}}{\sqrt[n]{y}}$	begge sider reelle, $y \neq 0$; for lige n kræves $x \geq 0, y > 0$
(3)	$\sqrt{x^2} = \lvert x \rvert$	hvor $x \in \mathbb{R}$
(4)	$(\sqrt[n]{x})^n = x$	for ulige n og alle x; for lige n og $x \geq 0$
(5)	$\sqrt[n]{-x} = -\sqrt[n]{x}$	hvor $x \geq 0$ og $n \in \mathbb{N}$ er ulige

Proposition A.3 (Potensregneregler for heltallige eksponenter)

Lad $n, m \in \mathbb{Z}$. Følgende identiteter gælder, når potenserne er definerede. Især skal et grundtal være forskelligt fra nul, når der optræder en negativ eksponent:

(1) $x^n \cdot x^m = x^{n+m}$,

(2) $\frac{x^n}{x^m} = x^{n-m}$ med $x \neq 0$,

(3) $x^n \cdot y^n = (xy)^n$,

(4) $\frac{x^n}{y^n} = \left(\frac{x}{y}\right)^n$ med $y \neq 0$,

(5) $(x^n)^m = x^{nm}$,

(6) $x^1 = x$.

Proposition A.4 (Flere potensregneregler for heltallige eksponenter)

Lad $n, m \in \mathbb{Z}$. Så gælder følgende (med $x, y \in \mathbb{R}$ og forskellige fra nul, hvor det er angivet):

(7) $x^0 = 1 \qquad x \neq 0$

(8) $\frac{1}{x^m} = x^{-m} \qquad x \neq 0$

Definition A.1 (Det udvidede potensbegreb)

Lad $m \in \mathbb{Z}$ og lad $n \in \mathbb{N}$, så $\frac{m}{n} \in \mathbb{Q}$. Så gælder følgende:

$x^{\frac{m}{n}} = \sqrt[n]{x^m} \qquad x > 0$

Mere specifikt gælder

$x^{\frac{1}{n}} = \sqrt[n]{x} \qquad x > 0$

Regler for omskrivning af formler

Følgende operationer kan udføres på begge sider af formlen:

- Læg den samme størrelse til på begge sider
- Træk den samme størrelse fra på begge sider
- Gang begge sider med den samme størrelse – husk at gange alle led
- Divider begge sider med den samme størrelse forskellig fra nul – husk at dividere alle led
- Anvend en funktion på begge sider. Det bevarer implikationen, men det bevarer kun ækvivalensen, når funktionen er injektiv på den relevante definitionsmængde. Kvadrering kan give falske løsninger, og at tage den reciprokke værdi kræver, at siderne er forskellige fra nul.

Definition A.2

Lad A og B være ikke-tomme mængder. En funktion f fra A til B er en tilordning, der til hvert element i A knytter præcis ét element i B. Vi skriver $f(a) = b$, hvis b er det entydige element i B, som funktionen f knytter til elementet a i A. Hvis f er en funktion fra A til B, skriver vi $f : A \to B$.

Definition A.3

Hvis f er en funktion fra A til B, siger vi, at A er **definitionsmængden** for f, og at B er **dispositionsmængden** for f. Hvis $f(a) = b$, siger vi, at b er **billedet** af a, og at a er et **urbillede** af b. **Værdimængden**, eller billedet, af f er mængden af alle billeder af elementer i A. Hvis f er en funktion fra A til B, siger vi desuden, at f **afbilder** A ind i B.

Definition A.4 (Injektive funktioner)

En funktion $f : A \to B$ kaldes **injektiv** (på engelsk **one-to-one**), hvis forskellige elementer i A afbildes over i forskellige elementer i B. Med andre ord: Hvis $f(a_1) = f(a_2)$, så er $a_1 = a_2$. Denne egenskab sikrer, at to forskellige elementer i A aldrig afbildes over i det samme element i B.

Definition A.5 (Surjektive funktioner)

En funktion $f : A \to B$ kaldes **surjektiv** (på engelsk **onto**), hvis ethvert element i B er billedet af mindst ét element i A. Med andre ord findes der for ethvert $b \in B$ mindst ét $a \in A$, så $f(a) = b$. Denne egenskab

sikrer, at funktionen »dækker« hele mængden B.

Definition A.6 (Inverse funktioner)

Lad f være en bijektion fra mængden A til mængden B. Den inverse funktion til f er den funktion, der til et element b i B knytter det entydige element a i A, for hvilket $f(a) = b$. Den inverse funktion til f betegnes f^{-1}. Altså er $f^{-1}(b) = a$, når $f(a) = b$.

Definition A.7

Lad $f : B \to C$ og $g : A \to B$ være to funktioner. Den **sammensatte funktion** af f og g, betegnet $f \circ g$, er en funktion fra A til C, der er defineret ved

$$(f \circ g)(x) = f(g(x)),$$

for ethvert $x \in A$.

Definition A.8 (Særlige eksponentialfunktioner)

$y = a \cdot 10^{bx}$ eksponentialfunktion med grundtal 10

$y = a \cdot e^{bx}$ naturlig eksponentialfunktion (grundtal e),

hvor a og b er konstanter, og definitionsmængden er alle reelle tal.

Definition A.9 (Titalslogaritmer)

$$\log x = y \iff 10^y = x, \qquad x > 0$$

Med ord: $\log x$ er den eksponent i potensen af 10, der giver x

Regneregler for titalslogaritmer

Følgende identiteter kræver $x > 0$ og $y > 0$:

- Logaritmen til et produkt:
  $$\log(xy) = \log x + \log y$$
  Med ord: log til et produkt er lig med summen af logaritmerne til faktorerne.
- Logaritmen til en kvotient:
  $$\log \frac{x}{y} = \log x - \log y$$
  Med ord: log til en kvotient er lig med logaritmen til tælleren minus log til nævneren.
- Logaritmen til en potens:
  $$\log x^y = y \log x$$
  Med ord: log til en potens er lig med eksponenten gange logaritmen til grundtallet.

Det vigtigste at huske om logaritmer er dette

En logaritme er en eksponent.

Definition A.10 (Logaritme med vilkårligt grundtal)

Algebraisk:

$$\log_b x = y \text{ hvis og kun hvis } b^y = x, \quad \text{hvor } b > 0, b \neq 1, \text{ og } x > 0$$

Med ord:

$\log_b x = y$ betyder, at y er den eksponent til b, der giver x som resultat.

Definition A.11 (Titalslogaritme og naturlig logaritme)

Titalslogaritme: Symbolet $\log x$ betyder $\log_{10} x$.

Naturlig: Symbolet $\ln x$ betyder $\log_e x$, hvor e er en konstant lig med $2.71828182845\ldots$

Reglen for basisskift for logaritmer

$$\log_a x = \frac{\log_b x}{\log_b a} \quad \text{eller} \quad \log_a x = \frac{1}{\log_b a}(\log_b x)$$

hvor $a, b, x > 0$ og $a \neq 1, b \neq 1$.

Logaritmeregneregler

Vi antager overalt, at logaritmernes grundtal er positive og forskellige fra 1, og at alle logaritmernes argumenter er positive. Logaritmen til en potens:

$$\log_b x^y = y \log_b x$$

Med ord: Logaritmen til en potens er lig med produktet af eksponenten og logaritmen til grundtallet. Logaritmen til et produkt:

$$\log_b(xy) = \log_b x + \log_b y$$

Med ord: Logaritmen til et produkt er lig med summen af logaritmerne til faktorerne. Logaritmen til en kvotient:

$$\log_b \frac{x}{y} = \log_b x - \log_b y$$

Med ord: Logaritmen til en kvotient er lig med logaritmen til tælleren minus logaritmen til nævneren.

Talsystem	Symboler	Grundtal	Yderligere information
Decimalt	0-9	10	-
Binært	0, 1	2	-
Hexadecimalt	0-9, A-F	16	$A \equiv 10, B \equiv 11, C \equiv 12, D \equiv 13, E \equiv 14, F \equiv 15$
Oktalt	0-7	8	-

Tabel A.1: Oversigt over almindelige talsystemer

Proposition A.5 (Regler for binær addition)

$0 + 0 = 0, \quad 0 + 1 = 1, \quad 1 + 0 = 1, \quad 1 + 1 = 10$

Decimalt tal		Som potenser af 2	Potens af 2				Binært tal
			3	2	1	0	
8	=	2^3	1	0	0	0	1000
7	=	$2^2 + 2^1 + 2^0$	0	1	1	1	111
6	=	$2^2 + 2^1$	0	1	1	0	110
5	=	$2^2 + 2^0$	0	1	0	1	101
4	=	2^2	0	1	0	0	100
3	=	$2^1 + 2^0$	0	0	1	1	11
2	=	2^1	0	0	1	0	10
1	=	2^0	0	0	0	1	1

Tabel A.2: Decimale tal i binær repræsentation

Proposition A.6 (Regler for binær multiplikation)

$0 \times 0 = 0$

$0 \times 1 = 0$

$1 \times 0 = 0$

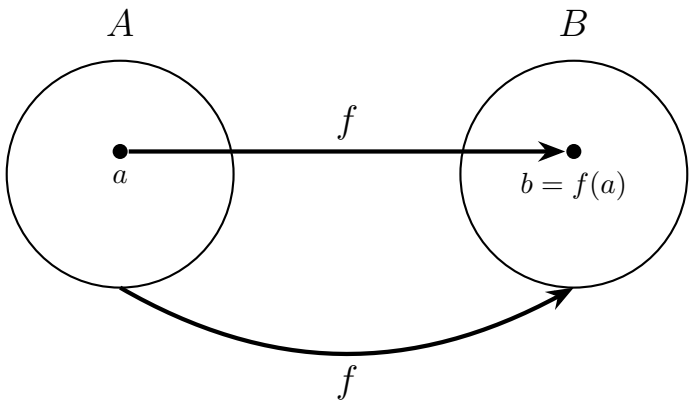
$1 \times 1 = 1$

$1 \times 10_2 = 10_2$ (multiplikation med grundtallet 10_2 tilføjer et 0 til sidst)

Proposition A.7 (XOR-operationen)

XOR giver 1, hvis de to bit, der sammenlignes, er forskellige, og 0, hvis de er ens:

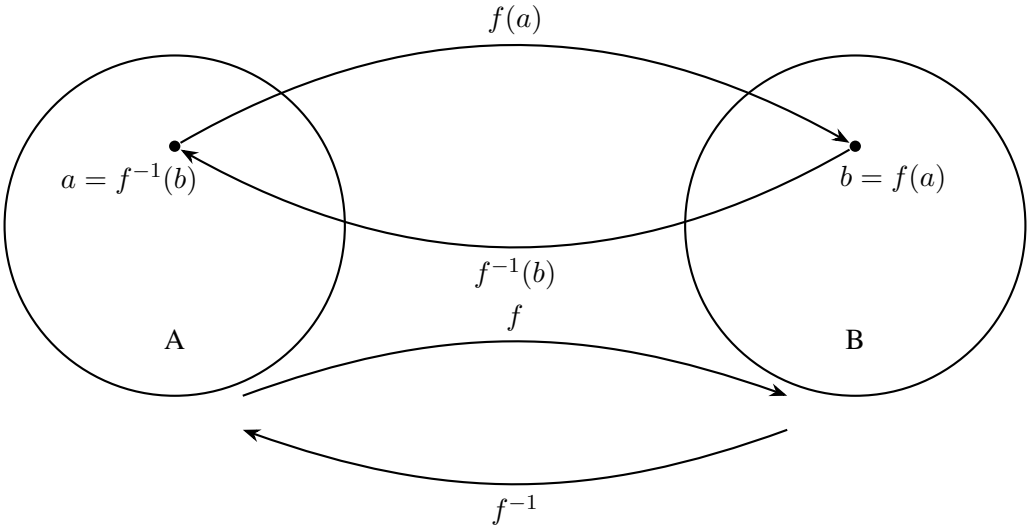
$0 \oplus 0 = 0, \quad 0 \oplus 1 = 1, \quad 1 \oplus 0 = 1, \quad 1 \oplus 1 = 0$



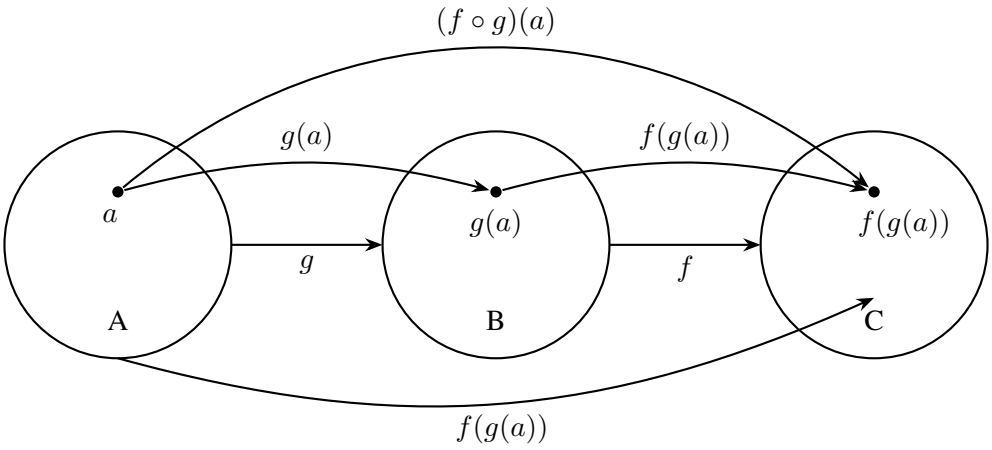
Figur A.1: En funktion f, der afbilder et element a fra mængden A over i et element $b = f(a)$ i mængden B.

Definition A.12 (Delmængde)

En mængde A er en **delmængde** af en mængde B, hvis ethvert element i A også er et element i B. Det skriver vi som $A \subseteq B$.



Figur A.2: Funktionen f^{-1} er den inverse til funktionen f.



Figur A.3: Sammensætningen af funktionerne f og g, betegnet $f \circ g$, er den funktion, der fremkommer ved først at anvende g og derefter f.

Definition A.13 (Ægte delmængde)

En mængde A er en **ægte delmængde** af en mængde B, hvis $A \subseteq B$ og $A \neq B$. Det betyder, at B skal indeholde mindst ét element, som ikke ligger i A. Det skriver vi som $A \subset B$.

Definition A.14 (Universalmængde)

Universalmængden, betegnet U, er mængden af alle mulige elementer, der betragtes i en given sammenhæng. Alle andre mængder i den sammenhæng betragtes som delmængder af universalmængden.

Definition A.15 (Den tomme mængde)

Den tomme mængde (eller **nulmængden**) er den entydige mængde, der ikke indeholder nogen elementer. Den betegnes $\emptyset$ eller $\{\}$.

Definition A.16 (Disjunkte mængder)

To mængder A og B er **disjunkte**, hvis de ikke har nogen elementer til fælles. Med andre ord er deres fællesmængde den tomme mængde: $A \cap B = \emptyset$.

Definition A.17 (Kardinalitet)

Kardinaliteten af en endelig mængde A, betegnet $|A|$, er antallet af forskellige elementer i mængden.

Definition A.18 (Potensmængde)

Potensmængden af en mængde A, betegnet $\mathcal{P}(A)$, er mængden af alle delmængder af A. Potensmængdens elementer er altså selv mængder.

Dualiteten mellem mængdelære og boolesk algebra

For en fast universalmængde er algebraen af dens delmængder med foreningsmængde, fællesmængde og komplement en boolesk algebra. Operationerne svarer til logisk OR, AND og NOT som følger:

Mængdelære		**Boolesk algebra / logik**
Foreningsmængde ($\cup$)	$\Longleftrightarrow$	Boolesk sum ($+$) eller `OR`
Fællesmængde ($\cap$)	$\Longleftrightarrow$	Boolesk produkt ($\cdot$) eller `AND`
Komplement (A^c)	$\Longleftrightarrow$	Komplementering ($\overline{x}$) eller `NOT`
Universalmængden (U)	$\Longleftrightarrow$	Sand (1)
Den tomme mængde ($\emptyset$)	$\Longleftrightarrow$	Falsk (0)

Definition A.19 (Fællesmængde)

Fællesmængden af mængderne A og B, betegnet $A \cap B$, er mængden af alle elementer, der ligger i både A og B.

$$A \cap B = \{x \mid x \in A \text{ og } x \in B\}$$

Definition A.20 (Foreningsmængde)

Foreningsmængden af mængderne A og B, betegnet $A \cup B$, er mængden af alle elementer, der ligger i A, i B eller i begge.

$$A \cup B = \{x \mid x \in A \text{ eller } x \in B\}$$

Definition A.21 (Mængdedifferens)

Differensen mellem mængden A og mængden B, betegnet $A \setminus B$, er mængden af alle elementer, der ligger i A, men ikke i B.

$$A \setminus B = \{x \mid x \in A \text{ og } x \notin B\}$$

Definition A.22 (Symmetrisk differens)

Den **symmetriske differens** af mængderne A og B, betegnet $A \oplus B$, er mængden af de elementer, der ligger i en af mængderne, men ikke i deres fællesmængde.

$$A \oplus B = (A \cup B) \setminus (A \cap B)$$

Definition A.23 (Komplement)

Komplementet til en mængde A, betegnet A^c, er mængden af alle elementer i universalmængden U, som ikke ligger i A.

$$A^c = U \setminus A$$

Definition A.24 (Kartesisk produkt)

Det **kartesiske produkt** af mængderne A og B, betegnet $A \times B$, er mængden af alle mulige ordnede par (a, b), hvor det første element a er fra A, og det andet element b er fra B.

$$A \times B = \{(a, b) \mid a \in A \text{ og } b \in B\}$$

Mængdeoperationer som bitvise operationer

Lad bitstrengene for mængderne A og B være s_A og s_B.

- **Foreningsmængde** ($A \cup B$) svarer til en bitvis `OR`-operation.
- **Fællesmængde** ($A \cap B$) svarer til en bitvis `AND`-operation.
- **Komplement** (A^c) svarer til en bitvis `NOT`-operation (enerkomplement).
- **Symmetrisk differens** ($A \oplus B$) svarer til en bitvis `XOR`-operation.

Definition A.25 (Stokastisk eksperiment)

Et stokastisk eksperiment er et eksperiment, der kan give forskellige resultater, selv om man gør alting på samme måde hver gang.

Definition A.26 (Udfaldsrum, udfald og hændelser)

Udfaldsrum: Mængden af alle mulige udfald af et stokastisk eksperiment kaldes udfaldsrummet og betegnes S. Udfaldene kan være diskrete eller kontinuerte afhængigt af eksperimentets natur.

Udfald: Et enkelt muligt resultat af et stokastisk eksperiment.

Hændelse: En vilkårlig delmængde af udfaldsrummet, som kan bestå af et eller flere udfald.

Sætning A.1 (Multiplikationsreglen)

Lad en operation være beskrevet som en følge af k trin. Antag, at følgende betingelser er opfyldt:

- Der er n_1 måder at udføre trin 1 på.
- Der er n_2 måder at udføre trin 2 på for hver måde at udføre trin 1 på.
- Der er n_3 måder at udføre trin 3 på for hver måde at udføre trin 2 på, og så videre.

Så er det samlede antal måder at udføre hele operationen på givet ved:

$$n_1 \times n_2 \times \cdots \times n_k.$$

♡

Definition A.27 (Optælling med og uden tilbagelægning)

Når vi tæller antallet af måder at udvælge objekter fra en mængde på, er der to almindelige scenarier:

- **Med tilbagelægning**: Et objekt kan udvælges mere end én gang.
- **Uden tilbagelægning**: Når et objekt er udvalgt, kan det ikke vælges igen.

♣

Definition A.28 (Permutation og kombination)

- **Permutation (rækkefølgen har betydning)**: Forskellige rækkefølger tælles som forskellige udfald, hvilket giver et større antal.
- **Kombination (rækkefølgen har ikke betydning)**: Rækkefølger behandles som ens, hvilket giver et mindre antal.

♣

Proposition A.8 (Permutationer af n forskellige objekter)

Antallet af ordnede arrangementer (permutationer) af n forskellige objekter er

$$n! = n \cdot (n-1) \cdot \cdots \cdot 2 \cdot 1.$$

♠

Sætning A.2 (Permutationer af delmængder)

For heltal $n \geq r \geq 0$ er antallet af ordnede udvalg af r forskellige objekter blandt n forskellige objekter

$$P_r^n = P(n,r) = n \cdot (n-1) \cdots (n-r+1) = \frac{n!}{(n-r)!}.$$

♡

Sætning A.3 (Kombinationer)

Antallet af kombinationer af r elementer udvalgt fra en mængde med n forskellige elementer er givet ved

$$C_r^n = \binom{n}{r} = \frac{n!}{r!(n-r)!}$$

♡

Sætning A.4 (Binomialsætningen)

I algebra bruges binomialkoefficienten til at udvikle potenser af binomier. Ifølge binomialsætningen gælder

$$(a+b)^n = \sum_{k=0}^{n} \binom{n}{k} a^k b^{n-k}$$

♡

Proposition A.9 (Med tilbagelægning)

For udvælgelser blandt n typer gælder:

- **Ordnet med tilbagelægning**: n^r udfald.

- **Uordnet med tilbagelægning**: $\binom{n+r-1}{r}$ udfald.

♠

Aksiom A.1 (Sandsynlighedsregningens aksiomer)

- **Aksiom 1:** For enhver hændelse A gælder $0 \leq P(A) \leq 1$.
- **Aksiom 2:** Sandsynligheden for udfaldsrummet S er $P(S) = 1$.
- **Aksiom 3:** Hvis $A_1, A_2, A_3, \cdots$ er disjunkte hændelser, så gælder $P\left(A_1 \cup A_2 \cup A_3 \cdots\right) = P\left(A_1\right) + P\left(A_2\right) + P\left(A_3\right) + \cdots$

♡

Definition A.29 (Sandsynligheden for en hændelse)

Et sandsynlighedsmål tildeler et tal $P(A) \in [0,1]$ til hver hændelse A i overensstemmelse med sandsynlighedsaksiomerne. I et endeligt udfaldsrum, hvor de elementære udfald er lige sandsynlige, reduceres dette til

$$P(A) = \frac{\text{Antal gunstige udfald}}{\text{Samlet antal udfald}}$$

♣

Sætning A.5 (Regneregler for sandsynligheder)

- **Komplementreglen:** Sandsynligheden for komplementet til hændelse A er
  $$P(A^c) = 1 - P(A)$$
- **Reglen for den tomme mængde:** Sandsynligheden for den tomme mængde er 0, dvs.
  $$P(\emptyset) = 0$$
- **Additionsreglen:** For to vilkårlige hændelser A og B er sandsynligheden for foreningsmængden af hændelserne A og B givet ved
  $$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$
- **Differensreglen:** Sandsynligheden for differensen mellem hændelserne A og B er givet ved
  $$P(A - B) = P(A) - P(A \cap B)$$
- **Delmængdereglen:** Hvis A er en delmængde af B ($A \subset B$), så gælder
  $$P(A) \leq P(B).$$

♡

Definition A.30 (Betinget sandsynlighed)

For to vilkårlige hændelser A og B med $P(B) > 0$ er den betingede sandsynlighed for A givet B defineret som:

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}, \qquad P(B) > 0$$

♣

Aksiom A.2 (Aksiomer for betinget sandsynlighed)

Fastlæg en hændelse B med $P(B) > 0$. Så er $A \mapsto P(A \mid B)$ et sandsynlighedsmål, og derfor gælder:

- **Aksiom 1:** For enhver hændelse A gælder $0 \leq P(A \mid B) \leq 1$.
- **Aksiom 2:** $P(S \mid B) = 1$ (og dermed $P(B \mid B) = 1$).
- **Aksiom 3:** Hvis $A_1, A_2, A_3, \cdots$ er disjunkte hændelser, så gælder
  $$P\left(A_1 \cup A_2 \cup A_3 \cdots \mid B\right) = P\left(A_1 \mid B\right) + P\left(A_2 \mid B\right) + P\left(A_3 \mid B\right) + \cdots$$

♡

Sætning A.6 (Regler for betinget sandsynlighed)

I de følgende regler antages $P(C) > 0$.

- **Komplementreglen:** Sandsynligheden for komplementet til hændelse A givet C er
$$P\left(A^c \mid C\right) = 1 - P(A \mid C)$$
- **Reglen for den tomme mængde:** Sandsynligheden for den tomme mængde givet en hændelse C er 0, dvs.
$$P(\emptyset \mid C) = 0$$
- **Additionsreglen:** For to vilkårlige hændelser A og B er sandsynligheden for foreningsmængden af hændelserne A og B givet en anden hændelse C
$$P(A \cup B \mid C) = P(A \mid C) + P(B \mid C) - P(A \cap B \mid C)$$
- **Differensreglen:** Sandsynligheden for differensen mellem hændelserne A og B givet en anden hændelse C er
$$P(A - B \mid C) = P(A \mid C) - P(A \cap B \mid C)$$
- **Delmængdereglen:** Hvis $A \subset B$, så gælder
$$P(A \mid C) \leq P(B \mid C)$$

♡

Sætning A.7 (Multiplikationsreglen)

Hvis $P(A) > 0$, så er $P(A \cap B) = P(A)P(B \mid A)$. Hvis $P(B) > 0$, så er $P(A \cap B) = P(B)P(A \mid B)$. Når begge sandsynligheder er positive, gælder derfor:
$$P(A \cap B) = P(A)P(B \mid A) = P(B)P(A \mid B)$$

♡

Sætning A.8 (Loven om total sandsynlighed)

For en vilkårlig hændelse A og en hændelse B med $0 < P(B) < 1$ giver partitionen B, B^c:
$$\begin{aligned} P(A) &= P(A \cap B) + P(A \cap B^c) \\ &= P(B)P(A \mid B) + P(B^c)P(A \mid B^c) \end{aligned}$$
Mere generelt gælder for enhver endelig partition $B_1, \ldots, B_n$ af S med $P(B_i) > 0$ for alle i:
$$P(A) = \sum_{i=1}^{n} P(A \mid B_i)P(B_i)$$

♡

Definition A.31 (Uafhængighed)

Hændelserne A og B er uafhængige, netop når
$$P(A \cap B) = P(A)P(B).$$
Hvis $P(B) > 0$, er dette ensbetydende med $P(A \mid B) = P(A)$; hvis $P(A) > 0$, er det ensbetydende med $P(B \mid A) = P(B)$.

♣

Sætning A.9 (Uafhængighed og De Morgans lov)

Hvis $A_1, A_2, \ldots, A_n$ er indbyrdes uafhængige, så gælder
$$P\left(A_1 \cup A_2 \cup \cdots \cup A_n\right) = 1 - \left(1 - P\left(A_1\right)\right)\left(1 - P\left(A_2\right)\right) \cdots \left(1 - P\left(A_n\right)\right)$$

♡

Definition A.32 (Uafhængighed af flere hændelser)

Hændelserne $A_1, \ldots, A_k$ er *indbyrdes uafhængige*, hvis der for enhver delmængde $J \subseteq \{1, \ldots, k\}$ af mindst to forskellige indeks gælder, at

$$P\left(\bigcap_{j \in J} A_j\right) = \prod_{j \in J} P(A_j).$$

Det er ikke tilstrækkeligt kun at kontrollere fællesmængden af alle k hændelser.

Sætning A.10 (Bayes' sætning)

For hændelser A og B med $P(A) > 0$ og $P(B) > 0$ giver Bayes' sætning den første lighed nedenfor. Den udvidede nævner forudsætter desuden $0 < P(B) < 1$, så begge de viste betingede sandsynligheder er definerede:

$$P(B \mid A) = \frac{P(A \mid B) \cdot P(B)}{P(A)} = \frac{P(A \mid B) \cdot P(B)}{P(A \mid B) \cdot P(B) + P(A \mid B^c) \cdot P(B^c)}$$

Hvis $B_1, B_2, \ldots$ udgør en endelig eller tællelig partition af S med $P(B_i) > 0$ for alle i, og $P(A) > 0$, så gælder

$$P\left(B_j \mid A\right) = \frac{P\left(A \mid B_j\right) \cdot P\left(B_j\right)}{\sum_i P\left(A \mid B_i\right) \cdot P\left(B_i\right)}$$

Definition A.33 (Population)

En **population** er den fuldstændige mængde af alle mulige observationer eller målinger, der er af interesse i en bestemt undersøgelse. I softwareudvikling kan det fx være alle mulige køretider for en algoritme, alle brugersessioner på et websted eller alle kodelinjer i et projekt.

Definition A.34 (Stikprøve)

En **stikprøve** er en delmængde af populationen, som vi faktisk observerer eller måler. På grund af praktiske begrænsninger arbejder vi ofte med stikprøver frem for hele populationer.

Definition A.35 (Kvalitative data)

Kvalitative data (også kaldet kategoriske data) består af ikke-numerisk information, der kan inddeles i kategorier. Eksempler er de programmeringssprog, der bruges i et projekt, brugernes vurdering af deres tilfredshed (tilfreds/neutral/utilfreds) eller fejlenes alvorlighedsgrad (kritisk/høj/middel/lav).

Definition A.36 (Kvantitative data)

Kvantitative data består af numeriske målinger, der kan ordnes, og som man kan udføre matematiske operationer på. De kan yderligere opdeles i:

- **Diskrete**: Tællelige værdier (antal fejl, kodelinjer, brugersessioner)
- **Kontinuerte**: Målbare værdier, der kan antage enhver værdi inden for et interval (svartider, hukommelsesforbrug, CPU-udnyttelse)

Definition A.37 (Hyppighedsfordeling)

En **hyppighedsfordeling** er en tabel, der viser hyppigheden (antallet) af hver værdi eller klasse af værdier i et datasæt. Den kan angives som:

- **Absolut hyppighed**: Det faktiske antal forekomster

- **Relativ hyppighed**: Andelen af det samlede antal observationer
- **Kumuleret hyppighed**: Den løbende sum af hyppighederne

Definition A.38 (Aritmetisk gennemsnit)

For et datasæt med n værdier $x_1, x_2, \ldots, x_n$ er det **aritmetiske gennemsnit** defineret som:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i = \frac{x_1 + x_2 + \cdots + x_n}{n}$$

Definition A.39 (Median)

For et datasæt med n værdier ordnet i stigende rækkefølge er medianen defineret som:

$$\text{median} = \begin{cases} x_{\frac{n+1}{2}} & \text{hvis } n \text{ er ulige} \\ \frac{x_{\frac{n}{2}} + x_{\frac{n}{2}+1}}{2} & \text{hvis } n \text{ er lige} \end{cases}$$

Definition A.40 (Typetal)

Typetallet er den værdi, der forekommer hyppigst i et datasæt. Et datasæt kan have:

- **Intet entydigt typetal**: Hvis alle observerede værdier har samme hyppighed, deler alle værdier den højeste hyppighed; efter nogle konventioner angives dette som »intet typetal«.
- **Ét typetal (unimodalt)**: Hvis én værdi forekommer hyppigst
- **Flere typetal (multimodalt)**: Hvis to eller flere værdier deler den højeste hyppighed

Definition A.41 (Variationsbredde)

Variationsbredden for et datasæt er forskellen mellem den største og den mindste værdi:

$$\text{Variationsbredde} = x_{\max} - x_{\min}$$

Definition A.42 (Stikprøvevarians og standardafvigelse)

For en stikprøve med n værdier er **stikprøvevariansen**, betegnet s^2, defineret som:

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2$$

Stikprøvestandardafvigelsen, betegnet s, er kvadratroden af variansen:

$$s = \sqrt{s^2} = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

Definition A.43 (Variationskoefficient)

For data på forholdsskala med et stikprøvegennemsnit forskelligt fra nul er **variationskoefficienten** forholdet mellem stikprøvestandardafvigelsen og gennemsnittet:

$$CV = \frac{s}{\bar{x}} \times 100\%$$

Definition A.44 (Percentil)

Den p'te **percentil** er en værdi, som cirka $p\%$ af de ordnede observationer ligger på eller under. Der findes flere interpolationskonventioner, så en numerisk percentilberegning bør angive, hvilken konvention der

er brugt.

Definition A.45 (Kvartiler)

Kvartiler deler et ordnet datasæt i fire lige store dele.

- **Q1 (første kvartil)**: 25.-percentilen.
- **Q2 (anden kvartil)**: 50.-percentilen. Det er datasættets **median**.
- **Q3 (tredje kvartil)**: 75.-percentilen.

Definition A.46 (Kvartilafstand)

Kvartilafstanden (IQR) måler spredningen af de midterste 50% af dataene og er defineret som

$$IQR = Q_3 - Q_1,$$

hvor Q_1 og Q_3 er første og tredje kvartil. En større IQR viser, at den centrale del af dataene er mere spredt, mens en mindre IQR viser, at den er tættere samlet.

Definition A.47 (Normalfordeling)

En **normalfordeling** er en symmetrisk, klokkeformet fordeling, hvor middelværdien, medianen og typetallet alle er ens og ligger i centrum. Fordelingens form er fuldstændig bestemt af dens middelværdi (μ) og standardafvigelse (σ).

Definition A.48 (Skævhed)

Skævhed måler asymmetrien i en datafordeling:

- **Positiv skævhed (højreskæv)**: Den længste hale strækker sig mod højre.
- **Negativ skævhed (venstreskæv)**: Den længste hale strækker sig mod venstre.
- **Symmetrisk**: De to sider har omtrent samme form.

Sammenligninger mellem gennemsnit og median kan være nyttige tommelfingerregler, men de definerer ikke skævhed.

Definition A.49 (Boksplottets bestanddele)

Et standardboksplot efter Tukey viser kvartilerne og knurhårene:

- boksen fra Q_1 til Q_3 og en linje ved medianen Q_2;
- knurhår, der strækker sig ud til de mest ekstreme observationer inden for $1.5\,IQR$ fra den pågældende kvartil; og
- observationer uden for knurhårene, som afsættes enkeltvis som mulige outliers.

Sætning A.11 (Store tals lov)

Hvis $X_1, X_2, \ldots$ er uafhængige og identisk fordelte stokastiske variable med endelig middelværdi μ, så konvergerer deres stikprøvegennemsnit i sandsynlighed mod μ, når stikprøvestørrelsen går mod uendelig.

Sætning A.12 (Den empiriske regel)

Hvis et datasæt er tilnærmelsesvis normalfordelt med stikprøvegennemsnit $\bar{x}$ og stikprøvestandardafvigelse s, så gælder:

- Cirka **68.3%** af observationerne ligger inden for 1 standardafvigelse fra gennemsnittet ($\bar{x} \pm s$).

- Cirka **95.4%** af observationerne ligger inden for 2 standardafvigelser fra gennemsnittet ($\bar{x} \pm 2s$).
- Cirka **99.7%** af observationerne ligger inden for 3 standardafvigelser fra gennemsnittet ($\bar{x} \pm 3s$).

Definition A.50 (Lineære ligninger)

En lineær ligning i variablene $x_1, x_2, \ldots, x_n$ er en ligning, der kan skrives på formen

$$a_1x_1 + a_2x_2 + \cdots + a_nx_n = b$$

hvor b og koefficienterne $a_1, \ldots, a_n$ er konstanter.

Definition A.51 (Lineære ligningssystemer)

Et **lineært ligningssystem** (eller kort et **lineært system**) er en samling af en eller flere lineære ligninger med de samme variable.

Definition A.52 (Løsninger til et lineært ligningssystem)

En **løsning** til et lineært ligningssystem i variablene $x_1, x_2, \ldots, x_n$ er en liste af tal $(s_1, s_2, \ldots, s_n)$, der opfylder alle systemets ligninger, når tallene indsættes på variablenes plads $x_1, x_2, \ldots, x_n$ i den nævnte rækkefølge. Mængden af alle mulige løsninger kaldes systemets **løsningsmængde**. To lineære ligningssystemer kaldes **ækvivalente**, hvis de har den samme løsningsmængde.

Definition A.53

En **matrix** er et rektangulært skema af tal ordnet i rækker og søjler. **Koefficientmatricen** for et lineært ligningssystem indeholder kun variablenes koefficienter, mens **totalmatricen** har en ekstra søjle med konstanterne fra ligningernes højre side.

Definition A.54

Størrelsen af en matrix er bestemt af antallet af rækker og søjler og angives som *rækker* $\times$ *søjler*. For eksempel har en koefficientmatrix med tre rækker og tre søjler størrelsen 3×3, og dens totalmatrix har størrelsen 3×4.

Definition A.55 (Echelonformer)

En matrix er på **echelonform**, hvis den opfylder følgende betingelser:

1. Alle nulrækker står nederst.
2. Hver ledende indgang i en række står i en søjle til højre for den ledende indgang i rækken ovenover.
3. Alle indgange i en søjle under en ledende indgang er nul.

Ud over echelonform kan en matrix også være på **reduceret echelonform** (RREF), som har følgende egenskaber:

4. Den ledende indgang i hver række, der ikke er en nulrække, er 1.
5. Hvert ledende 1-tal er den eneste indgang forskellig fra nul i sin søjle.

Sætning A.13 (Eksistenssætningen)

Et lineært ligningssystem er konsistent, hvis og kun hvis en echelonform af totalmatricen ikke har nogen række på formen

$$\begin{bmatrix} 0 & \ldots & 0 & b \end{bmatrix}$$

hvor b er forskellig fra nul.

Sætning A.14 (Entydighed af den reducerede echelonform)

Hver matrix er rækkeækvivalent med én og kun én matrix på reduceret echelonform

♡

Rækkereduktionsalgoritmen

Her beskriver vi en algoritme, der omdanner en vilkårlig matrix til en ækvivalent matrix på (reduceret) echelonform. Algoritmen er grundlaget for at løse lineære ligningssystemer ved hjælp af matricer.

1. Begynd med den søjle længst til venstre, der ikke er en nulsøjle. Det er en pivotsøjle, og pivotpositionen er øverst.
2. Vælg en indgang forskellig fra nul i pivotsøjlen som pivot. Byt om nødvendigt rækker for at flytte denne indgang til pivotpositionen.
3. Brug rækkeerstatninger til at skabe nuller i alle positioner under pivoten.
4. Anvend trin 1-3 på delmatricen af alle indgange under og til højre for pivotpositionen. Gentag processen, indtil der ikke er flere rækker forskellige fra nul at ændre. (Nu har vi nået en echelonform af matricen.)
5. Begynd med pivoten længst til højre, og arbejd opad og mod venstre: Skab nuller over hver pivot ved hjælp af rækkeoperationer. Hvis en pivot ikke er 1, gøres den til 1 ved en skalering. (Dette trin giver matricens reducerede echelonform.)

Sætning A.15 (Entydighedssætningen)

Hvis et lineært ligningssystem er konsistent, indeholder løsningsmængden enten

(i) en entydig løsning, når der ikke er nogen frie variable, eller

(ii) uendeligt mange løsninger, når der er mindst én fri variabel.

♡

Definition A.56

For ethvert positivt heltal n lader vi $\mathbb{R}^n$ betegne samlingen af ordnede n-tupler, hvor hver indgang ligger i $\mathbb{R}$. Vi skriver ofte disse elementer som $n \times 1$-matricer. Vi definerer addition og multiplikation med skalar for vektorer i $\mathbb{R}^n$ på samme måde som for $\mathbb{R}^2$. Det vil sige, at vi regner koordinat for koordinat.

♣

Definition A.57 (Linearkombinationer)

Givet vektorerne $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_p \in \mathbb{R}^n$ og skalarerne $c_1, c_2, \ldots, c_p \in \mathbb{R}$ kaldes vektoren $\boldsymbol{y}$ givet ved

$$\boldsymbol{y} = c_1\boldsymbol{v}_1 + \cdots + c_p\boldsymbol{v}_p$$

en linearkombination af $\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_p$ med vægtene $c_1, c_2, \ldots, c_p$.

♣

Definition A.58 (Udspændet af en mængde vektorer)

Hvis $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_p$ ligger i $\mathbb{R}^n$, betegnes mængden af alle linearkombinationer af $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_p$ med Span $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_p\}$, og den kaldes den delmængde af $\mathbb{R}^n$, der udspændes af $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_p$. Med andre ord består udspændet af $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_p$ af alle vektorer, der kan skrives på formen

$$c_1\boldsymbol{v}_1 + \cdots + c_p\boldsymbol{v}_p$$

hvor $c_1, \ldots, c_p$ er skalarer.

♣

Definition A.59 (Matrixligning)

Hvis A er en $m \times n$-matrix med søjlerne $\boldsymbol{a}_1, \ldots, \boldsymbol{a}_n$, og hvis $\boldsymbol{x} \in \mathbb{R}^n$, så er produktet af A og $\boldsymbol{x}$, betegnet $A\boldsymbol{x}$, givet ved

$$A\boldsymbol{x} = \begin{bmatrix} \boldsymbol{a}_1 & \boldsymbol{a}_2 & \ldots & \boldsymbol{a}_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = x_1\boldsymbol{a}_1 + x_2\boldsymbol{a}_2 + \cdots + x_n\boldsymbol{a}_n$$



Definition A.60

En indekseret mængde af vektorer $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_p\} \subset \mathbb{R}^n$ siges at være **lineært uafhængig**, hvis vektorligningen

$$x_1\boldsymbol{v}_1 + \cdots + x_p\boldsymbol{v}_p = \boldsymbol{0}$$

kun har den trivielle løsning, dvs. hvis den eneste løsning er $(x_1, \ldots, x_p) = (0, \ldots, 0)$. Tilsvarende siges mængden $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_p\}$ at være **lineært afhængig**, hvis der findes vægte $c_1, \ldots, c_p$, som ikke alle er nul, så

$$c_1\boldsymbol{v}_1 + \cdots + c_p\boldsymbol{v}_p = \boldsymbol{0}$$

Vi kalder en sådan ligning en *lineær afhængighedsrelation*, når vægtene ikke alle er nul.



Sætning A.16

Hvis A er en $m \times n$-matrix med søjlerne $\boldsymbol{a}_1, \ldots, \boldsymbol{a}_n \in \mathbb{R}^m$, og hvis $\boldsymbol{b} \in \mathbb{R}^m$, så har matrixligningen

$$A\boldsymbol{x} = \boldsymbol{b}$$

samme løsningsmængde som vektorligningen

$$x_1\boldsymbol{a}_1 + x_2\boldsymbol{a}_2 + \cdots + x_n\boldsymbol{a}_n = \boldsymbol{b}$$

som igen har samme løsningsmængde som det lineære ligningssystem med totalmatricen

$$\begin{bmatrix} \boldsymbol{a}_1 & \boldsymbol{a}_2 & \ldots & \boldsymbol{a}_n & \boldsymbol{b} \end{bmatrix}.$$



Sætning A.17

Lad A være en $m \times n$-matrix. Så er følgende udsagn ækvivalente:

(a) For hvert $\boldsymbol{b} \in \mathbb{R}^m$ har ligningen $A\boldsymbol{x} = \boldsymbol{b}$ en løsning.
(b) Hvert $\boldsymbol{b} \in \mathbb{R}^m$ er en linearkombination af søjlerne i A.
(c) Søjlerne i A udspænder $\mathbb{R}^m$.
(d) Matricen A har en pivotposition i hver række.



Sætning A.18

Hvis A er en $m \times n$-matrix, $\boldsymbol{u}$ og $\boldsymbol{v}$ er vektorer i $\mathbb{R}^n$, og c er en skalar, så gælder:

(a) $A(\boldsymbol{u} + \boldsymbol{v}) = A\boldsymbol{u} + A\boldsymbol{v}$
(b) $A(c\boldsymbol{u}) = c(A\boldsymbol{u})$



Sætning A.19

Antag, at $A\boldsymbol{x} = \boldsymbol{b}$ er konsistent for et bestemt $\boldsymbol{b}$, og lad $\boldsymbol{p}$ være en løsning. Så er løsningsmængden for $A\boldsymbol{x} = \boldsymbol{b}$ mængden af alle vektorer på formen

$$\boldsymbol{w} = \boldsymbol{p} + \boldsymbol{v}_h,$$

hvor $\boldsymbol{v}_h$ er en vilkårlig løsning til den homogene ligning $A\boldsymbol{x} = \boldsymbol{0}$.

Sætning A.20

(a) Hvis en mængde vektorer indeholder nulvektoren, er mængden lineært afhængig.

(b) Hvis en mængde vektorer indeholder et skalarmultiplum af en anden vektor, er mængden lineært afhængig.

(c) Hvis en mængde vektorer indeholder flere vektorer, end der er indgange i hver vektor, er mængden lineært afhængig.

Definition A.61 (Matrixaddition)

Lad A og B være $m \times n$-matricer. **Summen** af A og B, betegnet $A + B$, er den $m \times n$-matrix, hvis indgange fås ved at lægge de tilsvarende indgange i A og B sammen.
Givet matricerne

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}, \quad B = \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ b_{m1} & b_{m2} & \cdots & b_{mn} \end{bmatrix}$$

som begge har dimensionen $m \times n$, er summen $A + B$ altså defineret som

$$A + B = \begin{bmatrix} a_{11} + b_{11} & a_{12} + b_{12} & \cdots & a_{1n} + b_{1n} \\ a_{21} + b_{21} & a_{22} + b_{22} & \cdots & a_{2n} + b_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{m1} + b_{m1} & a_{m2} + b_{m2} & \cdots & a_{mn} + b_{mn} \end{bmatrix}$$

Definition A.62 (Multiplikation af en matrix med en skalar)

Lad A være en $m \times n$-matrix og c en skalar. **Produktet** af c og A, betegnet cA, er den $m \times n$-matrix, hvis indgange fås ved at gange hver indgang i A med c. Det er altså defineret som

$$cA = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} = \begin{bmatrix} ca_{11} & ca_{12} & \cdots & ca_{1n} \\ ca_{21} & ca_{22} & \cdots & ca_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ ca_{m1} & ca_{m2} & \cdots & ca_{mn} \end{bmatrix}$$

Definition A.63

For $A \in \mathbb{R}^{m \times n}$ og $B \in \mathbb{R}^{n \times p}$, hvor $B = \begin{bmatrix} \boldsymbol{b_1} & \boldsymbol{b_2} \cdots & \boldsymbol{b_p} \end{bmatrix}$, definerer vi produktet AB ved formlen

$$AB = \begin{bmatrix} A\boldsymbol{b_1} & A\boldsymbol{b_2} & \cdots & A\boldsymbol{b_p} \end{bmatrix}$$

Definition A.64

Lad A være en kvadratisk matrix, dvs. $A \in \mathbb{R}^{n \times n}$. Den k'te potens af A, betegnet A^k, er defineret som produktet af A med sig selv k gange. Det vil sige

$$A^k = \underbrace{AAA \cdots A}_{k \text{ gange}}$$

hvor A optræder k gange på højre side.

Definition A.65

Givet en matrix $A \in \mathbb{R}^{m \times n}$ er den transponerede af A den matrix A^T, hvis i'te søjle er den i'te række i A.

Definition A.66

En kvadratisk matrix $A \in \mathbb{R}^{n \times n}$ er invertibel (eller **ikke-singulær**), hvis der findes en matrix $C \in \mathbb{R}^{n \times n}$, så $AC = CA = I_n$. Matricen C kaldes den inverse til A og betegnes A^{-1}. Altså er $A^{-1}A = AA^{-1} = I_n$.

Sætning A.21

Lad A, B, C være matricer af samme størrelse, og lad α, β være skalarer. Så gælder

(a) $A + B = B + A$
(b) $(A + B) + C = A + (B + C)$
(c) $A + 0 = A$
(d) $\alpha(A + B) = \alpha A + \alpha B$
(e) $(\alpha + \beta)A = \alpha A + \beta A$
(f) $\alpha(\beta A) = (\alpha\beta)A$

Sætning A.22

Lad A, B, C være matricer med passende dimensioner, og lad α være en skalar. Så gælder

(a) $A(BC) = (AB)C$
(b) $A(B + C) = AB + AC$
(c) $(B + C)A = BA + CA$
(d) $\alpha(AB) = (\alpha A)B = A(\alpha B)$
(e) Hvis A er $m \times n$, så er $I_m A = A = AI_n$.

Sætning A.23

Lad A og B være matricer med passende dimensioner, og lad α være en skalar. Så gælder

(a) $(A^T)^T = A$
(b) $(A + B)^T = A^T + B^T$
(c) $(\alpha A)^T = \alpha A^T$
(d) $(AB)^T = B^T A^T$

Sætning A.24

Lad A og B være invertible $n \times n$-matricer. Så gælder:

(a) A^{-1} er invertibel, og
$$\left(A^{-1}\right)^{-1} = A$$
(b) Produktet AB er invertibelt, og
$$(AB)^{-1} = B^{-1}A^{-1}$$

(c) Den transponerede af A er også invertibel, dvs. A^T er invertibel, og
$$\left(A^T\right)^{-1} = \left(A^{-1}\right)^T$$

Sætning A.25

Lad $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$. Hvis $ad - bc \neq 0$, er den inverse til A givet ved
$$A^{-1} = \frac{1}{ad-bc}\begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

Sætning A.26

En $n \times n$-matrix A er invertibel, hvis og kun hvis A er rækkeækvivalent med I_n. I så fald vil enhver følge af elementære rækkeoperationer, der reducerer A til I_n, også omdanne I_n til A^{-1}.

Algoritme til at finde den inverse til en $n \times n$-matrix

Lad A være en $n \times n$-matrix. Følg disse trin for at finde den inverse til A:

1. Opstil totalmatricen $[A|I_n]$.
2. Udfør rækkeoperationer på $[A|I_n]$ for at reducere A til I_n.
3. Matricen i højre side af totalmatricen er A^{-1}.

Hvis A ikke er invertibel, vil algoritmen ikke kunne reducere A til I_n.

Sætning A.27

Lad A være en invertibel matrix. Så har ligningen $A\boldsymbol{x} = \boldsymbol{b}$ en entydig løsning, som er givet ved $\boldsymbol{x} = A^{-1}\boldsymbol{b}$.

Sætning A.28

Lad A være en $n \times n$-matrix. Følgende udsagn er ækvivalente:

(a) A er invertibel.
(b) A er rækkeækvivalent med I_n.
(c) A har n pivotpositioner (dvs. én for hver række og søjle).
(d) Ligningen $A\boldsymbol{x} = \boldsymbol{0}$ har kun den trivielle løsning.
(e) Søjlerne i A er lineært uafhængige.
(f) Ligningen $A\boldsymbol{x} = \boldsymbol{b}$ har en entydig løsning for hvert $\boldsymbol{b} \in \mathbb{R}^n$.
(g) Søjlerne i A udspænder $\mathbb{R}^n$.
(h) $\det(A) \neq 0$.
(i) Der findes en $n \times n$-matrix C, så $CA = I$.
(j) Der findes en $n \times n$-matrix D, så $AD = I$.
(k) A^T er invertibel.

Definition A.67 (Grænseværdi)

Lad $f(x)$ være en funktion, der er defineret i nærheden af et punkt c. Vi siger, at **grænseværdien** af $f(x)$ for x gående mod c er L, hvilket skrives

$$\lim_{x \to c} f(x) = L$$

hvis vi kan få værdien af $f(x)$ vilkårligt tæt på L ved at vælge et x, der ligger tilstrækkelig tæt på c, men ikke er lig med c.

Definition A.68 (Kontinuitet)

En funktion f er **kontinuert** i et punkt c, hvis følgende tre betingelser er opfyldt:

1. $f(c)$ er defineret.
2. $\lim_{x \to c} f(x)$ eksisterer.
3. $\lim_{x \to c} f(x) = f(c)$.

En funktion er kontinuert på et interval, hvis den er kontinuert i alle intervallets punkter.

Definition A.69 (Den afledte funktion)

Den **afledte funktion** af en funktion f med hensyn til x, betegnet $f'(x)$, er funktionen

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

forudsat at grænseværdien eksisterer. Hvis $f'(x)$ eksisterer, siger vi, at f er **differentiabel** i x.

Definition A.70

For en funktion $y = f(x)$ kaldes de punkter på grafen, hvor grafen har hældningen nul, for stationære punkter. Med andre ord er de stationære punkter dér, hvor $f'(x) = 0$.

Definition A.71 (Den anden afledte)

Den **anden afledte** af en funktion f, betegnet $f''(x)$, er funktionen

$$f''(x) = \frac{d}{dx}\left(f'(x)\right)$$

forudsat at grænseværdien eksisterer.

Sætning A.29 (Konstantreglen og potensreglen)

1. **Konstantreglen:** Hvis c er en konstant, så er $\frac{d}{dx}(c) = 0$,
2. **Potensreglen:** For ethvert reelt n gælder i de punkter, hvor den reelle potensfunktion er defineret og differentiabel, at $\frac{d}{dx} x^n = n x^{n-1}$.

Sætning A.30 (Skalarreglen og linearitetsreglen)

3. **Skalarreglen:** $\frac{d}{dx}[c \cdot f(x)] = c \cdot f'(x)$.
4. **Linearitetsreglen:** $\frac{d}{dx}[f(x) \pm g(x)] = f'(x) \pm g'(x)$.

Sætning A.31 (Produktreglen og kvotientreglen)

5. **Produktreglen:** Hvis $f(x)$ og $g(x)$ er differentiable, så gælder

$$\frac{d}{dx}(f(x)g(x)) = f'(x)g(x) + f(x)g'(x).$$

6. **Kvotientreglen:** Hvis $f(x)$ og $g(x)$ er differentiable, og $g(x) \neq 0$, så gælder
$$\frac{d}{dx}\left(\frac{f(x)}{g(x)}\right) = \frac{f'(x)g(x) - f(x)g'(x)}{[g(x)]^2}.$$

♡

Sætning A.32 (Kædereglen)

7. Hvis $h(x) = f(g(x))$ er en sammensat funktion, er dens afledte lig med den afledte af den ydre funktion (evalueret i den indre funktion) ganget med den afledte af den indre funktion.
$$h'(x) = f'(g(x)) \cdot g'(x).$$

♡

Sætning A.33 (Differentiationsregler)

1. **Konstantreglen:** Hvis c er en konstant, så er $\frac{d}{dx}c = 0$.
2. **Potensreglen:** For ethvert reelt n gælder i de punkter, hvor den reelle potensfunktion er defineret og differentiabel, at $\frac{d}{dx}x^n = nx^{n-1}$.
3. **Skalarreglen:** Hvis c er en konstant, så er $\frac{d}{dx}[c \cdot f(x)] = c \cdot f'(x)$.
4. **Linearitet:** For differentiable f, g og konstanter a, b gælder $\frac{d}{dx}(af + bg) = af' + bg'$.
5. **Produktreglen:** Hvis f, g er differentiable, så er $\frac{d}{dx}(fg) = f'g + fg'$.
6. **Kvotientreglen:** Hvis f, g er differentiable, og $g(x) \neq 0$, så er $\frac{d}{dx}\left(\frac{f}{g}\right) = \frac{f'g - fg'}{g^2}$.
7. **Kædereglen:** Hvis f, g er differentiable, så er $\frac{d}{dx}(f(g(x))) = f'(g(x))g'(x)$.

♡

Sætning A.34 (Første afledtes test)

Lad c være et kritisk punkt for en kontinuert funktion f.

- Hvis $f'(x)$ skifter fra negativ til positiv i c, har f et **lokalt minimum** i c.
- Hvis $f'(x)$ skifter fra positiv til negativ i c, har f et **lokalt maksimum** i c.
- Hvis $f'(x)$ ikke skifter fortegn i c, har f intet lokalt ekstremum i c. Det alene medfører ikke, at c er et vendepunkt.

♡

Sætning A.35 (Anden afledtes test)

Lad c være et stationært punkt for f (dvs. $f'(c) = 0$).

- Hvis $f''(c) > 0$, har f et **lokalt minimum** i c.
- Hvis $f''(c) < 0$, har f et **lokalt maksimum** i c.
- Hvis $f''(c) = 0$, giver testen ingen konklusion.

♡

Definition A.72 (Funktion af to variable)

En *funktion af to variable* er en regel, der til hvert ordnet par (x, y) i en mængde $D \subseteq \mathbb{R}^2$ knytter et entydigt reelt tal z, som vi skriver som $z = f(x, y)$. Mængden D kaldes *definitionsmængden* for f.

Værdimængden for f er mængden af alle reelle tal z, for hvilke der findes mindst ét $(x, y) \in D$, så $f(x, y) = z$. Det er illustreret i Figur 11.1.

Definition A.73 (Niveaukurve og konturkort)

En **niveaukurve** for en funktion $f(x, y)$ er mængden af alle punkter (x, y) i inputplanen, hvor funktionen har en konstant værdi, dvs. $f(x, y) = c$ for en konstant c.
En samling af niveaukurver for forskellige værdier af c udgør et **konturkort**.

Definition A.74 (Partielle afledte)

Den **partielle afledte** af $f(x, y)$ med hensyn til x, betegnet $\frac{\partial f}{\partial x}$ eller f_x, er

$$\frac{\partial f}{\partial x} = \lim_{h \to 0} \frac{f(x + h, y) - f(x, y)}{h}$$

Den **partielle afledte** med hensyn til y, betegnet $\frac{\partial f}{\partial y}$ eller f_y, er

$$\frac{\partial f}{\partial y} = \lim_{h \to 0} \frac{f(x, y + h) - f(x, y)}{h}$$

Definition A.75 (Gradienten)

Gradienten af en funktion $f(x, y)$, betegnet ∇f, er den vektorfunktion, der er defineret ved:

$$\nabla f(x, y) = \left\langle \frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right\rangle = \frac{\partial f}{\partial x}\mathbf{i} + \frac{\partial f}{\partial y}\mathbf{j}$$

Kommutative, associative og distributive love

- Kommutativitet: $a + b = b + a$
- Associativitet: $(a + b) + c = a + (b + c)$
- Den distributive lov: $a \times (b + c) = a \times b + a \times c$

Lukkethed

Addition: Mængden af hele tal er lukket under addition. Det betyder, at hvis man tager to vilkårlige hele tal og lægger dem sammen, vil summen altid være et helt tal. Hvis a og b for eksempel er hele tal, så er $a + b$ også et helt tal. Denne lukkethedsegenskab sikrer, at mængden af hele tal er stabil og fuldstændig under addition: Uanset hvor mange gange man lægger hele tal sammen, forbliver resultatet inden for mængden af hele tal.
Multiplikation: Mængden af hele tal er også lukket under multiplikation. Hvis man ganger to vilkårlige hele tal, vil produktet altid være et helt tal. Hvis a og b for eksempel er hele tal, så er $a \times b$ også et helt tal. Denne egenskab garanterer, at multiplikation ligesom addition ikke giver resultater uden for mængden af hele tal, og bevarer dermed mængdens integritet under multiplikation.

Ligheder og forskelle mellem rationale og irrationale tal

- **Repræsentation:** Rationale tal kan skrives som brøker, mens irrationale tal ikke kan. Denne forskel gør irrationale tal mere komplicerede at håndtere, især i regneoperationer.
- **Decimaludvikling:** Decimaludviklingen af et rationalt tal er enten endelig eller periodisk, mens decimaludviklingen af et irrationalt tal er uendelig og ikke-periodisk.
- **Lukkethed:** De rationale tal er lukkede under addition, subtraktion, multiplikation og division (undtagen division med nul). De irrationale tal er ikke lukkede under disse operationer; for eksempel kan summen eller produktet af to irrationale tal undertiden være rationalt.
- **Tæthed:** De rationale tal ligger tæt på den reelle tallinje, dvs. at der mellem to vilkårlige rationale tal findes et andet rationalt tal. De irrationale tal ligger også tæt, men på en komplementær måde: De udfylder de »huller«, som de rationale tal efterlader, og sikrer dermed, at den reelle tallinje er sammenhængende uden afbrydelser.

Sætning A.36 (Aritmetikkens fundamentalsætning)

Ethvert helt tal større end 1 kan skrives entydigt som et produkt af primtal, bortset fra faktorernes rækkefølge.

♡

Primære booleske operationer

- **Komplementering** (betegnet `NOT`, fx $\overline{0}$)
- **Boolesk sum** (betegnet `OR`, fx $1 + 0$)
- **Boolesk produkt** (betegnet `AND`, fx $1 \cdot 0$)

Grundlæggende booleske egenskaber

- **Dobbelt komplement:**
  $\overline{\overline{x}} = x$
- **Idempotenslove:**
  $x + x = x, \quad x \cdot x = x$
- **Identitetslove:**
  $x + 0 = x, \quad x \cdot 1 = x$
- **Dominanslove:**
  $x + 1 = 1, \quad x \cdot 0 = 0$
- **Enhedsegenskaben:**
  $\overline{x} + x = 1$
- **Nulegenskaben:**
  $\overline{x} \cdot x = 0$

Strukturelle booleske love

- **Kommutative love:**
  $x + y = y + x, \quad x \cdot y = y \cdot x$
- **Absorptionslove:**
  $x + (x \cdot y) = x, \quad x \cdot (x + y) = x, \quad \bar{x} \cdot \bar{y} + x = \bar{y} + x$
- **De Morgans love:**
  $\overline{x \cdot y} = \overline{x} + \overline{y}, \quad \overline{x + y} = \overline{x} \cdot \overline{y}$
- **Associative love:**
  $x + (y + z) = (x + y) + z, \quad x \cdot (y \cdot z) = (x \cdot y) \cdot z$
- **Distributive love:**
  $x \cdot (y + z) = (x \cdot y) + (x \cdot z), \quad x + (y \cdot z) = (x + y) \cdot (x + z)$

Avancerede booleske identiteter

- **Eksklusivt `OR` (`XOR`):**
  $x \oplus y = (x \cdot \overline{y}) + (\overline{x} \cdot y)$
- **Eksklusivt `NOR` (`XNOR`):**
  $\overline{x \oplus y} = (x \cdot y) + (\overline{x} \cdot \overline{y})$
- **Forsvindende modsætning:**
  $x + \overline{x} \cdot y = x + y$
- **FOIL-loven (First, Outer, Inner, Last – første, ydre, indre, sidste):**
  $(x + y) \cdot (z + w) = (x \cdot z) + (x \cdot w) + (y \cdot z) + (y \cdot w)$

I de følgende definitioner er f og g ikke-negative funktioner, og $g(n) > 0$ for alle tilstrækkelig store n.

Definition A.76 (Theta(Θ))

$f(n)$ er $\Theta(g(n))$, hvis der findes positive konstanter c_1, c_2 og k, så der for alle $n \geq k$ gælder

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

Det betyder, at $f(n)$ asymptotisk er begrænset både opadtil og nedadtil af $g(n)$, dvs. at $f(n)$ vokser med samme hastighed som $g(n)$, når n bliver tilstrækkelig stor.

Proposition A.10

Lad $f(n)$ og $g(n)$ være to ikke-negative funktioner med $g(n) > 0$ fra et vist trin, og antag, at der findes en konstant $c > 0$, så

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = c$$

Så er $f(n) = \Theta(g(n))$. Hvis desuden

$$\lim_{n \to \infty} \frac{f(n)}{g(n)}$$

eksisterer, men **ikke** er lig med en positiv konstant, så er $f(n) \neq \Theta(g(n))$.

Definition A.77 (Store $\mathcal{O}$)

$f(n)$ er $\mathcal{O}(g(n))$, hvis der findes positive konstanter c og k, så der for alle $n \geq k$ gælder

$$f(n) \leq c \cdot g(n)$$

Det betyder, at $f(n)$ asymptotisk er begrænset opadtil af $g(n)$, dvs. at $f(n)$ højst vokser lige så hurtigt som $g(n)$, når n bliver tilstrækkelig stor.

Definition A.78 (Omega-notation)

$f(n)$ er $\Omega(g(n))$, hvis der findes positive konstanter c og k, så der for alle $n \geq k$ gælder

$$f(n) \geq c \cdot g(n)$$

Det betyder, at $f(n)$ asymptotisk er begrænset nedadtil af $g(n)$, dvs. at $f(n)$ mindst vokser lige så hurtigt som $g(n)$, når n bliver tilstrækkelig stor.

Definition A.79 (Lille o)

$f(n)$ er $o(g(n))$, hvis der for enhver positiv konstant $\epsilon > 0$ findes en konstant $k \geq 0$, så der for alle $n \geq k$ gælder

$$f(n) < \epsilon \cdot g(n)$$

Det betyder, at $f(n)$ vokser strengt langsommere end $g(n)$, når n bliver tilstrækkelig stor.

Proposition A.11

Lad $f(n)$ og $g(n)$ være to ikke-negative funktioner med $g(n) > 0$ fra et vist trin, og antag, at

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$$

Så er $f(n) = o(g(n))$, dvs. at $f(n)$ vokser strengt langsommere end $g(n)$ for $n \to \infty$.

Definition A.80 (Lille ω)

$f(n)$ er $\omega(g(n))$, hvis der for enhver positiv konstant $\epsilon > 0$ findes en konstant $k \geq 0$, så der for alle $n \geq k$ gælder

$$f(n) > \epsilon \cdot g(n)$$

Det betyder, at $f(n)$ vokser strengt hurtigere end $g(n)$, når n bliver tilstrækkelig stor.

Proposition A.12

Lad $f(n)$ og $g(n)$ være to ikke-negative funktioner med $g(n) > 0$ fra et vist trin, og antag, at

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = \infty$$

Så er $f(n) = \omega(g(n))$, dvs. at $f(n)$ vokser strengt hurtigere end $g(n)$ for $n \to \infty$.

Proposition A.13

Lad $f(n)$ og $g(n)$ være to ikke-negative funktioner med $g(n) > 0$ fra et vist trin. Hvis

$$\limsup_{n\to\infty} \frac{f(n)}{g(n)} < \infty$$

så er $f(n) = \mathcal{O}(g(n))$, dvs. at $f(n)$ asymptotisk højst vokser lige så hurtigt som $g(n)$.

Proposition A.14

Lad $f(n)$ og $g(n)$ være to ikke-negative funktioner med $g(n) > 0$ fra et vist trin. Hvis

$$\liminf_{n\to\infty} \frac{f(n)}{g(n)} > 0$$

så er $f(n) = \Omega(g(n))$, dvs. at $f(n)$ asymptotisk mindst vokser lige så hurtigt som $g(n)$.

Uformel grænseværdimetode

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = L < \infty \quad \Rightarrow f(n) = \mathcal{O}(g(n))$$

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = L > 0 \quad \Rightarrow f(n) = \Omega(g(n))$$

Bemærk: Denne metode er en uformel rettesnor og ikke en streng matematisk definition. Der findes tilfælde, hvor den grænseværdibaserede metode ikke kan anvendes, men den er ofte et bekvemt værktøj til at bestemme asymptotisk opførsel.

Bilag B: Summer

Dette bilag præsenterer metoder til at beregne summer, som ofte optræder i analysen af algoritmer. Mange af formlerne her findes i enhver lærebog i infinitesimalregning, men det er praktisk at have metoderne samlet ét sted. Indholdet i bilaget bygger på *Introduction to Algorithms* Cormen m.fl., 2022. Bilaget afsluttes med et nyttigt oversigtsark til tidskompleksitetsanalyse.

Når en algoritme indeholder en iterativ kontrolstruktur som en løkke af typen **while** eller **for**, kan du udtrykke dens køretid som summen af den tid, der bruges på hver udførelse af løkkens krop. I Afsnit 15.3 argumenterede vi for eksempel for, at den i'te iteration af QUADRATIC-SUM i værste tilfælde tog tid proportional med i. Ved at lægge den tid sammen, der blev brugt på hver iteration, fik vi summen (eller rækken) $\sum_{i=1}^{n} i$. Beregningen af denne sum gav en grænse på $\mathcal{O}(n^2)$ for algoritmens køretid i værste tilfælde. Eksemplet viser, hvorfor du bør kunne regne med summer. Før vi går i detaljer, giver vi et overblik over de vigtigste summer og deres lukkede former.

Sum	**Lukket form**
$\sum_{k=0}^{n} ar^k \ (r \neq 1)$	$\frac{ar^{n+1} - a}{r - 1}$
$\sum_{k=1}^{n} k$	$\frac{n(n+1)}{2}$
$\sum_{k=1}^{n} k^2$	$\frac{n(n+1)(2n+1)}{6}$
$\sum_{k=1}^{n} k^3$	$\frac{n^2(n+1)^2}{4}$
$\sum_{k=0}^{n} x^k \ (x \neq 1)$	$\frac{x^{n+1} - 1}{x - 1}$
$\sum_{i=1,2,4,\ldots,n} i$ (n en potens af 2)	$2n - 1$
$\sum_{k=0}^{\infty} x^k, \ \lvert x \rvert < 1$	$\frac{1}{1 - x}$
$\sum_{k=1}^{\infty} kx^{k-1}, \ \lvert x \rvert < 1$	$\frac{1}{(1 - x)^2}$
$\sum_{k=1}^{n} \frac{1}{k}$	$\ln n + O(1)$
$\sum_{k=1}^{n} (a + bk)$	$an + b\frac{n(n+1)}{2}$

Tabel B.1: Nogle nyttige sumformler

Sumnotation

Betragt en følge af tal $a_1, a_2, \ldots, a_n$, hvor n er et ikke-negativt heltal. Summen af denne følge, $a_1 + a_2 + \cdots + a_n$, kan skrives med notationen $\sum_{k=1}^{n} a_k$. Når $n = 0$, defineres summen til at have værdien 0. Resultatet af en endelig sum er altid veldefineret, og den rækkefølge, leddene lægges sammen i, påvirker ikke slutværdien.

For en uendelig følge $a_1, a_2, \ldots$ af tal skriver vi den uendelige sum $a_1 + a_2 + \ldots$ med notationen $\sum_{k=1}^{\infty} a_k$, som svarer til $\lim_{n\to\infty} \sum_{k=1}^{n} a_k$. Hvis denne grænseværdi eksisterer, siges rækken at være konvergent; ellers er den divergent. I modsætning til endelige summer kan leddene i en konvergent række ikke nødvendigvis byttes om uden at påvirke resultatet. I en absolut konvergent række – dvs. en række, hvor også $\sum_{k=1}^{\infty} |a_k|$ konvergerer – kan leddene dog ombyttes uden at ændre summen.

$$\sum_{i=i_0}^{n} ca_i = c \sum_{i=i_0}^{n} a_i$$

hvor c er et vilkårligt tal. Vi kan altså sætte konstanter uden for en sum.

På samme måde kan vi dele en sum op i to summer:

$$\sum_{i=i_0}^{n} (a_i \pm b_i) = \sum_{i=i_0}^{n} a_i \pm \sum_{i=i_0}^{n} b_i$$

Bemærk, at vi har ladet rækkerne starte ved i_0 for at markere, at de kan starte ved en hvilken som helst værdi af i, vi har brug for. Bemærk også, at selv om vi kan dele summer og differenser op som ovenfor, kan vi ikke gøre det samme med produkter og kvotienter. Med andre ord:

$$\sum_{i=i_0}^{n} (a_i b_i) \neq \left(\sum_{i=i_0}^{n} a_i \right) \left(\sum_{i=i_0}^{n} b_i \right) \qquad \text{og} \qquad \sum_{i=i_0}^{n} \frac{a_i}{b_i} \neq \frac{\sum_{i=i_0}^{n} a_i}{\sum_{i=i_0}^{n} b_i}.$$

Summers linearitet

For ethvert reelt tal c og alle endelige følger $a_1, a_2, \ldots, a_n$ og $b_1, b_2, \ldots, b_n$ gælder

$$\sum_{k=1}^{n} (ca_k + b_k) = c \sum_{k=1}^{n} a_k + \sum_{k=1}^{n} b_k.$$

Linearitetsegenskaben gælder også for uendelige konvergente rækker.

Med ikke-negative funktioner og konstanter i Θ-grænserne, som er uniforme i k, gælder samme idé for summer, der indeholder asymptotisk notation. For eksempel er

$$\sum_{k=1}^{n} \Theta(f(k)) = \Theta\left(\sum_{k=1}^{n} f(k) \right).$$

I denne ligning gælder Θ-notationen på venstre side variablen k, mens den på højre side gælder n. Sådanne omskrivninger gælder også for uendelige konvergente rækker.

Aritmetiske rækker

Summen

$$\sum_{k=1}^{n} k = 1 + 2 + \cdots + n$$

er en aritmetisk række og har værdien

$$\sum_{k=1}^{n} k = \frac{n(n+1)}{2} \tag{B.1}$$

$$= \Theta\left(n^2\right). \tag{B.2}$$

En generel aritmetisk række har en additiv konstant $a \geq 0$ og en konstant koefficient $b > 0$ i hvert led, men har asymptotisk samme sum:

$$\sum_{k=1}^{n} (a + bk) = \Theta\left(n^2\right). \tag{B.3}$$

Summer af kvadrater og kubiktal

Følgende formler gælder for summer af kvadrater og kubiktal:

$$\sum_{k=0}^{n} k^2 = \frac{n(n+1)(2n+1)}{6}, \tag{B.4}$$

$$\sum_{k=0}^{n} k^3 = \frac{n^2(n+1)^2}{4}. \tag{B.5}$$

Geometriske rækker

For reelle $x \neq 1$ er summen

$$\sum_{k=0}^{n} x^k = 1 + x + x^2 + \cdots + x^n$$

en geometrisk række og har værdien

$$\sum_{k=0}^{n} x^k = \frac{x^{n+1} - 1}{x - 1}. \tag{B.6}$$

Den uendelige aftagende geometriske række opstår, når summen er uendelig, og $|x| < 1$:

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1 - x}. \tag{B.7}$$

Hvis vi antager, at $0^0 = 1$, gælder formlerne også, når $x = 0$.

Et **særtilfælde** af den geometriske række opstår, når leddene er potenser af 2. Antag, at n selv er en potens af 2, så leddene er $1, 2, 4, 8, \ldots, n$. I dette tilfælde er:

$$\sum_{i=1,2,4,\ldots,n} i = 1 + 2 + 4 + 8 + \cdots + n.$$

Denne række kan udledes af den generelle formel for en geometrisk række. Den generelle form for en geometrisk række er:

$$S = a + ar + ar^2 + \cdots + ar^{k-1},$$

hvor:

- a er det første led ($a = 1$ i dette tilfælde),
- r er kvotienten ($r = 2$ her),
- k er antallet af led i rækken.

Antallet af led, k, er bestemt af, hvor mange gange følgen $1, 2, 4, \ldots, n$ fordobles, indtil den når n. Da $n = 2^{k-1}$, har vi $k = \log_2(n) + 1$.

Med formlen for den geometriske række (B.6) får vi:

$$\sum_{i=1,2,4,\ldots,n} i = \frac{2^k - 1}{2 - 1} = 2^k - 1.$$

Ved at indsætte $k = \log_2(n) + 1$ får vi:

$$\sum_{i=1,2,4,\ldots,n} i = 2^{\log_2(n)+1} - 1 = 2n - 1.$$

Summen af potenserne af 2 op til n er altså:

$$\sum_{i=1,2,4,\ldots,n} i = 2n - 1.$$

Harmoniske rækker

For positive heltal n er det n'te harmoniske tal

$$\begin{aligned} H_n &= 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \cdots + \frac{1}{n} \\ &= \sum_{k=1}^{n} \frac{1}{k} && \text{(B.8)} \\ &= \ln n + O(1). && \text{(B.9)} \end{aligned}$$

Integration og differentiation

Ved at integrere eller differentiere formlerne ovenfor får man yderligere formler. Hvis man for eksempel differentierer begge sider af den uendelige geometriske række (B.7) og ganger med x, får man

$$\sum_{k=0}^{\infty} kx^k = \frac{x}{(1-x)^2} \qquad \text{(B.10)}$$

for $|x| < 1$.

Teleskoprækker

For enhver følge $a_0, a_1, \ldots, a_n$ gælder

$$\sum_{k=1}^{n} (a_k - a_{k-1}) = a_n - a_0, \tag{B.11}$$

da hvert af leddene $a_1, a_2, \ldots, a_{n-1}$ lægges til præcis én gang og trækkes fra præcis én gang. Vi siger, at summen teleskoperer. Tilsvarende gælder

$$\sum_{k=0}^{n-1} (a_k - a_{k+1}) = a_0 - a_n.$$

Som eksempel på en teleskopsum kan vi se på rækken

$$\sum_{k=1}^{n-1} \frac{1}{k(k+1)}$$

Når hvert led omskrives til

$$\frac{1}{k(k+1)} = \frac{1}{k} - \frac{1}{k+1},$$

får vi

$$\begin{aligned} \sum_{k=1}^{n-1} \frac{1}{k(k+1)} &= \sum_{k=1}^{n-1} \left(\frac{1}{k} - \frac{1}{k+1} \right) \\ &= 1 - \frac{1}{n}. \end{aligned} \tag{B.12}$$

Omindeksering af summer

En række kan undertiden forenkles ved at ændre dens indeks, ofte ved at vende summationsrækkefølgen om. Betragt rækken $\sum_{k=0}^{n} a_{n-k}$. Da leddene i denne sum er $a_n, a_{n-1}, \ldots, a_0$, kan vi vende indeksernes rækkefølge om ved at sætte $j = n - k$ og omskrive summen til

$$\sum_{k=0}^{n} a_{n-k} = \sum_{j=0}^{n} a_j$$

Generelt er det værd at overveje en omindeksering, hvis summationsindekset optræder med minustegn i summens led.

Betragt som eksempel summen

$$\sum_{k=1}^{n} \frac{1}{n-k+1}. \tag{B.13}$$

Indekset k optræder med negativt fortegn i $\dfrac{1}{(n-k+1)}$. Vi kan da også forenkle denne sum, denne gang ved at sætte $j = n - k + 1$, hvilket giver

$$\sum_{k=1}^{n} \frac{1}{n-k+1} = \sum_{j=1}^{n} \frac{1}{j}. \tag{B.14}$$

som netop er den harmoniske række (B.8).

Produkter

Det endelige produkt $a_1 a_2 \dots a_n$ kan skrives som

$$\prod_{k=1}^{n} a_k.$$

Hvis $n = 0$, defineres produktets værdi til at være 1 . Man kan omdanne en formel med et produkt til en formel med en sum ved hjælp af identiteten

$$\log\left(\prod_{k=1}^{n} a_k\right) = \sum_{k=1}^{n} \log a_k, \qquad a_k > 0.$$

Oversigtsark til tidskompleksitetsanalyse

Dette afsnit giver de vigtigste formler og genveje til asymptotisk analyse og vurdering af algoritmer. De supplerer de detaljerede summationsmetoder i dette bilag.

- **Logaritmisk tilnærmelse af fakultet (Stirlings approksimation):**
  $$\ln(n!) = n \ln n - n + \mathcal{O}(\ln n).$$
- **Reglen for logaritmisk eksponent:**
  $$2^{\log_2 n} = n.$$
- **Grundtal som potens med logaritmisk eksponent:** Hvis grundtallet er a^b, og eksponenten indeholder $\log_a$, så gælder:
  $$(a^b)^{\log_a(n^c)} = n^{bc}, \qquad a > 0,\ a \neq 1,\ n > 0.$$
  Eksempel: $4^{\log_2(n^2)} = n^4$ (da $4 = 2^2$).
- **Reglen for logaritmen til en potens:**
  $$\log(n^k) = k \log n, \quad \text{for } k > 0.$$
- **Logaritmisk vækst for potenser af 2:** For $n = 2^k$ gælder
  $$k = \log_2 n.$$
- **Forenkling af logaritmiske udtryk:**
  $$\log_a n = \frac{\log_b n}{\log_b a}, \quad \text{for } a, b, n > 0 \text{ med } a \neq 1 \text{ og } b \neq 1.$$
- **Almindelige asymptotiske sammenligninger:**
  $$n^c \ll c^n \quad \text{for enhver konstant } c > 1.$$
- **Fakultetets asymptotik (udvidet Stirlings approksimation):** Ifølge Stirlings formel er væksten af $n!$ tilnærmelsesvis:
  $$n! \sim \sqrt{2\pi n}\left(\frac{n}{e}\right)^n,$$
- **Sum af potenser af 2:** Hvis n er en potens af 2, og leddene er $1, 2, 4, 8, \dots, n$, er summen:
  $$\sum_{i=1,2,4,\dots,n} i = 2n - 1.$$